O'REILLY®



Vector Databases

A Practical Introduction

Nitin Borwankar

Vector Databases

A Practical Introduction

Nitin Borwankar



OceanofPDF.com

Vector Databases

by Nitin Borwankar

Copyright © 2026 Nitin Borwankar. All rights reserved.

Published by O’Reilly Media, Inc., 141 Stony Circle, Suite 195, Santa Rosa, CA 95401.

O’Reilly books may be purchased for educational, business, or sales promotional use. Online editions are also available for most titles (*http://oreilly.com*). For more information, contact our corporate/institutional sales department: 800-998-9938 or *corporate@oreilly.com*.

Acquisitions Editor: Andy Kwan

Development Editor: Sara Hunter

Production Editor: Beth Kelly

Copyeditor: Audrey Doyle

Proofreader: Krsta Technology Solutions

Indexer: WordCo Indexing Services, Inc.

Cover Designer: Susan Thompson

Cover Illustrator: José Marzan Jr.

Interior Designer: David Futato

Interior Illustrator: Kate Dullea

April 2026: First Edition

Revision History for the First Edition

- 2026-04-06: First Release

See *http://oreilly.com/catalog/errata.csp?isbn=9781098177591* for release details.

The O’Reilly logo is a registered trademark of O’Reilly Media, Inc. *Vector Databases*, the cover image, and related trade dress are trademarks of O’Reilly Media, Inc.

The views expressed in this work are those of the author and do not represent the publisher’s views. While the publisher and the author have used good faith efforts to ensure that the information and instructions contained in this work are accurate, the publisher and the author disclaim all responsibility for errors or omissions, including without limitation responsibility for damages resulting from the use of or reliance on this work. Use of the information and instructions contained in this work is at your own risk. If any code samples or other technology this work contains or describes is subject to open source licenses or the intellectual property rights of others, it is your responsibility to ensure that your use thereof complies with such licenses and/or rights.

978-1-098-17759-1

[LSI]

OceanofPDF.com

Preface

To repeat a well-worn phrase, this is the book I wish I had when I started exploring vector databases. I was looking for a single source of basic theory and practical applications of vector databases, but I found none.

Existing vector database resources are fragmented across vendor documentation, blog posts, YouTube videos, and GitHub code. I wanted practical working examples rather than theoretical coverage. I also have a personal preference for proceeding systematically from basics to simple working systems to help organize data locally—especially private data.

Thus began my journey to pull together information from multiple sources and multiple directions into a coherent whole. And this book is what emerged. It is meant to be useful to the intermediate developer, from the theoretically minded to the “just show me the code” type. There’s something for everyone.

The book follows a clear pedagogical arc: concepts → tools → applications → future directions. Each chapter builds on previous ones while remaining practical with working code examples. The progression moves from understanding vector databases to building working applications that are meant for personal use, hands-on interaction, and extension by developers who want to build personal data management systems. It culminates in a forward-looking proposal for industry standardization of a Vector Query Language modeled after SQL.

The somewhat opinionated part of this book’s approach is a focus on simple hybrid vector relational databases—created by adding vector extension plug-ins to the well-known open source databases SQLite3 and PostgreSQL. The reason for this is that these databases are well known among database developers. Simply adding a data type of “vector” opens up a whole new world of AI data applications. This makes onboarding as low-effort as possible.

> **NOTE**
>
> It can't be emphasized enough that these applications are for hands-on learning and not for web-scale production deployment. Modify them as you see fit; make them your own.

And now let's dive into the contents.

What's in This Book

Chapters 1 and 2 establish foundational concepts—vector databases, embeddings, and semantic search fundamentals:

Chapter 1, "Introduction to Vector Databases"

- Introduces vector databases as foundational technology for AI applications, explaining why unstructured data (text, images, audio, video) requires semantic search rather than keyword matching
- Covers the vector data type, similarity search, and key operations (cosine similarity, nearest-neighbor search, vector arithmetic)
- Contrasts vector databases with SQL and NoSQL databases, highlighting hybrid architectures that combine structured metadata with vector similarity search
- Establishes the conceptual foundation for understanding how embeddings preserve semantic meaning in vector space

Chapter 2, "Embeddings"

- Takes a deep dive into the history, evolution, and current usage of vector embeddings with code examples for language model applications
- Explains how embeddings bridge the gap between unstructured data and machine-processable representations, mapping raw data into

vectors that capture semantic relationships

- Covers the mathematical foundations and how proximity in vector space reflects semantic similarity
- Provides the conceptual framework needed for practical applications later in the book

Chapters 3 and 4 discuss practical implementations with Facebook AI Similarity Search (FAISS) and SQLite, showing how to build personal-scale search systems:

Chapter 3, "Similarity Search with FAISS"

- Explores FAISS as both a production similarity search engine and a flexible toolkit for building custom vector databases
- Is designed for developers interested in the internals of vector search and those wanting to extend capabilities of systems such as sqlite-vss
- Covers the mathematical foundations and implementation details of efficient similarity search at scale
- Focuses on CPU-based implementations for maximum accessibility

Chapter 4, "Semantic Search with SQLite3"

- Builds a personal knowledge management system using semantic search over Reddit content, demonstrating meaning-based search versus keyword matching
- Introduces sqlite-vss, a SQLite extension that wraps FAISS and integrates vector search into the relational database world
- Combines semantic retrieval with relational metadata filtering in a

single SQL workflow

- Uses an “overfetch-then-filter” pattern to handle the fact that metadata filters aren’t pushed down into FAISS search

Chapters 5 through 8 progress through increasingly sophisticated applications—ArXiv paper search, local retrieval-augmented generation (RAG) systems, scientific RAG, and personal conversation search—moving from public to private data:

Chapter 5, “Building an ArXiv Paper Search System with PostgreSQL pgvector”

- Constructs a scientific literature search system addressing the challenge of discovering relevant research across millions of papers
- Demonstrates how vector embeddings capture semantic relationships that keyword search misses (e.g., connecting “neural network optimization” with “gradient descent improvements”)
- Leverages PostgreSQL with the pgvector extension for enterprise-scale applications
- Shows how to handle technical terminology, structured academic content, and metadata in a unified search system

Chapter 6, “Building a Retrieval-Augmented Generation System with SQLite VSS and Ollama”

- Assembles a complete RAG system combining vector search with local large language model (LLM) capabilities
- Builds an entirely local, private system running on a single desktop using SQLite Vector Similarity Search (SQLite VSS) and Ollama
- Addresses the fundamental LLM limitation: knowledge frozen at

training time without access to private or recent information

- Creates a question-answering system that grounds LLM responses in actual retrieved data to reduce hallucinations

Chapter 7, "Building a Scientific RAG System with PostgreSQL and pgvector"

- Develops a RAG system specifically designed for scientific literature with unique challenges: technical terminology, structured content, citation networks, and evidence quality
- Enables semantic discovery, cross-paper synthesis, and contextual understanding across ArXiv's 15,000+ monthly publications
- Uses PostgreSQL with pgvector for scientific knowledge retrieval
- Demonstrates handling domain-specific language and structured academic conventions

Chapter 8, "Building a Complete Conversation Search and RAG System"

- Moves from searching public data to building a "second brain" for personal chat history with AI assistants
- Addresses unique characteristics of conversational data: contextual dependencies, conversational flow across exchanges, personal language patterns, and privacy requirements
- Handles the messiness of real conversational data where ideas develop across multiple message exchanges
- Creates a system for retrieving valuable insights, solutions, and learned knowledge from thousands of personal AI conversations

Chapter 9 proposes an experimental Vector Query Language (VQL) to standardize the fragmented vector database landscape:

Chapter 9, “Vector Query Language”

- Presents a prototype SQL-inspired query language for vector databases, designed to start community discussion and consensus building
- Addresses the needs of three communities: application developers seeking database abstractions, database maintainers needing cross-vendor tooling, and AI/machine learning researchers requiring efficient data manipulation
- Defines a data model (collections, tables, vectors, embeddings, metadata) and query syntax combining SQL familiarity with vector operations
- Covers similarity search, hybrid search, range search, batch operations, vector functions, and aggregations
- Proposes a path toward standardization across the fragmented vector database landscape

Who This Book Is For

This book's target audience comprises:

- Enterprise software developers building their first AI applications
- Data scientists implementing semantic search
- Machine learning engineers working on RAG systems
- Backend developers adding vector capabilities to existing systems

Additionally, it is geared toward people who learn by doing, so there are also practical applications focused on individual use.

Prerequisites include Python programming proficiency, a basic understanding of databases and SQL, and familiarity with running LLMs locally for the RAG chapters. Readers should also be comfortable with simple mathematical concepts.

The book is for those who want to learn how to build small- to medium-scale semantic search systems, implement RAG applications with local LLMs, optimize vector similarity search, choose the right indexes, and integrate vector search with relational databases.

The book focuses on Python implementations rather than other languages, covers CPU-based solutions rather than GPU optimization, and excludes cloud-managed vector database services in favor of self-hosted solutions. The scope of the applications and the theory is to cover enough to get started and do something useful with personal data as a first step. This is not a textbook on how to build web-scale production vector database applications. However, the lessons in the chapters can serve as a foundation and a springboard to take the next leap up in scope and complexity.

How to Use This Book

For the reader who is coming from a purely relational database background, reading the first two chapters is a must. Chapter 3 is more detailed on the innards of FAISS, but you can just use the code examples to dive right in and use it.

Chapters 4 through 8 use SQLite3 and Postgres to build semantic search and RAG applications using local data and local LLMs. Chapter 8 is especially topical, as it creates a semantic search over your LLM conversations—in this case, exported from Claude.

Finally, those interested in exploring similarities with SQL can dive into Chapter 9, which is a speculative design of a Vector Query Language to enable a unified, vendor-independent, client access metaphor for vector databases.

Software, Environment, and Resource Requirements

With the following resources, the example applications will run on standard developer laptops:

- Python version 3.11+
- Key dependencies: PostgreSQL, SQLite3, FAISS, Ollama
- OS: Linux, macOS, or WSL on Windows
- Minimum 16 GB RAM; preferably 24 GB

For a developer machine that can run PostgreSQL and an 8B LLM:

- Storage requirements: 512 GB to 1 TB

Conventions Used in This Book

The following typographical conventions are used in this book:

Italic

Indicates new terms, URLs, email addresses, filenames, and file extensions.

`Constant width`

Used for program listings, as well as within paragraphs to refer to program elements such as variable or function names, databases, data types, environment variables, statements, and keywords.

`Constant width bold`

Shows commands or other text that should be typed literally by the user.

`Constant width italic`

Shows text that should be replaced with user-supplied values or by values determined by context.

> **TIP**
>
> This element signifies a tip or suggestion.

> **NOTE**
>
> This element signifies a general note.

> **WARNING**
>
> This element indicates a warning or caution.

Using Code Examples

Supplemental material (code examples, exercises, etc.) is available for download at *https://github.com/nborwankar/VectorDatabaseBook*.

If you have a technical question or a problem using the code examples, please send email to *support@oreilly.com*.

This book is here to help you get your job done. In general, if example code is offered with this book, you may use it in your programs and documentation. You do not need to contact us for permission unless you're reproducing a significant portion of the code. For example, writing a program that uses several chunks of code from this book does not require permission. Selling or distributing examples from O'Reilly books does require permission. Answering a question by citing this book and quoting example code does not require permission. Incorporating a significant amount of example code from this book into your product's documentation does require permission.

We appreciate, but generally do not require, attribution. An attribution usually includes the title, author, publisher, and ISBN. For example: "*Vector Databases* by Nitin Borwankar (O'Reilly). Copyright 2026 Nitin Borwankar, 978-1-098-17759-1."

If you feel your use of code examples falls outside fair use or the permission given above, feel free to contact us at *permissions@oreilly.com*.

O'Reilly Online Learning

> **NOTE**
>
> For more than 40 years, *O'Reilly Media* has provided technology and business training, knowledge, and insight to help companies succeed.

Our unique network of experts and innovators share their knowledge and expertise through books, articles, and our online learning platform. O'Reilly's online learning platform gives you on-demand access to live training courses, in-depth learning paths, interactive coding environments,

and a vast collection of text and video from O’Reilly and 200+ other publishers. For more information, visit *https://oreilly.com*.

How to Contact Us

Please address comments and questions concerning this book to the publisher:

O’Reilly Media, Inc.

141 Stony Circle, Suite 195

Santa Rosa, CA 95401

800-889-8969 (in the United States or Canada)

707-827-7019 (international or local)

707-829-0104 (fax)

support@oreilly.com

https://oreilly.com/about/contact.html

We have a web page for this book, where we list errata and any additional information. You can access this page at *https://oreil.ly/vector-databases*.

For news and information about our books and courses, visit *https://oreilly.com*.

Find us on LinkedIn: *https://linkedin.com/company/oreilly-media*

Watch us on YouTube: *https://youtube.com/oreillymedia*

Acknowledgments

I want to thank all the technical reviewers, especially Sunil Sawant. Because of their efforts, the book is a much better product.

Sara Hunter, my development editor, deserves praise for her patience during the yo-yo delivery cadence of the chapters. I would also like to thank the rest of the O'Reilly editorial team—both the early release and final production teams—for putting up with the inexperience of a first-time author.

My wife, Garima, and daughter, Vanita, had to tolerate my minimal participation in family events during all of last year. I would like to thank them deeply for their support and understanding during the writing of "the book." It's finally done, guys!

Of course, none of this would be possible without the open source developer communities of PostgreSQL, SQLite, and the vector extensions, whose work enabled the example code.

OceanofPDF.com

Chapter 1. Introduction to Vector Databases

The current wave of generative AI (GenAI for short), especially in natural language applications, has brought the technology of vector databases to the forefront of developer attention. Unstructured data used to train models is vectorized, and data used in inference is often saved in vector databases. And yet the technology is not well understood or widely known to the average database application developer.

This chapter provides a high-level overview of vector databases, outlining their main components and offering a preview of the material ahead. Mostly, you'll learn what the technology is and why you need it. By the end of the chapter, you'll understand how to use vector databases; the key topic of embeddings will be covered in the next chapter.

Why Do You Need Vector Databases?

Even before the advent of GenAI, managing unstructured data such as text, images, audio, and video was difficult. It required specialized technology and knowledge to manage these assets at scale. Indexing and searching over such data were only possible at the level of the metadata—for example, the date of creation, size, author, and so on. But searching for an image similar to another one, or a paragraph with a similar meaning to another, was not something an SQL database could do. SQL databases are used to match on syntax, not semantics.

Now, with the emergence of vector databases into the mainstream, managing unstructured data and searching based on *semantics*—the meaning of the data—have become much easier. This is the world of vector databases, which opens up new possibilities for applications based on semantic or similarity search. This goes well beyond searching by keyword,

which often leads to confusion between, say, *apple* as a fruit and *Apple* as a computer company. Vector databases are a foundational technology for AI in the enterprise, just as relational databases are the foundational technology for business data. Applications are now being built that combine business data with the semantics of text, images, audio, and video to create rich, intelligent, interactive systems. To make these rich applications, you'll need a deeper understanding of vector databases than is broadly available today.

While specialized vector databases are currently available (circa 2025), they all differ slightly and require database-specific knowledge. Many such vector databases are available in the cloud—requiring far less specialized maintenance—while databases such as Postgres, enhanced with vector capability, are similarly available.

This book focuses on technology in which vector database capability is well integrated into the world of relational databases. This allows structured and unstructured data to be handled in familiar ways and managed using familiar tools for databases, such as backup, restore, partitioning, and access control, among others. Vector (i.e., unstructured) data is saved in familiar database tables without special treatment based on what these tables might contain.

Prior to vector databases, unstructured data was often stored as *BLOBs* (binary large objects) or directly in files, with a field in a relational database pointing to the file. In either case, the data was considered opaque—hence a BLOB. We were not able to do much with the data aside from saving and retrieving it, except perhaps with text, where keyword search was possible.

A New Data Type: Vector

So what is "vector data," anyway, and why can't we just use the BLOB as it is? We could—but with a vector, we get a superpower that allows us to add a semantic component to our data and also make it GenAI-ready. In the language model part of GenAI, words, sentences, paragraphs, and even documents may be represented as vectors—so to do the heavy lifting in the

GenAI space, it's essential to understand vector data. Let's dive a little deeper.

First, to start with a familiar domain, think of "vector" as a new data type in a database management system. As a concrete example, let's consider Postgres, which will be our mainstay for this book and is a well-known and popular database.

In Postgres, we can add new types to the type system via the Postgres extension system. In the case of the vector type, the pgvector extension provides the capability to create columns of type `VECTOR` in a table and offers a vector-native search capability—similarity search—over this data type. If you look at the content of a `VECTOR` column, as in Figure 1-1, you will see a list of floats—but there's more to this than meets the eye.

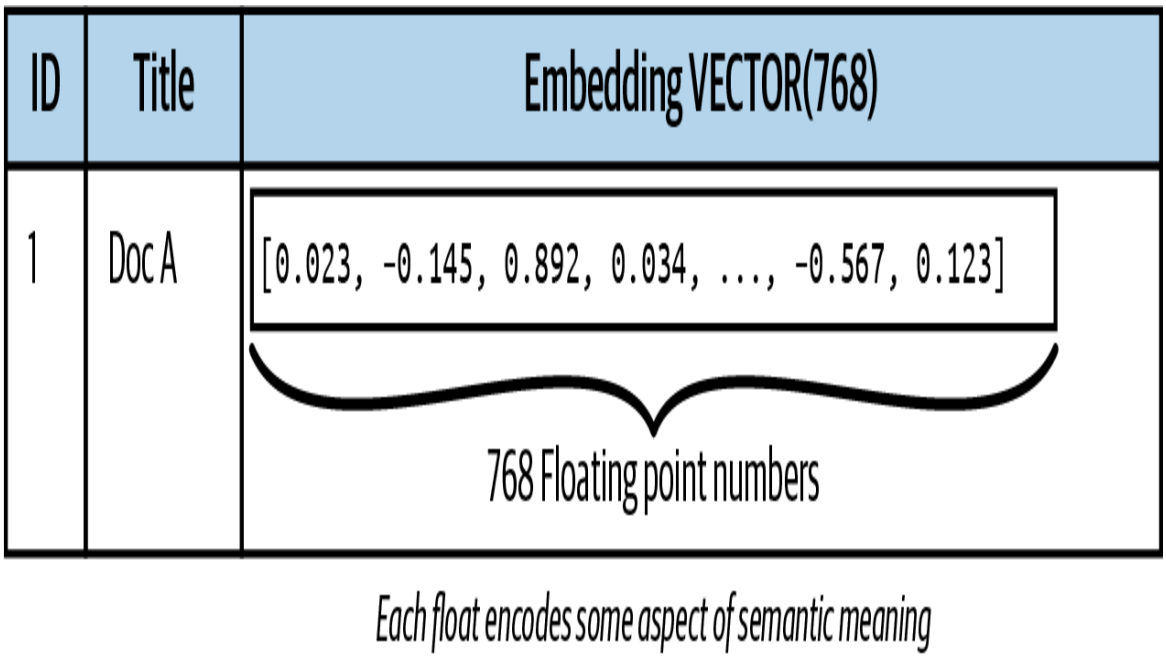


Figure 1-1. Vector data type contents

Similarity Search

The similarity search capability is what differentiates the `VECTOR` type from merely saving and fetching an ordered set of floats. It is this similarity search that allows you to find text similar in meaning to a given piece of text. This vector similarity capability forms the foundation of database

applications built with large language models (LLMs) such as ChatGPT and Claude, or local LLMs like Llama3.x. The vector similarity search has its own search operator, which the pgvector extension adds to the type system of Postgres. We will explore this in more detail in the next chapter on vector embeddings.

Similarity search example

Imagine you are searching a company's knowledge base, and you type, "How do I get my money back?" See Figure 1-2 for what happens next.

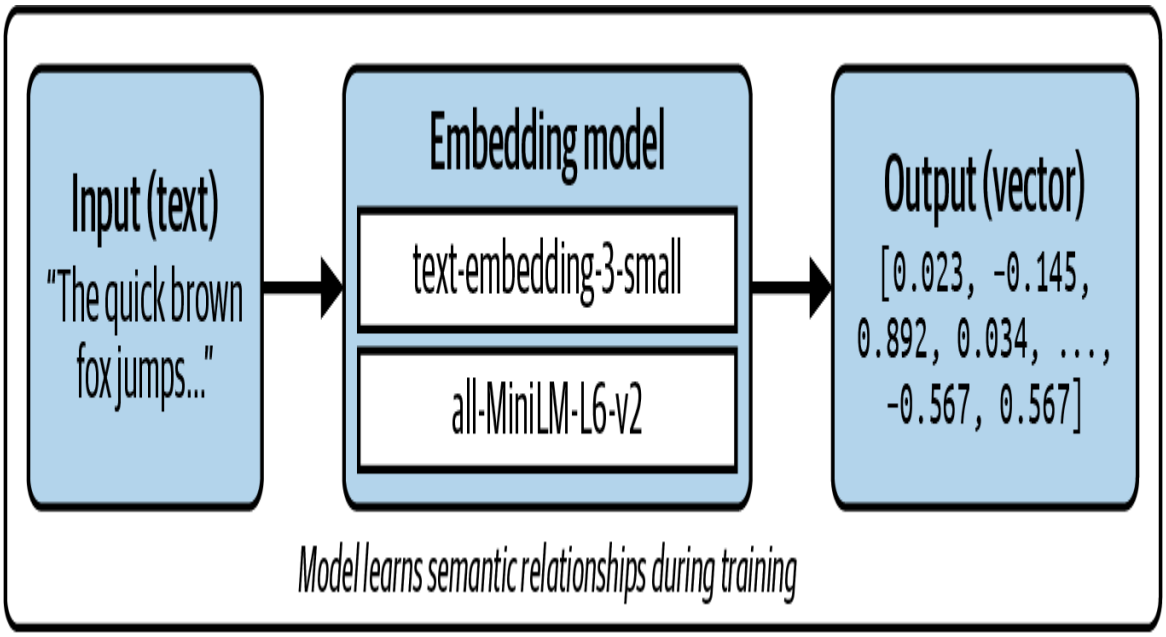


Figure 1-2. Embedding process flow

Keyword search (literal word matching)

In a *keyword search*, the system looks for the exact words you typed (or close variants). It performs well if a page literally contains words such as "money" and "back" or the phrase "get my money back." However, it may miss the best answer if the document uses different wording, such as:

- "Refund policy"
- "Return and exchange"
- "Reimbursement"

- “Cancel your order”

So a keyword search might return:

- A page about “money-back guarantee” (good, if such a page exists)
- Or irrelevant pages that mention “money” and “back” separately (e.g., “cash-back rewards”)

Semantic search (meaning matching)

Semantic search tries to understand what you mean. It recognizes that “get my money back” expresses the same intent as:

- “Request a refund”
- “Return an item”
- “Cancel and reimburse”
- “Charge reversal”

So semantic search is more likely to return:

- The “Refunds and Returns” page
- The exact section “How to request a refund after purchase”

One-sentence summary

At their essence, here’s what both searches are doing:

- Keyword search: “Find pages that contain these words.”
- Semantic search: “Find pages that answer this question, even if the wording is different.”

An additional query

Let’s look briefly at one more example. Imagine you search “My laptop won’t turn on.”

- A keyword search may miss articles titled “Device fails to boot” or “Power issues.”
- A semantic search will likely find them because it maps “won’t turn on” to “power/boot failure.”

Let’s take a look at how all this magic happens (Figure 1-3).

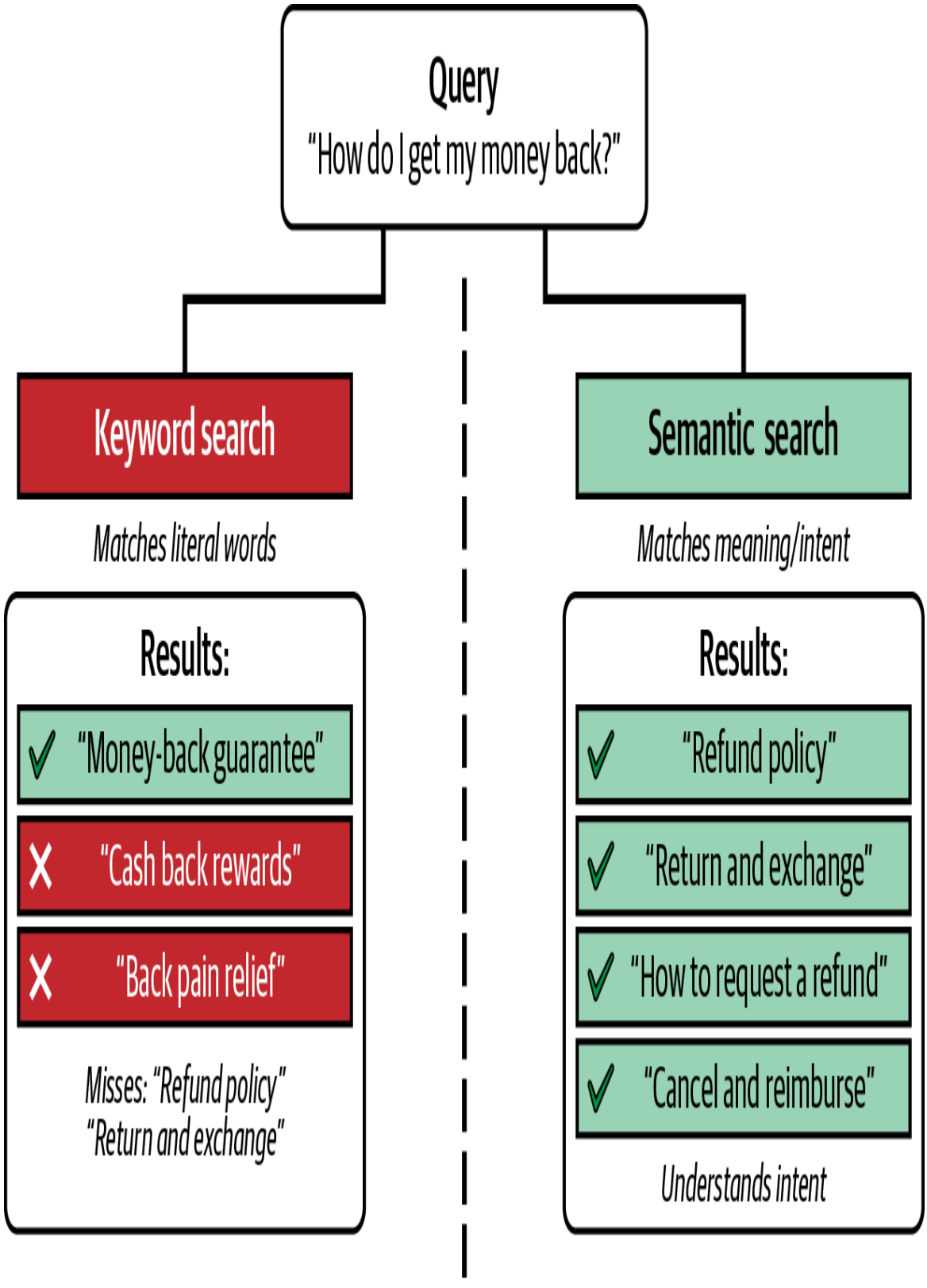


Figure 1-3. Semantic versus keyword search comparison

Embedding

There is a process called *embedding* that enables you to take chunks of unstructured (i.e., nontabular) data and map them to a vector—a list of floats (see Figure 1-3). This kind of mapping preserves the context in which a word or sentence occurs. Words close in meaning are mapped to vectors that lie close to one another in vector space. Very loosely speaking, the meaning of a word is strongly associated with the context in which it is used.

To revisit the earlier example of the word “apple”: when the word “apple,” or sentences containing “apple,” occur in the context of other sentences about fruit, orchards, or pie baking, you can say that the meaning—in this context—of “apple” is the fruit. If, on the other hand, the text you are examining appears in the context of descriptions of computers, you can conclude that “apple” refers to the company Apple. This explanation is somewhat imprecise at this point, but it conveys the general idea. Fear not—we will take a much deeper dive into embeddings in the next chapter. For now, take a look at Figure 1-4 to get a sense of how this works.

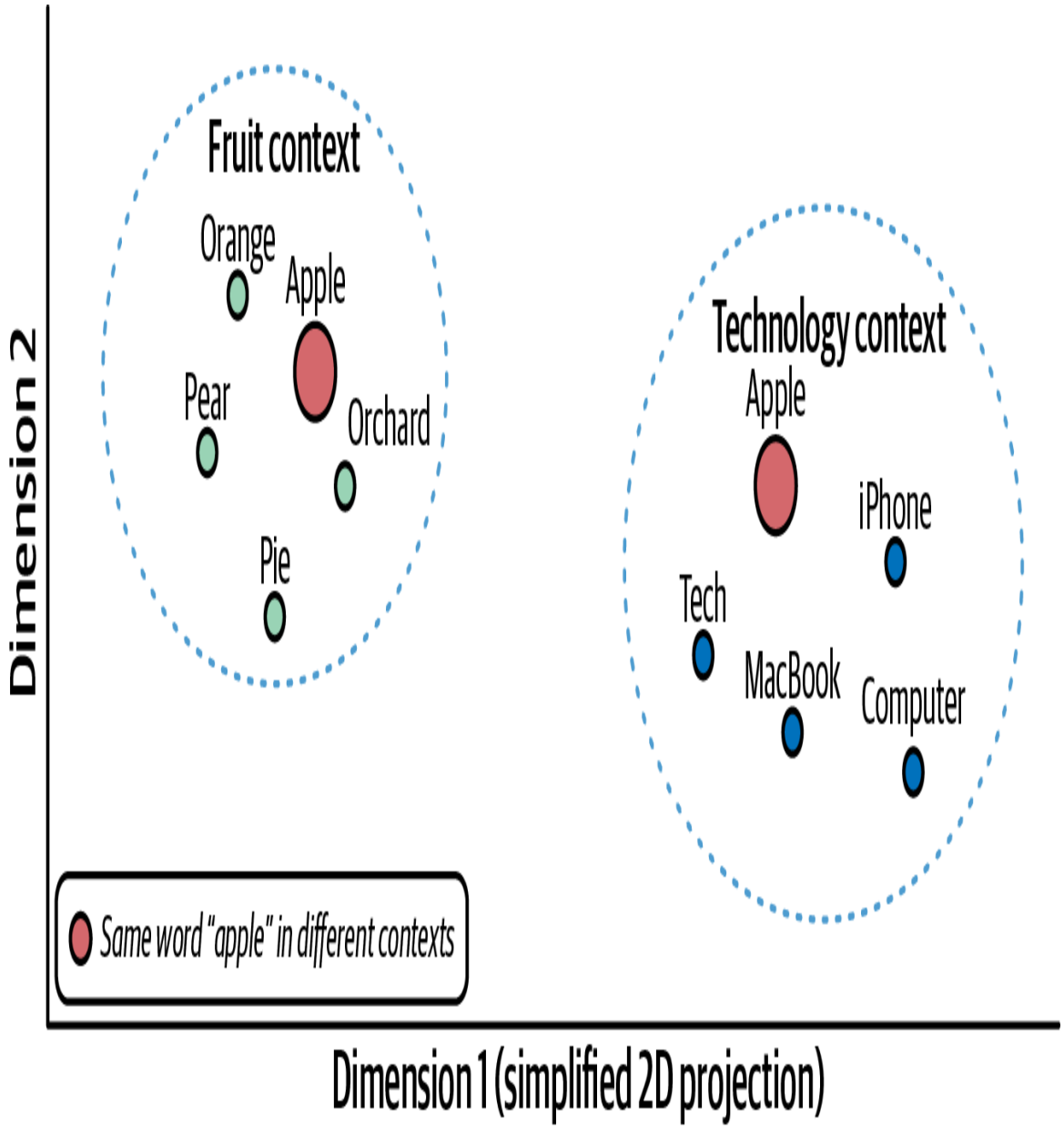


Figure 1-4. Vector space clustering

At this point, I should add that the word “embedding” is used with different meanings, and this can be confusing to newcomers.

First, it refers to the process by which data is mapped into a vector—and, more importantly, by which a collection of data is mapped into a vector space. Hence, it functions as a verb: the process of embedding. For example, a few pages of text, or several books collectively called a *corpus*,

can be transformed through the process of embedding into a set of vectors that follow the rules of a vector space.

Second, it is also a noun—an *embedding*. In this case, the term refers to the output of the embedding process—the set of vectors collectively called *an embedding*. Thus, the process of embedding (verb) produces, as output, an embedding (noun). A good way to remember this distinction is to think of a painter creating a painting: the process of painting produces a painting. Finally, "an embedding" can refer to either a single vector or the entire collection; which one is meant is usually clear from context.

What's Different About the Vector Type?

With this new data type comes a set of operations specifically designed for vector manipulation. These operations are fundamental to many AI and machine learning tasks, such as recommendation systems, image recognition, and natural-language understanding. Let's take a look at what these operations include.

Similarity comparison

The first operation is similarity comparison, where you determine whether a given vector is similar in meaning to a reference vector. A common method for this is *cosine similarity*—measuring the cosine of the angle between two vectors to determine their similarity (see Figure 1-5). Vectors close in meaning have cosine similarity values near 1 (the maximum, for identical vectors), while those that are far apart have low cosine similarity.

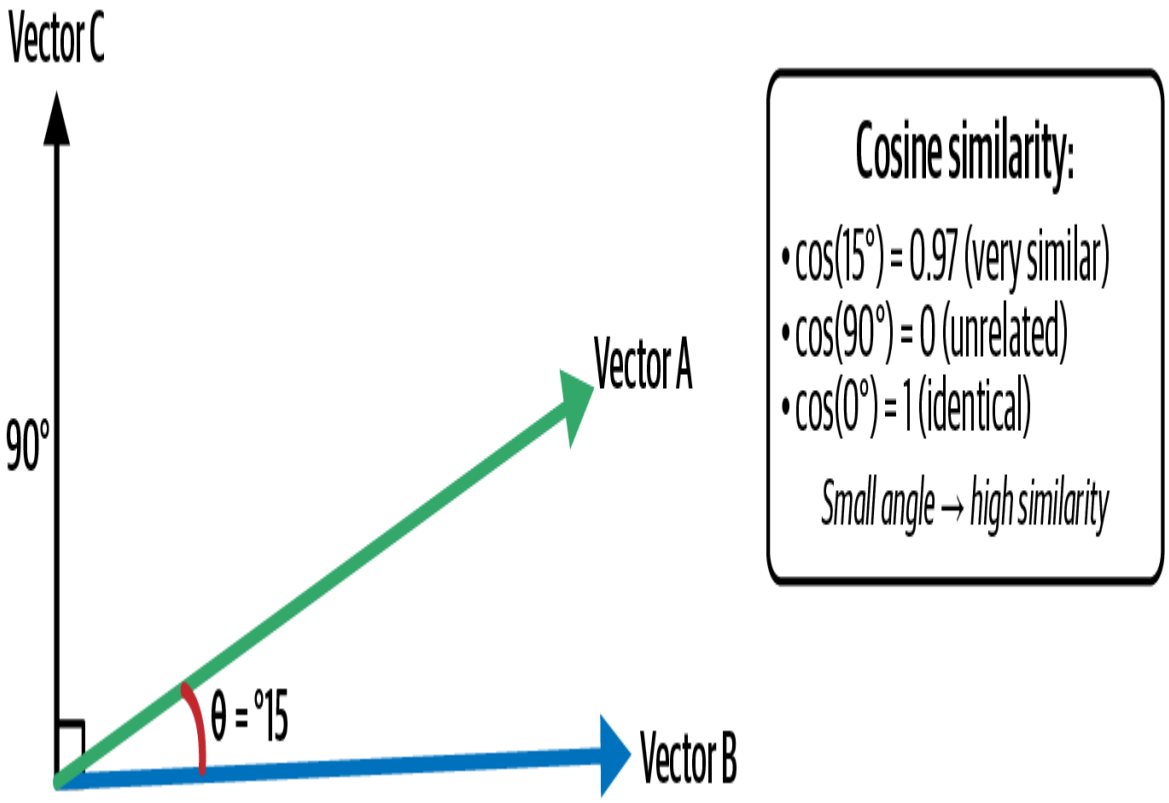


Figure 1-5. Cosine similarity geometry

Nearest-neighbor search

The next operation is *nearest-neighbor search*, in which you identify multiple vectors in the dataset that are close to a given vector. In vector-embedding space, there is a method called *ANN*—Approximate Nearest Neighbors—which provides approximate results. This approach is sufficient for GenAI applications (see Figure 1-6) because such applications are inherently probabilistic: an exact match is neither required nor desirable, as it can hinder the flexibility needed to allow multiple possible choices at each step of the generation process.

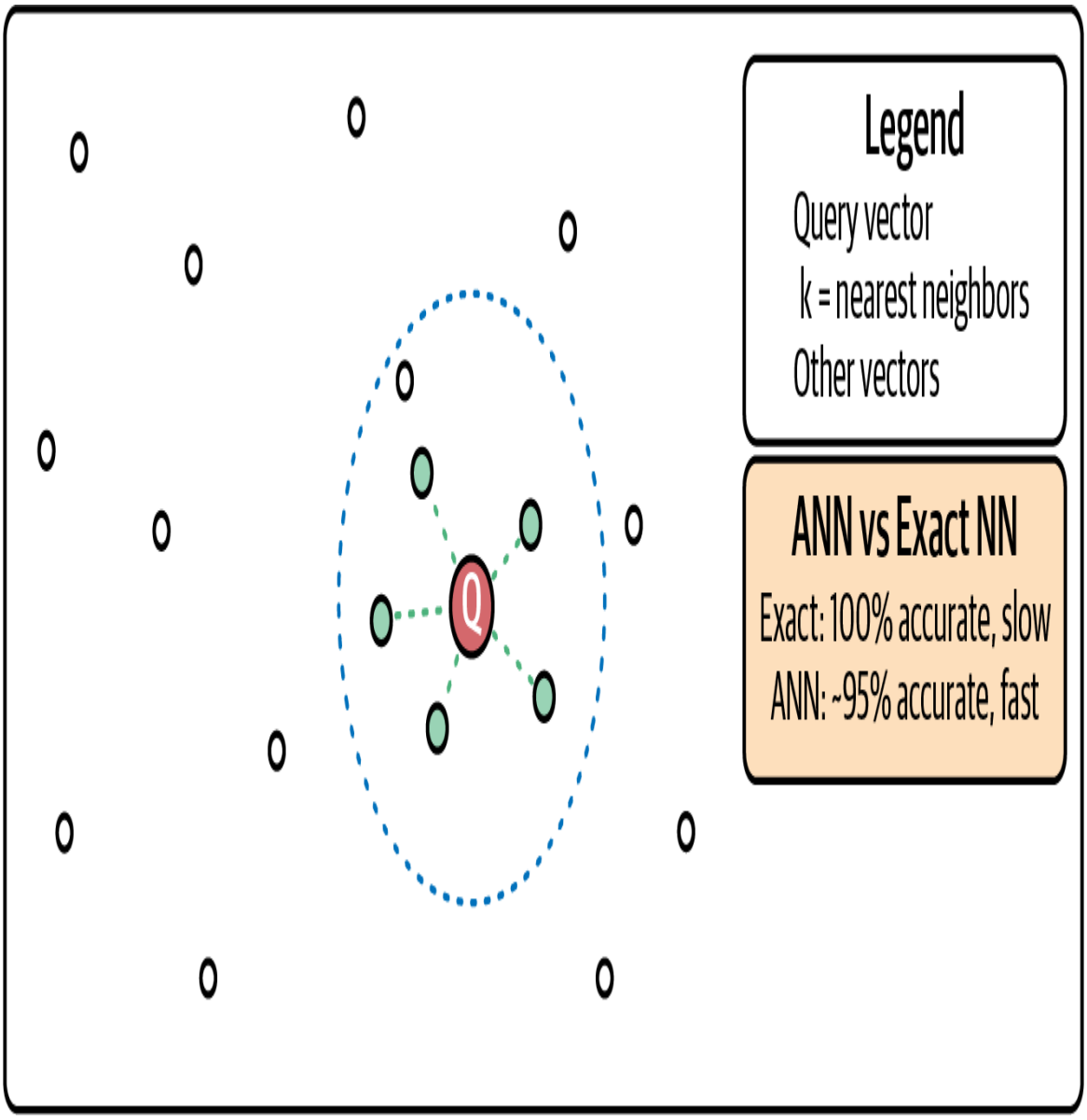


Figure 1-6. Nearest-neighbor search

Vector addition and subtraction

The final operation is *vector addition and subtraction*, in which you perform mathematical operations on vectors to discover relationships or create new representations. One seminal paper introducing such vector embeddings demonstrated a striking result:

```
Vec("Queen") ≈ Vec("King") – Vec("Man") +
Vec("Woman")
```

where `Vec(<word>)` represents the vector embedding for a given word (see Figure 1-7). For deeper exploration, see Mikolov et al. (2013).

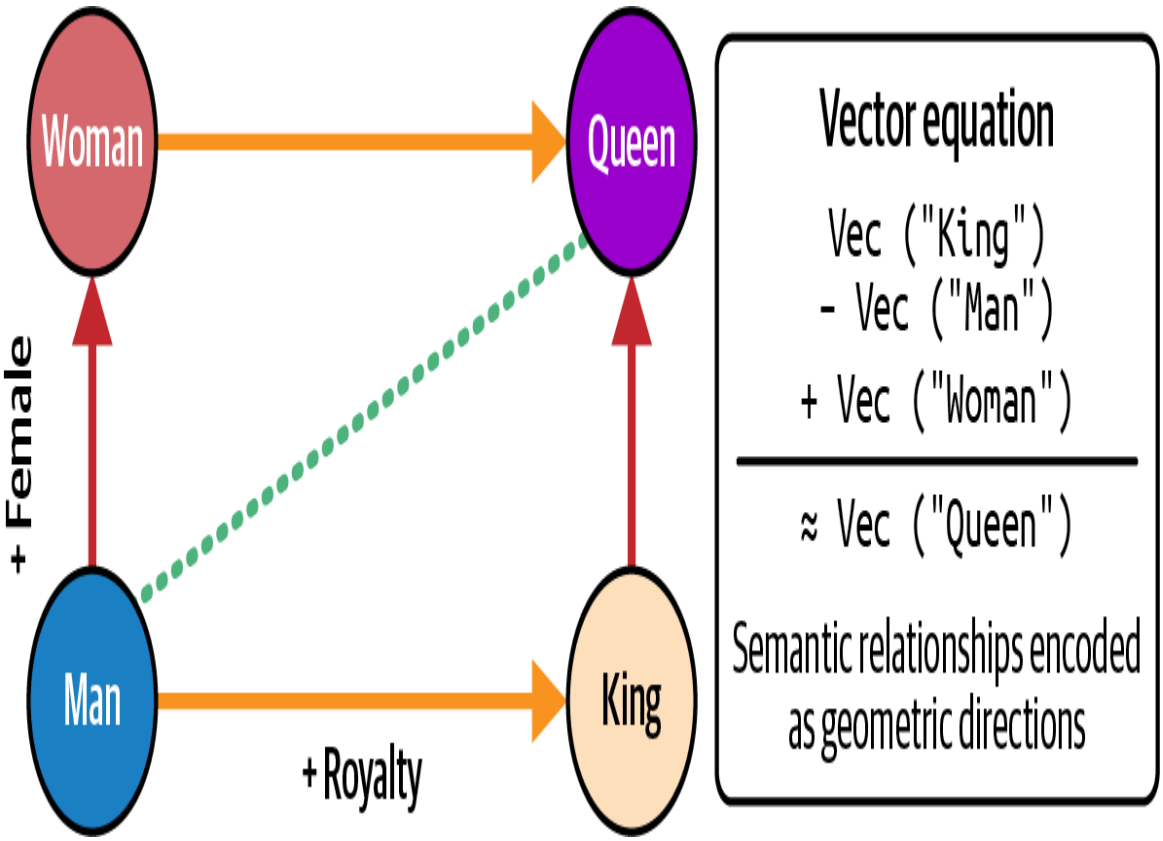


Figure 1-7. Vector arithmetic: King – Man + Woman ≈ Queen

Where Do You Use Vector Databases?

Vector databases are used across a wide range of applications and industries. Many of these use cases predate GenAI, and you've probably interacted with an application built on a vector database without realizing it. As GenAI—especially LLM-based applications—becomes more common, vector databases are increasingly taking center stage.

Here are some use cases familiar to most developers:

Semantic search

Enables intelligent queries that understand meaning and context rather than simply matching keywords

Recommendation system

: Delivers personalized product, content, or service recommendations based on user preferences and behaviors (e.g., ecommerce sites)

Image and video retrieval

: Efficiently stores and searches visual data for content-based image and video retrieval (e.g., Google Image Search)

Natural language processing (NLP)

: Supports tasks such as text classification, sentiment analysis, and language translation by representing text as vectors

Anomaly detection

: Identifies unusual patterns or behaviors in cybersecurity or fraud contexts

Bioinformatics

: Analyzes genetic sequences and protein structures in computational biology

Computer vision

: Performs object, facial, and scene recognition tasks

Audio processing

: Supports music recommendation, voice recognition, and audio fingerprinting

Chatbots and conversational AI

: Helps interpret user intent and provides more contextually relevant responses

Drug discovery

Assists in identifying potential drug candidates

Financial analysis

Aids in market trend analysis, risk assessment, and algorithmic trading

SQL Versus Vector Databases

To appreciate the unique value of vector databases, it helps to compare them with traditional SQL (Structured Query Language) databases, which have served as the backbone of enterprise data management for decades.

The Foundation of Business Math: Accounting Arithmetic

Relational databases—commonly called *SQL databases*—have long been the preferred solution for enterprise data management because they are built from the ground up on the mathematical principles of business: accounting arithmetic. This concept is reflected in the structure and capabilities of SQL databases:

Tabular data

SQL databases organize data into tables with rows and columns, mirroring the structure of spreadsheets and ledgers.

Data types

They primarily handle simple types such as integers, floats, strings, and binary-coded decimals (BCDs), which support exact currency calculations.

CRUD operations

SQL databases support Create, Read, Update, and Delete (CRUD) operations that map directly to common business processes.

Aggregations and joins

They provide powerful tools for summing values, joining related data, and supporting financial reporting and analysis.

Data warehousing

Modern SQL databases can handle denormalized tables—fact tables and star or snowflake schemas—for decision support queries.

Vector Representation in a Relational Database Management System

Although it's technically possible to represent vectors in a relational database management system (RDBMS), the approach is not ideal. A vector can be stored as a list of floating-point numbers, for example (see Figure 1-8):

```
```sql
CREATE TABLE vector_data (
    id INT PRIMARY KEY,
    vector FLOAT[]
);

INSERT INTO vector_data (id, vector) VALUES (1, ARRAY[0.1, 0.2,
..., 0.768]);
```
```

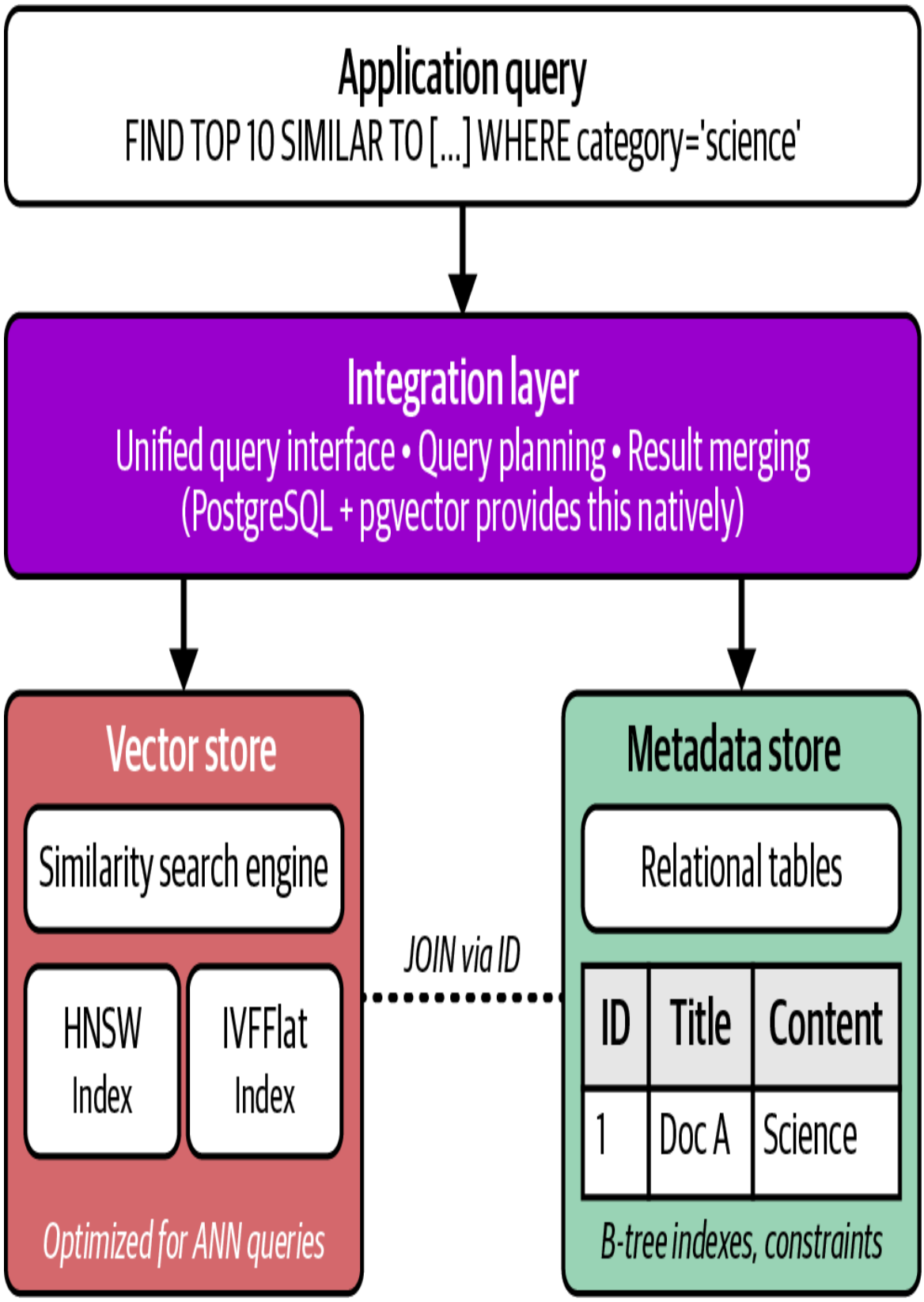


Figure 1-8. Hybrid database architecture

However, this approach has several limitations:

Performance

Querying and manipulating these vectors using SQL operations is inefficient—especially for high-dimensional data—because traditional indexes are not optimized for vector search.

Lack of vector operations

SQL databases don't natively support vector-specific operations such as similarity search or nearest-neighbor search.

Scalability issues

As vector counts and dimensions grow, SQL performance on vector queries degrades rapidly.

The Need for Vector-Specific Capabilities

To fully exploit the power of vector representations, we need databases that can:

Efficiently index high-dimensional data

Vector databases use specialized indexing techniques such as locality-sensitive hashing (LSH) or hierarchical navigable small world (HNSW) graphs to make similarity searches fast and scalable.

Perform vector operations natively

Vector databases offer built-in cosine similarity, Euclidean distance, and *k*-nearest-neighbor functions.

Optimize for ANN search

In many AI applications, finding ANNs is sufficient and much faster than performing an exact search. Vector databases are designed with this in mind.

Handle sparse vectors

Many AI models produce sparse vectors (vectors with many zero values), and vector databases can store and query them efficiently.

NoSQL Versus Vector Databases

As SQL databases struggled with unstructured and semistructured data, NoSQL databases emerged around 2005 as a partial solution. They were designed to handle formats such as JSON and XML that didn't fit cleanly into flat SQL tables.

With GenAI on the rise, it's worth asking whether NoSQL databases can serve as vector databases.

NoSQL Databases and Vector Storage

Several NoSQL vendors have introduced extensions for vector handling. Some well-known examples include:

- Redis RediSearch, with vector similarity search
- MongoDB Atlas, which now includes vector search functionality
- Elasticsearch, which has dense vector fields and vector functions
- Apache Cassandra, which can be extended with vector search capabilities through plug-ins

Limitations of Vector Extensions in NoSQL Databases

Although these NoSQL databases can handle vector similarity, they remain retrofits compared with purpose-built vector databases:

Performance

Vector operations are often less optimized, particularly for high-dimensional or large-scale datasets.

Functionality

Fewer operations and indexing methods are available than in specialized systems.

Scalability

Some NoSQL databases struggle to maintain performance as the volume of vector data grows into billions of vectors.

Integration

Vector operations may not blend smoothly with standard queries, creating complexity in queries that combine vector and nonvector data.

When to Choose NoSQL with Vector Extensions

NoSQL databases with vector extensions can work well in certain scenarios:

Mixed data types

When your application uses structured, semistructured, and vector data, with vectors as a secondary focus

Existing infrastructure

When you already use NoSQL extensively and want to add vector capability without a full system change

Moderate scale

When extreme performance or scale isn't critical

For most of this book, I'll use PostgreSQL with the pgvector extension for enterprise department–level applications—sufficient for small and midsize systems that don't require billion-vector scale.

Hybrid Approaches: Combining Structured and Vector Data

Many real-world systems require both vector data and structured business data, leading to hybrid designs that combine vector databases with traditional SQL databases.

The Need for Both Vector Data and Metadata

Most practical systems also require more associated metadata—structured information that describes vectors. For example:

- In a document retrieval system, each document might be represented as a vector, but we also need metadata such as the document's title, author, publication date, and categories.
- In an image search system, each image might be represented as a vector, but we also need metadata such as the image filename, size, creation date, and tags. Often, GPS coordinates and camera information are also part of the metadata.

Limitations of Pure Vector Storage

Storing metadata as vectors is not practical for several reasons:

Query flexibility

It's much easier to perform filtering and sorting operations on structured metadata using traditional database queries. Currently, there are no comparable filter and sort capabilities in vector storage systems.

Storage efficiency

Storing simple metadata as vectors would be wasteful in terms of storage space.

Indexing

Traditional databases provide efficient indexing mechanisms for structured data, which are crucial for quick lookups and filtering. Vector indexing is optimized for similarity and clustering, not for subsetting and filtering based on arbitrary predicates.

Data integrity

Relational databases offer features such as constraints and transactions, which are essential for maintaining the integrity of structured data.

Hybrid Database Architecture

To address these needs, most vector database systems now adopt a hybrid architecture that combines vector storage with traditional database capabilities (see Figure 1-9):

Vector component

This part of the database is optimized for storing and querying high-dimensional vectors. It uses specialized indexing structures and algorithms for efficient similarity search and other vector operations.

Metadata component

This is often a traditional relational database (such as SQLite) or a NoSQL database that stores the structured metadata associated with each vector.

Integration layer

This provides a unified interface for querying both vector and metadata components, enabling complex queries that combine vector similarity search with metadata filtering.

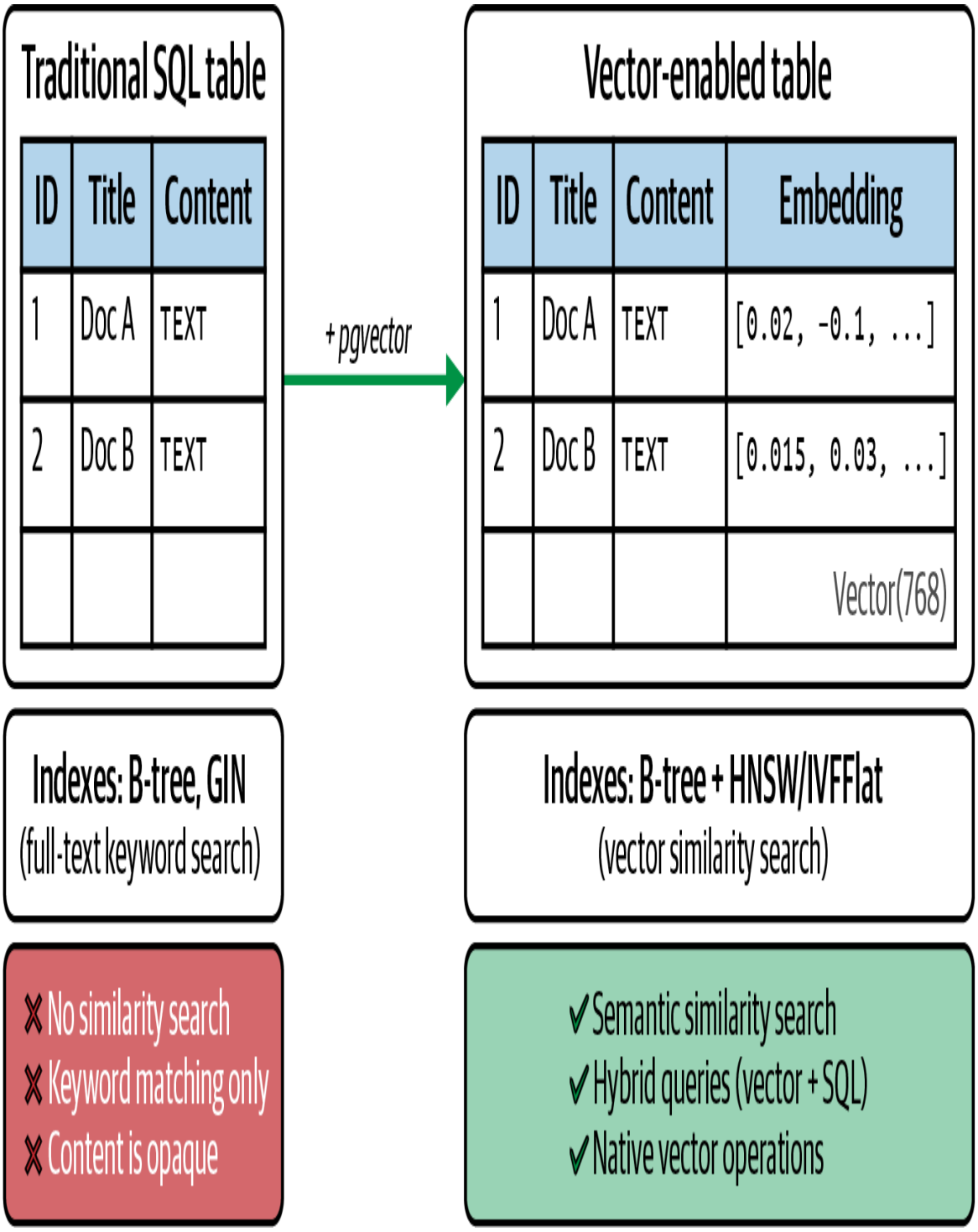


Figure 1-9. SQL table versus vector-enabled table

Example of a Hybrid Query

In a hybrid vector database system, a typical query might look like this:

```
FIND TOP 10 SIMILAR VECTORS TO [0.1, 0.2, ..., 0.5]
WHERE metadata.category = 'science'
AND metadata.publish_date > '2023-01-01'
ORDER BY similarity DESC, metadata.views DESC
```

This query combines a vector similarity search with metadata filtering and sorting, demonstrating the power of the hybrid approach. The syntax shown here is informal and meant only to illustrate the types of operations possible in a hybrid query; it is not the formal syntax of any specific hybrid database. However, a database that supports both SQL and vector operations will provide a formal syntax to achieve this result. In upcoming chapters, you will see how this is done using PostgreSQL with the pgvector extension.

Benefits of the Hybrid Approach

The hybrid model offers several key benefits, particularly for small and midsize databases:

Flexibility

> The hybrid architecture allows complex queries that leverage both vector similarity and traditional enterprise SQL data operations, enabling filtering by one and then the other.

Performance

> Each component (vector and metadata) can be optimized for its specific data type and query patterns. Indexes can be chosen separately for tabular data and vector data to optimize the performance of hybrid queries based on the columns being selected and filtered.

Data management

> It’s easier to manage, update, and maintain metadata separately from vector data. Usually, the metadata table is distinct from the vector data table, with a foreign key ID connecting each vector to its associated metadata.

For databases containing billions of objects, pure vector databases may be more appropriate.

Conclusion

Vector databases mark a major evolution in data technology, driven by the needs of GenAI and machine learning. They introduce concepts such as semantic search and vector embeddings for unstructured data—text, audio, images, and video. In hybrid designs, they complement rather than replace traditional databases.

The future of data management likely lies in hybrid systems that efficiently handle both structured and vector data. As AI continues to expand across domains, the importance of vector databases and hybrid architectures will only increase.

Understanding the strengths and limitations of vector databases—and how they integrate into the broader data management ecosystem—is essential for data scientists, engineers, and decision-makers building next-generation AI applications. As this field continues to evolve, you can expect to see further innovations that bridge the gap between vector-based and traditional data management approaches.

OceanofPDF.com

Chapter 2. Embeddings

At their core, vector embeddings solve a fundamental challenge in computing: how to represent complex, unstructured data in a way that machines can understand and efficiently process. In this chapter, you'll dive deeper into the history, evolution, and current usage of vector embeddings, with code examples showing their applications in language models. This is a conceptually meaty topic, and you may find yourself revisiting it more than once when faced with unfamiliar terminology or concepts in later chapters as you begin applying embeddings.

Understanding Vector Embeddings: Why We Need Them

Traditional data structures work well for structured information such as numbers, dates, and categorical values, but they fall short when dealing with the rich, nuanced content humans naturally create and consume—text, images, audio, and video. Embeddings bridge this gap by mapping raw data into vectors (ordered lists of numbers) that capture meaning and relationships within the data (see Figure 2-1).

This transformation from unstructured data into mathematical vectors isn't just a technical convenience—it's the foundation that enables modern AI systems, including LLMs and generative AI, to process and understand information in ways that approach human comprehension. Unlike traditional feature vectors based on hand-crafted statistics, embeddings are learned representations in which proximity in vector space reflects semantic or conceptual similarity. This mapping from meaning to structure arises from training on large text corpora fed to an algorithm or dedicated language model, depending on what is being embedded (i.e., mapped to vector space).

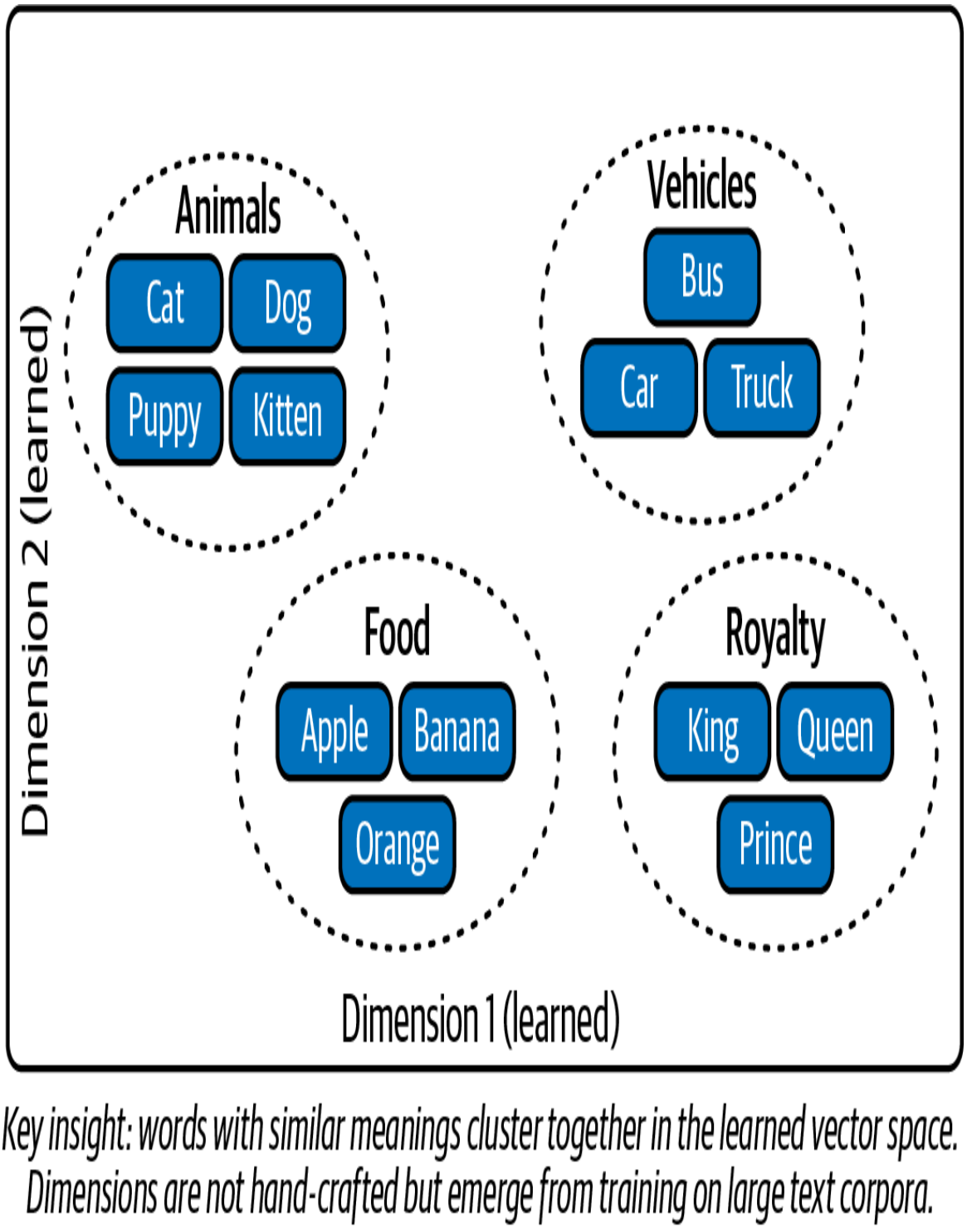


Figure 2-1. Vector space mapping

Vector databases have emerged as specialized systems to store and query these embeddings efficiently, providing built-in capabilities for finding similar vectors—and, by extension, similar content—at scale. This marks a

fundamental shift in how we can search and analyze information. Instead of relying on exact keyword matches or rigid categorical filters, vector embeddings enable search by meaning, or *semantic similarity*. For example, a search for “sunset by the ocean” can find relevant images even if they’re labeled differently. Likewise, a query about “renewable energy solutions” can surface documents discussing solar and wind power without requiring exact phrase matches. This ability to find similar vectors quickly and efficiently makes vector databases essential infrastructure for modern AI applications.

The impact of embeddings extends far beyond improved search capabilities. They underpin the data representations that power LLMs and GenAI systems. These models operate on enormous numbers of embeddings that capture relationships and patterns in language, images, and other modalities. By converting unstructured data into vector representations, we create a universal format that AI systems can process to understand context, generate meaningful responses, and make sophisticated connections across content types. This conversion of raw data into mathematically comparable vectors is so foundational that it’s fair to say modern artificial intelligence is built on vector embeddings.

Word2Vec: The Breakthrough That Changed Everything

Before Word2Vec emerged in 2013, the world of NLP was dominated by rule-based systems and simple statistical methods. Text was typically represented using techniques such as *bag of words* and term frequency–inverse document frequency (TF-IDF), where words were treated as discrete symbols without inherent relationships. Under these approaches, the words “king” and “queen” were as mathematically distinct as “king” and “bicycle.” As a result, machines struggled to understand semantics or linguistic nuance. Efforts to encode meaning often relied on hand-crafted rules and curated lexical databases such as WordNet, making language systems brittle and limited in their capabilities.

Word2Vec, introduced by Tomas Mikolov and his team at Google, represented a paradigm shift. Its insight was both elegant and powerful: words could be represented as dense vectors in a continuous vector space, where geometric relationships captured semantic relationships. The model predicted either a word from its context or the context from a word—known as the *skip-gram* and *continuous bag-of-words* models, respectively (see Figure 2-2B). Through this process, words appearing in similar contexts ended up with similar vector representations. The resultant embeddings exhibited almost magical properties: vector arithmetic such as "king – man + woman ≈ queen" became possible, demonstrating that the model had learned to encode genuine semantic and syntactic relationships (see Figure 2-2A). As linguist JR Firth once said, "You shall know a word by the company it keeps."

The mapping process itself was straightforward yet effective. Word2Vec trained a shallow neural network on a large text corpus, but instead of using it for prediction, it extracted the hidden-layer weights as word embeddings. These weights captured the statistical patterns of word co-occurrence in a way that preserved semantic relationships. For example, words appearing in similar contexts (like "coffee" and "tea") ended up with similar vectors, while unrelated words lay far apart in vector space. This approach was computationally efficient and trainable on massive amounts of text data without requiring labeled examples.

Word2Vec's influence cannot be overstated—it sparked a revolution that continues today. The paper in which Mikolov et al. introduced Word2Vec showed that meaningful language representations could be learned automatically from raw text, without human supervision or rule-crafting. This triggered a wave of advances: GloVe embeddings refined the method via explicit co-occurrence modeling, fastText extended it with subword information, and the concept of learned embeddings spread to sentences, paragraphs, and documents. Most significantly, learning contextual representations through prediction became a cornerstone of modern language models. The transformer architecture powering today's LLMs, such as GPT and BERT, directly descends from Word2Vec's insights—

scaled up and architecturally enriched. In many ways, the modern AI revolution traces its roots back to that seminal Word2Vec paper.

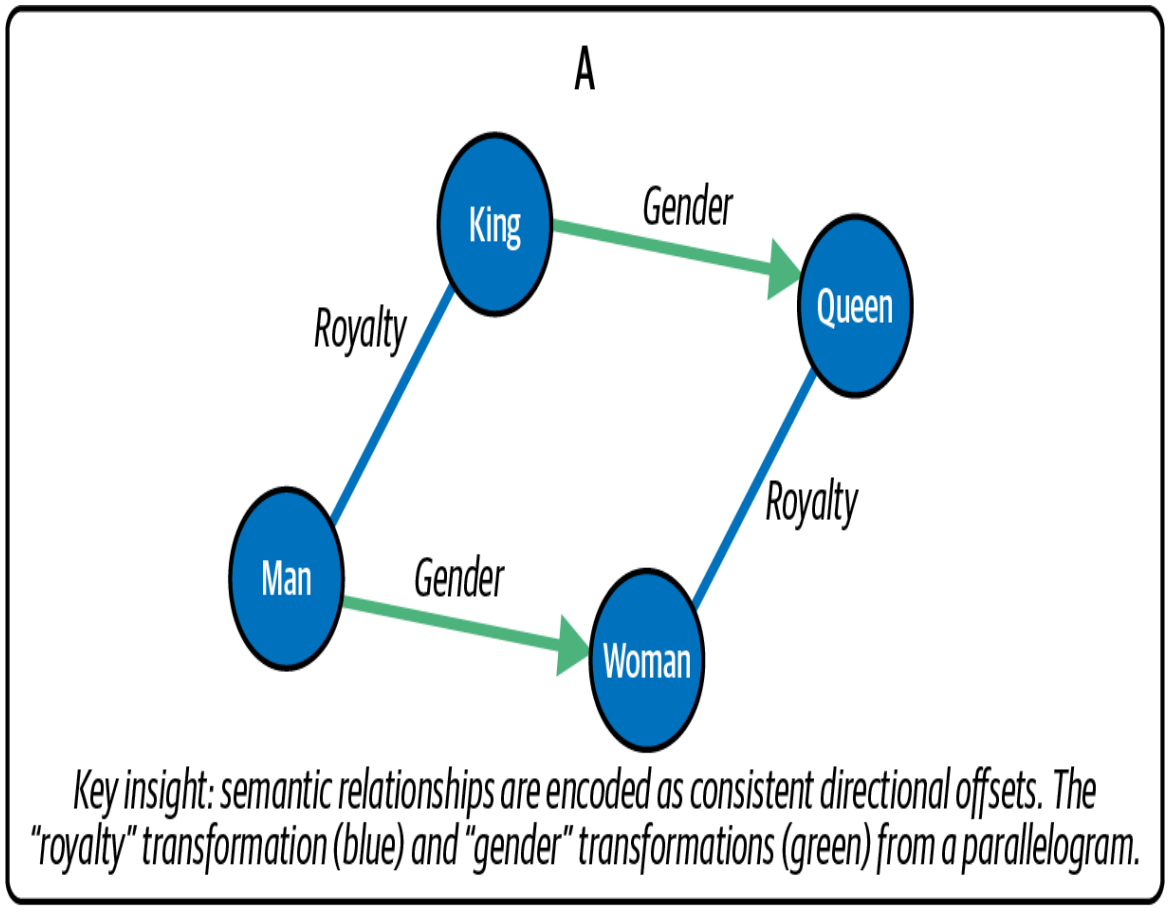

A
King
Gender
Queen
Royalty
Royalty
Man
Gender
Woman
Key insight: semantic relationships are encoded as consistent directional offsets. The "royalty" transformation (blue) and "gender" transformations (green) from a parallelogram.
B
Input sentence with sliding window (window size = 2)
The
quick
brown
fox
jumps
over
the
Training objective:
Given "fox" → predict "quick", "brown", "jumps", "over"
Words appearing in similar contexts get similar vector representations

Figure 2-2. Word2Vec vector arithmetic. (A) King – Man + Woman ≈ Queen (parallelogram analogy); (B) Skip-gram model predicting content from the center word

Doc2Vec: From Words to Documents

Building on Word2Vec, Mikolov and Le introduced Doc2Vec (originally called *Paragraph Vector*) in 2014 to represent larger text units as single vectors. While Word2Vec captured word-level semantics, combining word vectors to represent whole documents often lost context and order. Doc2Vec extended Word2Vec by learning fixed-length vector representations for variable-length text—from sentences to full documents.

The genius of Doc2Vec lay in its simple yet effective modification to the Word2Vec training process. In addition to learning word vectors, the model simultaneously learned a unique vector for each document in the training corpus. This document vector acted as a memory that captured the topics and global semantic content of the text. During training, it was treated like another word in the context but provided a consistent context across the document. This enabled the model to capture both the general topic of the document and the precise meaning of words in context—something averaging word vectors could not achieve (see Figure 2-3).

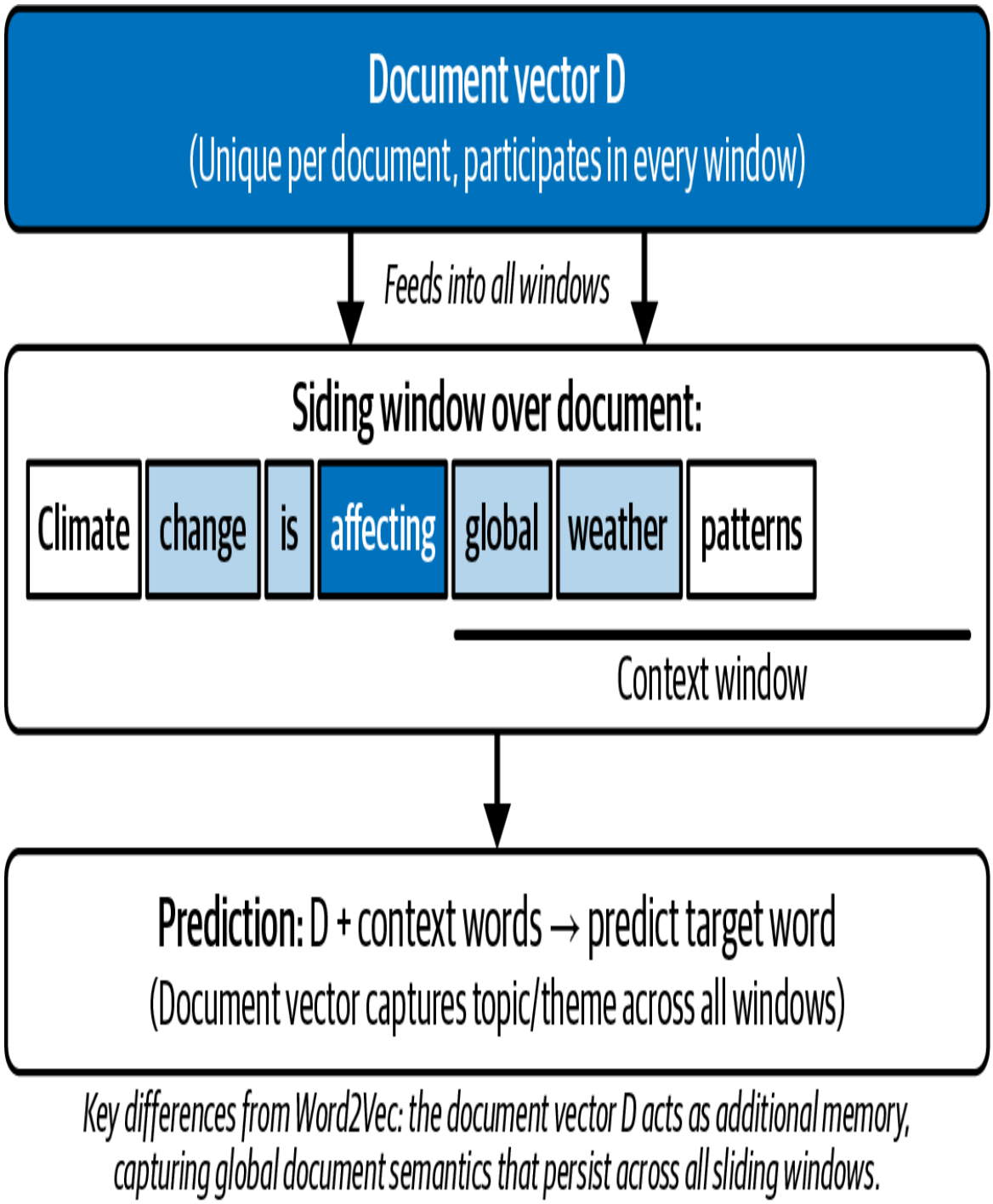


Figure 2-3. Doc2Vec: Document vector as persistent context

The differences between Word2Vec and Doc2Vec embeddings became apparent in their practical applications. While Word2Vec vectors could tell you that “cat” and “kitten” are related words, Doc2Vec could tell you that “The cat sat on the mat” and “A kitten rested on the rug” are similar

sentences, even with few overlapping words. This made Doc2Vec valuable for document classification, recommendation, and plagiarism detection. For example, two news stories about the same event but using different vocabulary would yield similar document embeddings, while unrelated stories would diverge in vector space, even if they shared some common words.

Consider these practical examples of the difference:

- A Word2Vec approach to representing the sentence “The bank is by the river” would struggle to disambiguate whether “bank” refers to a financial institution or a riverbank, as it would typically average or combine individual word vectors.
- Doc2Vec, on the other hand, would learn a representation that captures the full context, clearly encoding that this is a sentence about geography and not finance.
- Similarly, Doc2Vec could recognize that “The movie was fantastic” and “The film was excellent” express nearly identical sentiments, while simple word vector averaging might miss this equivalence due to the different word choices.

The implications of Doc2Vec extended far beyond document similarity. Doc2Vec provided one of the first demonstrations of successfully learning fixed-length representations for variable-length text—a concept that would become crucial for modern language models. The ability to encode entire documents as dense vectors that preserved semantic meaning paved the way for more sophisticated document embedding techniques like Universal Sentence Encoder and the attention-based embeddings used in transformer models.

While newer approaches have since surpassed Doc2Vec in terms of performance, its elegant solution to the document embedding problem remains foundational in how we think about representing text at different scales.

SPARSE VERSUS DENSE EMBEDDINGS: A TALE OF TWO REPRESENTATIONS

Before deep learning, sparse embeddings dominated NLP through one-hot encoding and TF-IDF. Representing the word “cat” in a 100,000-word vocabulary required a vector of length 100,000 with a single 1 for “cat” and zeros elsewhere. Similarly, a document would be represented by a sparse vector showing which words appear in it and how frequently. Such representations were intuitive but extremely inefficient: most of the vector was filled with zeros, with dimensionality proportional to vocabulary size.

Dense embeddings, introduced with Word2Vec, revolutionized this by mapping words or documents into compact spaces (typically from 100 to 1,000 dimensions) where each dimension carries meaningful information. Unlike sparse vectors, dense embeddings pack semantic information into every dimension, with each number in the vector contributing to the overall meaning. For example, a 300-dimensional Word2Vec embedding might use different dimensions to encode attributes such as gender, plurality, formality level, or semantic category—though these dimensions aren’t necessarily human-interpretable (see Figure 2-4).

The trade-offs between sparse and dense embeddings are significant:

Sparse embeddings

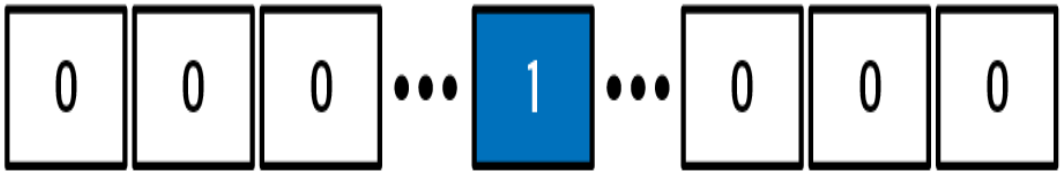
- Easy to interpret (each dimension corresponds to a specific word or feature)
- Simple to compute
- Memory-inefficient
- Weak at capturing relationships between words
- Suitable for traditional machine learning algorithms

Dense embeddings

- More compact and memory-efficient
- Better at capturing semantic relationships
- Enable meaningful vector arithmetic
- Harder to interpret individual dimensions
- Require substantial data and compute resources to train effectively

Sparse (one-hot) encoding:

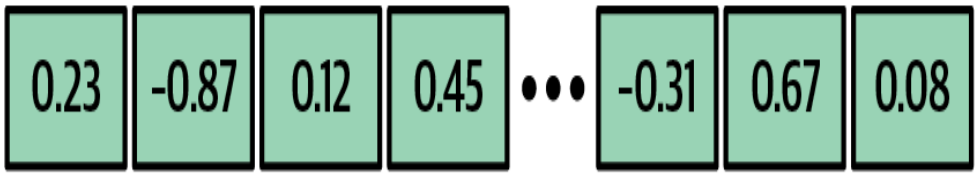
Word "cat" in vocaulary of 100,000:



- Dimensionality: 100,000
- Nonzero values: 1
- Memory: mostly wasted on zeros
- Semantics: none (just identity)
- "cat" and "kitten" are equidistant from "car"

Dense (Word2Vec) embedding:

Word "cat" as learned embedding:



- Dimensionality: 300
- Nonzero values: 300 (all meaningful)
- Memory: compact, efficient
- Semantics: encoded in every dimension
- "cat" closer to "kitten" than to "car"

Figure 2-4. Comparison of sparse and dense vector representations

In modern systems, both are often combined—dense embeddings for semantic matching and sparse vectors for exact keyword matches. Neural networks have firmly established dense embeddings as the dominant paradigm, but sparse representations remain useful in specific scenarios where interpretability or exact matching is crucial. Elasticsearch and Vespa use this hybrid approach.

From Embeddings to Modern Language Models: The Transformer Connection

Vector embeddings form the fundamental building blocks of modern LLMs, serving as the bridge between human language and machine understanding. Every word, sentence, or token that enters a language model must first be converted to a vector representation that the model can process. These embeddings aren't just input—they form the mathematical space where reasoning and generation occur. When GPT-4 writes a poem and Claude analyzes a legal document, they're performing complex mathematical operations on vectors, with each layer of the model transforming these representations in increasingly sophisticated ways. This deep connection between embeddings and language models is why understanding embeddings is crucial for anyone working with modern AI—they're not just a technical detail but the very medium in which artificial intelligence "thinks."

At the core of all modern LLMs is the transformer architecture—a powerful method for processing and generating sequences of embeddings. While Word2Vec and Doc2Vec provided static vector representations, transformers dynamically manipulate and generate these representations at scale. Let's explore the three main variants of transformer architectures and how they use embeddings in distinct ways.

Encoder-Only Transformers (BERT and Its Variants)

Encoder-only transformers such as BERT (Bidirectional Encoder Representations from Transformers) specialize in understanding text by creating rich, contextual embeddings. Unlike the static embeddings of Word2Vec, these models produce dynamic embeddings that change based on context—so “bank” gets different vector representations in “river bank” than in “bank account,” because the encoder processes the entire sequence bidirectionally. Each layer of the encoder progressively refines these embeddings, with the final layer producing highly contextualized representations. These models excel at text classification, named entity recognition, sentiment analysis, and question answering.

The embeddings they produce capture deep semantic relationships and can be used as sophisticated feature extractors for downstream tasks.

Decoder-Only Transformers (GPT Family)

Decoder-only transformers, such as GPT (Generative Pretrained Transformer) models, are optimized for text generation. They process embeddings sequentially, with each position attending only to previous positions—much like how humans write text from left to right. The embedding process here is particularly interesting because it must capture not just meaning but also predictive information about what tokens might come next.

In GPT models:

- Input embeddings combine token, position, and segment information.
- Each layer transforms these embeddings through self-attention and feed-forward networks.
- The final layer projects embeddings into probability distributions over the next token.

This architecture has proven remarkably effective at tasks such as text completion, creative writing, code generation, and conversational AI.

Encoder-Decoder Transformers (T5, BART)

Encoder-decoder transformers, also known as *sequence-to-sequence models*, combine both approaches for tasks requiring input understanding and output generation. Models such as T5 (Text-to-Text Transfer Transformer) use:

- An encoder to create contextual embeddings of the input
- A decoder to generate output while attending to both the encoded input and its own generated sequence
- Cross-attention mechanisms that allow the decoder to focus on relevant parts of the encoded input

This architecture shines in tasks such as translation, summarization, text simplification, and structured data generation.

Across all three architectures (see Figure 2-5):

- Input embeddings convert tokens into vectors the model can process.
- Positional embeddings encode sequence order information.
- Layer embeddings capture increasingly abstract features.
- Output embeddings (in generative models) map back to token probabilities.

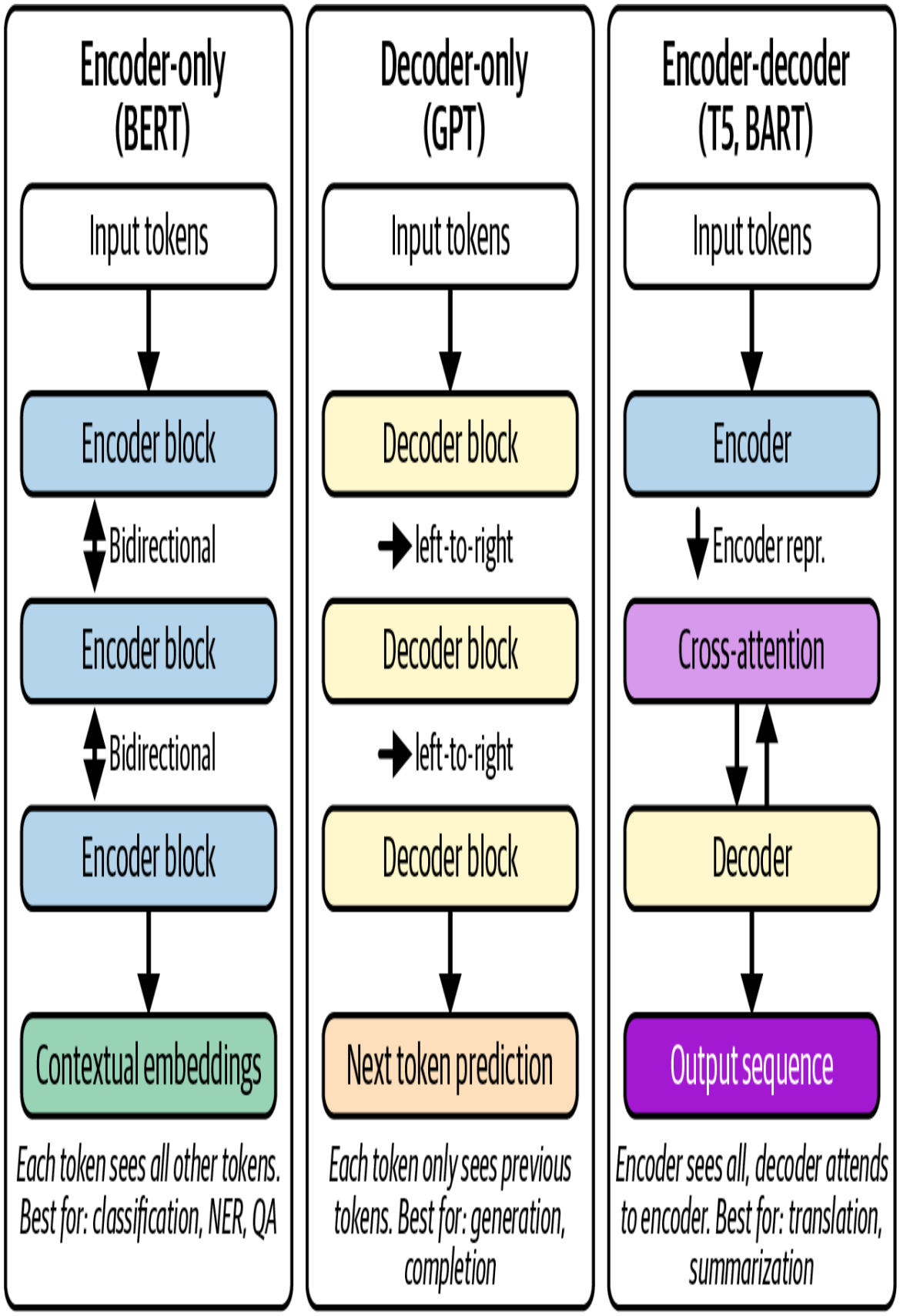

Encoder-only (BERT)
Input tokens
Encoder block
Bidirectional
Encoder block
Bidirectional
Encoder block
Contextual embeddings
Each token sees all other tokens. Best for: classification, NER, QA
Decoder-only (GPT)
Input tokens
Decoder block
left-to-right
Decoder block
left-to-right
Decoder block
Next token prediction
Each token only sees previous tokens. Best for: generation, completion
Encoder-decoder (T5, BART)
Input tokens
Encoder
Encoder repr.
Cross-attention
Decoder
Output sequence
Encoder sees all, decoder attends to encoder. Best for: translation, summarization

Figure 2-5. Three transformer architectures

The precision with which transformers manipulate these embeddings—through self-attention, cross-attention, and feed-forward networks—is what enables them to achieve their remarkable performance. The ability to dynamically update and refine embeddings based on context, while maintaining the mathematical properties that make vector operations meaningful, has proven to be a powerful combination that underlies much of modern AI's capabilities.

Embedding Models: The Specialized Vector Generators

So far in this chapter, I've discussed embeddings as components within larger language models. Let's now turn to dedicated embedding models, which have emerged as crucial standalone tools in modern AI architectures. These specialized models—such as OpenAI's text-embedding-ada-002 or Cohere's co-embed-multilingual-v3—are designed with a singular purpose: to create high-quality vector representations of text that can be used for downstream applications. Unlike general-purpose LLMs that generate text or perform various language tasks, embedding models focus exclusively on mapping text to vectors that capture semantic meaning with high fidelity.

Distinction from Traditional Models

The key differences between embedding models and embeddings as layers of LLMs and language-based applications are both architectural and philosophical:

Specialized architecture

> Unlike Word2Vec or Doc2Vec, modern embedding models are often based on transformer architectures but optimized specifically for embedding generation. They typically remove the language generation

components and focus entirely on creating meaningful vector representations.

Consistency

While LLMs create dynamic, context-dependent representations internally, embedding models are designed to generate consistent vectors for the same input, making them ideal for search and retrieval systems.

Computational efficiency

Because they don't need to generate text or perform complex language tasks, embedding models can be much smaller and faster than full LLMs while still producing high-quality embeddings.

Optimization target

These models are specifically trained to maximize the usefulness of their vector representations for similarity comparisons, often using techniques such as *contrastive learning* to ensure that similar texts are mapped to similar vectors. Contrastive learning uses algorithms that push similar vectors closer together and distinct vectors farther apart, thereby increasing semantic contrast.

Role in Modern LLM Applications

Embedding models have become essential components of what's often called the *RAG pattern* (see Figure 2-6), which is transforming how we build AI applications. When looking at the RAG pattern, you can see where embedding models have been integrated:

- Document processing pipeline
  - Documents are split into chunks.
  - Each chunk is processed by an embedding model to create a vector representation.

  - These vectors are stored in a vector database for efficient retrieval.

- Query processing
  - User queries are converted into vectors using the same embedding model.
  - Similar vectors are retrieved from the database.
  - Retrieved content provides context to the LLM for generating responses.

- Hybrid search systems
  - Embedding models enable semantic search capabilities.
  - They can be combined with traditional keyword search for better results.
  - They support multimodal search across text, images, and other data types.

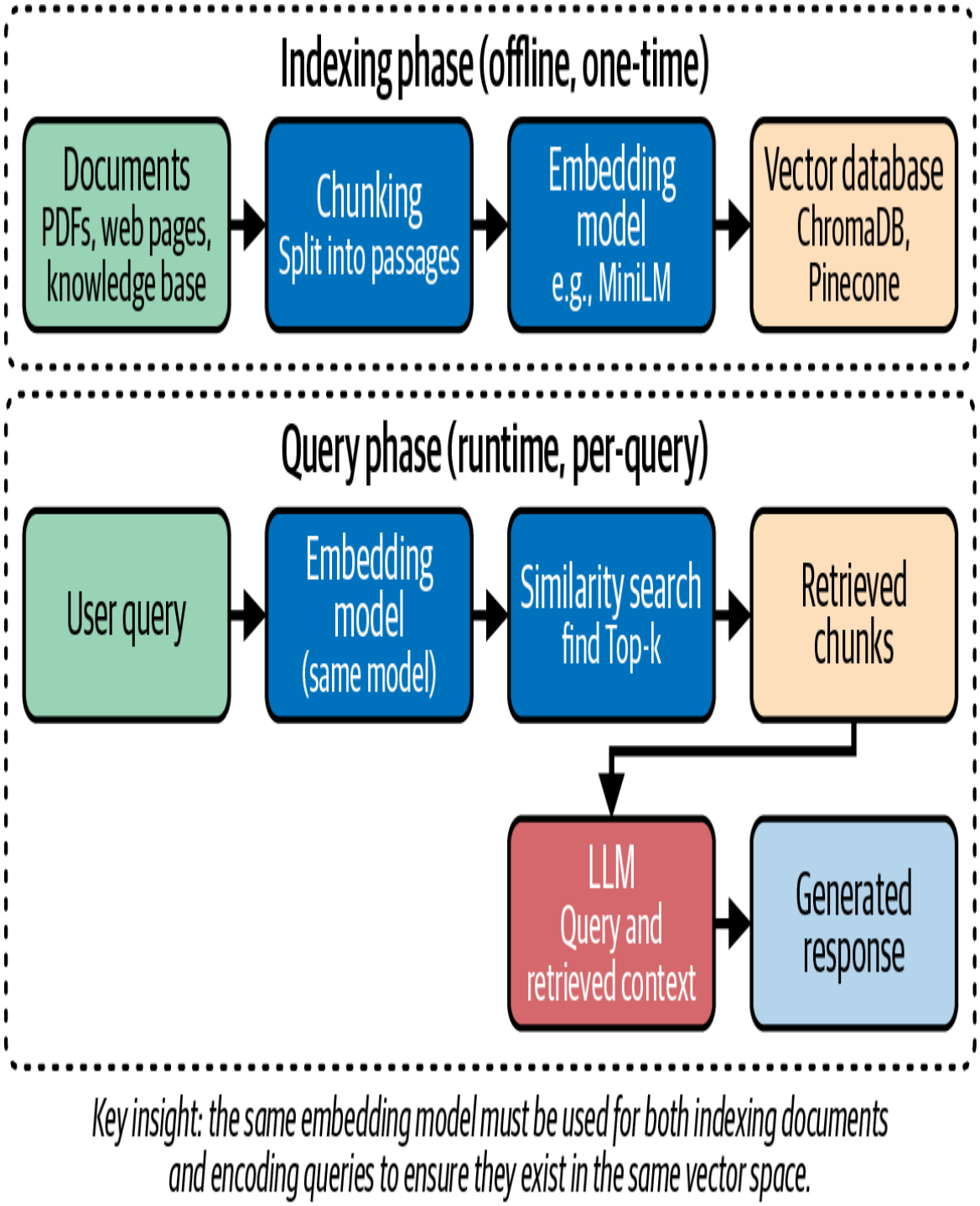


Figure 2-6. RAG architecture

Practical Applications and Use Cases

At this point, you may be wondering how these embedding models are utilized apart from their original role in LLMs. Modern embedding models

have enabled a wide range of applications:

Semantic search

Finding relevant documents based on meaning rather than keywords

Content recommendation

Suggesting similar articles, products, or media

Data deduplication

Finding and removing near-duplicate entries in databases

Knowledge base construction

Organizing and linking related pieces of information

Question answering

Finding relevant context for answering specific queries

Content clustering

Grouping similar documents or passages automatically

Simple RAG Pipeline

Here's a concrete example showing RAG in action—embedding content, retrieving by similarity, and using context for LLM responses:

```
# Example of a simplified RAG pipeline
from embedding_model import get_embedding  # Hypothetical
embedding model
from vector_db import VectorDB

# Document processing
document = "Climate change is affecting global weather
patterns..."
doc_embedding = get_embedding(document)
vector_db.store(doc_embedding)
```

```
# Query processing
query = "How does climate change affect weather?"
# Note that we use the same embedding model to create an
embedding of the
# query—this gives us an embedding vector that can be used to do
semantic
# search over the vector database. We then take the search
results and pass
  them to the LLM as context

query_embedding = get_embedding(query)
similar_docs = vector_db.search(query_embedding, top_k=3)

# Feed to LLM with context
llm_response = generate_response(query, context=similar_docs)
```

This conceptual example of RAG forms the core of the applications in this book. In future chapters, we will take a deep dive into RAG—but only after we have experience saving embeddings in vector database extensions to SQLite and PostgreSQL.

Understanding and effectively using embedding models has become a crucial skill for AI engineers, as these models form the foundation of many modern AI applications. While LLMs might receive more attention, it's often the careful use of embedding models that determines whether an AI application is practical and effective in production.

Here's a detailed example using the sentence-transformers library. Note that ChromaDB is an open source vector database, and there are many others that could be used in this example:

```
from sentence_transformers import SentenceTransformer
from chromadb import Client, Settings
import chromadb

# Initialize the embedding model
# all-MiniLM-L6-v2 is a lightweight but effective model, a good
balance of
  speed/quality
model = SentenceTransformer('all-MiniLM-L6-v2')

# Initialize ChromaDB as our vector store
```

```
# Using in-memory storage for this example
chroma_client = Client(Settings(is_persistent=False))
collection = chroma_client.create_collection(name="climate_docs")

# Example documents to process
documents = [
    "Climate change is affecting global weather patterns, causing
more extreme
      events.",
    "Rising sea levels threaten coastal communities worldwide.",
    "Greenhouse gas emissions continue to rise despite
international
      agreements."
]

# Create embeddings for our documents
# model.encode() converts text to dense vectors (embeddings)
embeddings = model.encode(documents)

# Store documents and their embeddings
# ChromaDB expects embeddings as lists, so we convert numpy
arrays
collection.add(
    embeddings=[e.tolist() for e in embeddings],
    documents=documents,
    ids=[f"doc_{i}" for i in range(len(documents))]
)

# Process a query
query = "How does climate change affect weather?"
query_embedding = model.encode(query)

# Search for similar documents
results = collection.query(
    query_embeddings=[query_embedding.tolist()],
    n_results=2
)

# Print results
for doc in results['documents'][0]:
    print(f"Retrieved document: {doc}")
```

Let’s break down each part.

Model initialization:

```
model = SentenceTransformer('all-MiniLM-L6-v2')
```

- A pretrained `SentenceTransformer` model is loaded.
- all-MiniLM-L6-v2 is a popular model that creates 384-dimensional embeddings.
- It provides a good balance of speed and quality for general use.

Vector store setup:

```
chroma_client = Client(Settings(is_persistent=False))
collection = chroma_client.create_collection(name="climate_docs")
```

- ChromaDB is initialized as an in-memory vector store.
- This creates a new collection to store our documents and embeddings.

Document processing:

```
embeddings = model.encode(documents)
```

- Each document is converted into a dense vector (embedding).
- NumPy arrays of shape (`n_documents`, `384`) are returned.
- The process is automatically batched for efficiency.

Document storage:

```
collection.add(
   embeddings=[e.tolist() for e in embeddings],
   documents=documents,
   ids=[f"doc_{i}" for i in range(len(documents))])
```

- Both the original documents and their embeddings are stored.

- Unique IDs are assigned to each document.
- NumPy arrays are converted to lists for ChromaDB compatibility.

Query processing:

```
query_embedding = model.encode(query)
```

- An embedding for the query is created using the same model.
- The query and the documents exist in the same vector space.

Similarity search:

```
results = collection.query(
   query_embeddings=[query_embedding.tolist()],
   n_results=2)
```

- The most similar documents to the query are retrieved.
- Cosine similarity is used by default.
- The top two most similar documents are returned.

This code demonstrates a simple but complete RAG pipeline. In a production environment, you might also want to add:

- Error handling
- Batch processing for large document sets
- Document chunking for long texts
- Persistent storage configuration
- Distance threshold filtering
- Metadata handling

This is a good point to note that images, audio, and video are also data types that can be mapped to vectors and even searched using a similarity search. However, those media types are much harder to map to vectors and understanding how it's done is beyond the scope of this book. Nevertheless, once you understand the basics of text embeddings in practice, it will be a solid foundational step toward understanding the embedding of richer media types.

While we have seen the sentence-transformers library briefly in action, it's a workhorse of text embedding, so it's worth looking at in more detail.

The sentence-transformers Library: The Swiss Army Knife of Text Embeddings

The sentence-transformers library, developed by SBERT.net, has emerged as one of the most practical and widely used collections of embedding models. What makes this library particularly valuable is its set of pretrained models, optimized specifically for creating sentence embeddings, along with its easy-to-use interface and strong performance across a variety of tasks. These models are much smaller in size and are highly focused on generating text embeddings only. They are all embedding models, as opposed to an embedding layer in an LLM. They are not generative models; that is, they are not used to create chatbot-like entities. They are well documented and actively maintained.

Most popular models and their characteristics

Let's take a look at the most popular text embedding models and their characteristics:

all-MiniLM-L6-v2:

- Dimensions: 384
- Model size: ~80 MB
- Speed: Very fast (2,000 sentences/second on a typical GPU)

- Pros:
  – Excellent speed-to-performance ratio
  – Small model size
  – Good general-purpose performance
- Cons:
  – Less powerful than larger models for specialized tasks
  – Lower accuracy compared to larger models
- Best for production environments where speed and efficiency matter

all-mpnet-base-v2:

- Dimensions: 768
- Model size: ~420 MB
- Speed: Moderate (1,000 sentences/second on a typical GPU)
- Pros:
  – State-of-the-art performance
  – Better semantic understanding
  – More robust across different domains
- Cons:
  – Larger model size
  – Slower inference time
- Best for applications where accuracy is critical

paraphrase-multilingual-mpnet-base-v2:

- Dimensions: 768
- Languages: 50+
- Model size: ~970 MB
- Pros:
  – Excellent cross-lingual capabilities
  – Consistent performance across languages
  – Can compare text across different languages
- Cons:
  – Large model size
  – Slower than monolingual models
- Best for multilingual applications

Training approach

These models are typically trained using a combination of techniques:

Knowledge distillation

Smaller models such as all-MiniLM-L6-v2 are trained to mimic larger models while being more efficient.

Contrastive learning

Models are trained on pairs of similar sentences (positive examples) and dissimilar sentences (negative examples).

Multiple training objectives

Some models have multiple training objectives, such as multilingual text and Q&A, as well as Natural Language Inference (NLI) data, paraphrase detection, Semantic Textual Similarity (STS) benchmarks, and question-answer pairs.

all-MiniLM-L6-v2 for RAG applications

For this book, I'll be using all-MiniLM-L6-v2 extensively in later chapters because it represents an optimal balance for enterprise applications. For specific needs—such as multilingual text or Q&A—you may see better results with other models from the sentence-transformers library.

all-MiniLM-L6-v2 is good enough, with the following properties:

- Fast enough for real-time applications
- Small enough to deploy easily
- Accurate enough for most use cases
- Excellent community support and documentation
- Production-ready performance

For a more detailed understanding of the trade-offs, see the discussions at SBERT.net.

Here's a practical example demonstrating both monolingual and multilingual capabilities:

```
from sentence_transformers import SentenceTransformer
import numpy as np

# Load our two models
monolingual_model = SentenceTransformer('all-MiniLM-L6-v2')
multilingual_model = SentenceTransformer('paraphrase-
multilingual-mpnet-base-v2')

# Example sentences in different languages
sentences = {
    'english': "The weather is beautiful today.",
```

```
    'spanish': "El clima está hermoso hoy.",
    'french': "Le temps est magnifique aujourd'hui.",
    'german': "Das Wetter ist heute wunderschön."
}

# Function to compute similarity between embeddings
def compute_similarity(emb1, emb2):
    return np.dot(emb1, emb2) / (np.linalg.norm(emb1) *
np.linalg.norm(emb2))

# Compare embeddings across languages
def compare_sentences(model, sentences):
    # Generate embeddings for all sentences
    embeddings = {
        lang: model.encode(text, convert_to_numpy=True)
        for lang, text in sentences.items()
    }

    # Compare each pair
    print(f"\nSimilarity scores for {model.__class__.__name__}:")
    for lang1 in sentences:
        for lang2 in sentences:
            if lang1 < lang2:  # Avoid duplicate comparisons
                sim = compute_similarity(embeddings[lang1],
embeddings[lang2])
                print(f"{lang1} vs {lang2}: {sim:.4f}")

# Test both models
for model in [monolingual_model, multilingual_model]:
    compare_sentences(model, sentences)

# Example of batch processing for efficiency
texts = list(sentences.values())
batch_embeddings = multilingual_model.encode(texts, batch_size=8)
```

When you run this code, you'll notice that:

- The multilingual model produces similar scores for the same sentence across languages.
- The monolingual model performs less consistently with non-English text.
- Batch processing is significantly faster than individual encoding.

Best practices when using sentence-transformers

Here are four best practices when using sentence-transformers, along with detailed explanations for each:

Batch processing

Always use batching for large datasets.

```
embeddings = model.encode(sentences, batch_size=32)
```

Length handling

Be aware of maximum sequence length.

```
# For long texts, consider chunking
max_seq_length = model.max_seq_length
```

Normalization

Consider normalizing embeddings for cosine similarity.

GPU utilization

Run on a GPU for faster processing.

```
model = SentenceTransformer('all-MiniLM-L6-v2', device='cuda')
# This is for Nvidia CUDA GPU environment, for Apple Silicon
use
# device = 'mps'
```

Best Practices for Using SentenceTransformers: A Detailed Guide

Let’s take a deeper dive into the best practices introduced earlier and add two more.

Batch processing

Encode a set of sentences together rather than one by one:

```
# Good practice
embeddings = model.encode(sentences, batch_size=32)

# Bad practice
embeddings = [model.encode(sentence) for sentence in sentences]
```

Why:

- Allows parallel processing of multiple sentences, making efficient use of GPU/CPU resources
- Can be 10x to 100x faster than processing sentences individually
- Reduces memory fragmentation and system overhead

Consequences of not batching:

- Significantly slower processing time
- Higher memory overhead due to repeated model loading
- Inefficient use of hardware capabilities, especially GPUs
- Can lead to higher costs in production environments

Length handling

Rather than embedding a whole document or even a whole paragraph, chunk the text into fixed-size segments:

```
# Good practice
max_seq_length = model.max_seq_length
def chunk_text(text, max_length=max_seq_length):
    # Split into sentences or chunks
    chunks = [text[i:i + max_length] for i in range(0, len(text),
max_length)]
    return chunks

# Process long document
long_text_embeddings = model.encode(chunk_text(long_document))
```

Why:

- Prevents truncation of important information
- Ensures consistent processing of long documents
- Maintains model performance on lengthy texts
- Avoids unexpected behavior or errors

Consequences of ignoring length limits:

- Silent truncation of input text
- Loss of potentially crucial information
- Inconsistent results for long documents
- Possible degradation of embedding quality

Normalization

Normalization reduces the length of all vectors to a single size, making them much easier to operate on and compare (see Figure 2-7):

```
# Good practice
embeddings = model.encode(sentences, normalize_embeddings=True)

# Or manually normalize if needed
from sklearn.preprocessing import normalize
embeddings = normalize(embeddings)
```

Why:

- Makes cosine similarity calculations more efficient
- Ensures consistent similarity scores across different lengths of text
- Reduces the impact of text length on similarity calculations
- Important for comparing embeddings across different documents

Consequences of skipping normalization:

- Inconsistent similarity scores
- Length bias in comparisons
- Less reliable search results
- More complex downstream processing

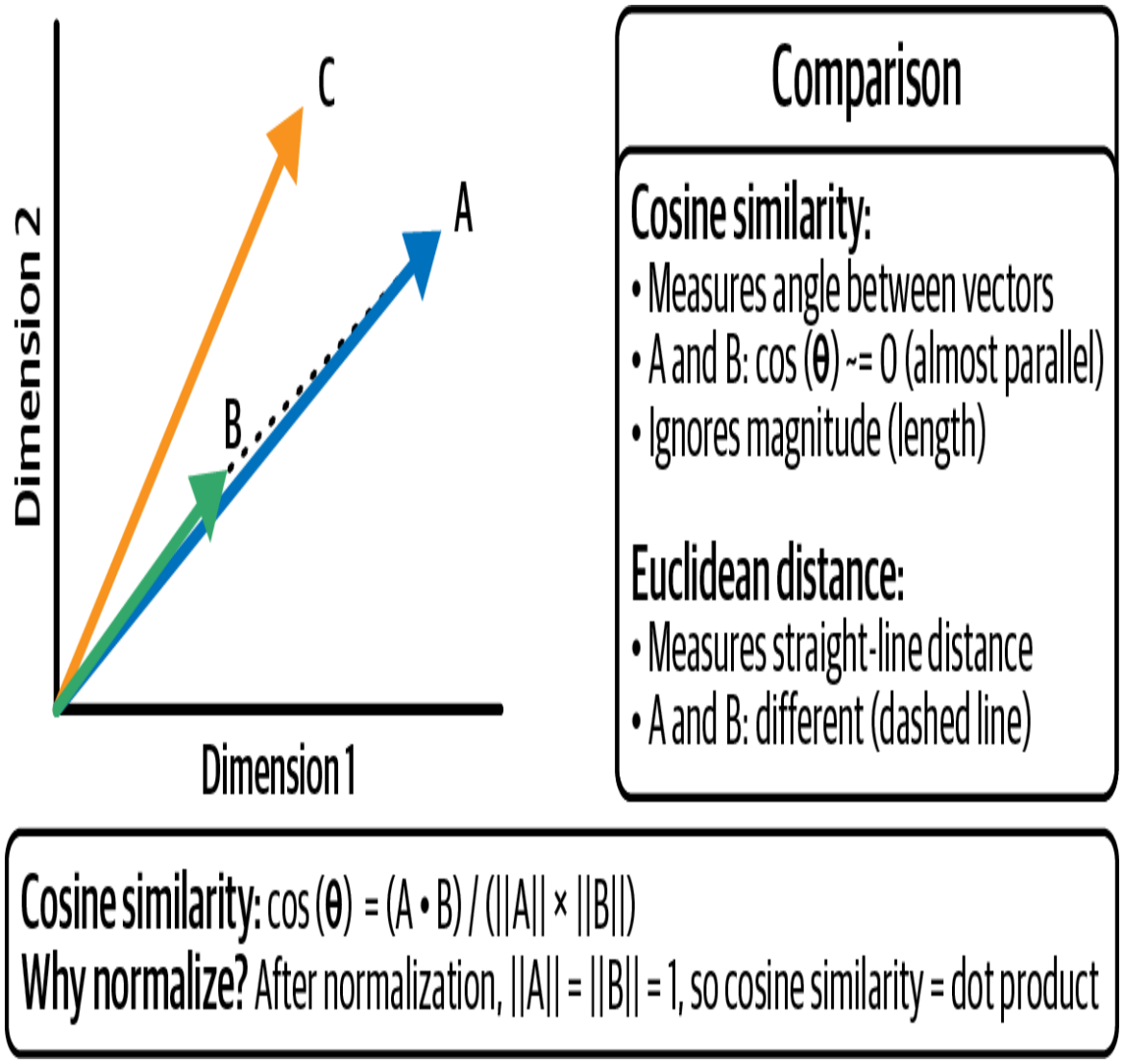


Figure 2-7. Cosine similarity versus Euclidean distance

GPU utilization

Use a GPU if one is available. GPUs are inherently high-speed vector processors, and they greatly accelerate embedding generation:

```
# Good practice
model = SentenceTransformer('all-MiniLM-L6-v2', device='cuda')

# With error handling
import torch
device = 'cuda' if torch.cuda.is_available() else 'cpu'
model = SentenceTransformer('all-MiniLM-L6-v2', device=device)
```

Why:

- Can be 10x to 50x faster than CPU processing
- Essential for processing large volumes of text
- Enables real-time applications
- More cost-effective in production environments

Consequences of not using a GPU when available:

- Significantly slower processing speed
- Higher operational costs
- Poor performance in real-time applications
- Inefficient resource utilization

Memory management

GPU memory is expensive, and we should ensure that we aren’t holding on to any unnecessarily:

```
# Good practice
import gc
import torch

def process_large_dataset(sentences, batch_size=32):
    embeddings = []
    for i in range(0, len(sentences), batch_size):
        batch = sentences[i:i + batch_size]
        batch_embeddings = model.encode(batch)
        embeddings.extend(batch_embeddings)

        # Clear GPU memory if needed
        if torch.cuda.is_available():
            torch.cuda.empty_cache()
        gc.collect()

return embeddings
```

Why:

- Prevents out-of-memory errors with large datasets
- Ensures stable performance over long-running periods
- Crucial for production environments
- Enables processing of very large datasets

Consequences of poor memory management:

- Random crashes due to memory exhaustion
- Degraded performance over time
- System instability
- Limited ability to process large datasets

Error handling

Creating embeddings for a large corpus can be computationally expensive and time-consuming. Handling known errors and edge cases prevents crashes and restarts:

```
# Good practice
from typing import List, Optional
import numpy as np

def safe_encode(
    model: SentenceTransformer,
    texts: List[str],
    batch_size: int = 32
) -> Optional[np.ndarray]:
    try:
        # Handle empty or invalid inputs
        if not texts or not isinstance(texts, list):
            return None

        # Remove empty strings and handle other edge cases
        valid_texts = [str(t) for t in texts if t and
str(t).strip()]
```

```
            if not valid_texts:
                return None

            return model.encode(valid_texts, batch_size=batch_size)

        except Exception as e:
            print(f"Error encoding texts: {e}")
            return None
```

Why:

- Prevents crashes from invalid inputs
- Makes the system more robust
- Easier to debug issues
- Better production reliability

Consequences of poor error handling:

- System crashes from unexpected inputs
- Difficult to diagnose issues
- Poor user experience
- Higher maintenance overhead

Now that we have a good understanding of embeddings, embedding models in general, and the sentence-transformers library in particular, let’s take a look at how embeddings work when they are part of an LLM. In an LLM, the embedding layer encodes tokens that can be thought of, loosely, as words or syllables. The embedding layer defines all the tokens that the LLM can handle—so if it encodes only English tokens, the model can process only English, and so on.

The Embedding Layer: The Gateway to Zero-

Shot Learning

The embedding layer in transformer models is more than just a lookup table for converting tokens to vectors—it's the foundation that enables these models to understand and generalize to new concepts without explicit training, a capability known as *zero-shot learning*.

Because these embedding layers encode not just what the token is but also where it is and how it functions in context, they set the stage for the model to reason compositionally—even about concepts it hasn't explicitly seen before.

Let's explore this crucial connection and understand how the embedding layers enable this remarkable capability.

Anatomy of Transformer Embeddings

A transformer's embedding layer actually consists of three components that work together: token embeddings, position embeddings, and segment embeddings. The following code is a simplified implementation (see also Figure 2-8):

```
# Simplified implementation of transformer embeddings
import torch
import torch.nn as nn

class TransformerEmbeddings(nn.Module):
    def __init__(self, vocab_size, embedding_dim,
max_sequence_length):
        super().__init__()

        # Token embeddings: convert words to vectors
        self.token_embeddings = nn.Embedding(vocab_size,
embedding_dim)

        # Position embeddings: add sequence order information
        self.position_embeddings =
nn.Embedding(max_sequence_length, \
            embedding_dim)
```

```
        # Segment embeddings: distinguish different parts of
input (e.g.,
        question vs context)
        self.segment_embeddings = nn.Embedding(2, embedding_dim)

        # Layer normalization for stable training
        self.layer_norm = nn.LayerNorm(embedding_dim)
        self.dropout = nn.Dropout(0.1)

    def forward(self, input_ids, segment_ids=None):
        # Get sequence length from input
        seq_length = input_ids.size(1)

        # Create position indices (0, 1, 2, ...)
        position_ids = torch.arange(seq_length,
device=input_ids.device)
        position_ids =
position_ids.unsqueeze(0).expand_as(input_ids)

        # If segment ids not provided, assume all zeros
        if segment_ids is None:
            segment_ids = torch.zeros_like(input_ids)

        # Combine all embeddings
        embeddings = (
            self.token_embeddings(input_ids) +      # Word
meaning
            self.position_embeddings(position_ids) + # Sequential
order
            self.segment_embeddings(segment_ids)     # Input
segment
        )

        # Normalize and apply dropout
        embeddings = self.layer_norm(embeddings)
        embeddings = self.dropout(embeddings)

      return embeddings
```

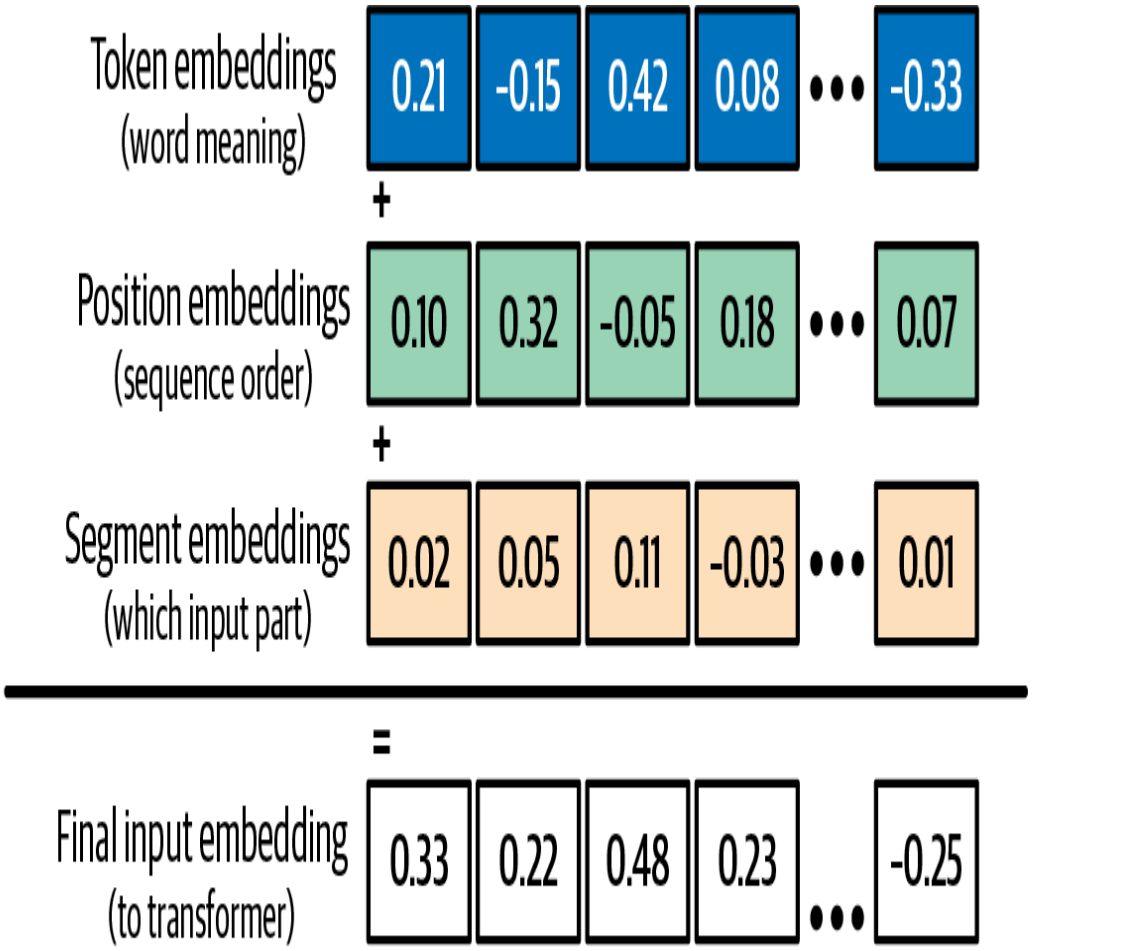


Figure 2-8. Composition of input embeddings

Connection to Zero-Shot Learning

Let’s take a look at what makes these embeddings special in their role of enabling zero-shot capabilities:

- Compositional nature
  - Embeddings capture semantic relationships between concepts.

  - New concepts can be understood through combinations of known embeddings.
  - This allows models to handle novel situations by combining familiar elements.

- Shared semantic space
  - Words with similar meanings cluster together in the embedding space.
  - Words that are different are mapped far apart (see Figure 2-9).

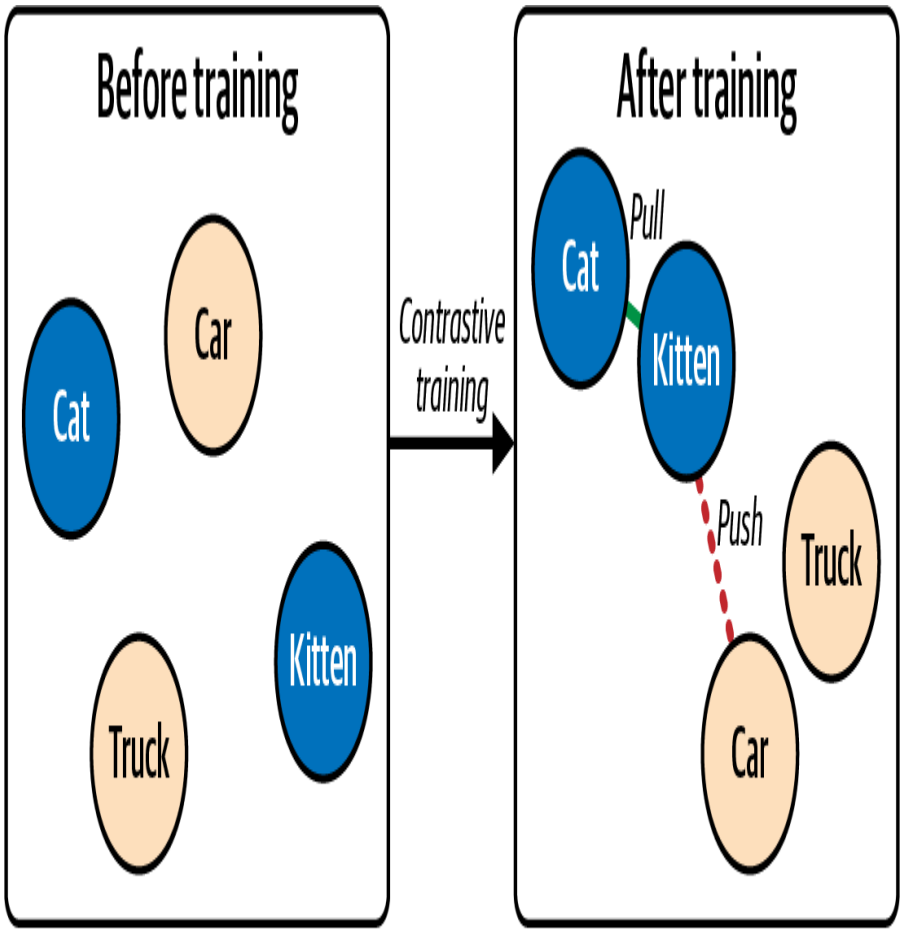


Figure 2-9. Contrastive learning: push and pull

– This clustering enables generalization to unseen examples.

– Models can infer relationships between concepts they've never explicitly seen together.

Here's an example demonstrating zero-shot classification using embeddings:

```
from sentence_transformers import SentenceTransformer, util
import torch

class ZeroShotClassifier:
```

```
    def __init__(self, model_name='all-mpnet-base-v2'):
        self.model = SentenceTransformer(model_name)

    def classify(self, text, candidate_labels):
        # Encode the input text
        text_embedding = self.model.encode(text,
convert_to_tensor=True)

        # Prepare label prompts
        label_prompts = [f"This text is about {label}" for label
in \
            candidate_labels]
        label_embeddings = self.model.encode(label_prompts, \
            convert_to_tensor=True)

        # Calculate similarities
        similarities = util.pytorch_cos_sim(text_embedding,
label_embeddings)[0]

        # Create results dictionary
        results = {
            label: float(score)
            for label, score in zip(candidate_labels,
similarities)
        }

        return results

# Example usage
classifier = ZeroShotClassifier()

text = "The new quantum computer can perform calculations in
seconds that would
       take classical computers thousands of years."
labels = ["technology", "sports", "cooking", "politics"]

results = classifier.classify(text, labels)
print("Zero-shot classification results:")
for label, score in sorted(results.items(), key=lambda x: x[1],
reverse=True):
      print(f"{label}: {score:.3f}")
```

Key Characteristics That Enable Zero-Shot Learning

Zero-shot learning is enabled by the effects of the training that the embedding has undergone (i.e., the data it has ingested). Once there is

“enough” data—an admittedly imprecise threshold—with sufficient variety and a wide range of examples of common patterns, the embedding implicitly contains a certain richness in language representation.

There are a few key characteristics of this richness that together enable zero-shot learning. It may help to note that much of this understanding is still qualitative. Research continues on internal representations of language in large models, so we don’t yet have precise definitions to make these mechanisms repeatable or reliable. What follows reflects the current working knowledge of embeddings and language modeling:

- Contextual understanding:

```
# Example of how context affects embeddings
texts = [
      "The bank by the river is eroding",
      "The bank approved my loan"
]
embeddings = model.encode(texts)
# These will have different embeddings despite sharing
the word "bank"
```

- Semantic relationships:

```
# Demonstration of semantic relationships
def find_analogies(model, a1, a2, b1):
      """Find word b2 that satisfies: a1 : a2 :: b1 :
b2"""
            e_a1 = model.encode(a1)
            e_a2 = model.encode(a2)
            e_b1 = model.encode(b1)

    # Calculate the relationship vector
    relationship = e_a2 - e_a1

    # Predict b2's embedding
    predicted_b2 = e_b1 + relationship
    return predicted_b2
```

- Cross-modal capabilities (modern embedding layers can handle multiple modalities):

```
from transformers import CLIPProcessor, CLIPModel

class MultiModalEmbeddings:
      def __init__(self):
        self.model= CLIPModel.from_pretrained("openai/clip-vit-
base-patch32")
        self.processor=
CLIPProcessor.from_pretrained("openai/clip-vit-base-patch32")

    def get_embeddings(self, text=None, image=None):
        inputs = self.processor(text=text, images=image,
return_tensors="pt")
        outputs = self.model(**inputs)

        # Get embeddings from the shared space
        if text:
            return outputs.text_embeds
        return outputs.image_embeds
```

While embeddings are a key component and are well understood, they are still not a plug-and-play, “works-great-every-time” sort of technology. There are a lot of variables, and it’s useful to understand how different aspects affect performance in practice.

Limitations and Considerations

Let’s take a look at some of the limitations of embeddings.

Quality of embeddings

More training data generally leads to better zero-shot performance, as zero-shot learning relies on the training data already embedded in the model. The more you have, the better the response. Domain-specific vocabulary may need special handling, since we want to preserve semantic context for specialized concepts—so if we possess domain-specific knowledge, we may require much more explanatory text to ensure that semantic search locates the relevant domain knowledge using more general-language associations.

In extreme specializations, quality may degrade for highly specialized or technical topics, because the rarer the subject, the fewer explanatory examples are available. Particular care is therefore needed to obtain effective search performance on highly specialized material.

Computational costs

Larger embedding dimensions provide better representation but increase memory usage. The greater the dimensionality of the embedding—that is, the larger the size of the embedding vector space—the more conceptual "space" there is for representing different ideas. However, using this extra space requires more computation. Depending on the quantity of data to be encoded, we need to balance embedding vector size with performance requirements. Avoid using higher-dimensional embeddings unless clearly needed—for example, in a multilingual model that must represent tokens from several languages.

Bias and fairness

As with all machine learning models, embeddings can inherit biases from their training data. It is therefore important to be aware of the contents of the training corpus and how it may affect the correctness and balance of responses. In zero-shot systems, which depend on the model's built-in "knowledge," it is crucial to validate performance across different domains and contexts and to obtain human feedback on perceived balance in the model's outputs, as appropriate for the target audience.

Latest Developments and Trends

The field of embedding models continues to evolve rapidly. Here are some of the latest developments at the time of writing:

Multimodal embeddings

> Newer models that can create compatible embeddings across different types of data (text, images, code)

Instruction-tuned embeddings

: Models that can adapt their embedding strategies based on specific instructions

Domain-specific embeddings

: Models that are optimized for particular fields such as legal, medical, or scientific text

Hierarchical embeddings

: Systems that create embeddings at multiple levels of granularity (words, sentences, paragraphs)

Efficient embedding

: Techniques that reduce embedding dimension while maintaining semantic quality

Vector Arithmetic with Word2Vec: A Hands-On Guide

In this section, I'll demonstrate the fascinating world of vector arithmetic using Word2Vec. You'll walk through the classic "king – man + woman ≈ queen" example and explore other interesting analogies that reveal the semantic relationships captured by word embeddings.

Step 1: Setup and Installation

First, let's install the required libraries:

```
pip install gensim
pip install numpy
pip install nltk
```

Step 2: Load Pretrained Word2Vec Model

Now let’s load the Word2Vec model, which is part of the gensim package. It comes pretrained, so we can start using it immediately for inference:

```
import gensim.downloader as api
import numpy as np
from typing import List, Tuple

# Download and load the pretrained Word2Vec model
# This might take a few minutes the first time as it downloads
~1.6 GB of data
print("Loading Word2Vec model...")
word2vec_model = api.load('word2vec-google-news-300')
print("Model loaded!")

def cosine_similarity(vec1: np.ndarray, vec2: np.ndarray) ->
float:
    """Calculate cosine similarity between two vectors"""
    return np.dot(vec1, vec2) / (np.linalg.norm(vec1) *
np.linalg.norm(vec2))

def find_similar_words(vector: np.ndarray, n: int = 5) ->
List[Tuple[str, \
    float]]:
    """Find n words with vectors most similar to the input
vector"""
    return word2vec_model.simial_by_vector(vector, topn=n)
```

Let’s break down what you just did in this step:

- You loaded Google’s pretrained Word2Vec model, trained on news articles.
- The model contains 300-dimensional vectors for about 3 million words and phrases.
- You defined helper functions for similarity calculations and for finding similar words.

Now we will create some convenience functions to do vector arithmetic programmatically and explore the classic king–queen analogy.

Step 3: Implement Vector Arithmetic Functions

Now that you have things set up, let’s get to the exciting part—treating words as vectors and doing arithmetic on words. It’s a brave new world!

```
def vector_arithmetic(*words_and_weights: Tuple[str, float]) ->
np.ndarray:
    """
    Perform vector arithmetic with words and weights
    Example: vector_arithmetic(("king", 1), ("man", -1),
("woman", 1))
    """
    resulting_vector = np.zeros(word2vec_model.vector_size)

    for word, weight in words_and_weights:
        if word not in word2vec_model:
            raise ValueError(f"Word '{word}' not found in
vocabulary")
        resulting_vector += weight * word2vec_model[word]

    return resulting_vector

def print_analogy_results(result_vector: np.ndarray,
                          original_words: List[str],
                          n_results: int = 5):
    """Print the results of vector arithmetic in a readable
format"""
    similar_words = find_similar_words(result_vector, n_results)

    print("\nVector arithmetic:", " + ".join(original_words))
    print("\nMost similar words:")
    for word, similarity in similar_words:
        if word not in original_words:  # Don't show input words
in results
            print(f”{word}: {similarity:.4f}”}
```

These functions help you:

- Perform vector arithmetic with any number of words and weights
- Format and display the results clearly
- Filter out input words from the list of results

Step 4: Classic King–Queen Analogy

You've finally arrived at the king–queen analogy. Let's put our machinery to work and do arithmetic on words! What we are doing here is the linguistic equivalent of the primordial soup from which meaning—and, eventually, language models—emerge:

```
def demonstrate_royal_analogy():
    print("\n=== Royal Analogy Demonstration ===")

    # king – man + woman = queen
    result = vector_arithmetic(
        ("king", 1),
        ("man", -1),
        ("woman", 1)
    )

    print_analogy_results(
        result,
        ["king", "man", "woman"],
        n_results=5
    )

# Run the demonstration
demonstrate_royal_analogy()
```

This should output something like the following:

```
=== Royal Analogy Demonstration ===

Vector arithmetic: king - man + woman

Most similar words:
queen: 0.7118
monarch: 0.6523
princess: 0.6342
kings: 0.6103
prince: 0.5932
```

Step 5: More Interesting Analogies

There are many more analogies to explore that show how word arithmetic works in surprising ways. Let's keep experimenting:

```
def explore_more_analogies():
    print("\n=== More Analogy Examples ===")

    # Example 1: Paris is to France as Berlin is to ...?
    result1 = vector_arithmetic(
        ("Paris", -1),
        ("France", 1),
        ("Berlin", 1)
    )
    print("\nParis : France :: Berlin : ?")
    print_analogy_results(result1, ["Paris", "France", "Berlin"])

    # Example 2: Walking is to walked as running is to ...?
    result2 = vector_arithmetic(
        ("walking", -1),
        ("walked", 1),
        ("running", 1)
    )
    print("\nwalking : walked :: running : ?")
    print_analogy_results(result2, ["walking", "walked",
"running"])

    # Example 3: Good is to better as bad is to ...?
    result3 = vector_arithmetic(
        ("good", -1),
        ("better", 1),
        ("bad", 1)
    )
    print("\ngood : better :: bad : ?")
    print_analogy_results(result3, ["good", "better", "bad"])

# Run the demonstrations
explore_more_analogies()
```

Step 6: Interactive Exploration Tool

Now that we've tested a few different ways to do arithmetic with words, let's create a simple tool to explore analogies interactively:

```
def explore_custom_analogy():
    """Interactive tool for exploring word analogies"""
    print("\n=== Custom Analogy Explorer ===")
    print("Format: word1 is to word2 as word3 is to ???")

    try:
        word1 = input("Enter word1: ").strip()
        word2 = input("Enter word2: ").strip()
        word3 = input("Enter word3: ").strip()

        result = vector_arithmetic(
            (word1, -1),
            (word2, 1),
            (word3, 1)
        )

        print(f"\n{word1} : {word2} :: {word3} : ?")
        print_analogy_results(result, [word1, word2, word3])

    except ValueError as e:
        print(f"Error: {e}")
        print("Make sure all words are in the vocabulary!")

# Let users try their own analogies
if __name__ == "__main__":
    explore_custom_analogy()
```

When you run this code, you'll be able to:

- Try the classic king–queen analogy.
- See several other interesting analogies.
- Explore your own analogies interactively.

The results demonstrate how Word2Vec captures semantic relationships in its vector space. Here are some interesting patterns you might observe:

- Gender relationships (king/queen, actor/actress)
- Verb tenses (walk/walked, run/ran)
- Country-capital relationships (France/Paris, Germany/Berlin)

- Comparative forms (good/better, bad/worse)

Final Words on Vector Arithmetic

Just as arithmetic is foundational but not the be-all and end-all of mathematics, vector arithmetic is only the first step—not the entire foundation—for embeddings and LLMs. Word embeddings alone are not enough to represent words with multiple meanings or to capture how meaning shifts through complex grammar. More sophisticated techniques build on these basics.

In short, word vector arithmetic is a necessary conceptual building block, but it is by no means sufficient.

Conclusion

This rather meaty chapter plunged headlong into the core technology underlying how unstructured data is represented in vector databases and, separately, in language models.

The chapter concluded with practical, hands-on experience applying the concepts from Mikolov et al.'s original paper by performing vector arithmetic in practice. Though technically and conceptually demanding, this foundation is essential for understanding vector database applications, and the effort invested here will repay itself many times over in the coming chapters. Feel free to bookmark and refer back to this chapter as we go deeper into vector database applications, especially RAG.

OceanofPDF.com

Chapter 3. Similarity Search with FAISS

FAISS, or Facebook AI Similarity Search, is an open source library developed at Meta (formerly Facebook) to efficiently perform similarity search and clustering of dense vectors. Its evolution reflects Meta's growing need for scalable solutions to handle the massive datasets driving its personalized recommendations, content filtering, and image/video search functionalities.

This chapter is for developers comfortable with mathematical foundations and interested in the internals of vector search. You can skim it if your main interest lies in building applications rather than writing your own vector database. If you want to experiment with creating your own vector database, FAISS is a great place to start. It is both a robust similarity search engine out of the box and a powerful, flexible toolkit for building a customized vector database and similarity search engine. You will also find it useful for extending the capabilities of sqlite-vss, the vector search extension for SQLite3, which we will explore in detail in the next chapter. For now, understand that FAISS is a powerful tool to have in your custom search toolset—it is heavy artillery. Reading this chapter will also give you an inside look at what makes vector search engines tick.

FAISS can use a GPU, but in this book we will only use the CPU version to make the code accessible to the widest audience.

Foundations

The core motivation behind FAISS stemmed from limitations in existing solutions for handling the sheer volume and dimensionality of data generated by Meta's platforms. Traditional approaches such as brute-force search, while conceptually simple, become computationally infeasible with

millions or billions of vectors. The challenge wasn't just speed but also resource efficiency—Meta needed a solution that could scale gracefully across multiple machines while minimizing memory footprint and power consumption.

FAISS addresses this challenge through a combination of indexing and search algorithms. Instead of comparing a query vector to every vector in the dataset (brute force), FAISS leverages various indexing structures to drastically reduce the search space. These structures intelligently organize the vectors, allowing FAISS to quickly eliminate irrelevant candidates and focus on a smaller subset most likely to contain the nearest neighbors.

Several key indexing techniques underpin FAISS's effectiveness, which we'll also discuss in this chapter. Quantization techniques—such as product quantization (PQ) and scalar quantization (SQ)—reduce the memory footprint by representing vectors with fewer bits. This enables faster comparisons and facilitates the storage of larger datasets in RAM. (This quantization is not to be confused with quantization in LLM models, where weights are reduced in precision to shrink the memory footprint of the model. The commonality is the reduction in precision to lower memory and disk usage.)

In addition, tree-based indexes, such as the inverted file (IVF) index, accelerate the search by partitioning the data into smaller clusters. The query vector is first routed to the relevant cluster(s) based on its characteristics, significantly reducing the number of comparisons needed. HNSW graphs provide another indexing method, offering a balance between search speed and accuracy.

Beyond basic indexing, FAISS offers advanced features to fine-tune performance for specific application needs. These include:

GPU acceleration

> FAISS leverages the parallel processing capabilities of GPUs to speed up both indexing and search operations—particularly beneficial for very large-scale datasets.

Clustering algorithms

FAISS includes various clustering algorithms that enable efficient grouping of similar vectors—useful for tasks such as anomaly detection and data exploration.

Hybrid approaches

FAISS supports combining multiple indexing structures to achieve optimal performance, allowing tailoring of the index to specific data characteristics and query patterns.

The evolution of FAISS is ongoing. Since its initial release, the library has undergone continuous improvement in addressing memory management, increasing ANN search accuracy, and adding support for new hardware architectures. This sustained development reflects Meta's commitment to a robust, scalable solution for similarity search—one that evolves to meet the demands of ever-growing data volumes and increasingly complex applications.

The open source nature of FAISS has also fostered community contributions, further enhancing its capabilities and accessibility to a wider range of researchers and developers. Its impact extends far beyond Meta, empowering diverse applications across multiple domains that rely on efficient similarity search.

At the heart of FAISS lie three fundamental concepts: vector representations, distance metrics, and index structures. Understanding these core elements is crucial for effectively implementing and optimizing similarity search systems.

This chapter breaks down the essential mathematical concepts behind FAISS. It covers vector spaces, different distance metrics (Euclidean, cosine, inner product), and the basic principles of index structures. It also provides the theoretical foundation needed to understand how FAISS works under the hood.

Vector Representations

In the context of FAISS, *vectors* are dense arrays of floating-point numbers that represent objects in a high-dimensional space. Each dimension corresponds to a feature or characteristic of the object. For instance, a 768-dimensional vector might represent a sentence processed through a BERT model, where each dimension captures some semantic aspect of the text. In computer vision applications, a 2,048-dimensional vector could represent an image processed through a ResNet model, with each dimension encoding visual features such as edges, textures, or complex patterns.

Mathematically, we represent a vector x in d-dimensional space as:

$$x = [x_1, x_2, \ldots, x_d] \in \mathbb{R}^d$$

Here, each x_i represents a coordinate in the i-th dimension. The notation $\mathbb{R}^d$ indicates that each coordinate can take any real-number value, and we're working in d-dimensional space. This representation is fundamental to how FAISS organizes and searches through data.

The collection of vectors we want to search through forms our dataset X:

$$X = [x^{(i)}, x^{(2)}, \ldots, x^{(n)}] \text{ where each } x^{(i)} \in \mathbb{R}^d$$

Each $x^{(i)}$ represents one item in our database of n items. For example, in an image search system, each $x^{(i)}$ would represent one image in our database of n images.

Distance Metrics

Distance metrics quantify the similarity (or dissimilarity) between vectors. The choice of metric is crucial because it defines what we mean by "similarity" in a search system. FAISS supports several distance metrics, each with its own mathematical properties and use cases.

Let's look at the main types in the following sections. See Figure 3-1 for more details.

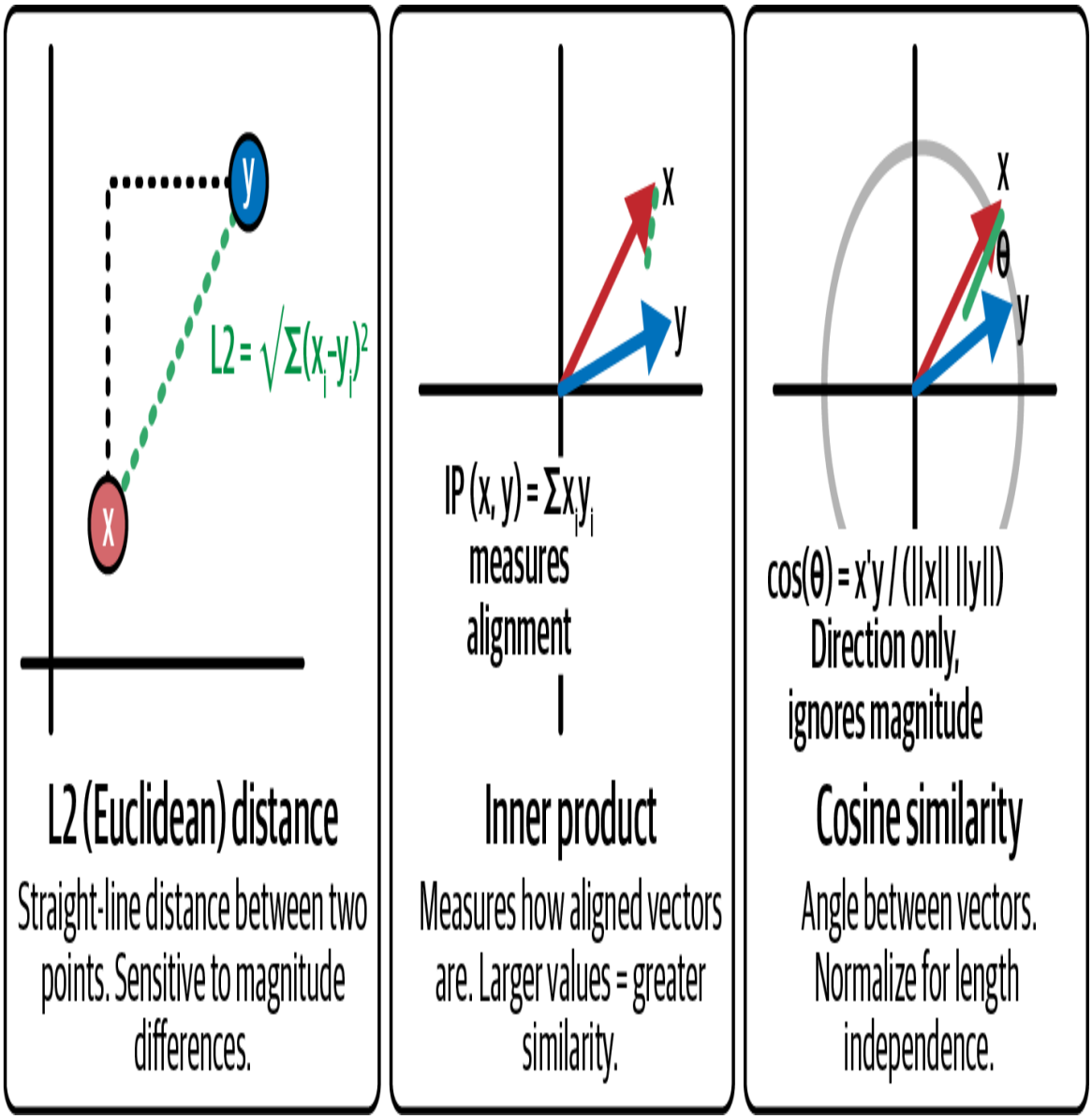


Figure 3-1. Comparison of distance metrics

L2 (Euclidean) distance

L2, or Euclidean, distance is the most commonly used metric. It is computed as:

$$L2(x, y) = \sqrt{\sum_{i=1}^{d} (x_i - y_i)^2}$$

This formula calculates the straight-line distance between two points in *d*-dimensional space. It's intuitive and works well for many applications but

can be sensitive to the scale of different dimensions. For example, if one dimension represents age (0–100) and another represents height in centimeters (150–200), the age dimension might dominate the distance calculation unless features are properly normalized.

Example use: k-means clustering (in its classic formulation). Minimizing within-cluster sums of squared distances is the k-means objective, so squared L2 is the natural fit.

When it tends to be good: When absolute magnitude differences are meaningful and when you want to penalize large coordinate errors heavily.

Inner-product distance

Inner-product distance is used when vectors are normalized, being equivalent to cosine similarity in this context. The inner product measures the degree of alignment between two vectors. When vectors are normalized to unit length, the inner product becomes equivalent to cosine similarity.

It is computed as:

$IP(x, y) = \sum_{i=1}^{d}(x_i y_i)$

The inner product measures how aligned two vectors are. When vectors are normalized (length = 1), larger inner products indicate greater similarity. This metric is particularly useful in NLP, where we care more about a vector’s direction than about its magnitude.

Example use: Recommender systems (matrix factorization/embeddings). User and item embeddings scored by the dot product; magnitude can encode confidence or popularity, while direction encodes preference.

When it tends to be good: When you want both alignment and magnitude to matter—for example, “strong preference” versus “weak preference” can be encoded by vector norm.

Cosine similarity

Cosine similarity is related to the inner product but is normalized by the vector magnitudes. It is computed as:

$$cos(x, y) = \frac{\sum_{i=1}^{d} x_i y_i}{\sqrt{\sum_{i=1}^{d} {x_i}^2} \sqrt{\sum_{i=1}^{d} {y_i}^2}}$$

Cosine similarity measures the cosine of the angle between two vectors, effectively normalizing for vector length. This is especially useful when we want to consider two vectors similar if they point in the same direction, regardless of their magnitudes. For example, in text similarity, two documents might be semantically alike even if one is much longer than the other.

Example use: Text/semantic similarity with embeddings. In many embedding models, direction carries semantics while norm may be less stable or less meaningful; cosine similarity eliminates norm effects.

When it tends to be good: Search over mixed-length content. If vector norms correlate with length or verbosity, cosine similarity avoids “longer text wins” bias.

Selection Heuristics

Here are some guidelines for when to use each metric:

- Use squared L2 when you’re optimizing least-squares-style objectives or need fast distance computations with identical nearest-neighbor rankings to L2.
- Use inner product when both direction and magnitude matter (for example, recommender-system scoring or learned-retrieval objectives).
- Use cosine similarity when only direction matters (common in NLP embeddings, TF-IDF, and other semantic-similarity tasks).

FAISS Indexes

FAISS offers a rich variety of index types, each designed for different use cases and performance trade-offs. They can be broadly categorized based on

their underlying techniques. Let’s break down some of the most important FAISS index types in the sections that follow.

Flat Indexes (Brute Force)

Flat indexes search over the entire space and do not intelligently optimize the amount of work performed—hence the term *brute force*. They compute the distance between a given vector and every vector in the database and return the best results, usually sorted by closeness.

FAISS provides several flat index implementations that perform exhaustive searches using different distance metrics:

`IndexFlatL2`

: The most basic index type, performing an exhaustive search using the L2 (Euclidean) distance. It compares the query vector to every vector in the dataset. While providing exact results, it’s computationally expensive and not suitable for very large datasets.

`IndexFlatIP`

: Similar to `IndexFlatL2` but uses the inner product (dot product) as the similarity metric. It is useful for normalized vectors.

`IndexFlat`

: A generic flat index that supports various distance metrics specified at search time.

Pros

- Exact results (guaranteed to find the true nearest neighbors)
- Simple to understand and use

Cons

- Slow for large datasets
- High memory consumption, as it stores all vectors in their original form

IVF-Based Indexes

These indexes partition the data into clusters and use an inverted list to store the mapping between cluster centroids and the data points assigned to each cluster. They provide approximate results but are significantly faster than flat indexes.

“Wait—what is an inverted list?” you might ask. Instead of storing documents and scanning them to determine whether they contain a term, you precompute the following:

Dictionary (vocabulary)

Every term you care about

Posting list (inverted list)

For each term, the list of document IDs containing that term (often with extra metadata)

The “inverted” aspect refers to flipping the natural relationship “document has terms” into “term has documents.” In other fields, this structure is also known as a *concordance*.

One widely used family of inverted indexes in FAISS combines inverted file lists with different vector encoding schemes, each offering a distinct trade-off between speed, memory footprint, and accuracy. See Figure 3-2 for details:

`IndexIVFFlat`

Combines IVF with a flat index for storing vectors within each cluster. The vectors within a cluster are searched exhaustively.

`IndexIVFPQ`

One of the most popular index types. It uses IVF for coarse quantization and PQ to compress vectors within each cluster, offering a strong balance of speed, accuracy, and memory use.

`IndexIVFPQFastScan`

An implementation of `IndexIVFPQ` optimized for speed over memory consumption. It is significantly faster than `IndexIVFPQ` by using a more cache-friendly reimplementation of product quantization.

`IndexIVFScalarQuantizer`

Similar to `IndexIVFPQ` but uses SQ instead of PQ for compressing vectors within clusters. SQ quantizes each dimension of a vector independently.

`IndexIVFPQR`

An extension of `IndexIVFPQ` that adds a refinement step after the initial PQ search, using a small portion of the original vectors to improve accuracy.

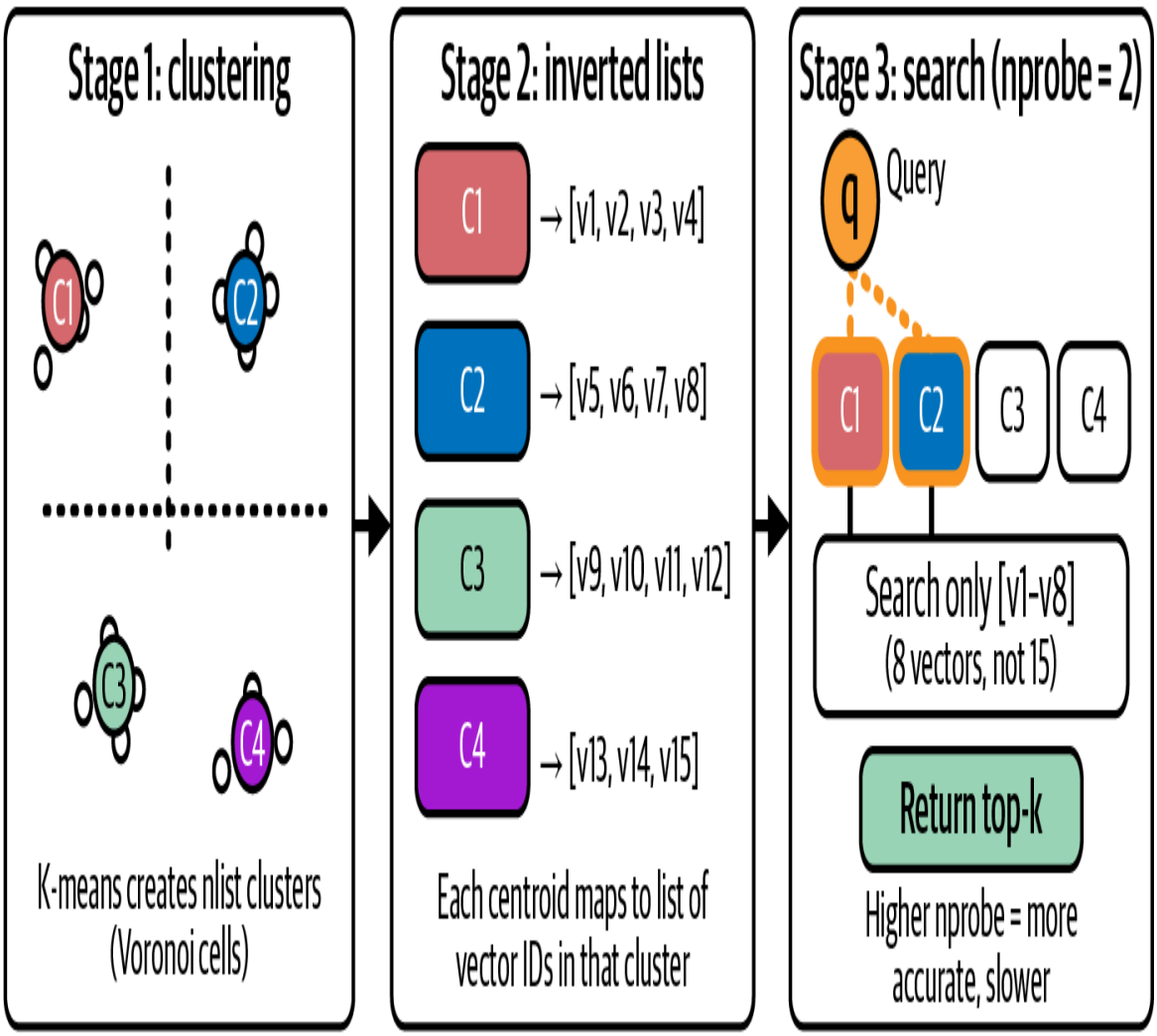


Figure 3-2. IVF index construction and search

Pros

- Much faster than flat indexes for large datasets
- Reduced memory footprint compared to flat indexes, especially with PQ

Cons

- Approximate results (may not always find the true nearest neighbors)
- Requires training to determine cluster centroids and quantization parameters

LSH-Based Indexes

Let’s look at one LSH-based index:

`IndexLSH`

Implements LSH, which uses random projections to hash similar vectors into the same buckets

Pros

- Relatively fast for datasets with certain properties

Cons

- Performance sensitive to parameter choice
- Generally less accurate than IVF-based methods for high-dimensional data

HNSW-Based Indexes

The HNSW index is one of the most powerful, especially for human language, which has a natural structure of hierarchically clustered concepts —from broader categories down to more refined ones.

The HNSW index searches for clusters of concepts nearest to the query concept and then performs a more detailed search within that cluster. This avoids searching large parts of the database by exploiting the nested structure of language.

It comes in the following variants:

`IndexHNSWFlat`

Combines HNSW with a flat index for storing vectors

`IndexHNSWPQ`

Combines HNSW with PQ for vector compression

`IndexHNSWSQ`

Combines HNSW with SQ

Pros

- State-of-the-art search speed, especially for high-recall scenarios
- Excellent performance on high-dimensional data

Cons

- Higher memory usage compared to IVF-based indexes
- Can be slower to build the index

Other Specialized Indexes

Here are a handful of other specialized indexes you should know about:

`IndexScalarQuantizer`

A simple index that performs scalar quantization without any clustering or graph structure

`IndexPQ`

Performs product quantization without clustering

`IndexBinaryFlat`

A flat index for binary vectors (vectors with only 0/1 values), using Hamming distance

`IndexBinaryIVF`

An IVF index for binary vectors

Composite and Transformative Indexes

FAISS allows for creating complex indexes by composing and transforming simpler ones. Some common examples include:

`IndexPreTransform`

Applies a linear transformation (e.g., principal component analysis [PCA]) to vectors before indexing them with another index

`IndexIDMap`

Adds an ID mapping on top of another index, allowing for efficient retrieval of original IDs

`IndexRefineFlat`

Refines results from another index using a flat index on a subset of data

`IndexShards`

Shards an index across multiple CPU or GPU instances

Choosing the Right Index

The optimal index type depends on several factors:

Dataset size

For small datasets, flat indexes might be sufficient; for larger datasets, IVF- or HNSW-based indexes are preferred.

Dimensionality

HNSW often performs best for high-dimensional data.

Desired accuracy

If exact results are crucial, use flat indexes; for approximate results, IVF or HNSW is suitable.

Memory constraints

PQ-based indexes offer significant memory savings.

Speed requirements

HNSW generally provides the fastest search speeds, followed by IVF.

Hardware

GPU acceleration can significantly improve performance for many index types.

Understanding these different index types and their trade-offs is essential for effectively utilizing FAISS to build efficient and accurate similarity search systems. Experimentation and careful consideration of your specific requirements are key to finding the best solution for your application. See Figure 3-3.

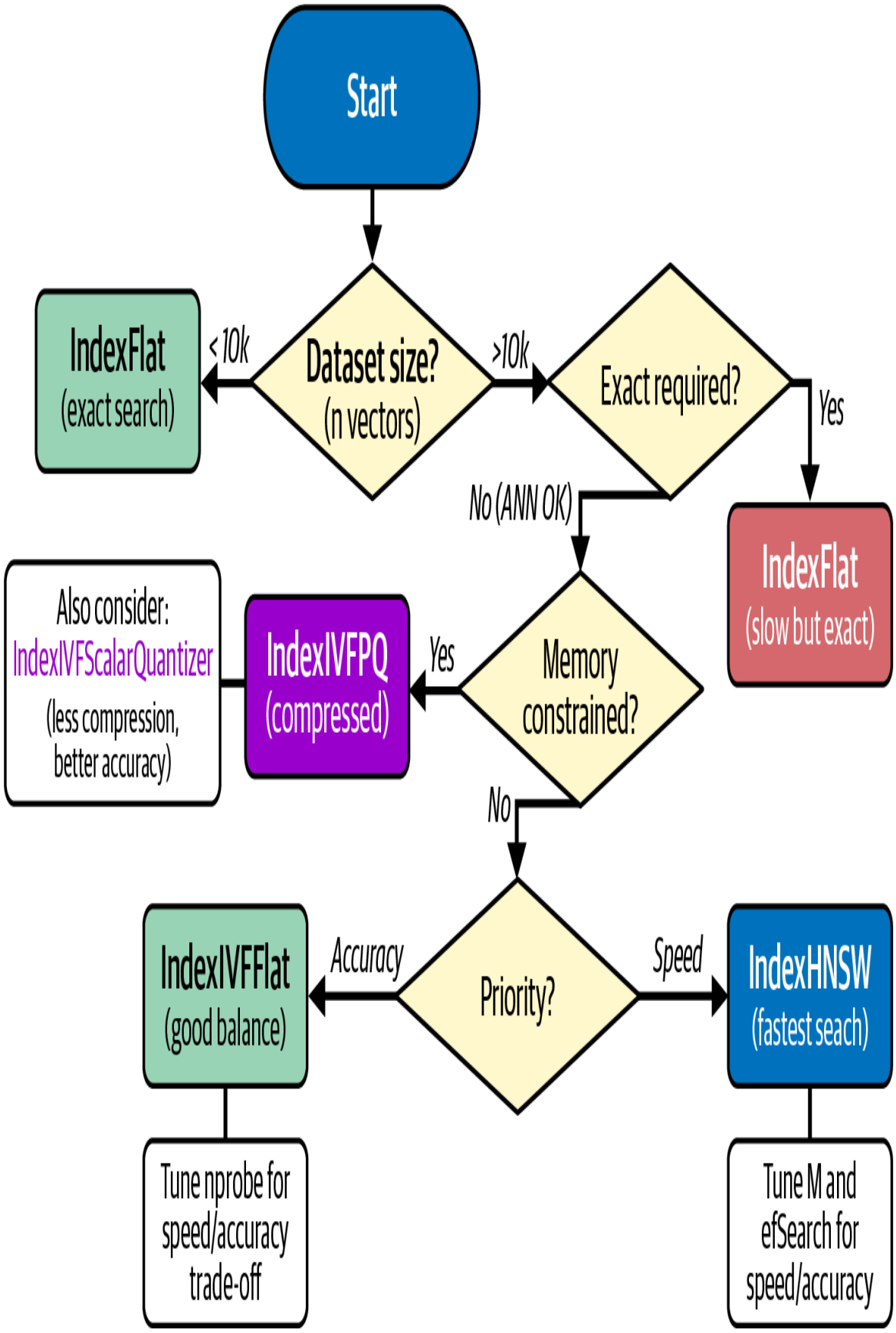

Start
Dataset size?
(n vectors)
< 10k
IndexFlat
(exact search)
>10k
Exact required?
Yes
IndexFlat
(slow but exact)
No (ANN OK)
Memory
constrained?
Yes
IndexIVFPQ
(compressed)
Also consider:
IndexIVFScalarQuantizer
(less compression,
better accuracy)
No
Priority?
Accuracy
IndexIVFFlat
(good balance)
Tune nprobe for
speed/accuracy
trade-off
Speed
IndexHNSW
(fastest seach)
Tune M and
efSearch for
speed/accuracy

Figure 3-3. FAISS index selection decision tree

The choice between flat and IVF indexes fundamentally comes down to balancing accuracy and speed. Flat indexes are ideal for smaller datasets where exact results are required, whereas IVF indexes excel in large-scale applications where approximate results are acceptable. Understanding these trade-offs is crucial for building efficient similarity search systems. The implementation details and best practices provided in this section serve as a foundation for more advanced FAISS features and optimizations.

See Table 3-1 for FAISS indexes and some of their properties.

Table 3-1. FAISS indexes and their properties (adapted from the Meta FAISS

Method	Class name	`index_factory`	Main parameters	Byt
Exact search for L2	`IndexFlatL2`	`"Flat"`	`d`	4*d
Exact search for inner product	`IndexFlatIP`	`"Flat"`	`d`	4*d
HNSW graph exploration	`IndexHNSWFlat`	`"HNSW,Flat"`	`d`, `M`	4*d 2 * 4
Inverted file with exact post-verification	`IndexIVFFlat`	`"IVFx,Flat"`	`quantizer`, `d`, `nlists`, `metric`	4*d
LSH (binary flat index)	`IndexLSH`	—	`d`, `nbits`	ceil(

Method	Class name	`index_factory`	Main parameters	Byte
SQ in flat mode	`IndexScalarQuantizer`	`"SQ8"`	`d`	d
PQ in flat mode	`IndexPQ`	`"PQx"`, `"PQ"M"x"nbits`	`d`, `M`, `nbits`	ceil(8)
IVF and scalar quantizer	`IndexIVFScalarQuantizer`	`"IVFx,SQ4"`, `"IVFx,SQ8"`	`quantizer`, `d`, `nlists`, `qtype`	SQf + 8, or S
IVFADC (coarse quantizer + PQ on residuals)	`IndexIVFPQ`	`"IVFx,PQ"y"x"nbits`	`quantizer`, `d`, `nlists`, `M`, `nbits`	ceil(nbits
IVFADC + R (same as IVFADC with re-ranking based on codes)	`IndexIVFPQR`	`"IVFx,PQy+z"`	`quantizer`, `d`, `nlists`, `M`, `nbits`, `M_refine`, `nbits_refine`	M+M

Quantization

Quantization is a technique used across many domains where data must be represented or searched efficiently, balancing space, time, and accuracy. In quantizing a value, its resolution is reduced (e.g., the number of bits used for representation), which saves memory and speeds up search by reducing the number of elements compared—but at the cost of accuracy. Depending on the application and the degree of quantization—that is, how coarsely the data is represented—this loss of accuracy can be accepted as a trade-off for gains in speed and space efficiency.

Let's look at the quantization methods supported by FAISS.

SQ

FAISS supports SQ as one of its indexing methods for reducing memory usage and improving search speed. SQ is a compression technique that represents each component of a vector with fewer bits. Instead of storing, say, 32-bit floats, each component might be represented with 8-bit integers (or even fewer bits).

This process involves three main steps:

Defining codebooks

> For each dimension (or a group of dimensions) of the vector, a codebook is created containing a limited number of representative values (centroids).

Encoding

> Each vector component is replaced by the index of the closest centroid in the corresponding codebook. This index typically requires fewer bits than the original floating-point value.

Decoding

> To reconstruct the approximate vector, the indexes are used to look up the corresponding centroids in the codebooks.

FAISS provides several SQ implementations, often used in conjunction with techniques such as PQ or IVF. Key aspects include:

`IndexScalarQuantizer`

This is the basic index for performing SQ in FAISS. It allows you to specify the number of bits per component (e.g., 8 bits for `QT_8bit`).

Quantization types

FAISS offers multiple SQ modes. These include:

`QT_8bit`

Each component is encoded with 8 bits (range 0–255).

`QT_4bit`

Each component is encoded with 4 bits (range 0–15).

`QT_fp16`

Components are stored as 16-bit floating-point numbers. This is less aggressive than `QT_8bit` and `QT_4bit` but still reduces memory usage compared to 32-bit floats.

`QT_8bit_uniform` *and* `QT_4bit_uniform`

These use a uniform distribution for the codebooks, making training faster than for nonuniform variants.

`QT_8bit_direct` *and* `QT_6bit_direct`

These map components directly to bit representations, eliminating the need for a codebook lookup but potentially reducing accuracy (see Figure 3-4).

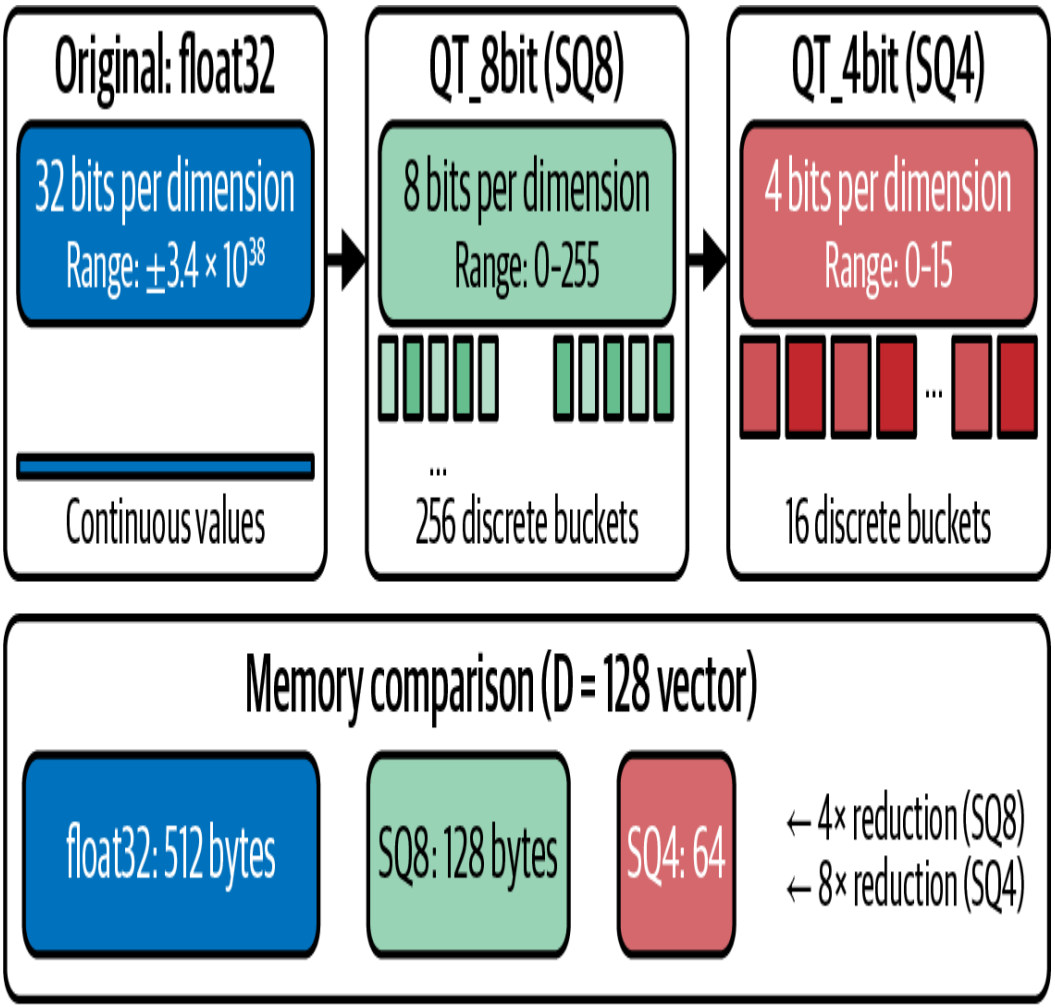


Figure 3-4. SQ compression

Combination with other indexes

SQ is often a preprocessing step before more complex indexing techniques. For instance:

`IndexIVFScalarQuantizer`

This uses an inverted filesystem whose centroids are determined by SQ, making it a strong choice for very large datasets.

`IndexPQ` *followed by* `IndexScalarQuantizer`

This combines PQ with SQ by first reducing dimensionality with PQ and then compressing the resultant codes further with SQ.

Benefits of SQ in FAISS

Let’s review some of the key benefits of SQ:

Reduced memory usage

SQ significantly reduces the memory footprint of vector databases by representing each vector with fewer bits.

Faster search speed

Smaller vectors mean faster computations, particularly for distance calculations during search operations.

Simplified implementations

SQ is relatively straightforward to implement and can be quicker to train than more complex quantization methods.

Trade-offs of SQ in FAISS

While SQ offers many benefits, there are some trade-offs:

Accuracy loss

Quantization inevitably introduces some information loss, which can affect search accuracy. The fewer bits used, the greater the potential loss in precision.

Choice of codebook

The quality of the codebook strongly influences accuracy. A well-chosen codebook that represents the data distribution faithfully is essential.

PQ

The other quantization technique used in FAISS is PQ, which takes SQ a step further. In PQ, a vector space is divided into subspaces. This means that each vector is partitioned into segments, and each segment is quantized separately with its own codebook. The same segmentation structure is applied across all vectors in the collection, though each segment may employ a distinct quantization scheme if needed.

The problem

Imagine you have a large database of vectors (e.g., image or text embeddings) with hundreds or even thousands of dimensions. Searching for the most similar vectors directly is computationally expensive and storing them in memory can become infeasible. PQ to the rescue! PQ addresses these challenges through:

Decomposition

> Dividing the high-dimensional vector space into multiple lower-dimensional subspaces

Quantization

> Quantizing the vectors within each subspace using a codebook

Indexing

> Creating a compressed representation of each original vector using the codebook indices

Walk-through of PQ

Here's a step-by-step explanation of how PQ works in FAISS (see Figure 3-5):

1. Subspace partitioning

> Suppose you have a *D*-dimensional vector (e.g., 128 dimensions). PQ divides this vector into *m* disjoint subvectors, each of dimension *D/m*

(assuming D is divisible by m). For example, if $m = 4$ and $D = 128$, each subvector would have 128 / 4 = 32 dimensions. Each subvector now represents a subspace of the original space.

2. Codebook learning (for each subspace)

Now, for each of the m subspaces, FAISS learns a codebook. A codebook is a set of k representative vectors (centroids) for the subspace, typically obtained using k-means clustering. Suppose $k = 256$ for each subspace. For each subspace, k-means is run to identify k clusters. The centroids of these clusters form the codebook—meaning that any vector in a cluster can be represented by its centroid.

3. Vector quantization (encoding)

For each original vector in the database, the vector is divided into m subvectors. For each subvector, FAISS finds the closest centroid (codebook entry) in the corresponding subspace's codebook. The index of this closest centroid is stored. Thus, for each of the m subvectors, you end up with an integer index between 0 and $k - 1$.

4. Compressed representation

The original high-dimensional vector is now represented by a sequence of m integer indices. These are much smaller than the original floating-point values, producing significant storage compression. This compressed form is called a *PQ code*.

5. Similarity search (approximate)

Query vector encoding

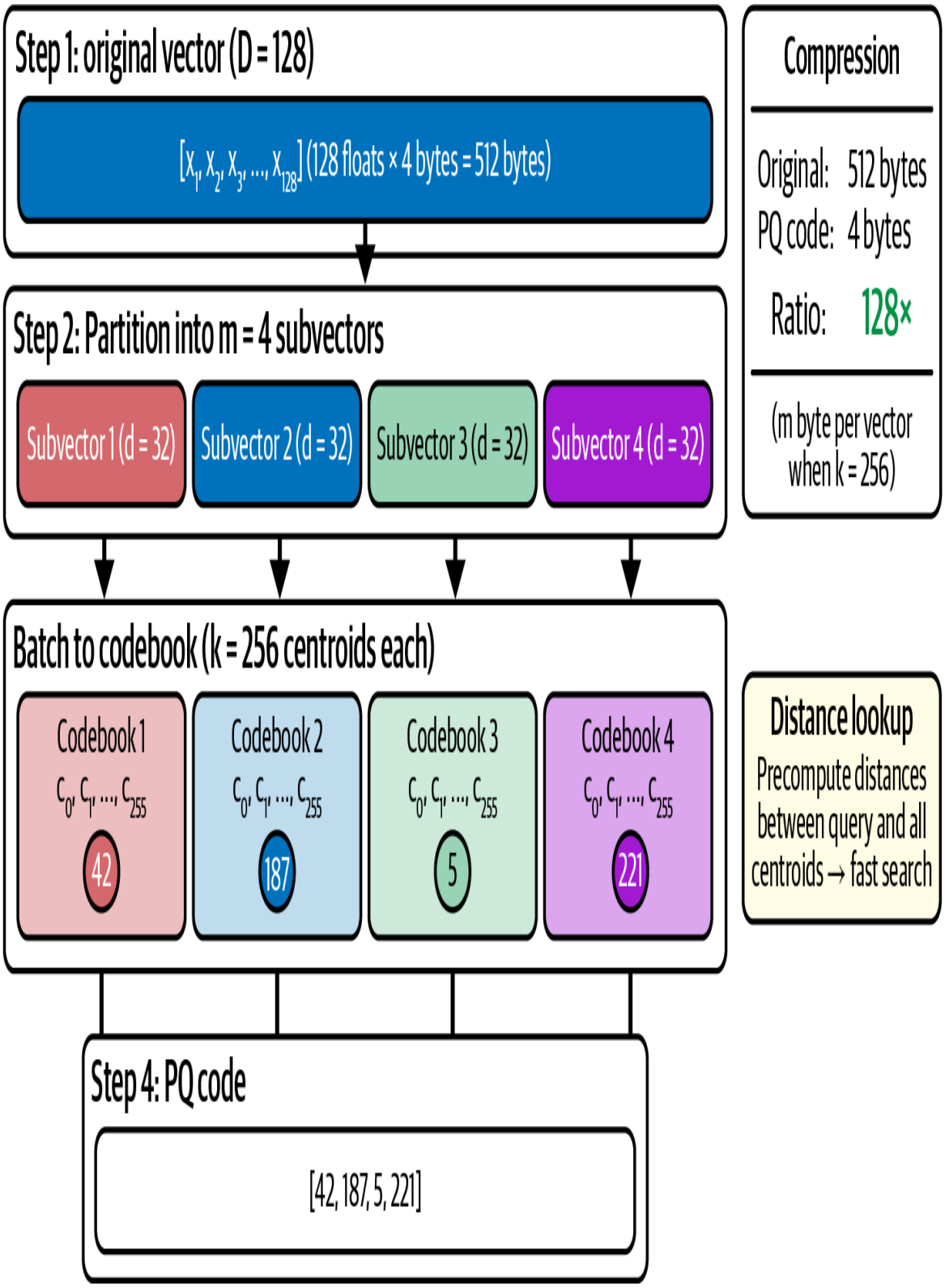
When a query vector arrives, it goes through the same process—it's split into subvectors, and each is mapped to a centroid index in the corresponding codebook.

Distance calculation

To compute an approximate distance between the query and database vectors, the distances are calculated between their compressed representations. This is typically done using a *lookup table* that precomputes distances between all codebook entries—a crucial optimization that avoids recalculating distances among high-dimensional vectors.

ANN search

Because comparisons use quantized vectors, the results are approximate. However, in practice, this method offers an excellent trade-off between speed and accuracy, especially for very large datasets.



PQ decomposes vectors into subspaces, quantizes each independently, achieving massive compression while enabling fast approximate distance computation via lookup tables.

Figure 3-5. PQ process

Details of PQ in FAISS

Let's examine some of the key details of this process:

Highly optimized

FAISS is renowned for its highly optimized C++ implementation, which makes it exceptionally fast. It leverages SIMD instructions and other techniques for maximum efficiency.

Multiple PQ variants

FAISS offers variations of PQ, including:

Optimized product quantization (OPQ)

Introduces a rotation step before PQ to improve accuracy

Inverted file product quantization (IVFPQ)

Combines PQ with an inverted file structure for further speed-ups by filtering out many vectors before performing PQ distance calculations

GPU support

FAISS can leverage GPUs for even faster computation, making it well suited to large-scale applications.

Key advantages of PQ in FAISS

Let's review the main advantages of using PQ in FAISS:

Compression

PQ significantly reduces the memory footprint of the database.

Speed

The compressed representation enables much faster distance calculations.

Scalability

It is capable of handling datasets containing millions or billions of vectors.

Trade-off

It allows fine-tuning of the balance between accuracy and speed according to application requirements.

When is PQ a good choice?

You may wonder when PQ is preferable—particularly in comparison with SQ. Here are the scenarios where PQ shines:

High-dimensional data

Especially effective for vectors with many dimensions

Large datasets

Ideal when indexing and searching enormous collections of vectors

ANN search

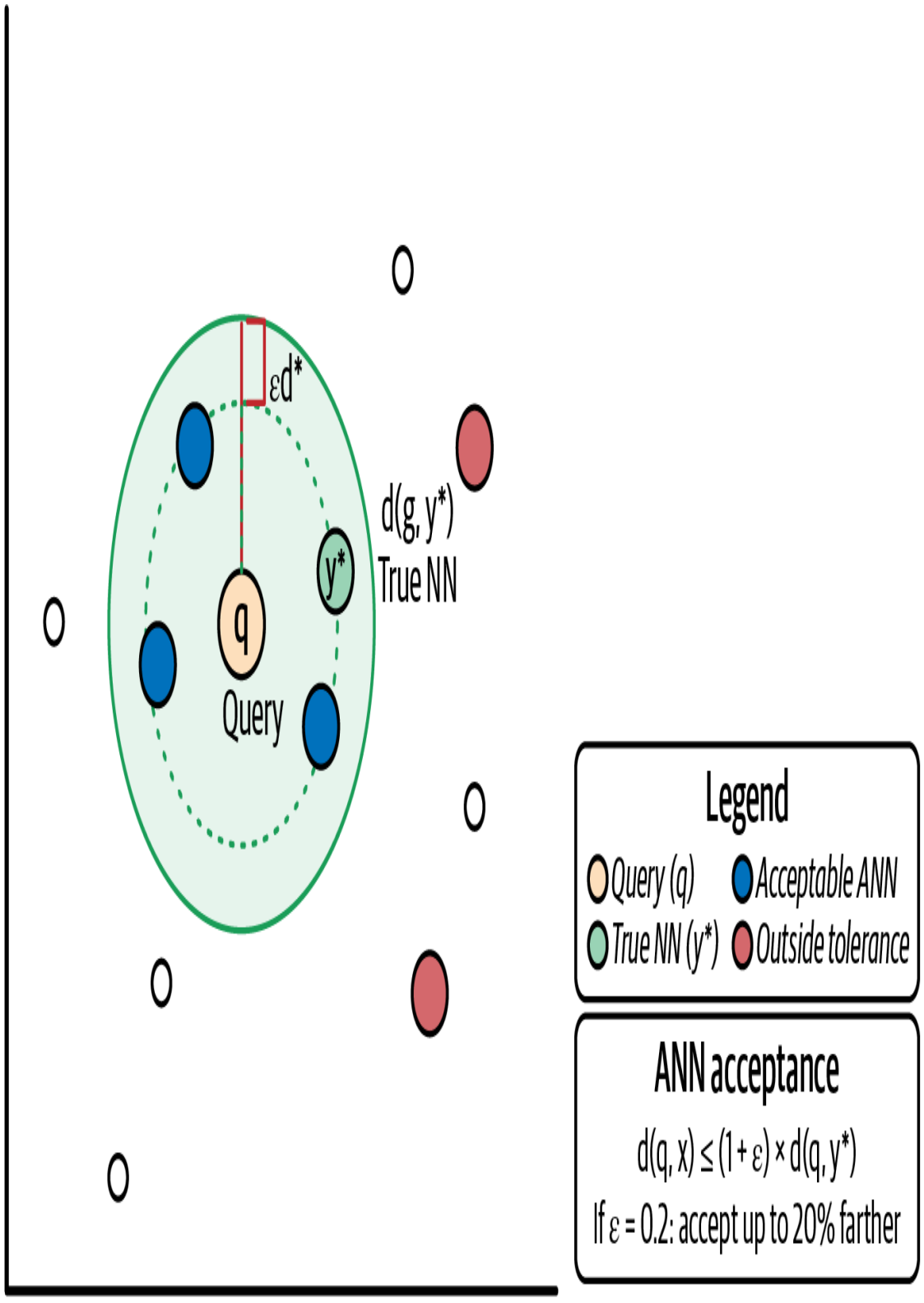
Appropriate when a small loss in accuracy can be traded for a substantial gain in speed

FAISS's PQ method is a powerful technique for making nearest-neighbor search both scalable and efficient. By decomposing vectors, quantizing subspaces, and using lookup tables for distance calculations, it delivers major performance improvements while reducing storage requirements. PQ is a key reason why FAISS has become one of the most widely used libraries for similarity search in AI and machine learning applications.

The ANN Problem

The fundamental problem FAISS addresses is ANN search. In essence, it accepts an approximate answer in exchange for a significant boost in response time. Most of the time, this trade-off is perfectly acceptable—much like settling for a not-quite-perfect version of a favorite sandwich because the *perfect* one is a half-hour drive away. Over the course of thousands or millions of repeated searches, the time savings increases dramatically.

ANN search is a core functionality of FAISS, designed to efficiently identify data points in a large dataset that are most similar to a given query point. Unlike exact nearest-neighbor search, which guarantees the absolute closest matches, ANN instead sacrifices a small amount of accuracy for a significant increase in speed. This trade-off makes ANN practical for massive datasets where exact search would be computationally prohibitive (see Figure 3-6).



ANN search accepts any point within (1+ε) times the true nearest neighbor distance.
The green zone shows acceptable approximation; red points are too far.

Figure 3-6. The ANN ε-tolerance zone

Now let's get into the math a little. Given a query vector q and a dataset X, the goal is to find:

$$ANN(q) = x \in X : d(q, x) \leq (1 + \epsilon) \min_{y \in X} d(q, y)$$

This formula needs a bit of unpacking. Let's break it down:

- q is the query vector—the representation of what we're searching for.
- X is the dataset of vectors, our search domain.
- $d(q, x)$ is the distance between the query vector q and a candidate vector x.
- $\min_{y \epsilon X} d(q, y)$ is the minimum distance—the distance to the *true* nearest neighbor.
- ϵ is the approximation factor, controlling how much error we're willing to accept.

The expression tells us that we'll accept any vector x whose distance from q is at most $(1 + \epsilon)$ times the distance to the true nearest neighbor. For example, if $\epsilon = 2$, we'll accept vectors up to 20% farther than the exact nearest neighbor.

In the sections that follow, we'll break down how ANN search works in FAISS, covering the key ideas and techniques involved.

The Problem

First, let's restate the problem that ANN aims to solve.

Given

- A large dataset of vectors (e.g., images, text embeddings, user profiles)

- A query vector

Goal

- Find the *k* vectors in the dataset that are closest to the query vector according to a chosen distance metric (e.g., Euclidean distance or cosine similarity).

Avoid Computational Cost

Exact nearest-neighbor search for high-dimensional data in large datasets suffers from the *curse of dimensionality*. The computational cost of comparing a query to every vector in the dataset grows linearly with dataset size and exponentially with dimensionality. This makes exact search impractical for many real-world applications.

ANN algorithms address this by:

Building an index

Preprocessing the dataset to create a data structure (the index) that enables efficient search

Approximation

Using techniques that avoid exhaustive comparisons, delivering faster search results while possibly missing a few exact nearest neighbors

Key ANN Techniques in FAISS

FAISS provides a wide range of indexing methods employing different strategies for ANN search. Here are some of the most important ones:

IVF

Clustering

The dataset is partitioned into clusters using algorithms such as k-means. Each cluster is represented by a centroid.

Inverted index

An inverted index maps each centroid to the list of data points assigned to that cluster.

Search

When a query arrives (see Figure 3-7), the following occurs:

1. The `nprobe` closest centroids to the query are identified.
2. Only the data points within those `nprobe` clusters are compared to the query.
3. The *k* closest points from these comparisons are returned as the ANNs.

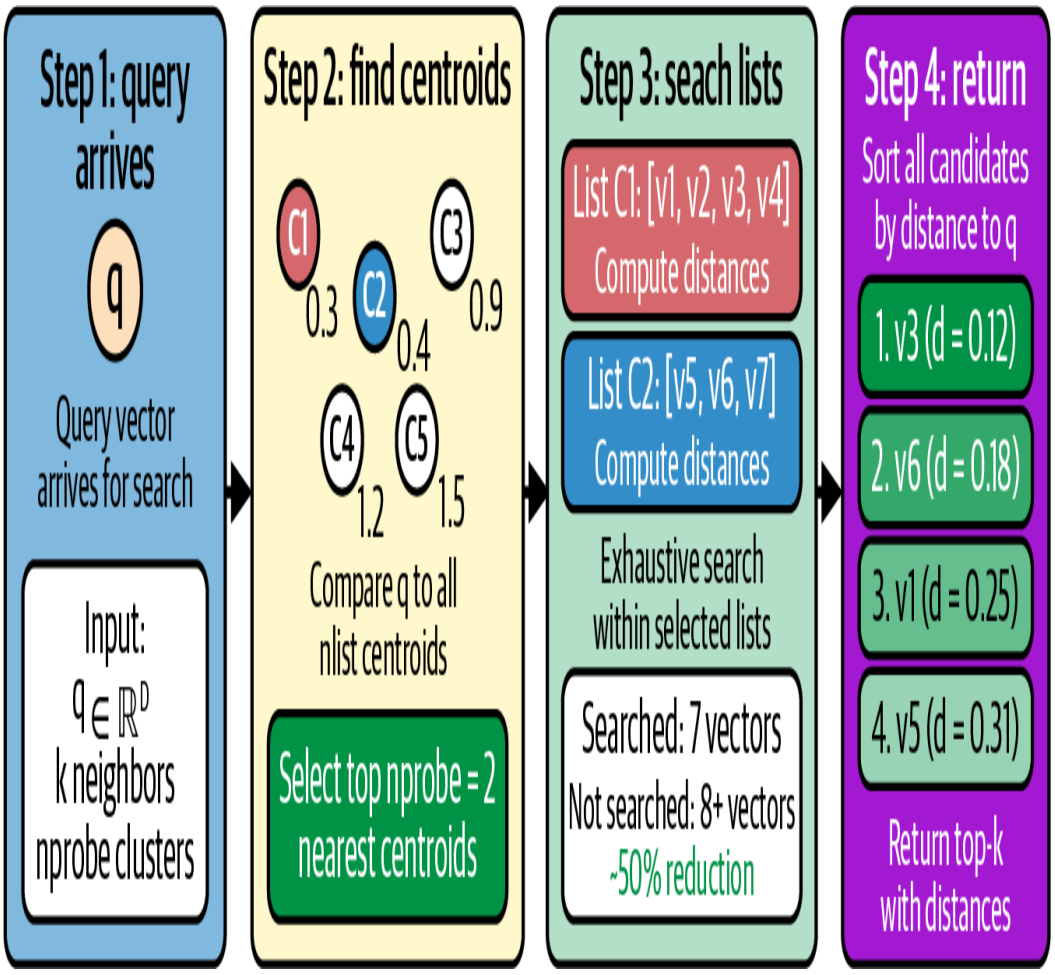


Figure 3-7. IVF search traversal steps

`nprobe` *parameter*

This controls the trade-off between speed and accuracy—larger `nprobe` values mean more clusters are searched, improving accuracy at the cost of speed.

PQ

Compression

Vectors are compressed by dividing them into subvectors and quantizing each subvector independently.

Search

Distance calculations are performed in the compressed space, which is much faster. FAISS uses asymmetric distance computation (ADC)—where the query is not quantized—for better accuracy.

Combination

PQ is often used with IVF (e.g., `IndexIVFPQ`). IVF handles coarse-grained partitioning, while PQ provides fine-grained distance computation within clusters.

See "PQ" for more information.

SQ

Compression

Each component of a vector is quantized independently.

Search

Like PQ, distances are computed in the compressed space.

Combination

SQ is often used with IVF (e.g., `IndexIVFScalarQuantizer`).

See "SQ" for more information.

LSH

Hashing

Vectors are hashed into buckets using specially designed hash functions so that similar vectors are likely to fall into the same bucket.

Search

The query is hashed, and only vectors in the same or nearby buckets are considered as potential nearest neighbors.

Usage

LSH is less common in FAISS than IVF and PQ, especially for high-dimensional data.

HNSW

Graph-based

HNSW builds a multilayered graph in which each layer is a proximity graph of the dataset.

Search

It begins at a high layer with long-range connections and greedily navigates toward the query, refining results at lower layers through shorter connections.

Performance characteristics

It provides excellent speed and accuracy but may require more memory and longer indexing times.

Choosing an Index in FAISS

The best index type in FAISS depends on several factors:

Dataset size and dimensionality

`IndexFlatL2` (exact search) is feasible only for small datasets. IVF-based indexes suit medium to very large datasets, while HNSW is ideal for large datasets when both high accuracy and speed matter.

Memory constraints

PQ and SQ offer effective memory compression.

Accuracy requirements

HNSW typically delivers the highest accuracy, followed by IVF with PQ/SQ.

Search speed requirements

HNSW often provides the fastest search, followed by IVF with PQ/SQ.

Indexing time

`IndexFlatL2` requires no indexing time; IVF-based indexes need moderate indexing time; HNSW may take longer to build.

Code Example

The following example demonstrates `IndexIVFFlat` (an IVF index paired with a flat index) for exhaustive search within clusters. The inline comments make the example largely self-explanatory. For further details—particularly on specific index types—see the FAISS documentation:

```
import faiss
import numpy as np

# Sample data
d = 64  # Dimension of vectors
nb = 100000  # Number of vectors in the dataset
nq = 1000  # Number of query vectors

np.random.seed(1234)

# Use synthetic data with random 32 bit floats
xb = np.random.random((nb, d)).astype('float32')
xq = np.random.random((nq, d)).astype('float32')

# Index parameters
nlist = 100  # Number of clusters
k = 4  # Number of nearest neighbors to retrieve
quantizer = faiss.IndexFlatL2(d)  # The index used for clustering
\
    (can also be an ANN index)
index = faiss.IndexIVFFlat(quantizer, d, nlist)
```

```
# Train the index
index.train(xb)
# Add vectors to the index
index.add(xb)

# Search parameters
index.nprobe = 10  # Number of clusters to search over at query
time \
    (trade-off: higher = more accurate but slower)

# Perform the search
D, I = index.search(xq, k)  # D: distances, I: indices

# Print the results
print(I[:5])  # Indices of the 4 nearest neighbors for the first
5 queries
print(D[:5])  # Distances of the 4 nearest neighbors for the
first 5 queries
```

ANN search in FAISS is a powerful technique for efficiently finding ANNs in large datasets. By employing indexing methods like IVF, PQ, SQ, and HNSW, FAISS achieves fast similarity search with tunable trade-offs between speed and accuracy—ideal for a wide range of real-world applications.

Understanding HNSW Indexes

This section provides a deep dive into one of FAISS's most powerful indexing methods. It explains how HNSW constructs navigable small-world graphs, their computational characteristics, and why they often deliver the best balance between search speed and accuracy.

What Is HNSW?

HNSW is like a multilevel highway system for navigating through your dataset. Imagine you're driving from one city to another:

- Highways connect major cities (top level).

- Major roads connect smaller towns (middle level).
- Local streets connect individual addresses (bottom level).

This is exactly how HNSW organizes your vectors.

The HNSW index is a graph structure that encodes vector similarity into a proximity graph (see Figure 3-8). During index construction, vectors (nodes) that are semantically similar are linked by short distances, while dissimilar vectors are farther apart. The resultant graph represents the semantic clustering of the corpus—showing which concepts are close together and which are not.

Think of it as a semantic highway map: roads correspond to edges in the graph, and cities (of varying sizes) correspond to data points (nodes).

During a search, given a query vector and a random starting point on this "map," the algorithm navigates toward the destination (the closest vector) by traveling across the network—first using broad, long-range connections ("highways") and then refining the route using shorter, local links. In practice, this means the algorithm may not reach the exact nearest neighbor but typically ends up in the right "neighborhood," close enough for most applications.

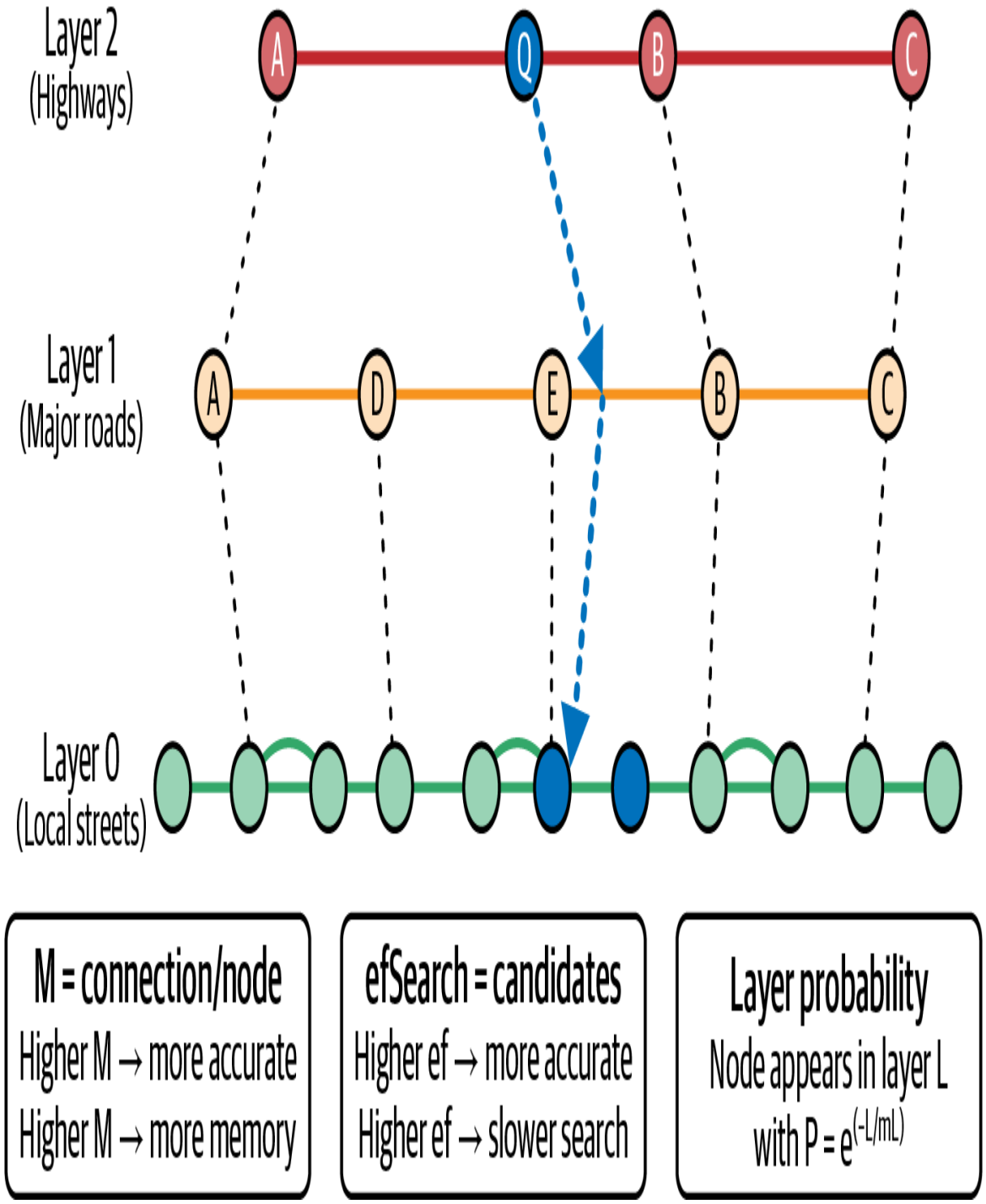


Figure 3-8. HNSW hierarchical graph structure

How HNSW Works

Let’s look more closely at how HNSW operates.

Building the index

In this example, we will generate some sample data (instead of a real corpus) and build a hierarchical index by identifying clusters (these are the “small worlds”) and arranging them hierarchically in the index. The FAISS indexing code automates this process.

The building process works as follows:

1. Each new vector is inserted starting from the top layer.
2. At each layer, it connects to *M* nearest neighbors.
3. The given vector may appear in multiple layers—like a city that appears on both highways and local roads.

```
import numpy as np
import faiss

# Generate sample data
num_vectors = 10000
vectors = np.random.random((num_vectors,
dimension)).astype('float32')


# Create an HNSW index
dimension = 128
M = 16  # Number of connections per layer
ef_construction = 40  # Number of candidates to consider during
construction

index = faiss.IndexHNSWFlat(dimension, M)
index.hnsw.efConstruction = ef_construction

# Add vectors to the index
index.add(vectors)
```

Searching the index

The search process resembles navigation:

1. Start at an entry point in the top layer.
2. Find the best direction to move.

3. Move down a layer when no better horizontal move exists.
4. Repeat until reaching the bottom layer.

```
# Set search parameters
ef_search = 16  # Number of candidates to consider during search
index.hnsw.efSearch = ef_search

# Perform a search
k = 5  # Number of nearest neighbors to find
query = np.random.random((1, dimension)).astype('float32')
distances, indices = index.search(query, k)
```

Key Parameters Explained

Let’s review the key parameters for HNSW in the preceding examples.

M (graph degree parameter)

Think of *M* as the number of roads connecting each city:

- Larger *M* = more connections = better accuracy but higher memory use.
- Smaller *M* = fewer connections = less memory use but potentially less accuracy.

In FAISS, `M` is the graph degree parameter for HNSW: it controls how many neighbor links each vector keeps in the HNSW graph (or more precisely, the maximum number of connections per node in the upper layers). Larger `M` generally improves recall and robustness but increases RAM usage and build cost.

Here are the practical implications:

Accuracy/recall

Higher `M` usually gives better recall at the same `efSearch`.

Memory

Memory grows roughly linearly with `M`. FAISS's own rule of thumb for HNSW RAM is about **(**d·4+M·2·4)(d·4+M·2·4) bytes per vector (plus overhead), showing the linear `M` dependence.

Build time

This increases with `M` because insertion must maintain more links:

```
M = 16  # Default value

Usage: index = faiss.IndexHNSWFlat(dimension, M)
```

efConstruction (construction factor)

`efConstruction` controls how thoroughly the index is built:

- Higher values = better index quality but slower construction.
- Lower values = faster construction but potentially less optimal connections.

```
# Number of candidates to consider during construction
ef_construction = 40
```

efSearch (search factor)

`efSearch` controls how exhaustive the search procedure is:

- Higher values = more accurate results but slower search.
- Lower values = faster searches but possible accuracy trade-offs.

```
# Set search parameters
ef_search = 16  # Number of candidates to consider during search
index.hnsw.efSearch = ef_search
```

Practical Example: Building a Similarity Search System

This example demonstrates using HNSW for a real-world similarity search application:

```
import numpy as np
import faiss
from typing import List, Tuple

class SimilaritySearcher:
    def __init__(self, dimension: int, M: int = 16):
        """Initialize HNSW index with given parameters."""
        self.dimension = dimension
        self.index = faiss.IndexHNSWFlat(dimension, M)

        # Set construction parameters
        self.index.hnsw.efConstruction = 40
        self.index.hnsw.efSearch = 16

        # Store original items for reference
        self.items = []

    def add_items(self, vectors: np.ndarray, items: List[str]):
        """Add vectors and their corresponding items to the
index."""
        assert vectors.shape[1] == self.dimension
        assert vectors.shape[0] == len(items)

        # Add to FAISS index
        self.index.add(vectors.astype('float32'))

        # Store original items
        self.items.extend(items)

def search(self, query_vector: np.ndarray, k: int = 5) ->
List[Tuple[str, \
    float]]:
"""Search for k most similar items."""
      # Ensure query vector has correct shape
      query_vector = query_vector.reshape(1, self.dimension)

      # Perform search
      distances, indices = self.index.search(
            query_vector.astype('float32'), k)

      # Return items and distances
      results = []
      for idx, dist in zip(indices[0], distances[0]):
           if idx != -1:  # FAISS returns -1 for invalid results
                results.append((self.items[idx], dist))

        return results
```

```
# Example usage
dimension = 128
searcher = SimilaritySearcher(dimension)

# Create sample data
num_items = 10000
vectors = np.random.random((num_items, dimension))
items = [f"item_{i}" for i in range(num_items)]

# Add to index
searcher.add_items(vectors, items)

# Perform a search
query = np.random.random(dimension)
results = searcher.search(query, k=5)

# Print results
for item, distance in results:
    print(f"{item}: distance = {distance:.4f}")
```

Performance Characteristics

A simple benchmark comparing HNSW with other index types follows:

```
def benchmark_indexes(dimension=128, num_vectors=100000, k=5):
vectors = np.random.random((num_vectors,
dimension)).astype('float32')

query = np.random.random((1, dimension)).astype('float32')

# Create indexes
     flat_index = faiss.IndexFlatL2(dimension)
     ivf_index = faiss.IndexIVFFlat(faiss.IndexFlatL2(dimension),
dimension, int(np.sqrt(num_vectors)))
     hnsw_index = faiss.IndexHNSWFlat(dimension, 16)

    # Train and add vectors

    flat_index.add(vectors)

    ivf_index.train(vectors)

    ivf_index.add(vectors)

    hnsw_index.add(vectors)
```

```
    # Benchmark search
    results = {}

    for name, index in [
        ('Flat', flat_index),
        ('IVF', ivf_index),
        ('HNSW', hnsw_index)
    ]:
        start = time.time()
        for _ in range(100):
            index.search(query, k)

        results[name] = (time.time() - start) / 100

    return results
```

Best Practices

Now that we've walked through HNSW, let's review some best practices to keep in mind when you're using HNSW.

Parameter selection:

- Start with $M = 16$.
- Increase `efConstruction` for higher-quality indexes.
- Tune `efSearch` to balance accuracy against speed.

Memory considerations:

- HNSW uses more memory than flat indexes.
- Memory usage grows with M and dataset size.
- Use smaller M values for very large datasets.

Performance optimization:

- Batch similar queries together.
- Use GPU acceleration when available.

- Monitor memory consumption during index construction.

FAISS Architecture and Components

In this section, we'll explore FAISS's modular design, including its index hierarchy, training requirements, and search methods. I'll explain how FAISS separates the concerns of data storage, indexing, and search, enabling flexible combinations of different approaches.

Foundation

The modular architecture of FAISS is built around the concept of *composable components* (see Figure 3-9). Each component—indexes, metrics, and storage—is implemented as a separate module that can be combined with others. This modularity provides exceptional flexibility when creating specialized search solutions.

The `Index` base class serves as the foundation for all index types in FAISS. It defines the core interface methods such as `add()`, `search()`, and `train()`. This consistent API makes it easier to interchange index implementations without modifying the surrounding code.

FAISS offers two main ways to combine components: direct programmatic composition and the `IndexFactory` string syntax. The string syntax approach is mainly used in advanced scenarios where custom indexes combine multiple underlying components. We simply note its existence here without exploring it further.

GPU support in FAISS is implemented through dedicated GPU versions of its core components. This design allows seamless switching between CPU and GPU implementations while maintaining the same interface. For our purposes in this book, we use only CPU versions to make the examples accessible to all readers. GPU use in FAISS requires NVIDIA hardware and CUDA, which can introduce additional setup complexity.

FAISS is highly extensible at the source-code level. Its modular design and well-documented API make it straightforward to extend its functionality (see Figure 3-9). Extensibility is achieved through well-defined base classes: developers can create custom components by inheriting from these base classes and implementing the necessary methods.

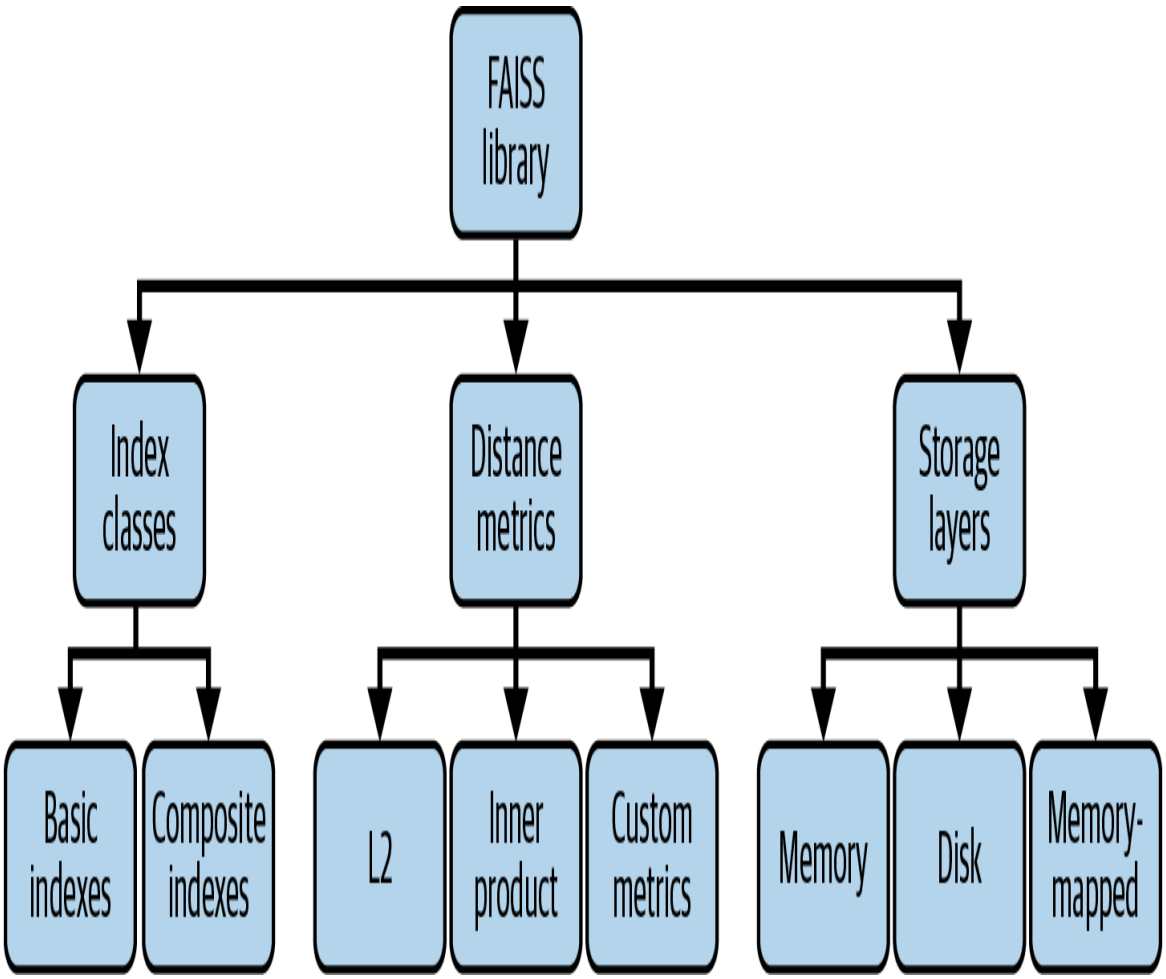


Figure 3-9. Module hierarchy in FAISS code

Developers can:

Implement new index types

> FAISS's architecture supports the creation and integration of novel indexing structures beyond the built-in ones (IVF, HNSW, PQ, etc.), enabling experimentation with state-of-the-art techniques.

Add new distance metrics

While FAISS includes several common metrics (e.g., Euclidean, inner product), developers can define custom metrics to fit specific application requirements. This is particularly important when dealing with specialized data representations or similarity measures.

Integrate with other libraries

FAISS's well-structured interfaces allow easy integration with other machine learning libraries or frameworks for seamless inclusion in existing workflows and pipelines.

Develop custom search strategies

Its search algorithms can be customized or extended—for example, implementing new traversal strategies or optimizing existing ones for particular data distributions or query patterns.

Extend quantization methods

Additional quantization strategies can be introduced to further optimize memory use and search speed for specific data types.

The open source nature of FAISS, combined with its clear codebase and comprehensive documentation, greatly enhances its extensibility. The developer community actively contributes improvements and new features. Extending FAISS, however, requires strong C++ skills and an understanding of its underlying algorithms.

In short, FAISS's modular architecture provides flexibility and extensibility while maintaining performance. Its composable design allows developers to create custom solutions that balance accuracy, speed, and memory usage. Consistent interfaces and GPU support make it feasible to scale from small prototypes to large-scale deployments. Understanding this architecture is key to using FAISS effectively in production environments.

Let's dig deeper into the FAISS architecture and components to better appreciate how it achieves its remarkable efficiency and scalability in

similarity search.

Core Concepts

At its core, FAISS builds indexes for fast ANN searches. It achieves this through three interrelated ideas:

Indexing

: Organizing high-dimensional vectors to enable fast retrieval of similar vectors

Quantization

: Representing vectors in a lower-memory form using techniques such as PQ

Search optimization

: Employing efficient traversal and approximation strategies to find nearest neighbors quickly

Key Components

The first essential set of components of FAISS are its index structures:

`Index` *(abstract base class)*

: The core building block, defining the API for adding vectors and searching for nearest neighbors. Many concrete implementations derive from it.

`IndexFlat`

: The simplest index; it stores vectors directly with no compression or structure. It is used for brute-force searches or as a base for more complex indexes (`IndexFlatL2` for Euclidean distance, `IndexFlatIP` for dot product).

`IndexIVF` *(inverted file index)*

An important index that partitions the vector space into Voronoi cells, maintaining inverted lists of vectors within each cell to speed search.

`IndexPQ`

Implements PQ by dividing vectors into subspaces and quantizing each subspace with its own codebook for compressed search.

`IndexIVFPQ`

Combines `IndexIVF` with `IndexPQ` for both compression and improved speed.

`IndexHNSW`

Uses HNSW graphs; offers high accuracy at higher memory cost.

`IndexBinary`

Similar to `IndexFlat` but designed to work with binary vectors. It includes `IndexBinaryFlat` and `IndexBinaryIVF`.

FAISS also includes specialized indexes such as `IndexLSH` (locality-sensitive hashing) and `IndexIDMap`. In addition, indexes can be chained sequentially using the `IndexPreTransform` class.

Also essential in FAISS are its quantization methods:

`ProductQuantizer`

Central to PQ. It learns subspace codebooks (cluster centroids) and encodes/decodes vectors.

`ScalarQuantizer`

Quantizes each dimension of the vector independently to save space.

`AdditiveQuantizer`

A generalized form of PQ allowing more flexible combinations of component vectors.

Rotation/transforms

Pre-index transformations such as learned rotations (used in OPQ) and PCA for dimensionality reduction.

Distance calculation

Supports multiple metrics including L2 (Euclidean), inner product (dot product), and cosine similarity (for normalized vectors). Distance calculation uses lookup tables in PQ to precompute distances between codebook centroids, enabling rapid computation on quantized vectors.

Search and filtering

Done through `search()`, the core method that takes a query vector and finds ANNs using the indexing and quantization algorithms; and `nprobe`, which is for `IndexIVF` and `IndexIVFPQ` and controls how many inverted file cells are searched (affecting speed versus accuracy). Optional filtering conditions can be applied to limit results.

GPU support

Provides GPU-enabled indexes for significantly faster search speeds. Most CPU indexes have corresponding GPU versions (e.g., `IndexFlatL2GPU`, `IndexIVFPQGPU`).

Preprocessing transforms

Done through `Transform`, a class that enables linear operations such as PCA or normalization before indexing (e.g., `PCAMatrix`, `Normalizer`).

Data management

Populating indexes incrementally with new data; removing vectors from an index; and performing serialization, that is, saving indexes to and

loading them from files for later use.

Common Workflow

Let’s review a common workflow that you would likely use with FAISS:

1. Select the appropriate index based on your data type and search requirements (e.g., `IndexFlat`, `IndexIVFPQ`, `IndexHNSW`).
2. Train the index (if required). Some indexes need training, such as learning codebooks for PQ or centroids for IVF.
3. Insert dataset vectors into the index.
4. Search for ANNs.
5. Evaluate the performance using metrics such as recall and query time.
6. Adjust parameters based on the evaluation.

To pull this together, let’s walk through a simple example of building and querying a FAISS index using some sample generated data. As before, you would replace the sample data with your own.

Illustrative Example

The following example uses the Python API:

```
import faiss
import numpy as np

# Create dummy data
d = 128  # Dimensionality
nb = 10000  # Number of vectors
xq = np.random.rand(10, d).astype('float32') # 10 query vectors
xb = np.random.rand(nb, d).astype('float32') # database

# 1. Choose an index (IVFPQ for example)
nlist = 100  # Number of Voronoi cells (clusters) used to
partition the datasets
m = 8        # Number of subvectors for PQ
```

```
k = 256     # Codebook size

quantizer = faiss.IndexFlatL2(d)  # Base index for IVF
index = faiss.IndexIVFPQ(quantizer, d, nlist, m, k)

# 2. Train the index on a sample of the data
index.train(xb)

# 3. Add vectors to the index
index.add(xb)

# 4. Search for nearest neighbors
D, I = index.search(xq, 5) # Search for 5 nearest neighbors

# Print results
print("Indices of the nearest neighbors:")
print(I)
print("Distances of the nearest neighbors")
print(D)
```

Key Takeaways

Let’s review the key FAISS takeaways:

- FAISS provides a collection of building blocks (indexes, quantizers, etc.) to handle various similarity search scenarios.
- It allows you to compose different index structures and quantization strategies.
- The library is highly optimized for speed and can run on both CPUs and GPUs.
- You can tune various parameters to balance speed and accuracy.
- All FAISS components are designed with a modular and extensible approach in mind.

Further Exploration

Here are some recommendations if you’d like to explore further:

- The official FAISS documentation is essential for in-depth information on all components and features.
- The FAISS GitHub repository includes numerous examples demonstrating index usage and configuration.

Conclusion

FAISS represents a sophisticated solution to the challenging problem of similarity search in high-dimensional spaces.

This chapter guided readers from foundational ideas through advanced implementations, concluding with practical deployment considerations. The chapter offered both theoretical grounding and hands-on knowledge of FAISS, while providing the necessary depth for implementing efficient similarity search systems in prototyping. The focus on both theoretical understanding and practical implementation makes the chapter particularly valuable for self-taught practitioners in the field.

OceanofPDF.com

Chapter 4. Semantic Search with SQLite3

In this chapter, we will build a personal knowledge management system using semantic search over Reddit content. You will learn how to search by meaning rather than by keywords by combining SQLite's familiar SQL interface with vector similarity search.

In Chapter 3, we explored FAISS as a dedicated vector similarity search engine. Here we use sqlite-vss, a SQLite extension that wraps FAISS, integrating vector search into the relational database world. This combination lets us perform semantic retrieval together with relational metadata filtering in a single SQL workflow. In practice, most sqlite-vss queries retrieve nearest-neighbor candidates first and then apply metadata filters as a second step (filters are not pushed down into the FAISS search), so we'll use an "overfetch-then-filter" pattern to reliably return enough results.

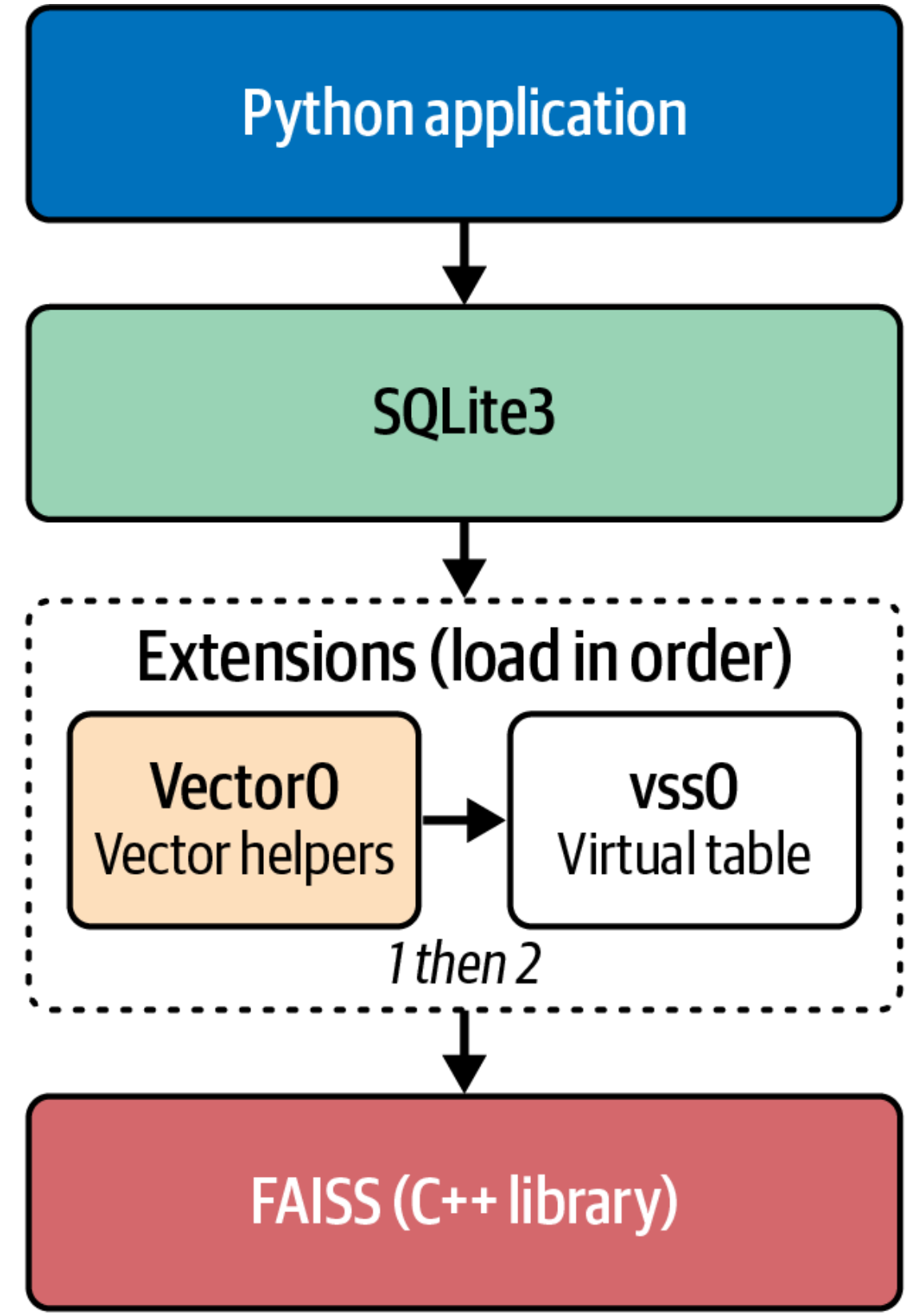
Our goal is practical: imagine that you have saved hundreds of Reddit posts across various topics. With this system, you can query for posts similar in meaning to "deep learning for edge devices," regardless of exact keywords, while filtering by time period or community. By the end of this chapter, you will have a working application demonstrating these capabilities.

Understanding the SQLite Vector Similarity Search Extension

The SQLite Vector Similarity Search (VSS) extension (sqlite-vss) adds vector storage and similarity search to SQLite. It transforms SQLite into a capable vector database suitable for personal-scale applications.

Core Capabilities

The extension consists of two components that work together (see Figure 4-1). The `vector0` extension provides vector data types and helper functions for building vectors from JSON or binary BLOBs. The `vss0` extension provides the virtual table that builds and queries the FAISS-backed similarity index. You must load both extensions, in that order, for the system to work.



vss0 wraps FAISS for nearest-neighbor search

Figure 4-1. Extension stack architecture

Vector storage allows you to store high-dimensional embedding vectors as binary BLOBs in regular SQLite tables. These vectors typically come from embedding models such as `SentenceTransformers`.

Distance functions calculate similarity between vectors. The extension supports L2 (Euclidean) distance, which measures straight-line distance between vectors and is the default metric. Throughout this chapter, we use L2 distance consistently—when I refer to "similarity," I mean vectors that are close in L2 space.

Vector indexing creates specialized index structures for fast ANN search. Without indexing, each query requires comparing against every vector in the database. With proper indexing, search complexity drops dramatically.

Architecture Overview

The sqlite-vss system uses a two-table pattern (see Figure 4-2). Your main content table stores text, metadata, and the raw embedding vector as a BLOB. A separate virtual table, created with the `vss0` module, builds an index over those vectors for fast similarity search. Queries join these tables to combine vector similarity with metadata filtering.

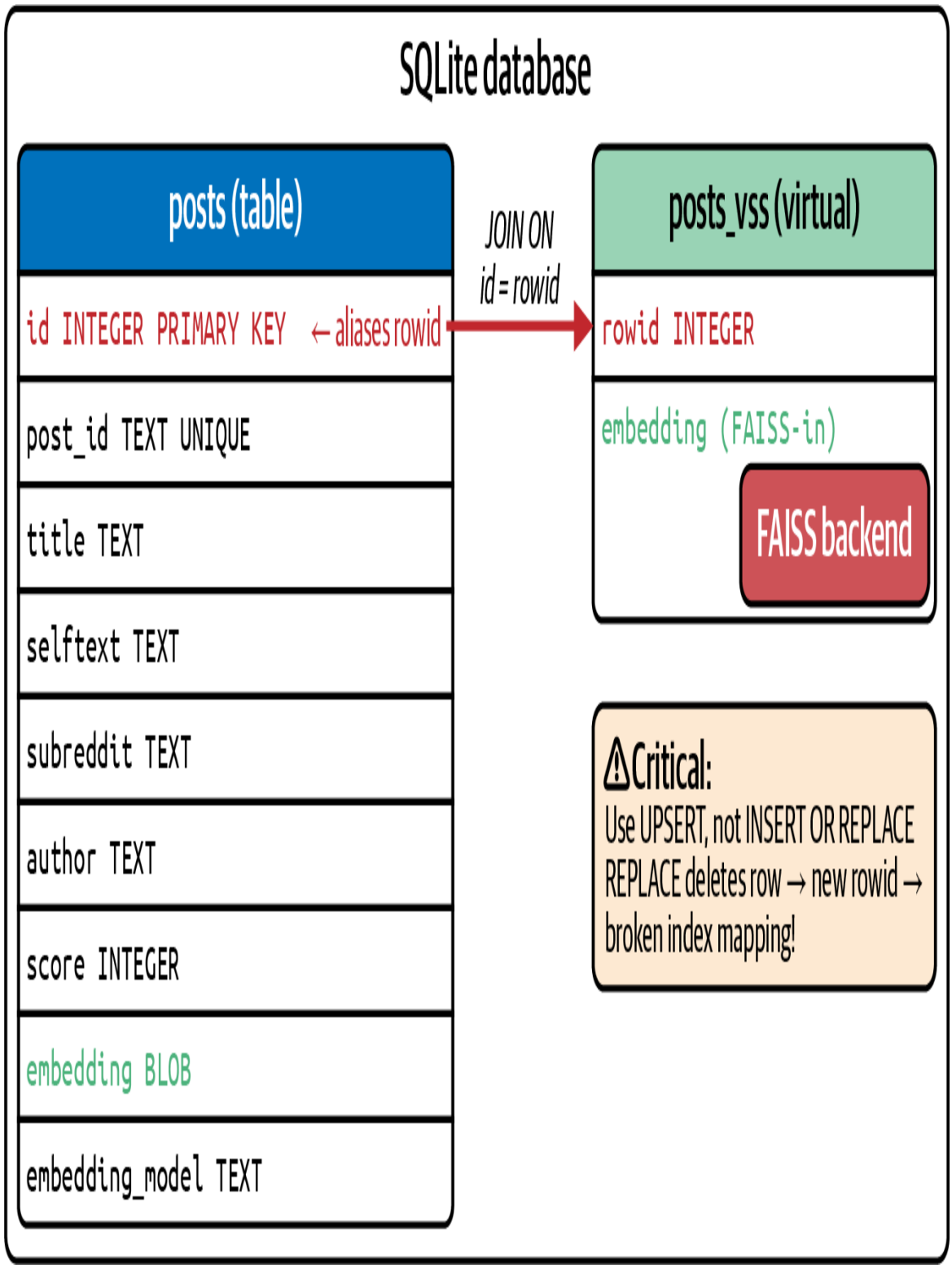


Figure 4-2. Two-table architecture

Critical implementation detail: The VSS virtual table stores vectors keyed by SQLite's implicit `rowid`. This means you must preserve stable `rowid`s when updating data. Using `INSERT OR REPLACE` will break the index because SQLite implements `REPLACE` as `DELETE` followed by `INSERT`, which assigns a new `rowid`. Always use `INSERT ... ON CONFLICT DO UPDATE` (UPSERT) to modify existing rows while preserving their `rowid`s.

Storing the embedding BLOB in the main `posts` table is optional for search—the index contains the vectors—but it makes rebuilding or migrating indexes possible without recomputing embeddings from scratch.

Limitations

The sqlite-vss extension works well for personal-scale datasets (thousands to low millions of vectors) but has constraints worth noting. One important limitation is that metadata predicates are typically not pushed down into the nearest-neighbor search itself. To combine semantic search with filters reliably, you generally fetch more candidates than needed and then filter and rank those candidates in SQL.

Vector operations are memory-intensive, so batch processing helps for large imports. Index files can grow large relative to the raw data. The extension requires runtime loading, which means ensuring version compatibility among SQLite, Python, and the extension binary.

For massive datasets or production deployments requiring high throughput, dedicated vector databases such as Pinecone or Weaviate are more appropriate. For personal knowledge management, sqlite-vss offers an excellent balance of capability and simplicity.

> **NOTE**
>
> The sqlite-vss extension is widely used for embedded or local vector search and serves as a good teaching tool because it wraps FAISS behind SQLite. However, extension ecosystems evolve; if you are starting a greenfield project, review the current state of maintained SQLite vector extensions and choose accordingly.

Setting Up the Development Environment

Before writing code, we need to install dependencies and verify that the sqlite-vss extension works correctly.

Installing Dependencies

Eventually these will end up in a *requirements.txt* file, but it is useful to do this step by hand as it can be tricky depending on your environment:

```
# Create a virtual environment
python -m venv reddit_search_env
source reddit_search_env/bin/activate

# Install Python packages
pip install praw sentence-transformers numpy

# Download sqlite-vss for your platform
# Check https://github.com/asg017/sqlite-vss/releases for latest
version
# Example for Linux x86_64:
wget https://github.com/asg017/sqlite-
vss/releases/download/v0.1.2/ \
    sqlite-vss-v0.1.2-linux-x86_64.tar.gz
tar -xzf sqlite-vss-v0.1.2-linux-x86_64.tar.gz
```

The extraction produces two files: *vector0.so* and *vss0.so* (or *.dylib* on macOS). Both must be loadable by Python's `sqlite3` module. On macOS and Linux, you may need to load the full filename (e.g., *vector0.dylib* or *vector0.so*) depending on the release packaging.

Verifying the Installation

Because extension loading can execute native code, enable it only long enough to load `vector0` and `vss0`, then disable it for the remainder of the session; this is a security best practice:

```
import sqlite3
import json

def verify_vss_installation(extension_path: str = ".") -> bool:
    """
    Verify sqlite-vss extension loads and works correctly.

    Args:
        extension_path: Directory containing vss0 and vector0
extensions

    Returns:
        True if verification succeeds, False otherwise
    """
    conn = sqlite3.connect(":memory:")
    conn.enable_load_extension(True)

    try:
        # Load both required extensions in order
        conn.load_extension(f"{extension_path}/vector0")
        conn.load_extension(f"{extension_path}/vss0")

        # Disable extension loading for security
        conn.enable_load_extension(False)

        # Test vector creation and L2 distance calculation
        # For orthogonal unit vectors, L2 distance = sqrt(2) ≈
1.414
        result = conn.execute("""
            SELECT vss_version(),
                   vss_distance_l2(
                       vector_from_json('[1.0, 0.0, 0.0]'),
                       vector_from_json('[0.0, 1.0, 0.0]')
                   )
        """).fetchone()

        version, distance = result
        expected_distance = 1.414  # sqrt(2) for orthogonal unit
vectors
```

```
            print(f"sqlite-vss version: {version}")
            print(f"L2 distance test: {distance:.3f} (expected \
                ~{expected_distance:.3f})")

            conn.close()
            return abs(distance - expected_distance) < 0.01

        except Exception as e:
            print(f"Verification failed: {e}")
            conn.close()
            return False

if __name__ == "__main__":
    if verify_vss_installation():
        print("Installation verified successfully.")
    else:
        print("Installation verification failed.")
```

If this script prints the version and correct distance, your environment is ready.

Operational Pragmas

For ingestion workloads, SQLite benefits from several configuration pragmas. We will set these when creating database connections:

```
def configure_connection(conn: sqlite3.Connection) -> None:
    """Apply recommended pragmas for performance and
correctness."""
    # Better concurrent read performance
    conn.execute("PRAGMA journal_mode=WAL;")
    # Faster writes, still crash-safe with WAL
    conn.execute("PRAGMA synchronous=NORMAL;")
    # Keep temp tables in memory
    conn.execute("PRAGMA temp_store=MEMORY;")
    # Enforce foreign key constraints
    conn.execute("PRAGMA foreign_keys=ON;")
```

Designing the Database Schema

A well-designed schema separates content storage from vector indexing while supporting efficient queries that combine both.

Schema Requirements

Our Reddit knowledge base needs to store post content and metadata for filtering, embedding vectors for similarity search, and supporting efficient joins between content and vector index tables. The schema should also track which embedding model generated each vector, since switching models invalidates existing embeddings.

Table Definitions

Let’s examine the table definitions:

```
import sqlite3
from typing import Optional

def create_database(db_path: str, extension_path: str = ".") -> \
    sqlite3.Connection:
    """
    Create the Reddit knowledge base database with proper schema.

    Args:
        db_path: Path to the SQLite database file
        extension_path: Directory containing vss0 and vector0
extensions

    Returns:
        Database connection with extensions loaded
    """
    conn = sqlite3.connect(db_path)
    conn.enable_load_extension(True)
    conn.load_extension(f"{extension_path}/vector0")
    conn.load_extension(f"{extension_path}/vss0")
    conn.enable_load_extension(False)  # Disable for security

    # Apply operational pragmas
    # Note: foreign_keys=ON is required for FK constraints to be
enforced
    conn.execute("PRAGMA journal_mode=WAL;")
    conn.execute("PRAGMA synchronous=NORMAL;")
    conn.execute("PRAGMA temp_store=MEMORY;")
    conn.execute("PRAGMA foreign_keys=ON;")

    # Main posts table with content and metadata
    # We use INTEGER PRIMARY KEY which is an alias for rowid,
```

```
    # giving us a stable identifier for VSS index joins
    conn.execute("""
        CREATE TABLE IF NOT EXISTS posts (
            id INTEGER PRIMARY KEY,
            post_id TEXT UNIQUE NOT NULL,
            title TEXT NOT NULL,
            selftext TEXT,
            url TEXT,
            subreddit TEXT NOT NULL,
            author TEXT,
            score INTEGER DEFAULT 0,
            num_comments INTEGER DEFAULT 0,
            created_utc INTEGER NOT NULL,
            embedding BLOB,
            embedding_model TEXT,
            created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
        )
    """)

    # Indexes for common filter operations
    conn.execute("CREATE INDEX IF NOT EXISTS idx_posts_subreddit
ON \
        posts(subreddit)")
    conn.execute("CREATE INDEX IF NOT EXISTS idx_posts_created ON
\
        posts(created_utc)")
    conn.execute("CREATE INDEX IF NOT EXISTS idx_posts_score ON
posts(score)")
    conn.execute("CREATE INDEX IF NOT EXISTS idx_posts_post_id ON
\
        posts(post_id)")

    conn.commit()
    return conn


def create_vector_index(conn: sqlite3.Connection, dimension: int)
-> None:
    """
    Create the VSS virtual table for vector similarity search.

    The virtual table uses FAISS under the hood. By default, it
creates
    a "Flat" index which performs exact (brute-force) search. For
larger
    datasets, you can specify approximate index types via factory
strings
    like "IVF100,Flat" or "HNSW16".
```

```
    Args:
        conn: Database connection with vss extension loaded
        dimension: Embedding vector dimension (e.g., 384 for all-
MiniLM-L6-v2)
    """
    # Drop existing index if present (required when changing
dimensions)
    conn.execute("DROP TABLE IF EXISTS posts_vss")

    # Create VSS virtual table with Flat (exact) index
    # The index stores vectors keyed by rowid for joining with
posts table
    conn.execute(f"""
        CREATE VIRTUAL TABLE posts_vss USING vss0(
            embedding({dimension})
        )
    """)

    conn.commit()
    print(f"Created vector index with dimension {dimension}")
```

Schema Design Decisions

We use `INTEGER PRIMARY KEY` for the `id` column, which in SQLite is an alias for the implicit `rowid`. This gives us a stable, explicit identifier that we control. The VSS virtual table indexes vectors by this `rowid`, so preserving it during updates is critical.

The `embedding` column stores the raw vector as a binary BLOB. This is redundant with the VSS index for search purposes but enables rebuilding the index without regenerating embeddings—useful when migrating to a new index type or recovering from corruption.

The `embedding_model` column tracks which model generated each embedding. If you later switch from all-MiniLM-L6-v2 to a different model, you can identify and regenerate affected rows rather than starting over.

Metadata indexes on `subreddit`, `created_utc`, and `score` enable fast filtering. SQLite's query planner can use these indexes to reduce the candidate set before or after vector search, depending on query structure.

Connecting to Reddit with the Python Reddit API Wrapper

The Python Reddit API Wrapper (PRAW) provides a clean interface to Reddit's API. We use read-only authentication, which requires minimal setup.

Creating Reddit API Credentials

To use the Reddit API, you need credentials from a registered application:

1. Log in to Reddit and visit *https://www.reddit.com/prefs/apps*.
2. Click "Create another app" at the bottom of the screen.
3. Select Script as the application type.
4. Fill in a name and description (the redirect URI can be *http://localhost*).
5. After creation, note your `client_id` (below the app name) and `client_secret`.

Your user agent string must follow Reddit's format to avoid rate limiting:

<platform>:<app_id>:<version> (by /u/<username>)

For example: `python:reddit_knowledge_base:v1.0 (by /u/your_username)`

Reddit rate limits and API rules can change; treat the rate limiter and exponential backoff as mandatory and consider caching locally to avoid repeated downloads of the same content.

PRAW Client Implementation

The following code will implement the PRAW client:

```
import praw
import time
```

```
from typing import Generator, Optional, Dict, Any
from prawcore.exceptions import PrawcoreException

class RedditClient:
    """Client for fetching Reddit content with rate limiting and
error \
        handling."""

    def __init__(
        self,
        client_id: str,
        client_secret: str,
        user_agent: str
    ):
        """
        Initialize Reddit client in read-only mode.

        Args:
            client_id: Reddit API client ID
            client_secret: Reddit API client secret
            user_agent: User agent string following Reddit's
format
        """
        self.reddit = praw.Reddit(
            client_id=client_id,
            client_secret=client_secret,
            user_agent=user_agent
        )
        # Verify we're in read-only mode
        assert self.reddit.read_only, "Client should be read-
only"

        self.request_delay = 1.0  # Seconds between requests
        self.last_request_time = 0.0

    def _rate_limit(self) -> None:
        """Enforce rate limiting between requests."""
        elapsed = time.time() - self.last_request_time
        if elapsed < self.request_delay:
            time.sleep(self.request_delay - elapsed)
        self.last_request_time = time.time()

    def _retry_with_backoff(
        self,
        func,
        max_retries: int = 3
    ) -> Any:
        """
```

```
        Execute function with exponential backoff on rate limit
errors.

        Args:
            func: Callable to execute
            max_retries: Maximum retry attempts

        Returns:
            Function result

        Raises:
            PrawcoreException: If all retries fail
        """
        for attempt in range(max_retries):
            try:
                self._rate_limit()
                return func()
            except PrawcoreException as e:
                if "429" in str(e) or "Too Many Requests" in
str(e):
                    wait_time = min(2 ** attempt * 10, 300)
                    print(f"Rate limited, waiting {wait_time}s
(attempt \
                        {attempt + 1})")
                    time.sleep(wait_time)
                    self.request_delay *= 1.5  # Slow down future
requests
                else:
                    raise
        raise PrawcoreException("Max retries exceeded")

    def get_subreddit_posts(
        self,
        subreddit_name: str,
        sort: str = "hot",
        limit: int = 100,
        time_filter: str = "all"
    ) -> Generator[Dict[str, Any], None, None]:
        """
        Fetch posts from a subreddit.

        Args:
            subreddit_name: Name of subreddit (without r/)
            sort: Sort method (hot, new, top, rising)
            limit: Maximum posts to fetch
            time_filter: Time filter for 'top' sort (hour, day,
week, month, \
                year, all)
```

```
        Yields:
            Dictionary containing post data
        """
        subreddit = self.reddit.subreddit(subreddit_name)

        if sort == "hot":
            posts = subreddit.hot(limit=limit)
        elif sort == "new":
            posts = subreddit.new(limit=limit)
        elif sort == "top":
            posts = subreddit.top(time_filter=time_filter,
limit=limit)
        elif sort == "rising":
            posts = subreddit.rising(limit=limit)
        else:
            raise ValueError(f"Unknown sort method: {sort}")

        def fetch_batch():
            return list(posts)

        fetched_posts = self._retry_with_backoff(fetch_batch)

        for submission in fetched_posts:
            yield {
                "post_id": submission.id,
                "title": submission.title,
                "selftext": submission.selftext,
                "url": submission.url,
                "subreddit": submission.subreddit.display_name,
                "author": str(submission.author) if
submission.author else \
                    "[deleted]",
                "score": submission.score,
                "num_comments": submission.num_comments,
                "created_utc": int(submission.created_utc),
                "is_self": submission.is_self
            }
```

Usage Example

Let’s look at a usage example:

```
# Initialize client
client = RedditClient(
    client_id="YOUR_CLIENT_ID",
```

```
    client_secret="YOUR_CLIENT_SECRET",
    user_agent="python:reddit_knowledge_base:v1.0 (by
/u/your_username)"
)

# Fetch top posts from a subreddit
for post in client.get_subreddit_posts("MachineLearning",
sort="top", limit=50):
    print(f"{post['score']:>5} | {post['title'][:60]}")
```

Content Extraction and Preprocessing

Raw Reddit content contains markdown formatting, URLs, and special characters that should be cleaned before embedding. Consistent preprocessing improves embedding quality.

Text Cleaning Pipeline

Let's walk through the text preprocessing pipeline as shown in Figure 4-3:

```
import re
from typing import Optional

class TextPreprocessor:
    """Clean and normalize Reddit text content for embedding."""

    def __init__(self):
        # Compile regex patterns once for efficiency
        self.url_pattern = re.compile(
            r'https?://[^\s<>"{}|\\^`\[\]]+'
        )
        self.reddit_link_pattern = re.compile(
            r'(?<!\w)/r/\w+|(?<!\w)/u/\w+'
        )
        self.markdown_link_pattern = re.compile(
            r'\[([^\]]+)\]\([^)]+\)'
        )
        self.whitespace_pattern = re.compile(r'\s+')

    def clean_markdown(self, text: str) -> str:
        """
        Remove or simplify markdown formatting.
```

```
        This is a lightweight best-effort cleaner using regex.
For perfect
        markdown handling you would use a proper parser, but that
adds
        complexity beyond the scope of this chapter.

        Args:
            text: Raw markdown text

        Returns:
            Cleaned text
        """
        # Replace markdown links with just the link text
        text = self.markdown_link_pattern.sub(r'\1', text)

        # Remove common markdown formatting
        # Bold
        text = re.sub(r'\*\*([^*]+)\*\*', r'\1', text)
        # Italic
        text = re.sub(r'\*([^*]+)\*', r'\1', text)
        # Strikethrough
        text = re.sub(r'~~([^~]+)~~', r'\1', text)
        # Inline code
        text = re.sub(r'`([^`]+)`', r'\1', text)
        # Headers
        text = re.sub(r'^#+\s*', '', text, flags=re.MULTILINE)
        # Quotes/lists
        text = re.sub(r'^[>\-*]\s*', '', text,
flags=re.MULTILINE)

        return text

    def handle_urls(self, text: str) -> str:
        """
        Replace URLs with a placeholder token.

        We use [URL] rather than removing URLs entirely because
the presence
        of a link is often semantically meaningful (e.g., "here's
a tutorial"
        vs just text). The placeholder preserves this signal
without letting
        long, unique URLs dominate the embedding.
        """
        return self.url_pattern.sub('[URL]', text)

    def clean_reddit_artifacts(self, text: str) -> str:
        """Remove Reddit-specific artifacts."""
```

```
        # Remove deleted/removed markers
        text = re.sub(r'\[removed\]|\[deleted\]', '', text)

        # Remove subreddit and user references (these add noise
for embeddings)
        text = self.reddit_link_pattern.sub('', text)

        # Decode HTML entities
        text = text.replace('&amp;', '&')
        text = text.replace('&lt;', '<')
        text = text.replace('&gt;', '>')
        text = text.replace(' ', ' ')

        return text

    def normalize_whitespace(self, text: str) -> str:
        """Collapse multiple whitespace to single spaces."""
        text = self.whitespace_pattern.sub(' ', text)
        return text.strip()

    def process(self, text: Optional[str]) -> str:
        """
        Apply full preprocessing pipeline.

        Args:
            text: Raw text content

        Returns:
            Cleaned and normalized text
        """
        if not text:
            return ""

        text = self.clean_markdown(text)
        text = self.handle_urls(text)
        text = self.clean_reddit_artifacts(text)
        text = self.normalize_whitespace(text)

        return text


class ContentPreparer:
    """Prepare Reddit content for embedding generation."""

    def __init__(self):
        self.preprocessor = TextPreprocessor()

    def prepare_post(self, post: dict) -> str:
```

```
        """
        Combine and clean post title and body for embedding.

        Args:
            post: Post dictionary from RedditClient

        Returns:
            Cleaned text suitable for embedding
        """
        title = self.preprocessor.process(post.get("title", ""))
        selftext = self.preprocessor.process(post.get("selftext",
""))

        # Combine title and body; title appears first for
emphasis
        if selftext:
            return f"{title}. {selftext}"
        return title

    def is_quality_content(
        self,
        text: str,
        min_length: int = 20,
        min_words: int = 3
    ) -> bool:
        """
        Check if content meets quality thresholds.

        Args:
            text: Preprocessed text
            min_length: Minimum character count
            min_words: Minimum word count

        Returns:
            True if content passes quality checks
        """
        if len(text) < min_length:
            return False
        if len(text.split()) < min_words:
            return False
        if text in ["[URL]", "", " "]:
            return False
        return True
```

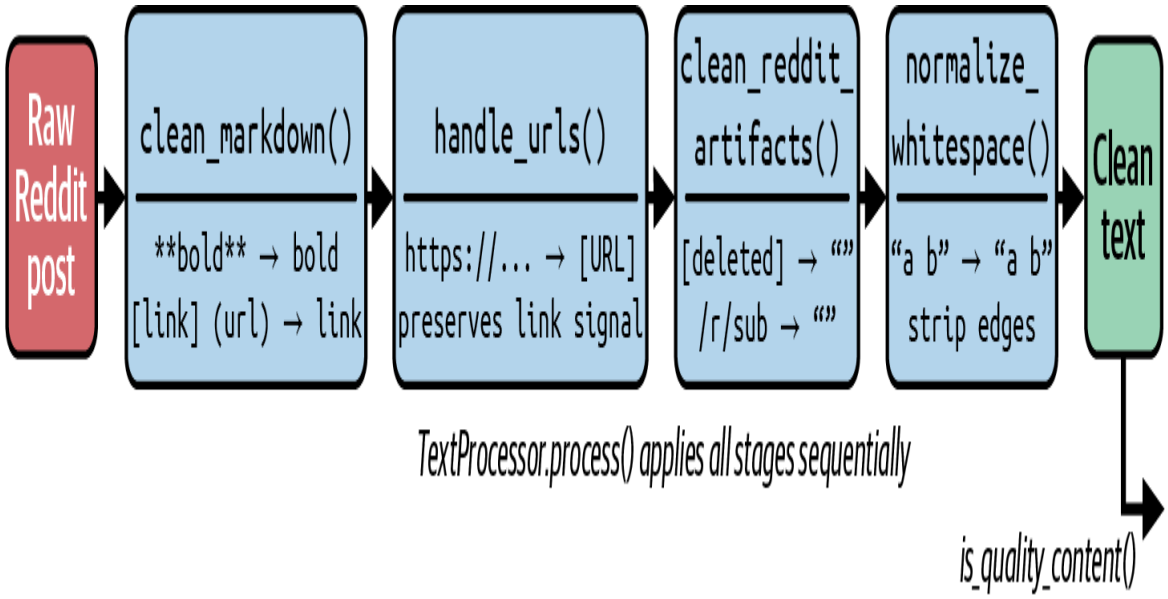


Figure 4-3. Text preprocessing pipeline

Quality Filtering

Not all Reddit content is worth storing. Very short posts, deleted content, and low-effort submissions add noise without value. The `is_quality_content` method provides basic filtering. You might extend this with score thresholds or content-type filters depending on your use case.

Generating and Storing Embeddings

With clean text prepared, we generate vector embeddings using `SentenceTransformers` and store them in SQLite as shown in Figure 4-4.

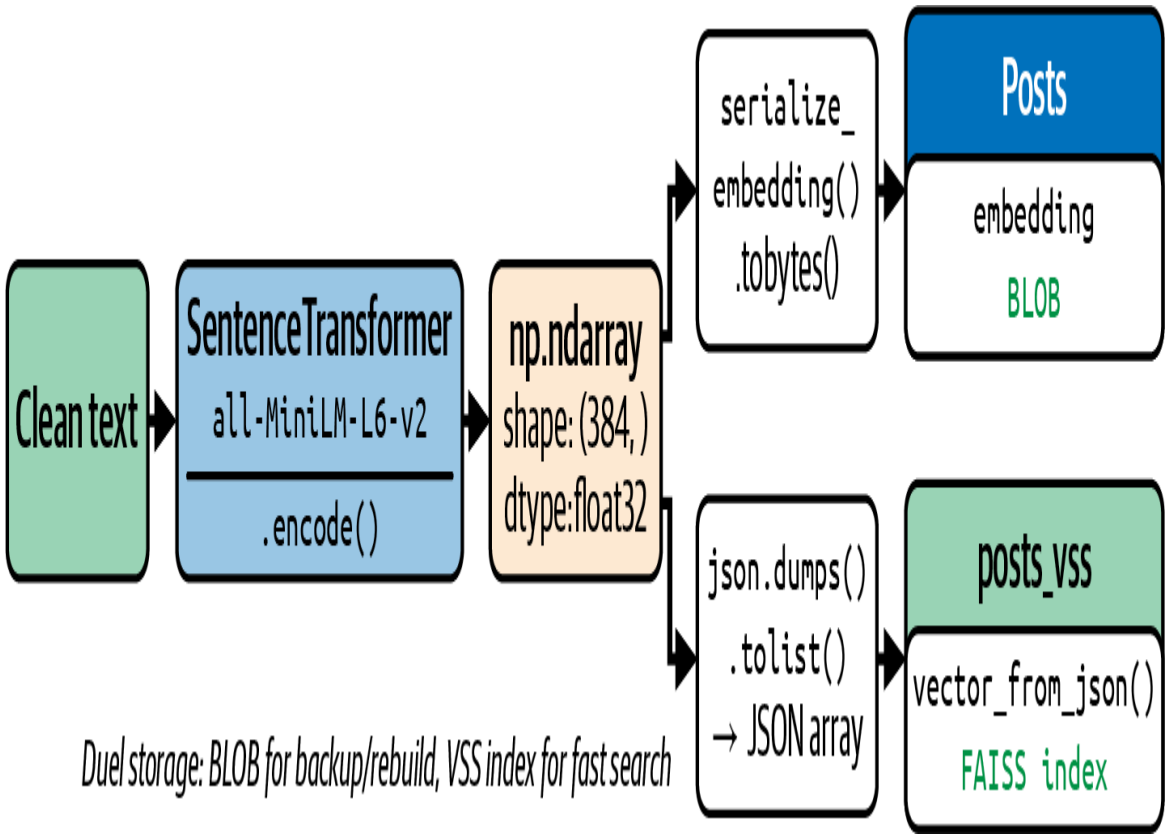


Figure 4-4. Embedding and storage flow

Embedding Generator

Let’s walk through the embedding generator:

```
import numpy as np
from sentence_transformers import SentenceTransformer
from typing import List, Union

class EmbeddingGenerator:
    """Generate embeddings using SentenceTransformers models."""

    def __init__(self, model_name: str = "all-MiniLM-L6-v2"):
        """
        Initialize embedding generator.

        For reproducibility, pin the SentenceTransformers model
version
        (and ideally transformers/torch versions) so embeddings
remain
        stable across different installations.
```

```
        Args:
            model_name: SentenceTransformers model identifier
        """
        self.model_name = model_name
        self.model = SentenceTransformer(model_name)
        self.dimension =
self.model.get_sentence_embedding_dimension()

        # Determine best available device
        self.device = self._select_device()
        self.model.to(self.device)

        print(f"Loaded {model_name} (dim={self.dimension}) on
{self.device}")

    def _select_device(self) -> str:
        """Select the best available compute device."""
        import torch

        if torch.cuda.is_available():
            return "cuda"
        elif hasattr(torch.backends, "mps") and \
            torch.backends.mps.is_available():
            return "mps"
        else:
            return "cpu"

    def encode(
        self,
        texts: Union[str, List[str]],
        batch_size: int = 32,
        show_progress: bool = False
    ) -> np.ndarray:
        """
        Generate embeddings for text(s).

        Args:
            texts: Single text or list of texts
            batch_size: Batch size for encoding
            show_progress: Show progress bar

        Returns:
            Numpy array of embeddings, shape (n_texts, dimension)
        """
        if isinstance(texts, str):
            texts = [texts]

        embeddings = self.model.encode(
```

```
            texts,
            batch_size=batch_size,
            show_progress_bar=show_progress,
            convert_to_numpy=True
        )

        return embeddings

    @staticmethod
    def serialize_embedding(embedding: np.ndarray) -> bytes:
        """
        Convert numpy embedding to bytes for SQLite storage.

        Args:
            embedding: 1D numpy array

        Returns:
            Bytes representation
        """
        return embedding.astype(np.float32).tobytes()

    @staticmethod
    def deserialize_embedding(blob: bytes, dimension: int) ->
np.ndarray:
        """
        Convert bytes back to numpy embedding.

        Args:
            blob: Bytes from SQLite
            dimension: Expected vector dimension

        Returns:
            Numpy array
        """
        return np.frombuffer(blob,
dtype=np.float32).reshape(dimension)
```

Database Storage

The storage class uses UPSERT (`INSERT ... ON CONFLICT DO UPDATE`) rather than `INSERT OR REPLACE`. This is critical: `REPLACE` deletes and reinserts the row, assigning a new `rowid` and breaking the VSS index mapping. UPSERT modifies the existing row in place, preserving its `rowid`:

```
import sqlite3
import json
from typing import List, Dict, Any, Optional
import numpy as np

class KnowledgeBaseStorage:
    """Handle storage and retrieval of Reddit content with
embeddings."""

    def __init__(
        self,
        db_path: str,
        extension_path: str = ".",
        embedding_dim: int = 384
    ):
        """
        Initialize storage connection.

        Args:
            db_path: Path to SQLite database
            extension_path: Directory containing vss extensions
            embedding_dim: Dimension of embedding vectors
        """
        self.db_path = db_path
        self.extension_path = extension_path
        self.embedding_dim = embedding_dim
        self.conn = self._connect()

    def _connect(self) -> sqlite3.Connection:
        """Establish database connection with extensions and
pragmas."""
        conn = sqlite3.connect(self.db_path)
        conn.enable_load_extension(True)
        conn.load_extension(f"{self.extension_path}/vector0")
        conn.load_extension(f"{self.extension_path}/vss0")
        conn.enable_load_extension(False)  # Disable for security

        # Apply operational pragmas
        conn.execute("PRAGMA journal_mode=WAL;")
        conn.execute("PRAGMA synchronous=NORMAL;")
        conn.execute("PRAGMA temp_store=MEMORY;")
        conn.execute("PRAGMA foreign_keys=ON;")

        conn.row_factory = sqlite3.Row
        return conn

    def insert_post(
        self,
```

```
        post: Dict[str, Any],
        embedding: Optional[np.ndarray],
        model_name: str
    ) -> int:
        """
        Insert or update a post with its embedding.

        Uses UPSERT to preserve rowid on conflict, which is
essential
        for maintaining VSS index consistency.

        Args:
            post: Post data dictionary
            embedding: Embedding vector (or None to skip)
            model_name: Name of embedding model used

        Returns:
            Row ID of inserted/updated post
        """
        embedding_blob = None
        if embedding is not None:
            embedding_blob =
EmbeddingGenerator.serialize_embedding(embedding)
        try:
            cursor = self.conn.execute("""
        INSERT INTO posts
        (post_id, title, selftext, url, subreddit, author,
         score, num_comments, created_utc, embedding,
embedding_model)
        VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?)
        ON CONFLICT(post_id) DO UPDATE SET
            title = excluded.title,
            selftext = excluded.selftext,
            url = excluded.url,
            subreddit = excluded.subreddit,
            author = excluded.author,
            score = excluded.score,
            num_comments = excluded.num_comments,
            created_utc = excluded.created_utc,
            embedding = excluded.embedding,
            embedding_model = excluded.embedding_model
        RETURNING id
    """, (
        post["post_id"],
        post["title"],
        post.get("selftext", ""),
        post.get("url", ""),
        post["subreddit"],
```

```
            post.get("author", "[unknown]"),
            post.get("score", 0),
            post.get("num_comments", 0),
            post["created_utc"],
            embedding_blob,
            model_name
        ))
        row_id = cursor.fetchone()[0]
    except sqlite3.OperationalError:
        # Older SQLite: no RETURNING support
        self.conn.execute("""
            INSERT INTO posts
            (post_id, title, selftext, url, subreddit, author,
             score, num_comments, created_utc, embedding,
    embedding_model)
            VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?)
            ON CONFLICT(post_id) DO UPDATE SET
                title = excluded.title,
                selftext = excluded.selftext,
                url = excluded.url,
                subreddit = excluded.subreddit,
                author = excluded.author,
                score = excluded.score,
                num_comments = excluded.num_comments,
                created_utc = excluded.created_utc,
                embedding = excluded.embedding,
                embedding_model = excluded.embedding_model
        """, (...same params...))
        row_id = self.conn.execute(
            "SELECT id FROM posts WHERE post_id = ?",
            (post["post_id"],)
        ).fetchone()[0]

    self.conn.commit()
    return row_id


    def insert_posts_batch(
        self,
        posts: List[Dict[str, Any]],
        embeddings: np.ndarray,
        model_name: str
    ) -> int:
        """
        Insert multiple posts with embeddings efficiently.

        Args:
            posts: List of post dictionaries
```

```
            embeddings: Array of embeddings, shape (n_posts, dim)
            model_name: Embedding model name

        Returns:
            Number of posts inserted
        """
        data = []
        for post, embedding in zip(posts, embeddings):
            embedding_blob =
EmbeddingGenerator.serialize_embedding(embedding)
            data.append((
                post["post_id"],
                post["title"],
                post.get("selftext", ""),
                post.get("url", ""),
                post["subreddit"],
                post.get("author", "[unknown]"),
                post.get("score", 0),
                post.get("num_comments", 0),
                post["created_utc"],
                embedding_blob,
                model_name
            ))

        self.conn.executemany("""
            INSERT INTO posts
            (post_id, title, selftext, url, subreddit, author,
             score, num_comments, created_utc, embedding,
embedding_model)
            VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?)
            ON CONFLICT(post_id) DO UPDATE SET
                title = excluded.title,
                selftext = excluded.selftext,
                url = excluded.url,
                subreddit = excluded.subreddit,
                author = excluded.author,
                score = excluded.score,
                num_comments = excluded.num_comments,
                created_utc = excluded.created_utc,
                embedding = excluded.embedding,
                embedding_model = excluded.embedding_model
        """, data)

        self.conn.commit()
        return len(data)

    def get_post_count(self) -> int:
        """Return total number of posts in database."""
```

```
        result = self.conn.execute("SELECT COUNT(*) FROM
posts").fetchone()
        return result[0]

    def close(self) -> None:
        """Close database connection."""
        self.conn.close()
```

Batch Processing Pipeline

Here is a complete pipeline for fetching, processing, and storing Reddit content:

```
class RedditIngestionPipeline:
    """End-to-end pipeline for ingesting Reddit content."""

    def __init__(
        self,
        reddit_client: RedditClient,
        storage: KnowledgeBaseStorage,
        embedding_generator: EmbeddingGenerator
    ):
        self.reddit = reddit_client
        self.storage = storage
        self.embedder = embedding_generator
        self.preparer = ContentPreparer()

    def ingest_subreddit(
        self,
        subreddit_name: str,
        sort: str = "top",
        limit: int = 100,
        time_filter: str = "month",
        batch_size: int = 32
    ) -> dict:
        """
        Ingest posts from a subreddit.

        Args:
            subreddit_name: Subreddit to fetch from
            sort: Sort method
            limit: Maximum posts
            time_filter: Time filter for 'top' sort
            batch_size: Embedding batch size

        Returns:
```

```
            Statistics dictionary
        """
        stats = {"fetched": 0, "processed": 0, "skipped": 0}

        posts_batch = []
        texts_batch = []

        for post in self.reddit.get_subreddit_posts(
            subreddit_name, sort=sort, limit=limit,
time_filter=time_filter
        ):
            stats["fetched"] += 1

            # Prepare text for embedding
            text = self.preparer.prepare_post(post)

            # Quality filter
            if not self.preparer.is_quality_content(text):
                stats["skipped"] += 1
                continue

            posts_batch.append(post)
            texts_batch.append(text)

            # Process in batches
            if len(posts_batch) >= batch_size:
                self._process_batch(posts_batch, texts_batch)
                stats["processed"] += len(posts_batch)
                posts_batch = []
                texts_batch = []

        # Process remaining
        if posts_batch:
            self._process_batch(posts_batch, texts_batch)
            stats["processed"] += len(posts_batch)

        return stats

    def _process_batch(
        self,
        posts: List[Dict],
        texts: List[str]
    ) -> None:
        """Generate embeddings and store a batch of posts."""
        embeddings = self.embedder.encode(texts)
        self.storage.insert_posts_batch(
            posts, embeddings, self.embedder.model_name
        )
```

Building the Vector Index

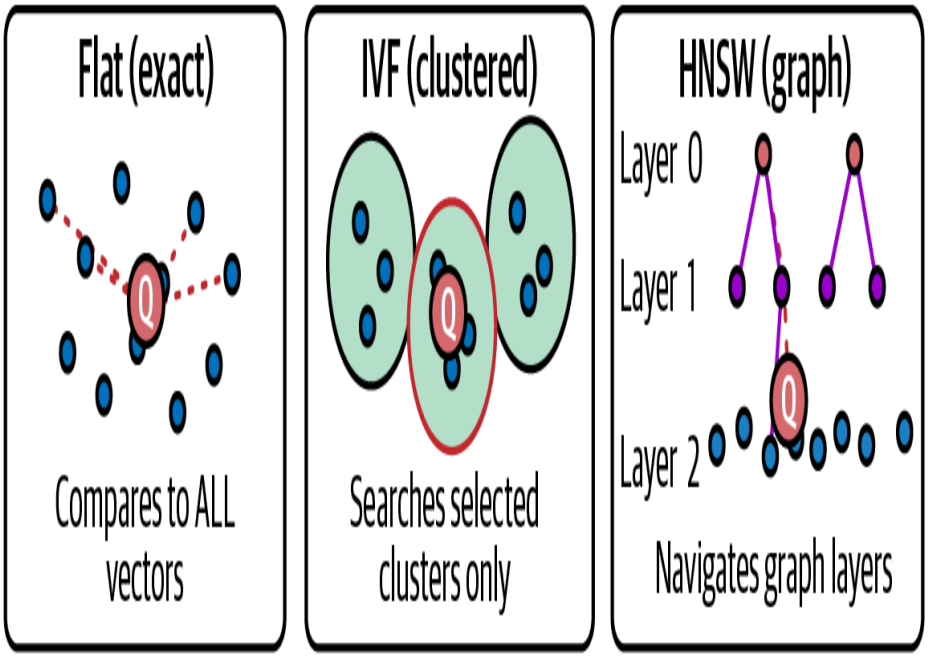
With posts and embeddings stored, we create a vector index for fast similarity search.

Understanding VSS Indexing

The sqlite-vss extension uses FAISS under the hood. By default, it creates a “Flat” index that performs exact (brute-force) search by comparing the query against every vector. This is accurate but becomes slow for large datasets. For larger collections, you can specify ANN algorithms via factory strings:

- `HNSW16...` creates an HNSW graph with 16 connections per node (fast approximate search for many workloads).
- `IVF...` creates an IVF index (clustered approximate search) and often requires a training step before it can be used.

Factory string syntax and requirements (such as whether `IDMap2` is needed or whether training is required) can vary across sqlite-vss releases and FAISS index types. Treat the strings as illustrative and confirm against the sqlite-vss documentation for your installed version. See Figure 4-5 for a comparison.



Index type	Search speed	Accuracy	Memory	Training required	Best for
Flat	O(n) – Slow	100% exact	Low	No	<100K vectors
IVF	O(n/k) – Fast	~95% approximate	Medium	Yes (clustering)	100K-10M vectors
HNSW	O(log n) – Very fast	~98% approximate	High	No	Any scale, low latency

Figure 4-5. Index type comparison

For personal knowledge bases with thousands to tens of thousands of posts, exact search is often fast enough. We will start with the simpler approach.

Index Management

For simplicity, we rebuild the index from stored embeddings whenever it needs updating. An incremental approach would insert or update vectors in `posts_vss` as new posts arrive, but that adds complexity. The rebuild strategy shown here is appropriate for a personal knowledge base where you might run ingestion periodically (e.g., weekly) rather than continuously:

```
class VectorIndexManager:
    """Manage the VSS vector index."""

    def __init__(self, storage: KnowledgeBaseStorage):
        self.storage = storage
        self.conn = storage.conn
        self.dimension = storage.embedding_dim

    def create_index(self, factory_string: str = "Flat") -> None:
        """
        Create or recreate the vector search index.
        Note: default factory_string may be version dependent

        Args:
            factory_string: FAISS index type. Options include:
                - "Flat": Exact search (default, best for <100k
vectors)
                - "IVF100,Flat": Approximate search with 100
clusters
                - "HNSW16": Graph-based approximate search
        """
        # Drop existing index
        self.conn.execute("DROP TABLE IF EXISTS posts_vss")

        # Create new VSS virtual table
        if factory_string == "Flat":
            self.conn.execute(f"""
                CREATE VIRTUAL TABLE posts_vss USING vss0(
                    embedding({self.dimension})
                )
            """)
        else:
            self.conn.execute(f"""
                CREATE VIRTUAL TABLE posts_vss USING vss0(
                    embedding({self.dimension}) factory="
{factory_string}"
```

```
                )
            """)

        self.conn.commit()
        print(f"Created vector index: {factory_string}, \
            dimension={self.dimension}")

    def populate_index(self, batch_size: int = 500) -> int:
        """
        Populate index from existing post embeddings.

        Note: This implementation prioritizes clarity over
performance.
        For better performance, you could:
        - Use executemany with larger batches
        - Use binary vector insertion APIs if available
        - Wrap the entire operation in a single transaction

        Args:
            batch_size: Number of rows to process at once

        Returns:
            Number of vectors indexed
        """
        # Count posts with embeddings
        total = self.conn.execute("""
            SELECT COUNT(*) FROM posts WHERE embedding IS NOT
NULL
        """).fetchone()[0]

        if total == 0:
            print("No embeddings to index")
            return 0

        indexed = 0
        offset = 0

        while offset < total:
            # Fetch batch of posts with embeddings
            rows = self.conn.execute("""
                SELECT id, embedding FROM posts
                WHERE embedding IS NOT NULL
                ORDER BY id
                LIMIT ? OFFSET ?
            """, (batch_size, offset)).fetchall()

            if not rows:
                break
```

```
                # Insert batch into VSS index
                # We use id (which aliases rowid) for the join
                insert_data = []
                for row in rows:
                    row_id = row["id"]
                    embedding_blob = row["embedding"]
                    embedding =
EmbeddingGenerator.deserialize_embedding(
                        embedding_blob, self.dimension
                    )
                    vector_json = json.dumps(embedding.tolist())
                    insert_data.append((row_id, vector_json))

                self.conn.executemany("""
                    INSERT INTO posts_vss (rowid, embedding)
                    VALUES (?, vector_from_json(?))
                """, insert_data)

                self.conn.commit()
                indexed += len(rows)
                offset += batch_size

                if indexed % 1000 == 0 or indexed == total:
                    print(f"Indexed {indexed}/{total} vectors")

            return indexed

        def rebuild_index(self, factory_string: str = "Flat") -> int:
            """Drop and rebuild the entire index."""
            self.create_index(factory_string)
            return self.populate_index()
```

Implementing Semantic Search

With content stored and indexed, we implement the search functionality that combines vector similarity with metadata filtering.

Search Result Container

Let’s start with the search result container:

```
from dataclasses import dataclass
from datetime import datetime
```

```
@dataclass
class SearchResult:
    """Container for a single search result."""
    rowid: int
    post_id: str
    title: str
    selftext: str
    subreddit: str
    author: str
    score: int
    num_comments: int
    created_utc: int
    distance: float

    @property
    def similarity_score(self) -> float:
        """
        Convert L2 distance to a similarity score for display.

        This is a presentation heuristic, not a mathematically
rigorous
        similarity measure. Lower distance = higher similarity.
The formula
        maps distance to a 0-1 range where 1 is most similar.
        """
        return 1.0 / (1.0 + self.distance)

    @property
    def created_date(self) -> str:
        """Format creation timestamp as date string."""
        return
datetime.utcfromtimestamp(self.created_utc).strftime("%Y-%m-%d")
```

Search Engine

The exact `vss_search` signature varies by sqlite-vss version. The following code works with recent releases, but check it against your installed version if you encounter errors (see Figure 4-6 before diving in):

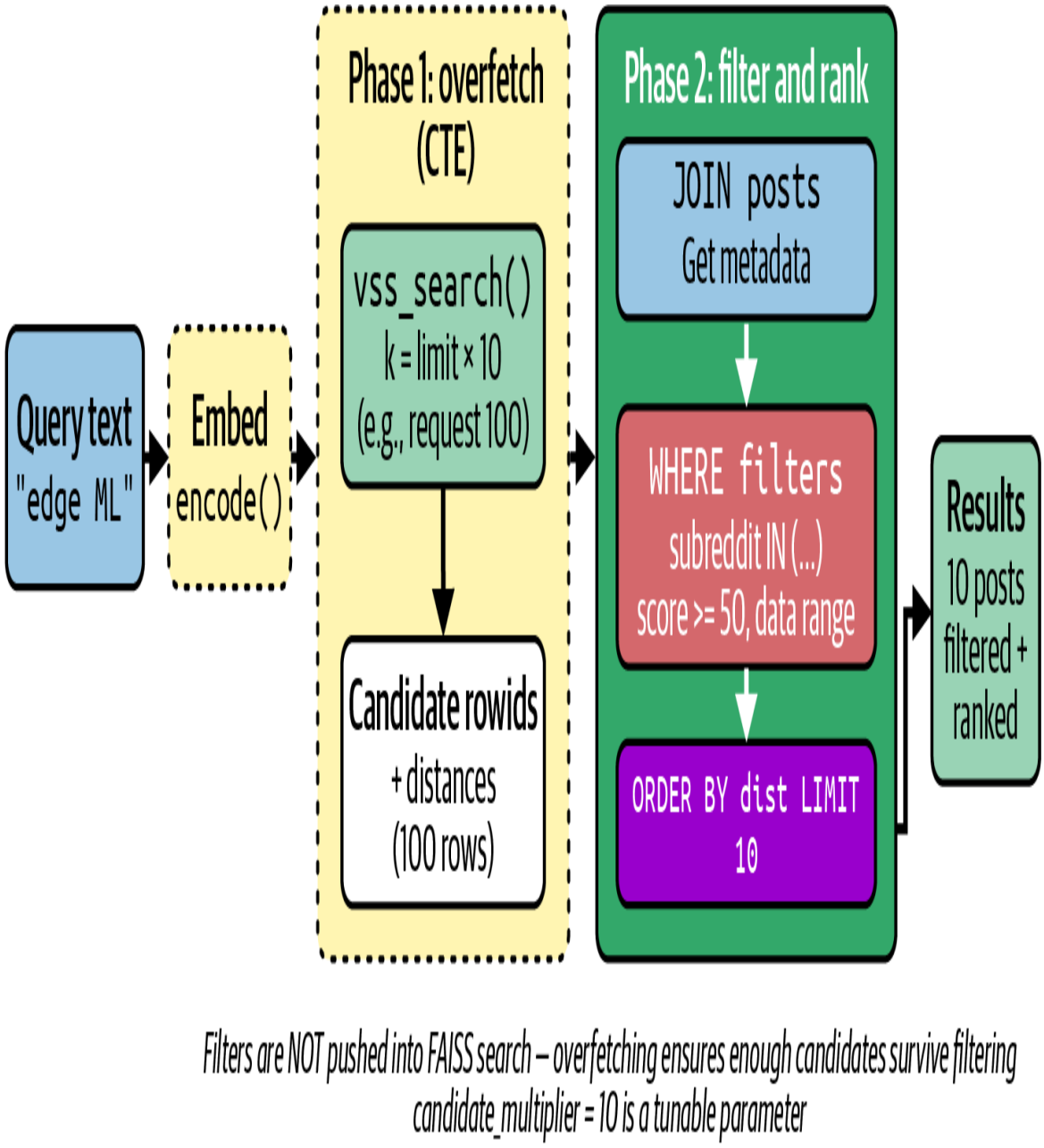


Figure 4-6. Query execution flow

```
import json
from typing import List, Optional

class SemanticSearchEngine:
    """Semantic search over Reddit knowledge base."""

    def __init__(
        self,
        storage: KnowledgeBaseStorage,
```

```
        embedding_generator: EmbeddingGenerator
        candidate_multiplier: int = 10
    ):
        self.storage = storage
        self.conn = storage.conn
        self.embedder = embedding_generator
        self.preparer = ContentPreparer()

    def search(
        self,
        query: str,
        limit: int = 10,
        subreddits: Optional[List[str]] = None,
        min_score: Optional[int] = None,
        after_date: Optional[datetime] = None,
        before_date: Optional[datetime] = None
    ) -> List[SearchResult]:
        """
        Search for posts semantically similar to query.

        The distance returned by sqlite-vss is raw L2 distance.
Its scale
        depends on the embedding model and content. Use the
similarity_score
        property for a normalized display value.

        Args:
            query: Natural language search query
            limit: Maximum results to return
            subreddits: Filter to specific subreddits
            min_score: Minimum post score
            after_date: Only posts after this date
            before_date: Only posts before this date

        Returns:
            List of SearchResult objects ordered by similarity
(closest first)
        """
        # Clean and embed the query
        clean_query = self.preparer.preprocessor.process(query)
        query_embedding = self.embedder.encode(clean_query)[0]
        query_vector_json = json.dumps(query_embedding.tolist())

        # We overfetch candidates, then apply metadata filters in
SQL.
        # This improves result stability because filters are not
pushed down
        # into the nearest-neighbor search in many sqlite-vss
```

```
versions.

k = max(limit * candidate_multiplier, limit)

sql = """
    WITH candidates AS (
        SELECT rowid, distance
        FROM posts_vss
        WHERE vss_search(embedding, vector_from_json(?))
        ORDER BY distance ASC
        LIMIT ?
    )
    SELECT
        p.id,
        p.post_id,
        p.title,
        p.selftext,
        p.subreddit,
        p.author,
        p.score,
        p.num_comments,
        p.created_utc,
        c.distance
    FROM candidates c
    INNER JOIN posts p ON p.id = c.rowid
    WHERE 1=1
"""
params: List = [query_vector_json, k]

if subreddits:
    placeholders = ",".join("?" * len(subreddits))
    sql += f" AND p.subreddit IN ({placeholders})"
    params.extend(subreddits)

if min_score is not None:
    sql += " AND p.score >= ?"
    params.append(min_score)

if after_date is not None:
    sql += " AND p.created_utc >= ?"
    params.append(int(after_date.timestamp()))

if before_date is not None:
    sql += " AND p.created_utc <= ?"
    params.append(int(before_date.timestamp()))

sql += " ORDER BY c.distance ASC LIMIT ?"
params.append(limit)
```

```
        # Execute and parse results
        rows = self.conn.execute(sql, params).fetchall()

        results = []
        for row in rows:
            results.append(SearchResult(
                rowid=row["id"],
                post_id=row["post_id"],
                title=row["title"],
                selftext=row["selftext"],
                subreddit=row["subreddit"],
                author=row["author"],
                score=row["score"],
                num_comments=row["num_comments"],
                created_utc=row["created_utc"],
                distance=row["distance"]
            ))

        return results

    def search_with_facets(
        self,
        query: str,
        limit: int = 50
    ) -> dict:
        """
        Search with faceted result breakdown.

        Args:
            query: Search query
            limit: Maximum results

        Returns:
            Dictionary with results and facet counts
        """
        results = self.search(query, limit=limit)

        # Compute facets
        subreddit_counts = {}
        score_ranges = {"low": 0, "medium": 0, "high": 0}

        for r in results:
            # Subreddit facet
            subreddit_counts[r.subreddit] =
subreddit_counts.get(r.subreddit, \
                0) + 1
```

```
            # Score range facet
            if r.score < 10:
                score_ranges["low"] += 1
            elif r.score < 100:
                score_ranges["medium"] += 1
            else:
                score_ranges["high"] += 1

        return {
            "results": results,
            "facets": {
                "subreddits": dict(sorted(
                    subreddit_counts.items(),
                    key=lambda x: x[1],
                    reverse=True
                )),
                "score_ranges": score_ranges
            },
            "total": len(results)
        }

    def find_similar_posts(
        self,
        post_id: str,
        limit: int = 10,
        exclude_same_subreddit: bool = False
    ) -> List[SearchResult]:
        """
        Find posts similar to a given post.

        Args:
            post_id: ID of the reference post
            limit: Maximum results
            exclude_same_subreddit: Whether to exclude posts from
same subreddit

        Returns:
            List of similar posts
        """
        # Get the reference post's embedding
        row = self.conn.execute("""
            SELECT id, embedding, subreddit FROM posts WHERE
post_id = ?
        """, (post_id,)).fetchone()

        if not row or not row["embedding"]:
            return []
```

```
        embedding = EmbeddingGenerator.deserialize_embedding(
            row["embedding"], self.embedder.dimension
        )
        vector_json = json.dumps(embedding.tolist())
        source_subreddit = row["subreddit"]
        source_id = row["id"]

        # Search for similar posts
        sql = """
            SELECT
                p.id, p.post_id, p.title, p.selftext,
p.subreddit,
                p.author, p.score, p.num_comments, p.created_utc,
                vss.distance
            FROM posts_vss vss
            INNER JOIN posts p ON p.id = vss.rowid
            WHERE vss_search(vss.embedding, vector_from_json(?))
            AND p.id != ?
        """
        params: List = [vector_json, source_id]

        if exclude_same_subreddit:
            sql += " AND p.subreddit != ?"
            params.append(source_subreddit)

        sql += " ORDER BY vss.distance ASC LIMIT ?"
        params.append(limit)

        rows = self.conn.execute(sql, params).fetchall()

        return [SearchResult(
            rowid=r["id"],
            post_id=r["post_id"],
            title=r["title"],
            selftext=r["selftext"],
            subreddit=r["subreddit"],
            author=r["author"],
            score=r["score"],
            num_comments=r["num_comments"],
            created_utc=r["created_utc"],
            distance=r["distance"]
        ) for r in rows]

    def cross_subreddit_analysis(
        self,
        query: str,
        limit_per_subreddit: int = 5
```

```
    ) -> dict:
        """
        Analyze how a topic is discussed across subreddits.

        Args:
            query: Topic to analyze
            limit_per_subreddit: Max results per subreddit

        Returns:
            Dictionary mapping subreddits to relevant posts
        """
        # Get broad results
        results = self.search(query, limit=100)

        # Group by subreddit
        by_subreddit = {}
        for r in results:
            if r.subreddit not in by_subreddit:
                by_subreddit[r.subreddit] = []

            if len(by_subreddit[r.subreddit]) <
limit_per_subreddit:
                by_subreddit[r.subreddit].append(r)

        # Sort subreddits by total relevance (sum of similarity
scores)
        subreddit_relevance = {
            sub: sum(r.similarity_score for r in posts)
            for sub, posts in by_subreddit.items()
        }

        sorted_subreddits = sorted(
            subreddit_relevance.keys(),
            key=lambda x: subreddit_relevance[x],
            reverse=True
        )

        return {
            "subreddits": sorted_subreddits,
            "posts_by_subreddit": {
                sub: by_subreddit[sub] for sub in
sorted_subreddits
            },
            "relevance_scores": subreddit_relevance
        }
```

Putting It All Together

Figure 4-7 shows the end-to-end system architecture.

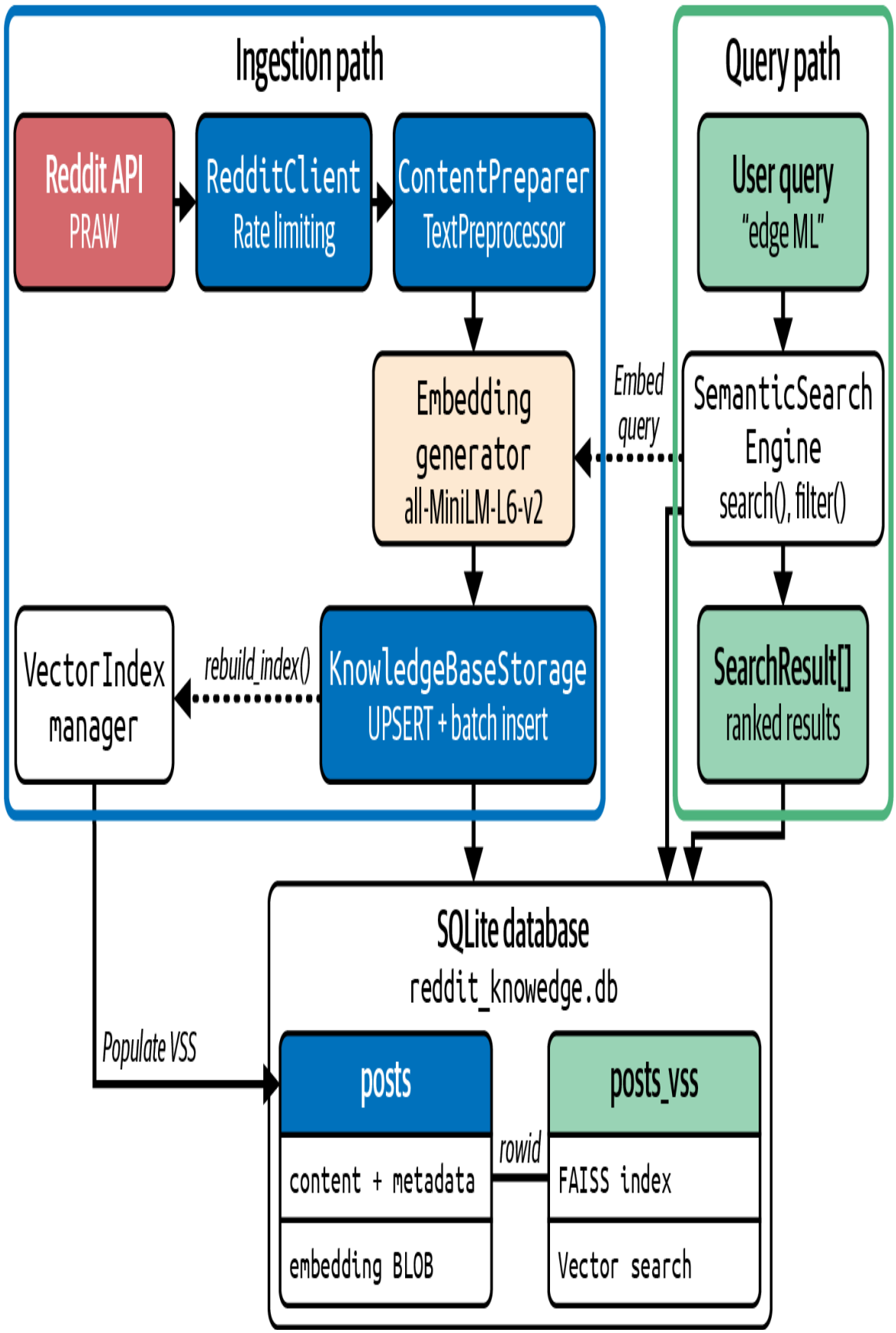

Ingestion path
Reddit API
PRAW
RedditClient
Rate limiting
ContentPreparer
TextPreprocessor
Embedding generator
all-MiniLM-L6-v2
VectorIndex manager
rebuild_index()
KnowledgeBaseStorage
UPSERT + batch insert
Query path
User query
"edge ML"
Embed query
SemanticSearch Engine
search(), filter()
SearchResult[]
ranked results
SQLite database
reddit_knowedge.db
Populate VSS
posts
content + metadata
embedding BLOB
rowid
posts_vss
FAISS index
Vector search

Figure 4-7. Complete system architecture showing ingestion (left) and query (right) paths with shared components

Workflow Example

Here is a complete example demonstrating the full workflow:

```
import json

def main():
    """Complete example: ingest Reddit content and perform
semantic search."""

    # Configuration
    DB_PATH = "reddit_knowledge.db"
    EXTENSION_PATH = "."  # Directory containing vss0 and vector0
    MODEL_NAME = "all-MiniLM-L6-v2"

    # Reddit credentials (use environment variables in
production)
    REDDIT_CLIENT_ID = "your_client_id"
    REDDIT_CLIENT_SECRET = "your_client_secret"
    REDDIT_USER_AGENT = "python:reddit_kb:v1.0 (by
/u/your_username)"

    # Initialize components
    print("Initializing components...")

    # Create database and schema
    conn = create_database(DB_PATH, EXTENSION_PATH)
    conn.close()

    # Initialize embedding generator
    embedder = EmbeddingGenerator(MODEL_NAME)

    # Initialize storage
    storage = KnowledgeBaseStorage(
        DB_PATH,
        EXTENSION_PATH,
        embedding_dim=embedder.dimension
    )

    # Initialize Reddit client
    reddit = RedditClient(
        REDDIT_CLIENT_ID,
        REDDIT_CLIENT_SECRET,
```

```
        REDDIT_USER_AGENT
    )

    # Create ingestion pipeline
    pipeline = RedditIngestionPipeline(reddit, storage, embedder)

    # Ingest content from multiple subreddits
    print("\nIngesting content...")
    subreddits = ["MachineLearning", "Python", "DataScience"]

    for subreddit in subreddits:
        print(f"\nFetching from r/{subreddit}...")
        stats = pipeline.ingest_subreddit(
            subreddit,
            sort="top",
            limit=100,
            time_filter="month"
        )
        print(f"  Fetched: {stats['fetched']}, "
              f"Processed: {stats['processed']}, "
              f"Skipped: {stats['skipped']}")

    print(f"\nTotal posts in database:
{storage.get_post_count()}")

    # Build vector index
    print("\nBuilding vector index...")
    index_manager = VectorIndexManager(storage)
    index_manager.create_index()
    indexed = index_manager.populate_index()
    print(f"Indexed {indexed} vectors")

    # Initialize search engine
    search_engine = SemanticSearchEngine(storage, embedder)

    # Example searches
    print("\n" + "=" * 60)
    print("SEMANTIC SEARCH EXAMPLES")
    print("=" * 60)

    # Basic semantic search
    query = "deep learning for edge devices"
    print(f"\nQuery: '{query}'")
    print("-" * 40)

    results = search_engine.search(query, limit=5)
    for i, r in enumerate(results, 1):
        print(f"{i}. [{r.subreddit}] {r.title[:60]}...")
```

```
        print(f"   Score: {r.score} | Similarity:
{r.similarity_score:.3f} | \
            Distance: {r.distance:.3f}")

    # Search with filters
    query = "neural network optimization"
    print(f"\nQuery: '{query}' (filtered to r/MachineLearning,
score >= 50)")
    print("-" * 40)

    results = search_engine.search(
        query,
        limit=5,
        subreddits=["MachineLearning"],
        min_score=50
    )
    for i, r in enumerate(results, 1):
        print(f"{i}. {r.title[:60]}...")
        print(f"   Score: {r.score} | Date: {r.created_date}")

    # Cross-subreddit analysis
    query = "python best practices"
    print(f"\nCross-subreddit analysis: '{query}'")
    print("-" * 40)

    analysis = search_engine.cross_subreddit_analysis(query)
    for sub in analysis["subreddits"][:3]:
        relevance = analysis["relevance_scores"][sub]
        print(f"\nr/{sub} (relevance: {relevance:.2f}):")
        for r in analysis["posts_by_subreddit"][sub][:2]:
            print(f"  - {r.title[:50]}...")

    # Cleanup
    storage.close()
    print("\nDone!")


if __name__ == "__main__":
    main()
```

Example Output

Running the preceding code produces output similar to the following:

```
Initializing components...
Loaded all-MiniLM-L6-v2 (dim=384) on cuda
```

```
Ingesting content...

Fetching from r/MachineLearning...
  Fetched: 100, Processed: 87, Skipped: 13

Fetching from r/Python...
  Fetched: 100, Processed: 92, Skipped: 8

Fetching from r/DataScience...
  Fetched: 100, Processed: 85, Skipped: 15

Total posts in database: 264

Building vector index...
Created vector index: Flat, dimension=384
Indexed 264/264 vectors
Indexed 264 vectors

============================================================
SEMANTIC SEARCH EXAMPLES
============================================================

Query: 'deep learning for edge devices'
----------------------------------------
1. [MachineLearning] TinyML: Running Neural Networks on
Microcontrollers...
   Score: 342 | Similarity: 0.847 | Distance: 0.181
2. [MachineLearning] Efficient inference on mobile devices with
quantization...
   Score: 156 | Similarity: 0.812 | Distance: 0.232
3. [DataScience] Deploying models to IoT sensors - lessons
learned...
   Score: 89 | Similarity: 0.798 | Distance: 0.253
```

Extension: Incremental Indexing

The rebuild approach works well for periodic batch updates, but for continuous ingestion, you may want to add vectors to the index immediately. The following code shows how to extend the storage class:

```
# Add these methods to KnowledgeBaseStorage

def insert_post_with_index(
```

```
    self,
    post: Dict[str, Any],
    embedding: np.ndarray,
    model_name: str
) -> int:
    """
    Insert a post and immediately add it to the VSS index.

    This enables incremental indexing without full rebuilds.
    """
    # Insert post (UPSERT preserves rowid)
    embedding_blob =
EmbeddingGenerator.serialize_embedding(embedding)

    cursor = self.conn.execute("""
        INSERT INTO posts
        (post_id, title, selftext, url, subreddit, author,
         score, num_comments, created_utc, embedding,
embedding_model)
        VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?)
        ON CONFLICT(post_id) DO UPDATE SET
            title = excluded.title,
            selftext = excluded.selftext,
            url = excluded.url,
            subreddit = excluded.subreddit,
            author = excluded.author,
            score = excluded.score,
            num_comments = excluded.num_comments,
            created_utc = excluded.created_utc,
            embedding = excluded.embedding,
            embedding_model = excluded.embedding_model
        RETURNING id
    """, (
        post["post_id"],
        post["title"],
        post.get("selftext", ""),
        post.get("url", ""),
        post["subreddit"],
        post.get("author", "[unknown]"),
        post.get("score", 0),
        post.get("num_comments", 0),
        post["created_utc"],
        embedding_blob,
        model_name
    ))

    row = cursor.fetchone()
    row_id = row[0]
```

```
        # Add to VSS index (or update if exists)
        vector_json = json.dumps(embedding.tolist())

        # Delete existing entry if present (for updates)
        self.conn.execute(
            "DELETE FROM posts_vss WHERE rowid = ?",
            (row_id,)
        )

        # Insert new entry
        self.conn.execute("""
            INSERT INTO posts_vss (rowid, embedding)
            VALUES (?, vector_from_json(?))
        """, (row_id, vector_json))

        self.conn.commit()
        return row_id
```

Conclusion

This chapter demonstrated building a semantic search system over Reddit content using SQLite and the sqlite-vss extension. The key components include:

- Content ingestion using PRAW to fetch posts with proper rate limiting and error handling. The Reddit API provides rich metadata that enables filtering alongside semantic search.
- Text preprocessing to clean markdown, URLs, and Reddit-specific artifacts before embedding. Consistent preprocessing improves embedding quality and search relevance.
- Embedding generation using `SentenceTransformers`, with batch processing for efficiency. The all-MiniLM-L6-v2 model provides a good balance of quality and speed for this application.
- Vector storage and indexing using sqlite-vss, which wraps FAISS in a SQL-friendly interface. The two-table pattern separates content storage from vector indexing while enabling efficient joins.

To ensure correctness, always use UPSERT rather than `INSERT OR REPLACE` to preserve `rowid` stability.

- Semantic search combining vector similarity (L2 distance) with traditional SQL filters on metadata like subreddit, date, and score. This hybrid approach lets you narrow candidates via metadata while still ranking by meaning—a capability that is difficult to achieve cleanly in pure vector databases without duplicating relational features.

The same architecture applies to other content sources. Replace the PRAW client with connectors for Twitter, email, notes, or any text-based content, and the rest of the pipeline remains largely unchanged. The principles of text preprocessing, batch embedding, and hybrid search transfer directly.

For personal knowledge management, this SQLite-based approach offers significant advantages: the entire database lives in a single portable file, requires no server infrastructure, and provides familiar SQL for ad hoc queries. While it won’t scale to millions of documents like dedicated vector databases, it handles personal-scale collections of thousands to tens of thousands of items with ease.

OceanofPDF.com

Chapter 5. Building an ArXiv Paper Search System with PostgreSQL pgvector

This chapter will teach you how to build an ArXiv paper search system with PostgreSQL pgvector, so let's dive in.

The Challenge of Searching Scientific Literature

In the rapidly evolving landscape of scientific research, staying current with the latest developments has become increasingly challenging. ArXiv alone publishes thousands of new papers each month across physics, mathematics, computer science, and other fields. Traditional keyword-based search often fails to capture the semantic relationships between papers, missing relevant work that uses different terminology or approaches the same problem from a different angle.

Consider a researcher investigating "neural network optimization techniques." A keyword search might miss papers discussing "gradient descent improvements" or "backpropagation efficiency," even though these are directly relevant. This semantic gap between search queries and relevant content is where vector databases excel: understanding the meaning behind text rather than just matching keywords.

Why ArXiv Makes an Ideal Data Source

ArXiv presents unique advantages for building a vector search system. First, it provides free, open access to scientific papers with consistent metadata through its API. Second, the papers follow relatively standard

academic formatting, making text extraction more predictable than general web content. Third, the technical nature of the content benefits significantly from semantic search—mathematical concepts, scientific terminology, and research methodologies often have multiple valid expressions that vector embeddings can connect.

The preprint nature of ArXiv also means researchers can track emerging ideas before formal publication, making timely discovery even more critical. A personal research assistant that understands the semantic content of papers can surface connections across disciplines, identify methodological similarities, and track the evolution of ideas through citation networks.

Real-World Use Cases

This system addresses several critical research workflows. During literature reviews, it can find semantically similar papers even when they use different terminology—essential when exploring interdisciplinary topics. For researchers entering new fields, it identifies foundational papers and tracks how concepts evolved. When writing papers, it helps ensure comprehensive related-work sections by finding papers that might be missed through keyword search alone.

The system also enables serendipitous discovery. By understanding paper content at a semantic level, it can suggest unexpected connections—perhaps a technique from quantum physics that applies to machine learning, or a mathematical framework from economics relevant to biological modeling. These cross-pollination opportunities are often where breakthrough insights emerge.

Technology Stack Rationale

PostgreSQL with pgvector offers an ideal foundation for a personal research tool. Unlike specialized vector databases that require additional infrastructure, PostgreSQL is likely already familiar to many developers and researchers. It provides battle-tested reliability, ACID compliance, and

rich SQL capabilities for complex queries combining vector similarity with traditional filters.

The pgvector extension seamlessly integrates vector operations into PostgreSQL's query planner, allowing sophisticated hybrid searches that combine semantic similarity with metadata filtering. You can search for papers similar to a query while restricting results to specific date ranges, authors, or categories—all in a single, efficient SQL query. This integration eliminates the complexity of synchronizing separate databases and provides a single source of truth for all paper data.

For a personal research tool, this approach offers the right balance of capability and simplicity. The system can scale from hundreds to tens of thousands of papers without architectural changes, and PostgreSQL's mature ecosystem provides excellent tooling for backup, monitoring, and management.

> **NOTE**
>
> This chapter outlines an architecture via class and method definitions. In the interests of architecture focus, brevity, and page count, code implementing these is available on the accompanying GitHub repository mentioned in the Preface.

Architecture Overview

Let's first take a look at the architecture of the ArXiv paper search system.

System Components

The ArXiv paper search system consists of six interconnected components, each handling a specific aspect of the data pipeline from paper discovery to semantic search (see Figure 5-1).

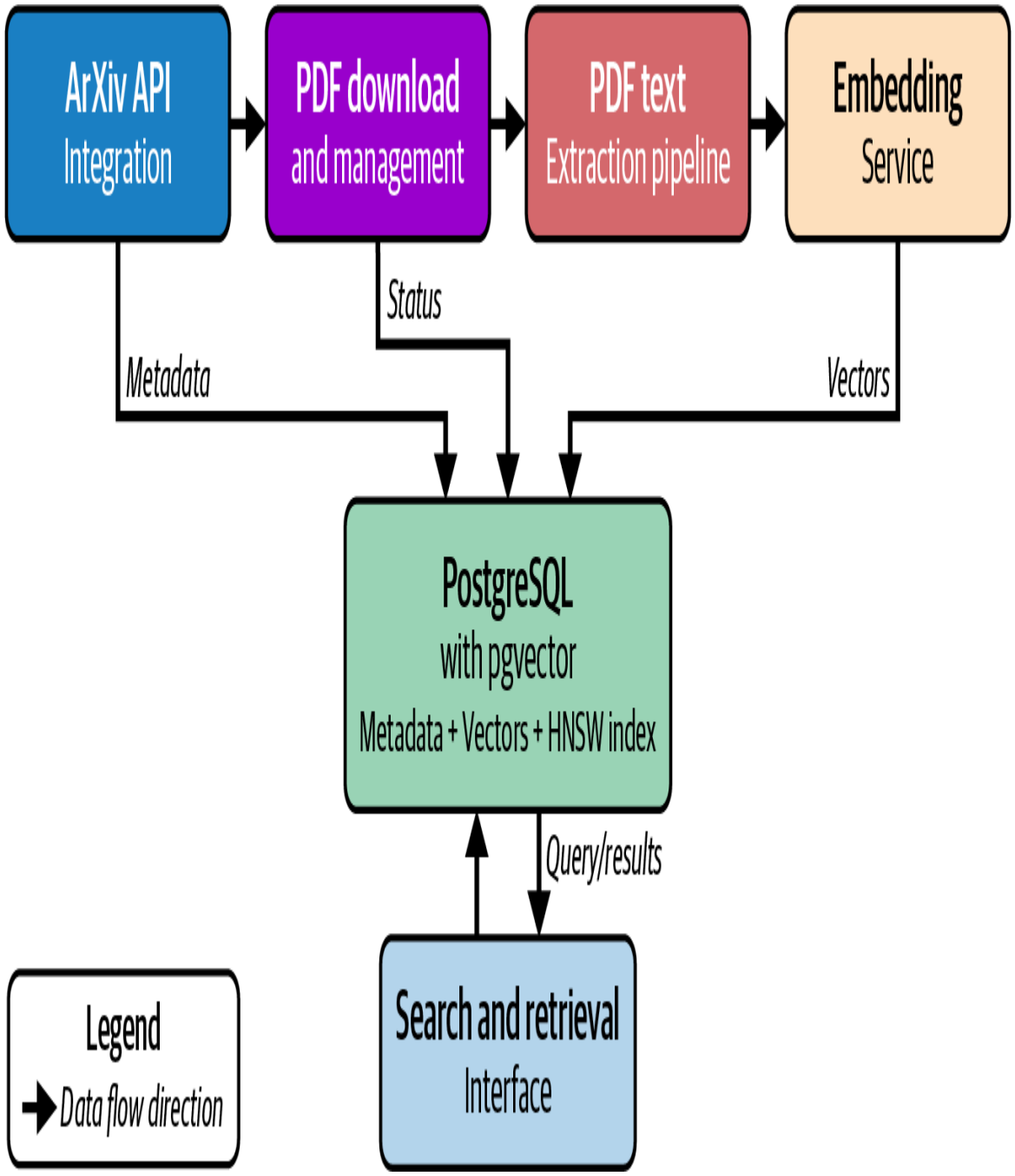


Figure 5-1. System architecture diagram

The *ArXiv API Integration component* serves as the entry point, fetching paper metadata and download URLs. It handles rate limiting, query construction, and result pagination while maintaining compliance with ArXiv's usage guidelines. This component runs periodically to check for new papers in specified categories or from particular authors.

The *PDF Download and Management subsystem* efficiently retrieves papers, organizing them in a structured filesystem hierarchy. It implements retry logic for failed downloads, validates PDF integrity, and maintains a download queue to respect rate limits. The system tracks which papers have been successfully processed to avoid redundant downloads.

The *PDF Text Extraction pipeline* tackles one of the most challenging aspects: converting academic PDFs into clean, searchable text. Using PyMuPDF, it handles multicolumn layouts, extracts text while preserving logical reading order, and attempts to clean up common PDF extraction artifacts. Special attention is paid to maintaining paragraph boundaries and section structures, which are crucial for generating meaningful embeddings.

The *PostgreSQL database with pgvector* serves as the central storage system. It maintains paper metadata, stores extracted text chunks with their vector embeddings, and provides high-performance similarity search through HNSW indexing. The database schema captures relationships among papers, authors, and categories while optimizing for both vector and traditional queries.

The *Embedding Generation Service* transforms text into high-dimensional vectors using `SentenceTransformers`. It processes text in batches for efficiency, maintains a singleton model instance to avoid memory overhead, and generates embeddings for both paper chunks and search queries. The service handles text preprocessing to ensure consistent embedding quality.

Finally, the *Search and Retrieval interface* provides the user-facing functionality. It accepts natural language queries, generates query embeddings, performs hybrid searches combining vector similarity with metadata filters, and presents results with relevant snippets and scores.

Data Flow

The data pipeline begins when the ArXiv API integration discovers new papers matching specified criteria. Metadata—including title, abstract, authors, and categories—is immediately stored in PostgreSQL. The PDF

URL is added to a download queue and processed asynchronously to manage rate limits and network issues. See Figure 5-2 for the full data flow.

Once downloaded, each PDF undergoes text extraction. The extraction pipeline reads the PDF page by page, identifying text blocks, and determines reading order. The extracted text is cleaned to remove formatting artifacts, normalize whitespace, and handle special characters. The complete text is then segmented into overlapping chunks—typically from 512 to 1,024 tokens each—preserving sentence boundaries where possible.

Each text chunk is processed through the embedding model, generating a 384-dimensional vector (using all-MiniLM-L6-v2). These embeddings, along with the original text and metadata, are stored in PostgreSQL. The pgvector extension automatically maintains the HNSW index, enabling efficient similarity searches across millions of vectors.

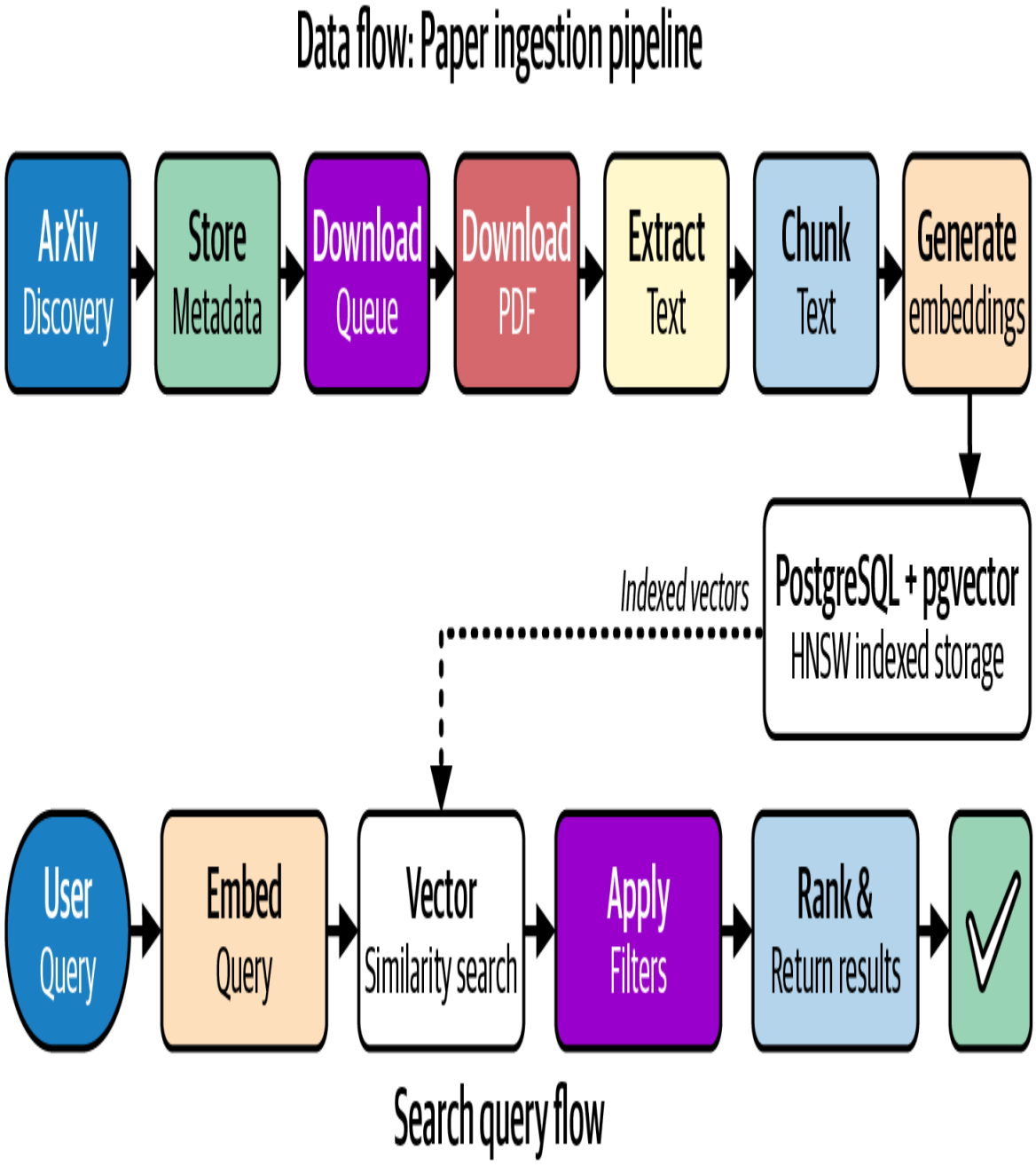


Figure 5-2. Data flow diagram

When a user submits a search query, it follows a similar but streamlined path. The query text is embedded using the same model to ensure compatibility with stored vectors. The search engine then queries PostgreSQL, using pgvector's operators to find similar chunks while applying any metadata filters. Results are aggregated at the paper level,

combining scores from multiple chunks and retrieving full paper metadata for display.

Design Philosophy

The architecture prioritizes simplicity and reliability over raw performance, recognizing that a personal research tool must be maintainable by a single person. Each component has clear boundaries and responsibilities, making the system easy to understand, debug, and extend. Error handling is comprehensive but pragmatic—the system gracefully handles common failures such as network timeouts or malformed PDFs without halting the entire pipeline.

Data quality takes precedence over processing speed. The text extraction pipeline includes validation steps to ensure that meaningful content is indexed. Rather than blindly processing every PDF page, the system can identify and skip problematic content such as complex figures or tables that don't extract well, preventing garbage text from polluting search results.

The system is designed for incremental growth. Starting with a handful of papers for testing, it scales smoothly to thousands of documents without architectural changes. Database indexes are configured for typical personal collection sizes (1,000–10,000 papers) rather than optimized for millions of documents, keeping query planning simple and predictable.

Environment Setup and Dependencies

Now let's set up the environment and dependencies for this system.

PostgreSQL and pgvector Installation

Setting up PostgreSQL with pgvector requires attention to compatibility. PostgreSQL 13 or later is recommended for optimal pgvector performance, as it includes improvements to parallel query execution and index building that significantly benefit vector operations.

On macOS, Homebrew provides the simplest installation path:

```
brew install postgresql@15
brew services start postgresql@15
```

For Linux systems, use the official PostgreSQL repositories for the latest stable version:

```
sudo apt-get update
sudo apt-get install postgresql-15 postgresql-server-dev-15
```

Installing pgvector requires building from source to ensure compatibility with your PostgreSQL version. Clone the official repository and build:

```
git clone https://github.com/pgvector/pgvector.git
cd pgvector
make
make install  # May require sudo
```

After installation, enable pgvector in your database:

```
CREATE EXTENSION IF NOT EXISTS vector;
```

Verify the installation by checking available vector operations:

```
SELECT * FROM pg_extension WHERE extname = 'vector';
```

Note that the code uses the psycopg2 library to interact with PostgreSQL. PostgreSQL does not automatically translate a Python list or `numpy.ndarray` into the vector type. Consequently, when you try to run an insert like `INSERT INTO paper_chunks (embedding) VALUES (%s)`, psycopg2 will throw a `can't adapt type 'numpy.ndarray'` error.

The fix: Import the pgvector helper and register it with the connection:

```
from pgvector.psycopg2 import register_vector
register_vector(conn)
```

Python Environment Configuration

Python 3.8 or later is required for compatibility with modern machine learning libraries. Create a dedicated virtual environment to isolate dependencies:

```
python -m venv arxiv_search_env
# On Windows: arxiv_search_env\Scripts\activate
source arxiv_search_env/bin/activate
```

The core dependencies each serve specific purposes in the pipeline:

arxiv (1.4.8+)

> The official Python client for the ArXiv API. It handles query construction, result parsing, and automatic retries for failed requests. The library respects rate limits and provides clean abstractions for paper metadata.

PyMuPDF (1.23.0+)

> Also imported as fitz. This library excels at PDF text extraction. Unlike simpler PDF libraries, PyMuPDF provides access to text positioning information—crucial for maintaining reading order in multicolumn academic papers. It also handles encrypted PDFs and complex font encodings common in scientific documents.

psycopg2 (2.9.0+)

> The PostgreSQL adapter for Python, offering both low-level database access and connection pooling capabilities. Binary wheels are available for most platforms, but building from source may be necessary for ARM-based systems.

sentence-transformers (2.2.0+)

> Provides pretrained models for generating text embeddings. The library handles model downloading, caching, and efficient batch processing. It

automatically uses GPU acceleration when available, significantly speeding up embedding generation.

requests (2.31.0+)

Used for downloading PDFs with proper timeout handling, retry logic, and progress tracking. Its session management reduces connection overhead when downloading multiple papers.

Install all dependencies:

```
pip install arxiv==2.1.0 PyMuPDF==1.23.8 psycopg2-binary==2.9.9 \
            sentence-transformers==2.2.2 requests==2.31.0
numpy==1.24.3 \
            tqdm==4.66.1 python-dotenv==1.0.0
```

Directory Structure and Configuration

Organize the project with clear separation between code, data, and configuration:

```
arxiv_search/
├── config/
│   ├── .env                  # Database credentials, API keys
│   └── settings.py           # Application configuration
├── data/
│   ├── pdfs/                # Downloaded PDF files
│   │   ├── 2024/           # Organized by year
│   │   └── 2025/
│   ├── cache/               # Embedding cache, temporary files
│   └── logs/                # Application logs
├── src/
│   ├── __init__.py
│   ├── arxiv_client.py     # ArXiv API integration
│   ├── pdf_processor.py    # PDF extraction pipeline
│   ├── embeddings.py       # Embedding generation
│   ├── database.py         # PostgreSQL operations
│   └── search.py           # Search interface
└── scripts/
    ├── setup_db.py         # Database initialization
    ├── fetch_papers.py     # Paper download script
    └── build_index.py      # Index building utilities
```

Create a *.env* file for configuration:

```
# Database Configuration
DB_HOST=localhost
DB_PORT=5432
DB_NAME=arxiv_papers
DB_USER=postgres
DB_PASSWORD=your_password

# ArXiv Configuration
ARXIV_MAX_RESULTS=100
ARXIV_RATE_LIMIT=3  # Seconds between requests

# Model Configuration
EMBEDDING_MODEL=all-MiniLM-L6-v2
EMBEDDING_BATCH_SIZE=32
EMBEDDING_DEVICE=cpu  # or 'cuda' if GPU available

# Storage Configuration
PDF_STORAGE_PATH=./data/pdfs
CACHE_PATH=./data/cache
LOG_PATH=./data/logs
```

Verification and Testing

Create a simple verification script to ensure that all components are properly installed:

```
#!/usr/bin/env python3
"""Verify environment setup for ArXiv search system."""

import sys
from pathlib import Path

def check_imports():
    """Verify all required packages are importable."""
    # Implementation omitted - see full chapter
    pass

def check_postgresql():
    """Verify PostgreSQL connection and pgvector extension."""
    # Implementation omitted - see full chapter
    pass

def check_model_download():
```

```
    """Verify the embedding model can be loaded."""
    # Implementation omitted - see full chapter
    pass

def main():
    """Run all verification checks."""
    # Implementation omitted - see full chapter
    pass

if __name__ == "__main__":
    sys.exit(main())
```

Run the verification script to ensure that your environment is correctly configured. Address any issues before proceeding with the implementation. Common problems include PostgreSQL connection permissions (check *pg_hba.conf*), missing system libraries for psycopg2 compilation, or insufficient disk space for model downloads.

With the environment verified, you're ready to begin building the ArXiv paper search system, starting with the database schema design in the next section.

Database Design for Scientific Papers

Let's look at database design specifically for research papers.

Schema Design Principles

Designing a database schema for scientific papers requires balancing several competing concerns. The schema must efficiently store both structured metadata and unstructured text content, support fast vector similarity searches, enable complex filtering queries, and maintain referential integrity across related entities. For a personal research tool, we optimize for query flexibility and ease of understanding over pure performance. See Figure 5-3 for the entity relationship diagram of the schema.

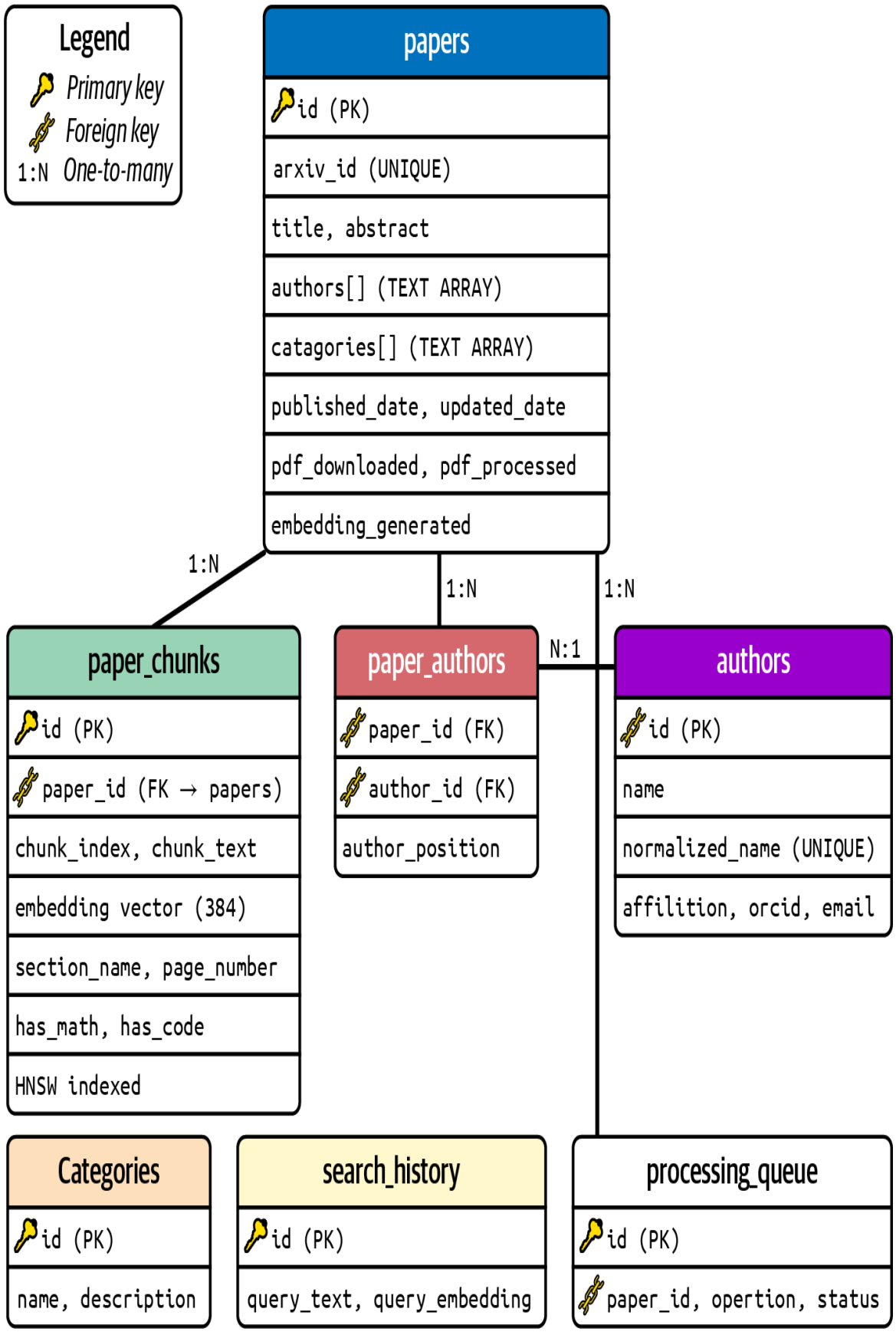

Legend
Primary key
Foreign key
1:N One-to-many
papers
id (PK)
arxiv_id (UNIQUE)
title, abstract
authors[] (TEXT ARRAY)
catagories[] (TEXT ARRAY)
published_date, updated_date
pdf_downloaded, pdf_processed
embedding_generated
1:N
1:N
1:N
paper_chunks
id (PK)
paper_id (FK → papers)
chunk_index, chunk_text
embedding vector (384)
section_name, page_number
has_math, has_code
HNSW indexed
paper_authors
paper_id (FK)
author_id (FK)
author_position
N:1
authors
id (PK)
name
normalized_name (UNIQUE)
affilition, orcid, email
Categories
id (PK)
name, description
search_history
id (PK)
query_text, query_embedding
processing_queue
id (PK)
paper_id, opertion, status

Figure 5-3. Entity-relationship diagram

The schema follows a hybrid approach, combining normalized tables for metadata with denormalized storage for vectors and text chunks. This design recognizes that vector searches return chunk-level results that must be aggregated to paper-level insights. By storing both the chunk text and its vector in the same row, we minimize joins during search operations while maintaining the ability to reconstruct full papers when needed.

Core Tables Structure

Let's go through each of the core tables shown in Figure 5-3:

```
-- Enable required extensions
CREATE EXTENSION IF NOT EXISTS vector;
CREATE EXTENSION IF NOT EXISTS pg_trgm; -- For text similarity
CREATE EXTENSION IF NOT EXISTS btree_gin; -- For composite
indexes


-- ============================================================
-- Main papers table storing metadata
-- ============================================================
CREATE TABLE papers (
    id SERIAL PRIMARY KEY,
    arxiv_id VARCHAR(50) UNIQUE NOT NULL,
    title TEXT NOT NULL,
    abstract TEXT,
    authors TEXT[],                -- Array of author names
    categories TEXT[],             -- Array of category codes
    primary_category VARCHAR(50),
    published_date DATE,
    updated_date DATE,
    pdf_url TEXT,
    comment TEXT,                  -- Version info, page count,
etc.
    journal_ref TEXT,
    doi VARCHAR(100),

    -- Processing status
    pdf_downloaded BOOLEAN DEFAULT FALSE,
    pdf_processed BOOLEAN DEFAULT FALSE,
    embedding_generated BOOLEAN DEFAULT FALSE,
    processing_error TEXT,
```

```
    -- Timestamps
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

-- Indexes for common queries
CREATE INDEX idx_papers_published_date ON papers (published_date
DESC);
CREATE INDEX idx_papers_categories ON papers USING GIN
(categories);
CREATE INDEX idx_papers_authors ON papers USING GIN (authors);

-- ============================================================
-- Chunks table storing paper segments with embeddings
-- ============================================================
CREATE TABLE paper_chunks (
    id SERIAL PRIMARY KEY,
    paper_id INTEGER REFERENCES papers(id) ON DELETE CASCADE,
    chunk_index INTEGER NOT NULL,   -- Position within paper
    chunk_text TEXT NOT NULL,
    chunk_tokens INTEGER,           -- Token count for this chunk

    -- Vector embedding
    embedding vector(384),          -- all-MiniLM-L6-v2 produces
384 dimensions

    -- Metadata for filtering and ranking
    section_name VARCHAR(255),      -- Introduction, Methods,
etc.
    page_number INTEGER,
    char_start INTEGER,             -- Character position in
original text
    char_end INTEGER,

    -- Quality indicators
    has_math BOOLEAN DEFAULT FALSE,
    has_code BOOLEAN DEFAULT FALSE,
    has_references BOOLEAN DEFAULT FALSE,
    language VARCHAR(10) DEFAULT 'en',

    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,

    -- Composite unique constraint
    UNIQUE (paper_id, chunk_index)
);

-- Indexes for vector + hybrid search
```

```
CREATE INDEX idx_chunks_embedding ON paper_chunks
USING hnsw (embedding vector_cosine_ops)
WITH (m = 16, ef_construction = 64);

CREATE INDEX idx_chunks_paper_id ON paper_chunks (paper_id);

CREATE INDEX idx_chunks_text_trgm ON paper_chunks
USING GIN (chunk_text gin_trgm_ops);

-- ============================================================
-- Authors table for detailed author information
-- ============================================================
CREATE TABLE authors (
    id SERIAL PRIMARY KEY,
    name TEXT NOT NULL,
    normalized_name TEXT,           -- Lastname, Firstname format
    affiliation TEXT,
    orcid VARCHAR(50),
    email VARCHAR(255),

    UNIQUE (normalized_name)
);

CREATE INDEX idx_authors_name_trgm ON authors
USING GIN (name gin_trgm_ops);

-- ============================================================
-- Many-to-many relationship between papers and authors
-- ============================================================
CREATE TABLE paper_authors (
    paper_id INTEGER REFERENCES papers(id) ON DELETE CASCADE,
    author_id INTEGER REFERENCES authors(id) ON DELETE CASCADE,
    author_position INTEGER,        -- Order in author list
    is_corresponding BOOLEAN DEFAULT FALSE,

    PRIMARY KEY (paper_id, author_id)
);

-- (Optional but often helpful)
CREATE INDEX idx_paper_authors_author_id ON paper_authors
(author_id);
CREATE INDEX idx_paper_authors_paper_id_position ON paper_authors
(paper_id, \
    author_position);

-- ============================================================
-- Categories table for ArXiv taxonomy
-- ============================================================
```

```
CREATE TABLE categories (
    code VARCHAR(20) PRIMARY KEY,
    name TEXT NOT NULL,
    description TEXT,
    parent_category VARCHAR(20)
);

-- (Optional) parent lookup index
CREATE INDEX idx_categories_parent_category ON categories
(parent_category);

-- ============================================================
-- Search history for query analysis and caching
-- ============================================================
CREATE TABLE search_history (
    id SERIAL PRIMARY KEY,
    query_text TEXT NOT NULL,
    query_embedding vector(384),
    result_count INTEGER,
    execution_time_ms INTEGER,
    filters JSONB,                  -- Store filter criteria as
JSON
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

CREATE INDEX idx_search_history_created_at ON search_history
(created_at DESC);

-- (Optional) if you will do similarity search over query
embeddings too
CREATE INDEX idx_search_history_query_embedding ON search_history
USING hnsw (query_embedding vector_cosine_ops)
WITH (m = 16, ef_construction = 64);

-- ============================================================
-- Processing queue for async operations
-- ============================================================
CREATE TABLE processing_queue (
    id SERIAL PRIMARY KEY,
    paper_id INTEGER REFERENCES papers(id) ON DELETE CASCADE,
    operation VARCHAR(50) NOT NULL, -- 'download_pdf',
'extract_text', \
        'generate_embedding'
    status VARCHAR(20) DEFAULT 'pending', -- 'pending',
'processing', \
        'completed', 'failed'
    priority INTEGER DEFAULT 0,
    retry_count INTEGER DEFAULT 0,
```

```
    error_message TEXT,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    started_at TIMESTAMP,
    completed_at TIMESTAMP
);

CREATE INDEX idx_queue_status_priority ON processing_queue
(status, priority \
    DESC);

-- (Optional) common queue worker lookup patterns
CREATE INDEX idx_queue_paper_id ON processing_queue (paper_id);
CREATE INDEX idx_queue_operation_status ON processing_queue
(operation, status);
```

Vector Storage Strategy

The vector storage strategy significantly impacts both search quality and system performance. The system should use 384-dimensional vectors from the all-MiniLM-L6-v2 model, which provides an excellent balance between quality and speed. These vectors are stored directly in PostgreSQL using the pgvector extension's vector type, which automatically handles serialization and provides optimized operators for similarity calculations.

Chunk size is critical for search relevance. Through experimentation with academic papers, chunks of 512 to 1,024 tokens work best, roughly corresponding to one to three paragraphs. This size preserves enough context for meaningful semantic representation while keeping chunks focused enough to match specific queries. Implement a sliding-window approach with 20% overlap between consecutive chunks, ensuring that important information spanning chunk boundaries isn't lost:

```
def calculate_chunk_parameters(text: str,
                               target_tokens: int = 768,
                               overlap_tokens: int = 128) ->
List[dict]:
    """Calculate optimal chunking parameters for text."""
    pass
```

Indexing Strategy

The HNSW index provides superior query performance compared to IVFFlat for our use case. HNSW builds a multilayer graph structure that enables logarithmic search complexity, making it ideal for collections up to 10 million vectors. For a personal research tool with thousands of papers, HNSW offers near-instantaneous search with minimal memory overhead.

The index parameters are tuned for our specific needs. The `m` parameter (16) controls the number of bidirectional links per node, affecting both index size and search quality. This value provides a balance between search speed and index size on your local disk.

The `ef_construction` parameter (64) determines the search breadth during index construction. These values provide good recall (>0.95) while keeping index build time reasonable for periodic updates.

```
-- Monitor index statistics
CREATE OR REPLACE FUNCTION get_index_stats()
RETURNS TABLE (
    index_name TEXT,
    table_name TEXT,
    index_size TEXT,
    number_of_scans BIGINT,
    tuples_read BIGINT,
    tuples_fetched BIGINT
) AS $$
BEGIN
    -- Implementation omitted - see full chapter
END;
$$ LANGUAGE plpgsql;

-- Automatic index maintenance
CREATE OR REPLACE FUNCTION maintain_indexes() RETURNS void AS $$
BEGIN
    -- Implementation omitted - see full chapter
END;
$$ LANGUAGE plpgsql;
```

The schema also includes traditional B-tree and GIN indexes for metadata filtering. The GIN indexes on array columns (authors, categories) enable efficient containment queries, while trigram indexes support fuzzy text

matching. This combination allows hybrid searches that leverage both semantic similarity and traditional filters—essential for research queries such as “neural network papers by Yoshua Bengio published after 2020.”

Notice the `pdf_downloaded` and `pdf_processed` flags in the `papers` table. These are the “gears” of our system. In the following section, we will write the Python logic that checks these flags to determine what needs to be downloaded or indexed next.

ArXiv Integration and PDF Management

Now let’s integrate ArXiv and create a system for managing PDFs.

ArXiv API Client Implementation

The ArXiv API provides programmatic access to paper metadata and download URLs. The client implementation should respect rate limits, handle pagination elegantly, and recover from transient failures. The API uses the OAI-PMH protocol with custom query syntax for sophisticated searches:

```
import time
import logging
from typing import List, Dict, Optional, Generator
from datetime import datetime, timedelta
import arxiv
from dataclasses import dataclass
from urllib.parse import quote

logger = logging.getLogger(__name__)

@dataclass
class ArxivPaper:
    """Structured representation of an ArXiv paper."""
    arxiv_id: str
    title: str
    abstract: str
    authors: List[str]
    categories: List[str]
    primary_category: str
```

```
    published_date: datetime
    updated_date: datetime
    pdf_url: str
    comment: Optional[str] = None
    journal_ref: Optional[str] = None
    doi: Optional[str] = None

class ArxivClient:
    """Robust ArXiv API client with rate limiting and error
handling."""

    def __init__(self,
                 rate_limit_seconds: float = 3.0,
                 max_results_per_query: int = 100):
        pass

    def _rate_limit(self):
        """Enforce rate limiting between API calls."""
        pass

    def search_papers(self,
                      query: str,
                      max_results: int = 100,
                      sort_by: arxiv.SortCriterion = \
                          arxiv.SortCriterion.SubmittedDate,
                      sort_order: arxiv.SortOrder =
arxiv.SortOrder.Descending
                      ) -> Generator[ArxivPaper, None, None]:
        """
        Search for papers matching the query.

        Query examples:
        - 'cat:cs.LG' - Machine learning papers
        - 'au:Bengio' - Papers by author
        - 'ti:transformer' - Title contains word
        - 'cat:cs.LG AND (ti:attention OR ti:transformer)' -
Complex query
        """
        pass

    def fetch_by_ids(self, arxiv_ids: List[str]) ->
List[ArxivPaper]:
        """Fetch specific papers by their ArXiv IDs."""
        pass

    def search_recent_papers(self,
                             categories: List[str],
                             days_back: int = 7) ->
```

```
Generator[ArxivPaper, None, \
                                None]:
        """Fetch papers from specified categories published in
the last \
            N days."""
        pass
```

PDF Download Pipeline

PDF downloading requires careful handling of network issues, rate limits, and storage management. The pipeline should implement exponential backoff for retries, validate downloaded files, and organize them hierarchically for easy browsing:

```
import os
import hashlib
import requests
from pathlib import Path
from typing import Optional, Tuple
import logging

logger = logging.getLogger(__name__)

class PDFDownloader:
    """Manages PDF downloads with retry logic and
organization."""

    def __init__(self,
                 storage_path: str = "./data/pdfs",
                 max_retries: int = 3,
                 timeout: int = 30):
        pass

    def _get_pdf_path(self, arxiv_id: str, published_date:
datetime) -> Path:
        """Generate organized storage path for PDF."""
        pass

    def download_pdf(self,
                     pdf_url: str,
                     arxiv_id: str,
                     published_date: datetime,
                     force: bool = False) -> Tuple[bool,
Optional[Path], \
                         Optional[str]]:
```

```
        """
        Download PDF with retry logic.

        Returns: (success, pdf_path, error_message)
        """
        pass

    def _validate_pdf(self, pdf_path: Path) -> bool:
        """Check if file is a valid PDF."""
        pass

    def get_storage_stats(self) -> Dict[str, any]:
        """Get statistics about stored PDFs."""
        pass

    def cleanup_old_pdfs(self, days_to_keep: int = 90):
        """Remove PDFs older than specified days."""
        pass
```

Batch Processing System

Efficiently processing multiple papers requires a queue-based system that handles downloads, text extraction, and embedding generation asynchronously. This approach prevents any single failure from stopping the entire pipeline and allows for easy resumption after interruptions:

```
import asyncio
from concurrent.futures import ThreadPoolExecutor, as_completed
from typing import List, Optional
import psycopg2
from psycopg2.extras import RealDictCursor

class PaperProcessor:
    """Orchestrates the complete paper processing pipeline."""

    def __init__(self,
                 db_config: dict,
                 arxiv_client: ArxivClient,
                 pdf_downloader: PDFDownloader,
                 max_workers: int = 4):
        pass

    def process_papers_batch(self,
                             query: str,
                             max_papers: int = 100,
```

```
                            skip_existing: bool = True) -> dict:
        """Process a batch of papers from search query."""

        pass

def _upsert_paper(self, cursor, paper: ArxivPaper): -> int
   """Insert or update paper metadata, return paper ID."""

    # 1. Insert into the main 'papers' table (Populates the
TEXT[] array)
    cursor.execute("""
        INSERT INTO papers (arxiv_id, title, authors, categories,
...)
        VALUES (%s, %s, %s, %s, ...)
        ON CONFLICT (arxiv_id) DO UPDATE SET title =
EXCLUDED.title
        RETURNING id
    """, (paper.arxiv_id, paper.title, paper.authors,
paper.categories))

    paper_db_id = cursor.fetchone()['id']

    # 2. Sync the Relational Tables (Keep them in lock-step)
    for author_name in paper.authors:
        # Create the author if they don't exist
        cursor.execute("""
            INSERT INTO authors (name) VALUES (%s)
            ON CONFLICT (name) DO NOTHING
            RETURNING id
        """, (author_name,))

        # If the author existed, we need to fetch their ID
        res = cursor.fetchone()
        if res:
            author_id = res['id']
        else:
            cursor.execute("SELECT id FROM authors WHERE name =
%s", \
                (author_name,))
            author_id = cursor.fetchone()['id']

        # Link them in the junction table
        cursor.execute("""
            INSERT INTO paper_authors (paper_id, author_id)
            VALUES (%s, %s) ON CONFLICT DO NOTHING
        """, (paper_db_id, author_id))

    return paper_db_id
```

```
    def _process_queue(self, conn, stats: dict):
        """Process items from the queue with parallel
execution."""
        pass

    def _process_download(self, item: dict) -> bool:
        """Process a single download task."""
        pass
```

Once the `ArxivClient` retrieves the metadata and the `PDFDownloader` saves the file to disk, we move from structured JSON data to the "wild west" of unstructured PDF layouts.

PDF Text Extraction and Processing

Let's address how we tame these layouts into clean, searchable strings of text.

PDF Extraction Challenges

Academic PDFs present unique extraction challenges. Multicolumn layouts must be read in the correct order. Mathematical equations often render as images or use special fonts. Figures and tables interrupt text flow. References sections have different formatting. Headers, footers, and page numbers add noise. Some papers use nonstandard fonts or encodings, especially for mathematical symbols.

The extraction pipeline should address these challenges through intelligent heuristics and fallback strategies. Implement PyMuPDF's advanced layout analysis to identify reading order, filter out obvious noise like page numbers, and preserve structure markers that indicate section boundaries (see Figure 5-4 for the full pipeline):

```
import fitz  # PyMuPDF
import re
from typing import List, Dict, Optional, Tuple
```

```
import logging
from dataclasses import dataclass

logger = logging.getLogger(__name__)

@dataclass
class ExtractedPage:
    """Structured representation of extracted page content."""
    page_num: int
    text: str
    blocks: List[Dict]  # Text blocks with positions
    has_columns: bool
    has_math: bool
    has_tables: bool
    confidence: float  # Extraction confidence score

class PDFExtractor:
    """Advanced PDF text extraction for academic papers."""

    def __init__(self):
        pass

    def extract_paper_text(self, pdf_path: str) -> Dict[str,
any]:
        """Extract complete text from PDF with structure
preservation."""
        pass

    def _extract_page(self, page, page_num: int) ->
ExtractedPage:
        """Extract text from a single page with layout
analysis."""
        pass

    def _detect_columns(self, blocks: Dict) -> bool:
        """Detect if page has multi-column layout."""
        pass

    def _extract_multicolumn(self, blocks: Dict) -> List[Dict]:
        """Extract text from multi-column layout."""
        pass

    def _extract_singlecolumn(self, blocks: Dict) -> List[Dict]:
        """Extract text from single-column layout."""
        pass

    def _extract_block_text(self, block: Dict) -> str:
        """Extract text from a single block."""
```

```
        pass

    def _group_by_columns(self, blocks: Dict) ->
List[List[Dict]]:
        """Group blocks into columns based on x-position."""
        pass

    def _combine_blocks(self, blocks: List[Dict]) -> str:
        """Combine text blocks into continuous text."""
        pass

    def _is_noise(self, text: str) -> bool:
        """Detect if text is likely noise (header/footer/page
number)."""
        pass

    def _classify_block(self, text: str) -> str:
        """Classify block type (heading, body, caption, etc.)."""
        pass

    def _detect_tables(self, blocks: Dict) -> bool:
        """Detect presence of tables in the page."""
        pass

    def _identify_sections(self, text: str) -> List[Dict]:
        """Identify document sections and their positions."""
        pass

    def _clean_extracted_text(self, text: str) -> str:
        """Clean extracted text for better processing."""
        pass

    def _calculate_page_confidence(self,
                                   text: str,
                                   has_columns: bool,
                                   page_num: int) -> float:
        """Calculate confidence score for page extraction."""
        pass

    def _calculate_confidence(self, pages: List[ExtractedPage]) -
> float:
        """Calculate overall extraction confidence."""
        pass
```

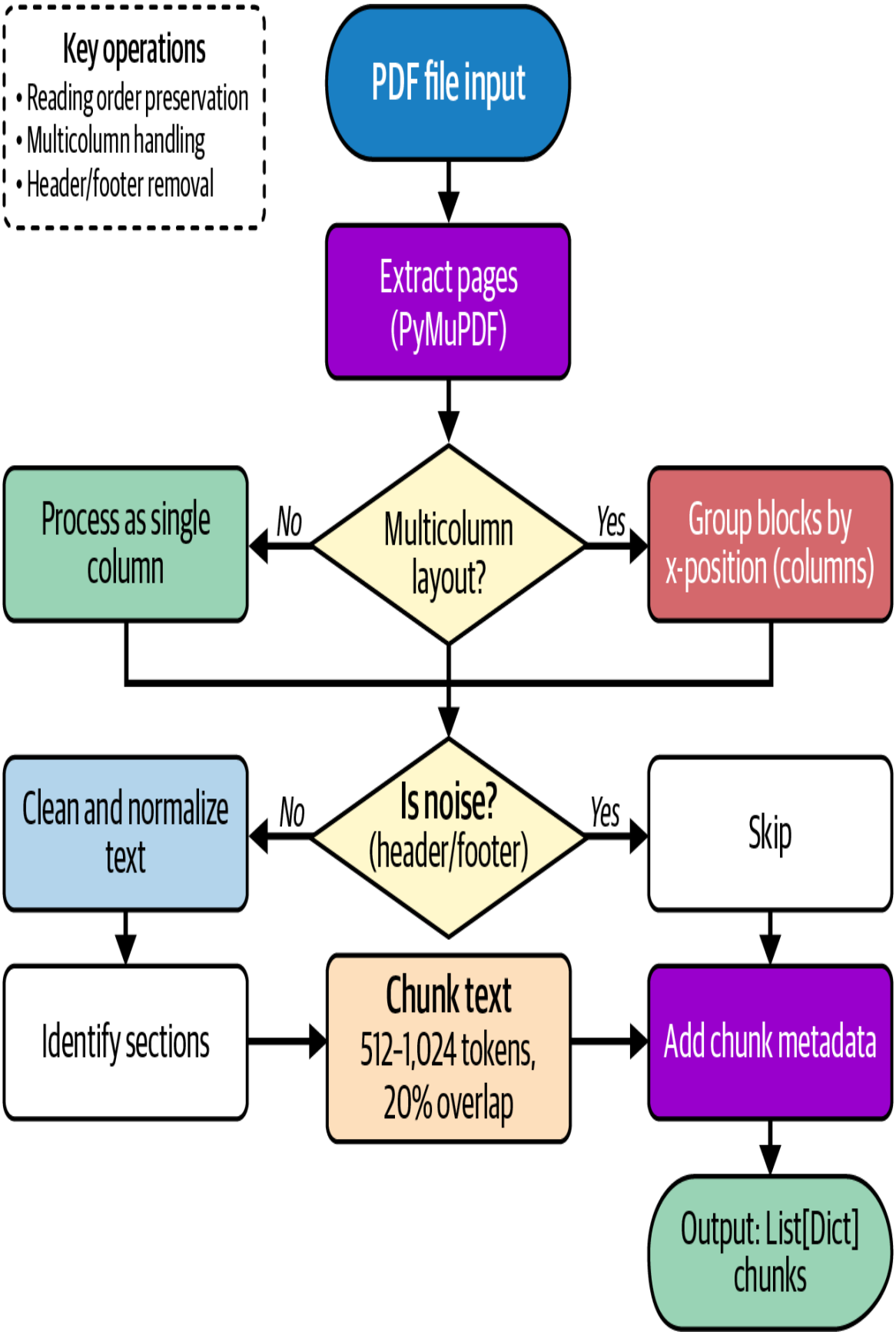

Key operations
• Reading order preservation
• Multicolumn handling
• Header/footer removal
PDF file input
Extract pages (PyMuPDF)
Multicolumn layout?
No
Yes
Process as single column
Group blocks by x-position (columns)
Is noise? (header/footer)
No
Yes
Clean and normalize text
Skip
Identify sections
Chunk text
512-1,024 tokens, 20% overlap
Add chunk metadata
Output: List[Dict] chunks

Figure 5-4. PDF processing pipeline

Intelligent Text Chunking

Chunking strategy significantly impacts search quality. Academic papers have natural structure—sections, paragraphs, sentences—that should be preserved when creating chunks. The implementation should use sliding windows with overlap, respect sentence boundaries, and maintain section context:

```
from typing import List, Dict, Optional
import nltk
from nltk.tokenize import sent_tokenize, word_tokenize
import logging

# Download required NLTK data
nltk.download('punkt', quiet=True)

logger = logging.getLogger(__name__)

class TextChunker:
    """Intelligent text chunking for academic papers."""

    def __init__(self,
                 target_chunk_size: int = 768,
                 min_chunk_size: int = 256,
                 max_chunk_size: int = 1024,
                 overlap_size: int = 128):
        pass

    def chunk_paper(self,
                    text: str,
                    sections: List[Dict],
                    preserve_sections: bool = True) -> List[Dict]:
        """
        Chunk paper text intelligently.

        Returns list of chunks with metadata.
        """
        pass

    def _chunk_text(self,
                    text: str,
                    section_name: Optional[str] = None) ->
```

```
List[Dict]:
        """Chunk text with sliding window and overlap."""
        pass

    def _get_overlap_sentences(self,
                               sentences: List[str],
                               target_tokens: int) -> List[str]:
        """Get sentences from end of chunk for overlap."""
        pass
```

Embedding Generation and Storage

Clean text is only half the battle. To enable semantic search, we must convert the human-readable chunks produced by our `TextChunker` into the 384-dimensional vectors expected by our PostgreSQL `vector` column. This is the role of the `EmbeddingGenerator`.

Embedding Model Strategy

The choice of embedding model profoundly impacts search quality. For academic papers, we need models that understand technical vocabulary, mathematical concepts, and research methodology. The all-MiniLM-L6-v2 model offers an excellent starting point with its 384-dimensional embeddings, providing good quality while remaining computationally efficient for a personal tool:

```
from sentence_transformers import SentenceTransformer
import numpy as np
from typing import List, Dict, Optional
import torch
import logging
from tqdm import tqdm
import psycopg2
from psycopg2.extras import execute_batch

logger = logging.getLogger(__name__)



class EmbeddingGenerator:
 """Efficient embedding generation with caching and batch
```

```
processing."""

    _instance = None      # Stores the actual object

    def __new__(cls, *args, **kwargs):

 if cls._instance is None:

 # 1. Create the instance
                      cls._instance = super(EmbeddingGenerator,
cls).__new__(cls)

                      # 2. Explicitly create the flag on the new
instance
                      cls._instance._initialized = False

           return cls._instance

    def __init__(self, model_name: str = 'all-MiniLM-L6-v2'):

# 3. Check our custom flag
           if self._initialized:
                      # We've been here before. Do nothing and
exit.
                      return

# 4. Perform the heavy lifting
        self.model = SentenceTransformer(model_name)

        # 5. Set the flag so we never do this again
        self._initialized = True
return


    def generate_embeddings(self,
                           texts: List[str],
                           show_progress: bool = True) ->
np.ndarray:
        """Generate embeddings for a list of texts."""
        pass

    def _preprocess_text(self, text: str) -> str:
        """Preprocess text before embedding generation."""
        pass

    def generate_query_embedding(self, query: str) -> np.ndarray:
        """Generate embedding for a search query."""
        pass
```

Batch Processing Pipeline

Processing thousands of paper chunks requires efficient batch operations. The pipeline should minimize memory usage, handle failures gracefully, and provide progress feedback:

```
class EmbeddingPipeline:
    """Complete pipeline for generating and storing
embeddings."""

    def __init__(self,
                 db_config: dict,
                 embedding_generator: EmbeddingGenerator,
                 batch_size: int = 100):
        pass

    def process_paper(self,
                      paper_id: int,
                      chunks: List[Dict]) -> Dict[str, any]:
        """Process all chunks for a single paper.
 chunks' here comes from the TextChunker in Section 6
           we are now preparing to send them to the DB defined in
Section 4"""

        pass

    def _store_chunks_with_embeddings(self,
                                      paper_id: int,
                                      chunks: List[Dict],
                                      embeddings: np.ndarray):
        """Store chunks and their embeddings in database."""
        pass

    def _detect_math(self, text: str) -> bool:
        """Detect mathematical content in text."""
        pass

    def _detect_code(self, text: str) -> bool:
        """Detect code snippets in text."""
        pass

    def _detect_references(self, text: str) -> bool:
        """Detect reference citations in text."""
        pass

    def process_pending_papers(self, limit: int = 10) ->
```

```
Dict[str, any]:
        """Process papers that need embedding generation."""
        pass
```

Similarity Search Implementation

With our database populated and our vectors indexed, we can finally move beyond keyword matching. The `PaperSearchEngine` implemented in this section is where the magic happens: it takes a user's natural language question and finds the most mathematically similar research papers in a few seconds. See Figure 5-5 for the full search architecture.

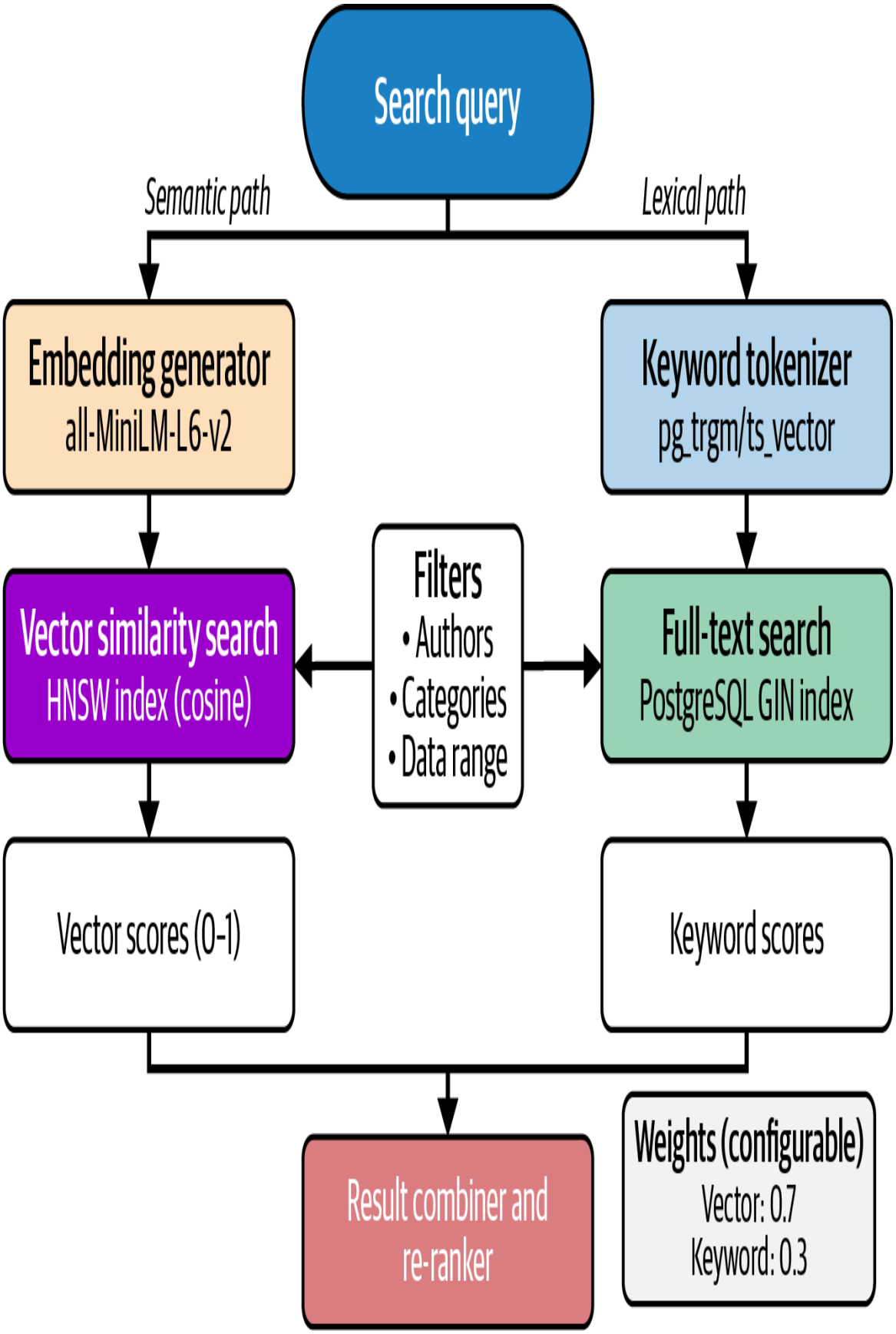

Search query
Semantic path
Lexical path
Embedding generator
all-MiniLM-L6-v2
Keyword tokenizer
pg_trgm/ts_vector
Filters
• Authors
• Categories
• Data range
Vector similarity search
HNSW index (cosine)
Full-text search
PostgreSQL GIN index
Vector scores (0-1)
Keyword scores
Result combiner and re-ranker
Weights (configurable)
Vector: 0.7
Keyword: 0.3

Figure 5-5. Hybrid search architecture

The search implementation should combine vector similarity with traditional filtering, enabling complex queries such as “papers about transformers by authors from MIT published after 2022.” The architecture should support multiple search modes—pure vector search for semantic similarity, hybrid search combining vectors with keyword matching, and metadata-filtered search for precise requirements:

```
import numpy as np
from typing import List, Dict, Optional, Tuple
import psycopg2
from psycopg2.extras import RealDictCursor
import logging
from dataclasses import dataclass
from enum import Enum

logger = logging.getLogger(__name__)

class SearchMode(Enum):
    VECTOR = "vector"
    HYBRID = "hybrid"
    KEYWORD = "keyword"

@dataclass
class SearchResult:
    """Structured search result."""
    paper_id: int
    arxiv_id: str
    title: str
    abstract: str
    authors: List[str]
    score: float
    matched_chunks: List[Dict]
    published_date: str
    categories: List[str]

class PaperSearchEngine:
    """Advanced search engine for academic papers."""

    def __init__(self,
                 db_config: dict,
                 embedding_generator: EmbeddingGenerator):
        pass
```

```
    def search(self,
               query: str,
               mode: SearchMode = SearchMode.HYBRID,
               limit: int = 10,
               filters: Optional[Dict] = None) ->
List[SearchResult]:
        """
        Main search interface supporting multiple modes.

        Filters can include:
        - authors: List[str] - Author names to filter
        - categories: List[str] - ArXiv categories
        - date_from: str - Minimum publication date
        - date_to: str - Maximum publication date
        - min_score: float - Minimum similarity score
        """
        pass

    def _vector_search(self,
                       query: str,
                       limit: int,
                       filters: Optional[Dict]) ->
List[SearchResult]:
        """Pure vector similarity search."""
        pass

    def _hybrid_search(self,
                       query: str,
                       limit: int,
                       filters: Optional[Dict]) ->
List[SearchResult]:
        """Hybrid search combining vector and keyword
matching."""
        pass

    def _keyword_search(self,
                        query: str,
                        limit: int,
                        filters: Optional[Dict]) ->
List[SearchResult]:
        """Traditional keyword search using PostgreSQL text
search."""
        pass

    def _build_filter_clause(self,
                             filters: Optional[Dict]) ->
Tuple[str, List]:
        """Build SQL filter clause from filter dictionary."""
```

```
        pass

    def _combine_results(self,
                         vector_results: List[SearchResult],
                         keyword_results: List[SearchResult],
                         vector_weight: float,
                         keyword_weight: float) ->
List[SearchResult]:
        """Combine and re-rank results from different search
methods."""
        pass

    def find_similar_papers(self,
                            paper_id: int,
                            limit: int = 10) ->
List[SearchResult]:
        """Find papers similar to a given paper."""
        pass
```

Interactive Application and UI

To search the scientific papers, we'll need an interactive application and UI. The CLI provides an intuitive interface for searching papers, managing the database, and exploring results. It supports multiple search modes, result export, and interactive paper exploration:

```
#!/usr/bin/env python3
"""
ArXiv Paper Search System - Interactive CLI
"""

import click
import sys
from typing import Optional, List
from tabulate import tabulate
from datetime import datetime, timedelta
import json

# Import our modules
from arxiv_client import ArxivClient
from pdf_processor import PDFDownloader, PDFExtractor
from text_chunker import TextChunker
from embeddings import EmbeddingGenerator, EmbeddingPipeline
from search import PaperSearchEngine, SearchMode
```

```
from database import DatabaseManager

@click.group()
@click.pass_context
def cli(ctx):
    """ArXiv Paper Search System - Personal Research Tool"""
    pass

@cli.command()
@click.option('--query', '-q', required=True, help='Search
query')
@click.option('--mode', '-m',
              type=click.Choice(['vector', 'hybrid', 'keyword']),
              default='hybrid',
              help='Search mode')
@click.option('--limit', '-l', default=10, help='Number of
results')
@click.option('--authors', '-a', multiple=True, help='Filter by
authors')
@click.option('--categories', '-c', multiple=True, help='Filter
by categories')
@click.option('--days', '-d', type=int, help='Papers from last N
days')
@click.option('--export', '-e', help='Export results to file')
@click.pass_context
def search(ctx, query, mode, limit, authors, categories, days,
export):
    """Search for papers"""
    pass

def display_results(results: List):
    """Display search results in a formatted table."""
    pass

def explore_results(ctx, results: List):
    """Interactive result exploration."""
    pass

def show_paper_details(result):
    """Show detailed information about a paper."""
    pass

def find_similar(ctx, result):
    """Find papers similar to the selected one."""
    pass

def download_papers(ctx, results):
    """Download PDFs for selected papers."""
```

```
    pass

def export_results(results: List, filename: str):
    """Export search results to JSON file."""
    pass

@cli.command()
@click.option('--categories', '-c', multiple=True, required=True,
              help='ArXiv categories to fetch (e.g., cs.LG,
cs.AI)')
@click.option('--days', '-d', default=7, help='Fetch papers from
last N days')
@click.option('--max-papers', '-m', default=100, help='Maximum
papers to fetch')
@click.pass_context
def fetch(ctx, categories, days, max_papers):
    """Fetch recent papers from ArXiv"""
    pass

@cli.command()
@click.pass_context
def stats(ctx):
    """Show database statistics"""
    pass

@cli.command()
@click.option('--init', is_flag=True, help='Initialize database
schema')
@click.option('--rebuild-index', is_flag=True, help='Rebuild
vector indexes')
@click.option('--cleanup', is_flag=True, help='Clean up old
data')
@click.pass_context
def manage(ctx, init, rebuild_index, cleanup):
    """Database management operations"""
    pass

if __name__ == '__main__':
    cli()
```

Docker Packaging for Local Deployment

To deploy the application, we’re going to use Docker packaging.

Container Architecture

Docker packaging enables consistent deployment across different environments and prepares the application for potential cloud migration. Our multicontainer architecture separates concerns—PostgreSQL with pgvector handles data persistence, while the Python application container manages processing and serving:

```
# Dockerfile for the application
FROM python:3.9-slim as builder

# Install build dependencies
RUN apt-get update && apt-get install -y \
    gcc \
    g++ \
    git \
    && rm -rf /var/lib/apt/lists/*

# Create virtual environment
RUN python -m venv /opt/venv
ENV PATH="/opt/venv/bin:$PATH"

# Install Python dependencies
COPY requirements.txt .
RUN pip install --upgrade pip && \
    pip install --no-cache-dir -r requirements.txt

# Download the embedding model during build
RUN python -c "from sentence_transformers import
SentenceTransformer; \
    model = SentenceTransformer('all-MiniLM-L6-v2'); \
    model.save('/opt/models/all-MiniLM-L6-v2')"

# Final stage
FROM python:3.9-slim

# ... (remainder of Dockerfile)

ENTRYPOINT ["python", "arxiv_search_cli.py"]
```

Docker Compose Configuration

Docker Compose orchestrates our multicontainer application, managing networking, volumes, and environment configuration:

```
# docker-compose.yml
version: '3.8'

services:
  postgres:
    image: pgvector/pgvector:pg15
    container_name: arxiv_postgres
    environment:
      POSTGRES_USER: ${DB_USER:-postgres}
      POSTGRES_PASSWORD: ${DB_PASSWORD:-password}
      POSTGRES_DB: ${DB_NAME:-arxiv_papers}
    volumes:
      - postgres_data:/var/lib/postgresql/data
      - ./init_scripts:/docker-entrypoint-initdb.d
    ports:
      - "${DB_PORT:-5432}:5432"
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U postgres"]
      interval: 10s
      timeout: 5s
      retries: 5
    restart: unless-stopped

  arxiv_app:
    build:
      context: .
      dockerfile: Dockerfile
    container_name: arxiv_search
    depends_on:
      postgres:
        condition: service_healthy
    environment:
      DB_HOST: postgres
      DB_PORT: 5432
      DB_NAME: ${DB_NAME:-arxiv_papers}
      DB_USER: ${DB_USER:-postgres}
      DB_PASSWORD: ${DB_PASSWORD:-password}
    volumes:
      - ./data/pdfs:/data/pdfs
      - ./data/cache:/data/cache
      - ./data/logs:/data/logs
    networks:
      - arxiv_network
    restart: unless-stopped

volumes:
  postgres_data:
    driver: local
```

```
networks:
  arxiv_network:
    driver: bridge
```

See Figure 5-6 for the Docker container topology.

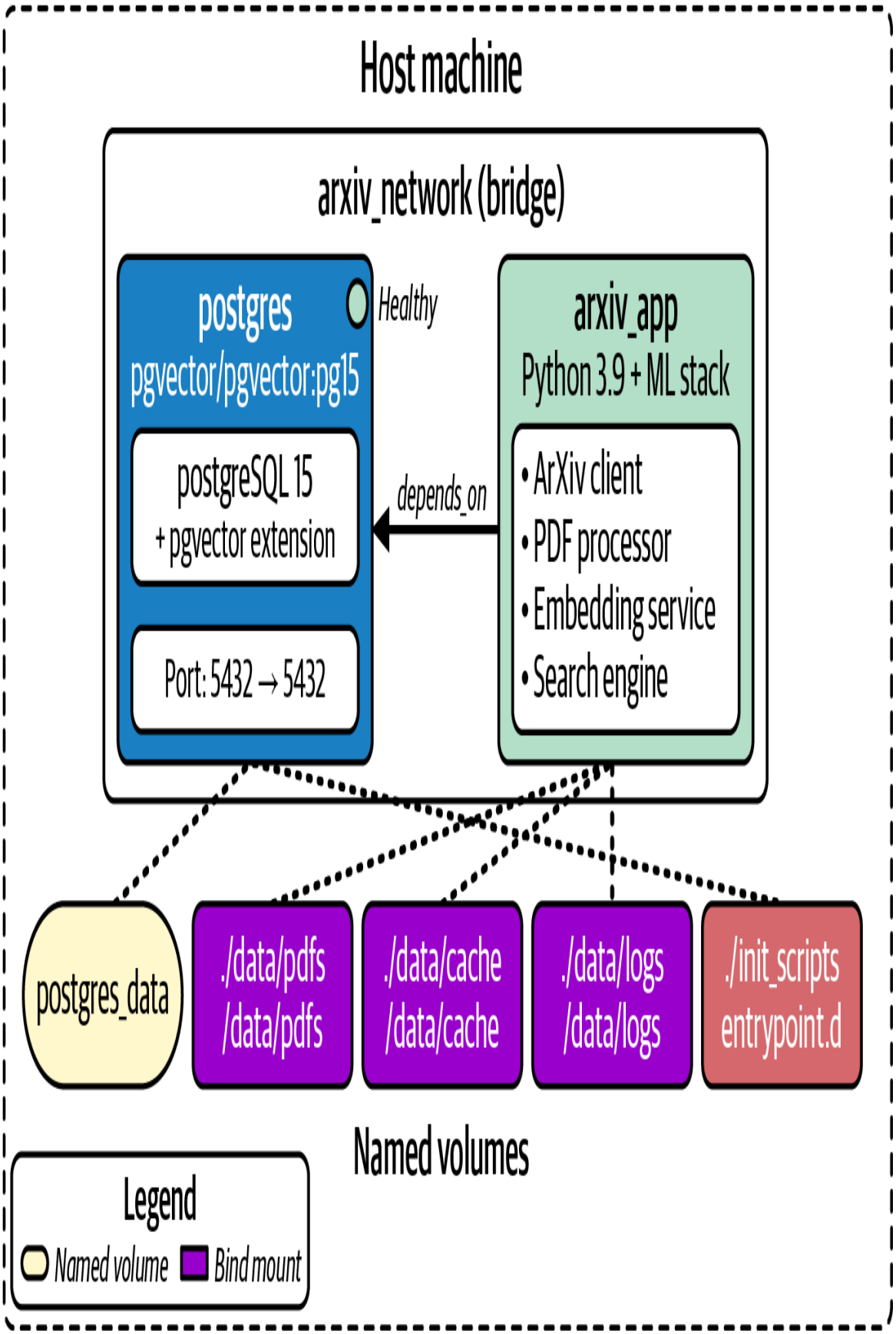

Host machine
arxiv_network (bridge)
postgres
pgvector/pgvector:pg15
Healthy
postgreSQL 15
+ pgvector extension
Port: 5432 → 5432
depends_on
arxiv_app
Python 3.9 + ML stack
• ArXiv client
• PDF processor
• Embedding service
• Search engine
postgres_data
./data/pdfs
/data/pdfs
./data/cache
/data/cache
./data/logs
/data/logs
./init_scripts
entrypoint.d
Named volumes
Legend
Named volume
Bind mount

Figure 5-6. Docker container topology

Database Initialization Scripts

Automated database setup ensures consistent schema creation and initial configuration:

```
-- init_scripts/01_create_extensions.sql
CREATE EXTENSION IF NOT EXISTS vector;
CREATE EXTENSION IF NOT EXISTS pg_trgm;
CREATE EXTENSION IF NOT EXISTS btree_gin;

-- init_scripts/02_create_schema.sql
-- Include the complete schema from Section 4

-- init_scripts/03_create_functions.sql
CREATE OR REPLACE FUNCTION update_updated_at()
RETURNS TRIGGER AS $$
BEGIN
    NEW.updated_at = CURRENT_TIMESTAMP;
    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

-- Apply trigger to all tables with updated_at
CREATE TRIGGER update_papers_updated_at
    BEFORE UPDATE ON papers
    FOR EACH ROW
    EXECUTE FUNCTION update_updated_at();
```

Development Workflow

The Docker setup supports both development and production workflows with hot reloading and debugging capabilities:

```
# docker-compose.dev.yml
version: '3.8'

services:
  arxiv_app:
    build:
      context: .
      dockerfile: Dockerfile.dev
      target: development
```

```
    volumes:
      - ./src:/app/src
      - ./scripts:/app/scripts
      - ./arxiv_search_cli.py:/app/arxiv_search_cli.py
    environment:
      PYTHONDONTWRITEBYTECODE: 1
      FLASK_ENV: development
      DEBUG: "true"
    command: python -m debugpy --listen 0.0.0.0:5678 \
        --wait-for-client arxiv_search_cli.py
    ports:
      - "5678:5678"  # Debugger port
```

Cloud-Ready Design

The containerized architecture facilitates cloud deployment with minimal changes:

```
#!/bin/bash
# deploy.sh - Deployment script

# Build and tag images
docker build -t arxiv-search:latest .
docker tag arxiv-search:latest ${REGISTRY}/arxiv-
search:${VERSION}

# Push to registry
docker push ${REGISTRY}/arxiv-search:${VERSION}

# Deploy to cloud (example for AWS ECS)
aws ecs update-service \
    --cluster arxiv-cluster \
    --service arxiv-search-service \
    --force-new-deployment

# Or deploy to Kubernetes
kubectl apply -f k8s/deployment.yaml
kubectl rollout status deployment/arxiv-search
```

Basic Performance Tuning

Here is some guidance to improve the performance of your new tool.

Index Configuration

For a personal research tool, index configuration focuses on balancing search quality with reasonable resource usage:

```
def optimize_indexes(conn):
    """Optimize indexes for typical personal use patterns."""
    pass
```

Query Performance

```
Simple optimizations significantly improve search responsiveness:
class QueryOptimizer:
    """Query performance optimizations."""

    def __init__(self, db_config):
        pass

    def cached_search(self, query: str, **kwargs):
        """Search with simple caching."""
        pass
```

Resource Management

You can manage memory and disk usage for sustainable long-term operation:

```
def manage_resources():
    """Monitor and manage system resources."""
    pass
```

Next Steps

Let’s talk about next steps now that we have a functional tool.

Current Limitations

While functional, the system has several limitations to acknowledge. PDF extraction sometimes struggles with complex layouts, especially papers

with extensive mathematical notation or unusual formatting. The system currently processes only English-language papers effectively. Updates require manual triggering rather than automatic monitoring of new publications.

Enhancement Ideas

The system provides a solid foundation for numerous enhancements, organized by area of improvement.

Content improvements could include better equation handling using specialized OCR for mathematical notation, extraction of figure captions and their association with relevant text, parsing of reference sections for citation graph construction, and highlighting of key findings in abstracts.

Search enhancements offer rich possibilities: implementing search history tracking to understand usage patterns, creating a favorites collection for important papers, adding custom tags and notes for personal organization, and developing query templates for common search patterns.

Workflow features would streamline research tasks: watching specific authors for new publications, tracking new papers in selected categories, exporting to reference managers like Zotero or Mendeley, and integration with note-taking applications like Obsidian or Roam.

Quality-of-life improvements focus on daily usability: better duplicate detection using fuzzy matching, smarter chunking that preserves more context, relevance feedback to improve search results over time, and a personal reading queue with progress tracking.

Deployment options prepare for different use cases: cloud migration to AWS, GCP, or Azure for accessibility; Kubernetes orchestration for scalability; managed database services for maintenance reduction; and CDN integration for faster PDF access.

What We Did

We've built a complete ArXiv paper search system that transforms PDF academic papers into a searchable knowledge base. The system successfully addresses the core challenge of semantic search in scientific literature, enabling researchers to find relevant papers based on meaning rather than just keywords.

System Achievements

The application delivers a fully functional pipeline from the ArXiv API to semantic search results. It handles real-world challenges like PDF text extraction, intelligent text chunking, and efficient vector storage. The hybrid search capability combines the best of semantic and keyword matching, while the Docker packaging ensures consistent deployment across environments.

Technical Skills Gained

Building this system provides hands-on experience with modern vector database techniques, from embedding generation to similarity search. You've learned to handle unstructured data at scale, implement error handling, and create intuitive interfaces for complex functionality. The PostgreSQL pgvector integration demonstrates how traditional databases can be enhanced with vector capabilities.

Practical Research Tool

Beyond the technical learning, you've created a genuinely useful tool for academic research. The system can help with literature reviews, finding related work, discovering cross-disciplinary connections, and staying current with rapid developments in your field. The personal focus means it's tailored to individual research needs rather than trying to serve everyone.

Foundation for Advanced Systems

This implementation provides the groundwork for more sophisticated applications. The patterns established here—text chunking, embedding generation, vector storage, and hybrid search—form the basis for RAG systems explored in later chapters. The modular architecture makes it straightforward to swap components, experiment with different models, or add new features.

Future Potential

The system's potential extends far beyond current capabilities. With the foundation in place, you could build citation networks, implement research trend analysis, create automated literature review assistants, or develop collaborative research tools. The Docker packaging and cloud-ready design mean the system can grow from a personal tool to a team resource when needed.

The journey from PDF to semantic search encapsulates the power of vector databases in making unstructured information accessible and useful. This ArXiv search system demonstrates that sophisticated AI-powered tools are within reach of individual developers, democratizing access to advanced research capabilities.

Conclusion

By building this for your personal computer, you have implemented a core architecture similar to the ones used by larger-scale AI engines. You have successfully navigated the "unstructured-to-structured pipeline"—a foundational skill in modern AI engineering.

OceanofPDF.com

Chapter 6. Building a Retrieval-Augmented Generation System with SQLite VSS and Ollama

In previous chapters, we focused on individual components of vector search: creating embeddings and performing similarity queries. Now it’s time to assemble these pieces into a functional retrieval-augmented generation (RAG) system.

Unlike production-scale web applications that rely on distributed cloud clusters, our goal here is to build a high-performance, private, entirely local RAG system that runs on a single desktop. We will use SQLite VSS for our search engine and Ollama as our local LLM “brain.”

RAG systems solve a fundamental limitation of LLMs: their knowledge is frozen at training time, preventing access to private or recent information. By augmenting LLMs with a retrieval mechanism, we create a system that answers questions using up-to-date, domain-specific knowledge.

Our goal is to build a question-answering system that responds intelligently to queries about Reddit content. When a user asks a question, the system will (1) search stored Reddit posts to find the most relevant information, (2) retrieve the best-matching content chunks, (3) provide this context to an LLM, and (4) generate a natural language answer based solely on the retrieved information.

This approach ensures factual accuracy by grounding the LLM’s responses in actual data, significantly reducing (though not eliminating) hallucinations and enabling the system to work with private or specialized content absent from the LLM’s training data.

System Architecture Overview

Our RAG system consists of five main components working in concert:

Vector database layer (SQLite VSS)

: Stores content chunks with their embeddings, enabling fast similarity search

Embedding engine (`SentenceTransformers`*)*

: Converts text into dense vector representations that capture semantic meaning

Hybrid search system

: Combines semantic vector search with traditional keyword search for optimal retrieval

LLM integration (Ollama)

: Provides local LLM inference for generating natural language responses

RAG pipeline orchestrator

: Coordinates the retrieval and generation process

The data flow follows this path: content ingestion → text chunking → embedding generation → vector storage → query processing → hybrid search → context retrieval → prompt construction → LLM generation. See Figure 6-1.

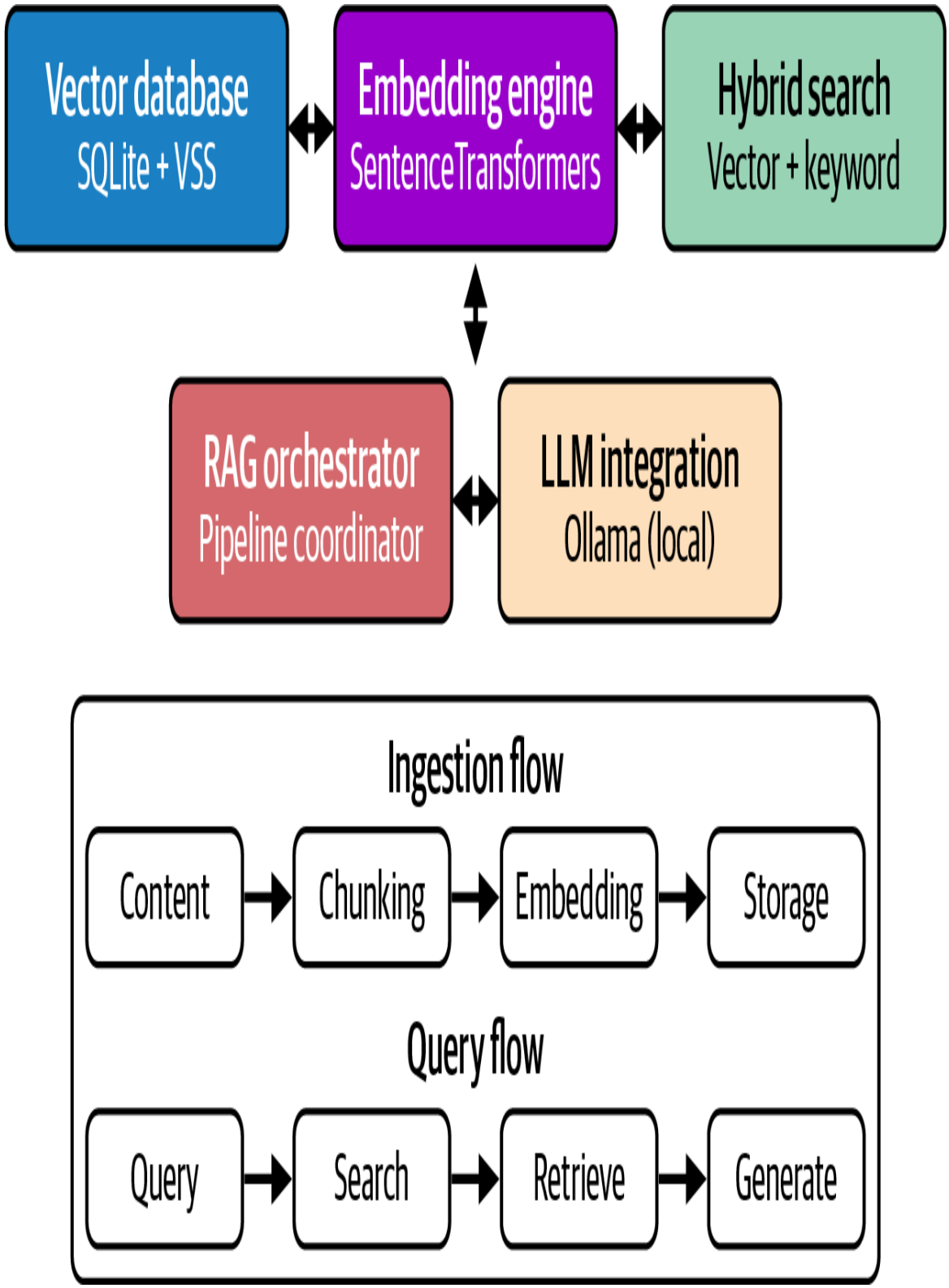


Figure 6-1. System architecture overview

Let’s build each component step by step.

Database Foundation with Vector Support

First, we need to set up our database infrastructure. We're using SQLite with the VSS extension, which adds vector search capabilities to SQLite. This gives us a lightweight, embedded solution perfect for RAG applications:

```
# Chapter 6: Minimal RAG System with SQLite VSS and Ollama
# Simplified version demonstrating core retrieval-augmented
generation
import sqlite3
import ollama
import requests
import json
import time
import hashlib
from sentence_transformers import SentenceTransformer
from typing import List, Dict, Optional
```

The imports reveal our technology stack: SQLite for storage, requests for API calls to Ollama, `SentenceTransformers` for embeddings, and standard Python libraries for data handling.

Setting Up the Vector-Enabled Database

Because we are building an experimental desktop app, we are using SQLite's powerful extension system to "teach" it vector math. SQLite VSS is not part of the standard Python sqlite3 library; it requires loading an external compiled library. This effectively transforms our local file-based database into a high-performance semantic engine.

The database setup creates the foundation for both traditional data storage and vector operations:

```
def setup_database(db_path='reddit_rag.db'):
    """Setup SQLite with VSS extension and create tables"""
    conn = sqlite3.connect(db_path)
    conn.row_factory = sqlite3.Row  # Return rows as dictionaries
```

```
    # Enable and load VSS extension
    conn.enable_load_extension(True)
    try:
        conn.load_extension("./vss0")
                conn.load_extension("./vector0")
        print("VSS extension loaded successfully")
    except Exception as e:
        print(f"Error loading VSS: {e}")
        print("Download from: https://github.com/asg017/sqlite-
vss/releases")
        raise
```

The VSS extension transforms SQLite from a traditional relational database into a hybrid vector database. Without this extension, we couldn't perform efficient similarity searches on embeddings. The `row_factory` setting makes our code cleaner by returning query results as dictionaries instead of tuples.

> **NOTE**
>
> sqlite-vss is not officially supported on Windows as of this edition's release. You can run the Linux version of sqlite-vss inside WSL2. This is the most stable way to use it on a Windows machine.
>
> For the definitive current way to load the extension, see the GitHub repository at https://github.com/asg017/sqlite-vss.

Schema Design for RAG

Our schema needs to support three key requirements—storing original content, managing chunked text, and enabling both vector and keyword search:

```
    cursor = conn.cursor()

    # Create posts table
    cursor.execute("""
        CREATE TABLE IF NOT EXISTS posts (
            post_id TEXT PRIMARY KEY,
```

```
            created_at TIMESTAMP DEFAULT CURRENT TIMESTAMP,
            title TEXT,
            content TEXT,
            subreddit TEXT,
            author TEXT,
            score INTEGER
        )
    """)
```

The `posts` table stores the original Reddit content with metadata. This serves as our source of truth and provides context for the chunks:

```
    # Create chunks table with vector support
    cursor.execute("""
        CREATE TABLE IF NOT EXISTS content_chunks (
            chunk_id INTEGER PRIMARY KEY,
            created_at TIMESTAMP DEFAULT CURRENT TIMESTAMP,
            post_id TEXT,
            chunk_index INTEGER,
            content TEXT,
            chunk_vector BLOB,  -- Store as BLOB for VSS
            FOREIGN KEY (post_id) REFERENCES posts(post_id)
        )
    """)
```

The `content_chunks` table is where the magic happens. Each piece of content is broken into smaller chunks (essential for context window management), and each chunk has its own embedding stored as a BLOB. The foreign key relationship maintains the connection to the original post.

Creating Search Indexes

To enable efficient search, we create two specialized indexes:

```
    # Create VSS virtual table for vector search
    cursor.execute("""
        CREATE VIRTUAL TABLE IF NOT EXISTS chunk_vss USING vss0(
            chunk_vector(384),
            chunk_id INTEGER
        )
```

```
    """)

    # Create FTS5 for keyword search
    cursor.execute("""
        CREATE VIRTUAL TABLE IF NOT EXISTS chunks_fts USING fts5(
            content,
            content='content_chunks',
            content_rowid='chunk_id'
        )
    """)

    conn.commit()
    return conn
```

The VSS virtual table enables vector similarity search with 384-dimensional vectors (matching our embedding model). The FTS5 virtual table provides full-text search capabilities. Together, they power our hybrid search system.

VSS uses a flat brute-force index, which will work well for small, personal datastores but will hit performance limits as the data size approaches ~100,000 vectors (see Figure 6-2).

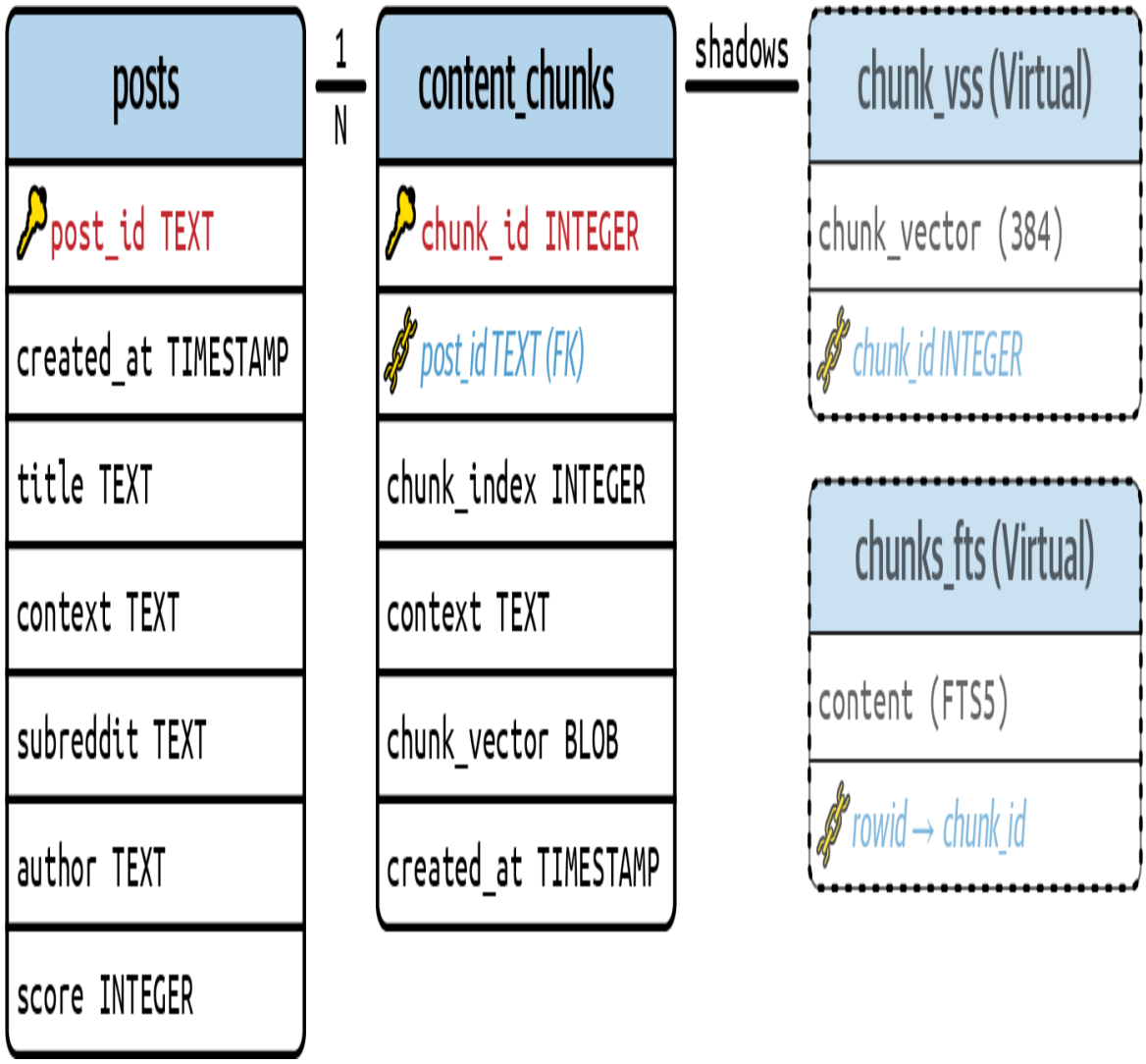


Figure 6-2. Entity relationship diagram

Text Processing and Embedding Generation

RAG systems need to break content into manageable chunks and convert them into searchable vectors. This section handles both tasks.

The full end-to-end ingestion pipeline is shown in Figure 6-3.

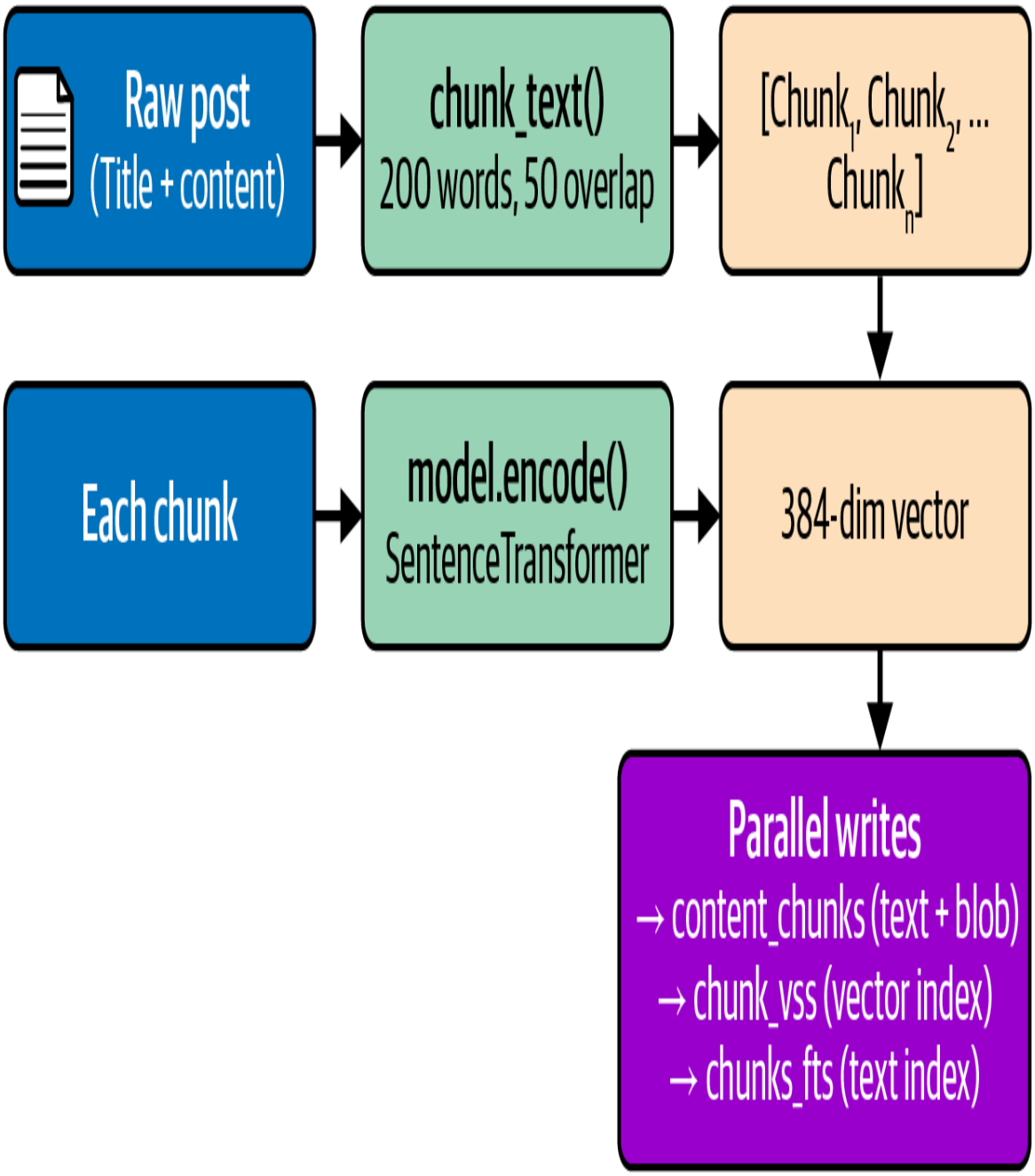


Figure 6-3. Data ingestion pipeline

Embedding Model Management

We use a singleton pattern for the embedding model to avoid the overhead of loading it multiple times:

```
# Initialize embedding model globally
```

```
embedding_model = None

def get_embedding_model():
    """Get or initialize the embedding model
Note: first call will take longer as it is loaded from disk"""
    global embedding_model
    if embedding_model is None:
        embedding_model = SentenceTransformer('all-MiniLM-L6-v2')
        print(f"Loaded embedding model (dimension: \

{embedding_model.get_sentence_embedding_dimension()})")
    return embedding_model
```

The all-MiniLM-L6-v2 model is chosen for its balance of quality and speed. It produces 384-dimensional embeddings and is small enough to run efficiently on a CPU.

Intelligent Text Chunking

Chunking is critical for RAG performance. Too small and we lose context; too large and we dilute relevance:

```
def chunk_text(text, chunk_size=200, overlap=50):
    """Simple text chunking by word count"""
    words = text.split()
    chunks = []

    for i in range(0, len(words), chunk_size - overlap):
        chunk = ' '.join(words[i:i + chunk_size])
        if chunk:
            chunks.append(chunk)

    return chunks
```

The overlap ensures continuity between chunks, preventing important information from being split across boundaries. A 200-word chunk with 50-word overlap provides good context while maintaining specificity.

> **NOTE**
>
> This simple chunking approach is for initial prototyping. If you find retrieved data unsatisfactory, consider using semantic chunking (e.g., LangChain's `RecursiveCharacterTextSplitter`) that respects sentence boundaries. For initial learning this may not be necessary, but it's a great way to extend the system's capabilities gradually and make it your own. There is ongoing research and experimentation in tuning chunking strategies, so this is by no means a settled solution.

See Figure 6-4 for a visual representation of chunking with overlap.

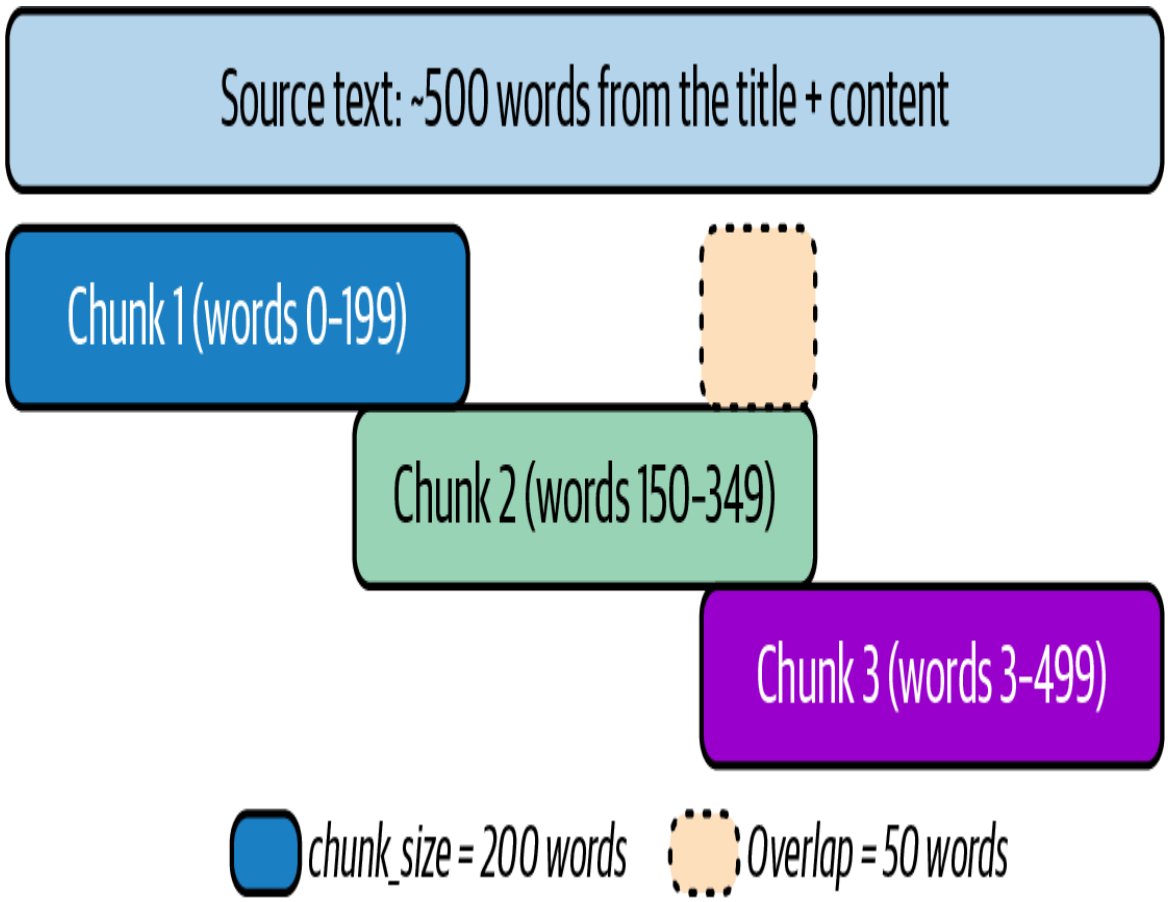


Figure 6-4. Text chunking with overlap

Storing Content with Embeddings

This function orchestrates the entire ingestion pipeline:

```
def store_post_with_chunks(conn, post_data):
    """Store a post and its chunks with embeddings"""
    cursor = conn.cursor()

    # Store post
    cursor.execute("""
        INSERT OR REPLACE INTO posts (post_id, title, content,
subreddit, \
            author, score)
        VALUES (?, ?, ?, ?, ?, ?)
    """, (
        post_data['post_id'],
        post_data['title'],
        post_data['content'],
        post_data['subreddit'],
        post_data['author'],
        post_data.get('score', 0)
    ))
```

First, we store the original post with all its metadata. The `INSERT OR REPLACE` pattern handles updates gracefully:

```
    # Create chunks from title + content
    full_text = f"{post_data['title']}\n\n{post_data['content']}"
    chunks = chunk_text(full_text)

    # Get embedding model
    model = get_embedding_model()
```

We combine title and content because the title often contains important context that helps us understand the content:

```
    # Store each chunk with embedding
    for idx, chunk_content in enumerate(chunks):
        # Generate embedding
        embedding = model.encode(chunk_content)

        # Store chunk
        cursor.execute("""
            INSERT INTO content_chunks (post_id, chunk_index,
content, \
```

```
                chunk_vector)
            VALUES (?, ?, ?, ?)
        """, (
            post_data['post_id'],
            idx,
            chunk_content,
            embedding.tobytes()
        ))

        chunk_id = cursor.lastrowid
```

Each chunk is embedded and stored with its vector representation. The `tobytes()` conversion is necessary for BLOB storage:

```
        # Add to VSS index
        vector_str = f"vss_create_vector({','.join(map(str,
embedding))})"
        cursor.execute(f"""
            INSERT INTO chunk_vss (chunk_id, chunk_vector)
            VALUES (?, {vector_str})
        """, (chunk_id,))

        # Add to FTS index
        cursor.execute("""
            INSERT INTO chunks_fts (rowid, content)
            VALUES (?, ?)
        """, (chunk_id, chunk_content))

    conn.commit()
    print(f"Stored post {post_data['post_id']} with {len(chunks)}
chunks")
```

We populate both search indexes—VSS for semantic search and FTS5 for keyword search. This dual indexing enables our hybrid search approach.

Hybrid Search Implementation

A common mistake in RAG is assuming that simple vector search is always enough. To build a robust system, we need to handle cases where a user might be searching for a specific technical term or an exact phrase that embeddings might “blur” together.

Hybrid search combines the strengths of semantic and keyword search. Semantic search understands meaning and context, while keyword search ensures that specific terms aren’t missed.

Hybrid Search Algorithm

Here’s the hybrid search algorithm:

```
def hybrid_search(conn, query_text, limit=5,
semantic_weight=0.7):
    """Perform hybrid search combining vector and keyword
search"""
    cursor = conn.cursor()
    model = get_embedding_model()

    # Generate query embedding
    query_embedding = model.encode(query_text)
    vector_str = f"vss_create_vector({','.join(map(str,
query_embedding))})"
```

We start by converting the query into an embedding using the same model used for the chunks. This ensures compatibility in the vector space:

```
def hybrid_search(query_text, query_vector, limit=5):
    sem_res = get_semantic_results(query_vector, limit)
    key_res = get_keyword_results(query_text, limit)

    # Normalize keyword scores by dividing by the maximum score
in the set
    max_key = max([r['score'] for r in key_res]) if key_res else
1.0
    for r in key_res: r['score'] /= max_key

    # Merge and weight: (Semantic * 0.7) + (Keyword * 0.3)
    merged = {}
    for r in sem_res:
        merged[r['id']] = r['score'] * 0.7
    for r in key_res:
        merged[r['id']] = merged.get(r['id'], 0) + (r['score'] *
0.3)
```

```
    # Sort by the final hybrid score
    sorted_ids = sorted(merged.items(), key=lambda x: x[1],
reverse=True)[:limit]
    return [id for id, score in sorted_ids]
```

See “Score Fusion and Ranking” for an explanation of the 0.7 and 0.3 coefficients.

Semantic Search Component

Here’s how to do the semantic search component:

```
    def get_semantic_results(query_vector, limit=5):
       # Semantic search using VSS
    cursor = conn.cursor()
    # We fetch limit * 2 to create a larger 'candidate pool' for
our hybrid merge
    later.
    query = """
        SELECT rowid, vss_cosine_distance(embedding, ?) as
distance
        FROM vss_posts
        WHERE vss_search(embedding, ?)
        ORDER BY distance ASC
        LIMIT ?
    """
    cursor.execute(query, (query_vector, query_vector, limit *
2))
    return [{"id": r[0], "score": 1 - r[1]} for r in
cursor.fetchall()]
```

In the preceding query, we use the `vss_cosine_distance` function. It is important to distinguish between distance and similarity. SQLite VSS returns cosine distance, where a value of `0` means the vectors are identical. To use this in a hybrid ranking system where higher is better, we convert it to cosine similarity using the formula `1 - distance`.

You’ll also notice we fetch `limit * 2` results. Since we are later merging these results with a separate keyword search, we pull a larger “candidate pool” from both systems. This ensures that even if the two search methods

return completely different sets of documents, we still have enough high-quality samples to produce a meaningful top-5 list for the LLM.

See Figure 6-5 for an intuitive representation of semantic search.

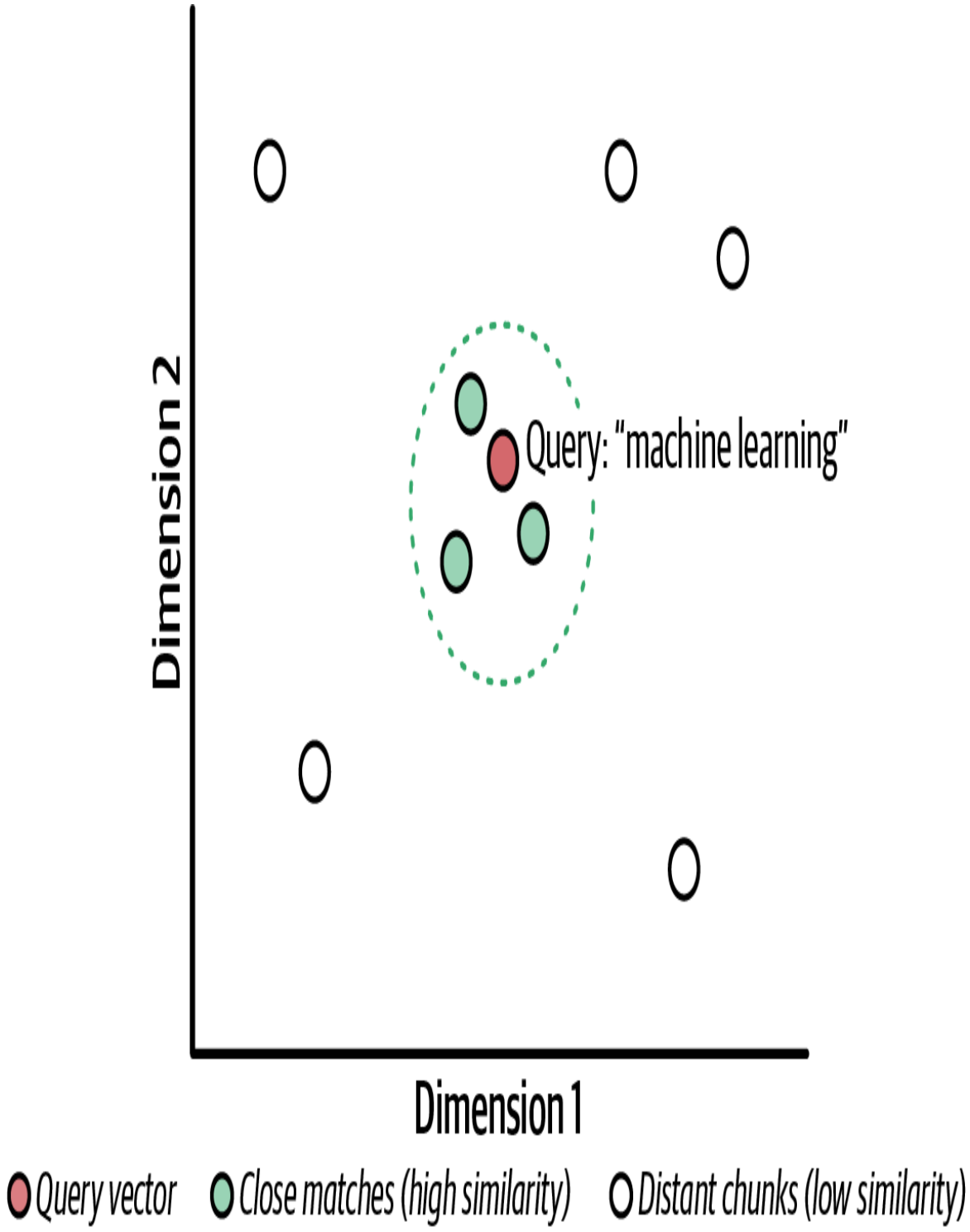


Figure 6-5. Vector space similarity search

Keyword Search Component

Here’s how to do the keyword search using FTS5:

```
    # Keyword search using FTS5
    keyword_results = {}

    def get_keyword_results(query_text, limit=5):
    cursor = conn.cursor()
    try:
        # FTS5 uses the bm25() function for ranking
        query = "SELECT rowid, bm25(posts_fts) FROM posts_fts
WHERE posts_fts \
            MATCH ? LIMIT ?"
        cursor.execute(query, (query_text, limit * 2))
        results = cursor.fetchall()
        # BM25 scores are usually negative in SQLite; we invert
for a 'higher is
        better' score
        return [{"id": r[0], "score": -r[1]} for r in results]
    except sqlite3.OperationalError as e:
        print(f"Warning: Keyword search failed. Ensure FTS5 index
is \
            populated. Error: {e}")
        return []
```

The FTS5 full text search extension uses Best Matching 25 (BM25). While simple term frequency just counts how many times a word appears, BM25 is a probabilistic ranking function. It accounts for saturation (mentioning a word 100 times doesn’t make a document 100 times more relevant) and document length (shorter documents are rewarded for being concise). This makes it significantly more effective than basic keyword matching for technical datasets.

The `try-except` block handles cases where the keyword query returns no results.

Score Fusion and Ranking

The fusion strategy (see Figure 6-6) normalizes scores from both search methods and combines them using weighted averaging. The `semantic_weight` parameter (default 0.7) gives more importance to semantic similarity while still considering exact keyword matches.

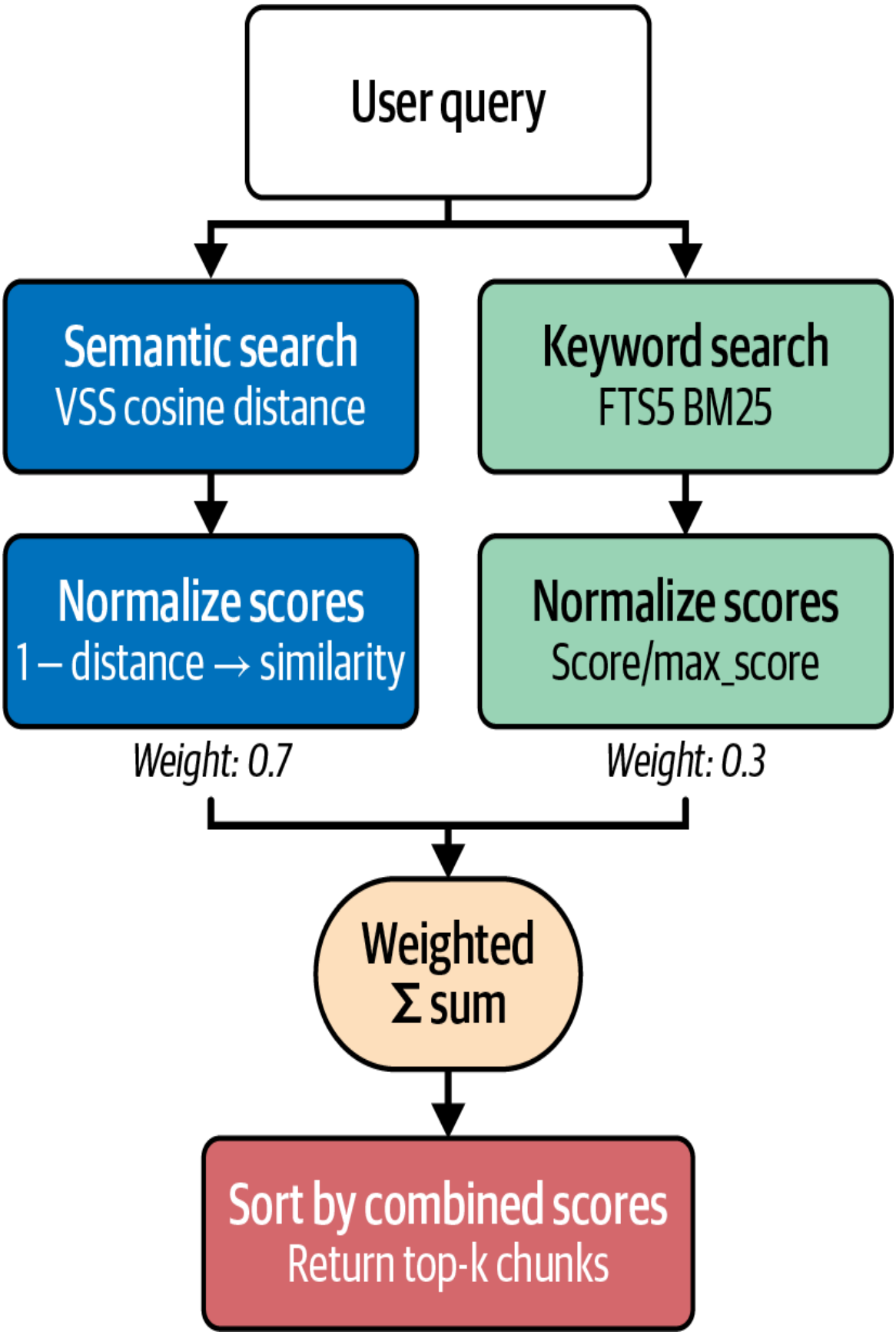

User query
Semantic search
VSS cosine distance
Keyword search
FTS5 BM25
Normalize scores
1 – distance → similarity
Normalize scores
Score/max_score
Weight: 0.7
Weight: 0.3
Weighted
Σ sum
Sort by combined scores
Return top-k chunks

Figure 6-6. Hybrid search score fusion

A common pitfall in hybrid search is *score dominance*. BM25 scores are unbounded (often ranging from 0 to 20+), while cosine similarity is bounded between 0 and 1. To compare them fairly, we normalize the keyword scores by dividing each by the maximum score in the current result set.

We then apply a *weighted average*. In the code for `hybrid_search` earlier, we use a `semantic_weight` of `0.7` and a `keyword_weight` of `0.3`. This 70/30 split is a standard starting point for RAG systems, prioritizing conceptual meaning while allowing specific technical keywords to "nudge" the results. If you find your search is too vague, consider increasing the keyword weight.

LLM Integration with Ollama

Ollama provides local LLM inference, ensuring privacy and eliminating API costs. Our integration is straightforward but robust.

Ollama API Client

The following code calls the Ollama API client:

```
def call_ollama(prompt, model="llama3.1:8b", temperature=0.1):
    """Simple Ollama API call"""
    url = http://localhost:11434/api/generate

    payload = {
        "model": model,
        "prompt": prompt,
        "stream": False,
        "options": {
            "temperature": temperature,
            "top_p": 0.9
        }
    }

    try:
```

```
        response = requests.post(url, json=payload)
        response.raise_for_status()
        return response.json()['response']
    except Exception as e:
        print(f"Ollama error: {e}")
        return f"Error calling Ollama: {str(e)}"
```

We use a low temperature (0.1) to ensure consistent, factual responses based on the provided context. The model choice (llama3.1:8b) balances quality with inference speed.

Temperature is a measure of randomness in LLM generation—higher temperature creates more variability, which is useful for creative output, but in this case we want the facts and concepts in our data returned as faithfully as possible without modification of meaning. So we keep it low.

> **NOTE**
>
> Ollama defaults to 2,048 tokens. If your prompt + retrieved chunks exceed this, they will be truncated silently. Add `"num_ctx": 4096` (or higher) to `options` to expand the context.

Health Check Function

Here is the health check function:

```
def test_ollama():
    """Test if Ollama is running"""
    try:
        response =
requests.get("http://localhost:11434/api/tags")
        models = response.json()
        print("Available Ollama models:", [m['name'] for m in \
            models.get('models', [])])
        return True
    except:
        print("Ollama not running. Start with: ollama serve")
        return False
```

This utility function verifies Ollama is running and shows available models, helping with debugging and setup.

The RAG Pipeline

This is where everything comes together. The RAG pipeline orchestrates retrieval and generation to answer questions.

See Figure 6-7 for the sequence of steps constituting the RAG pipeline.

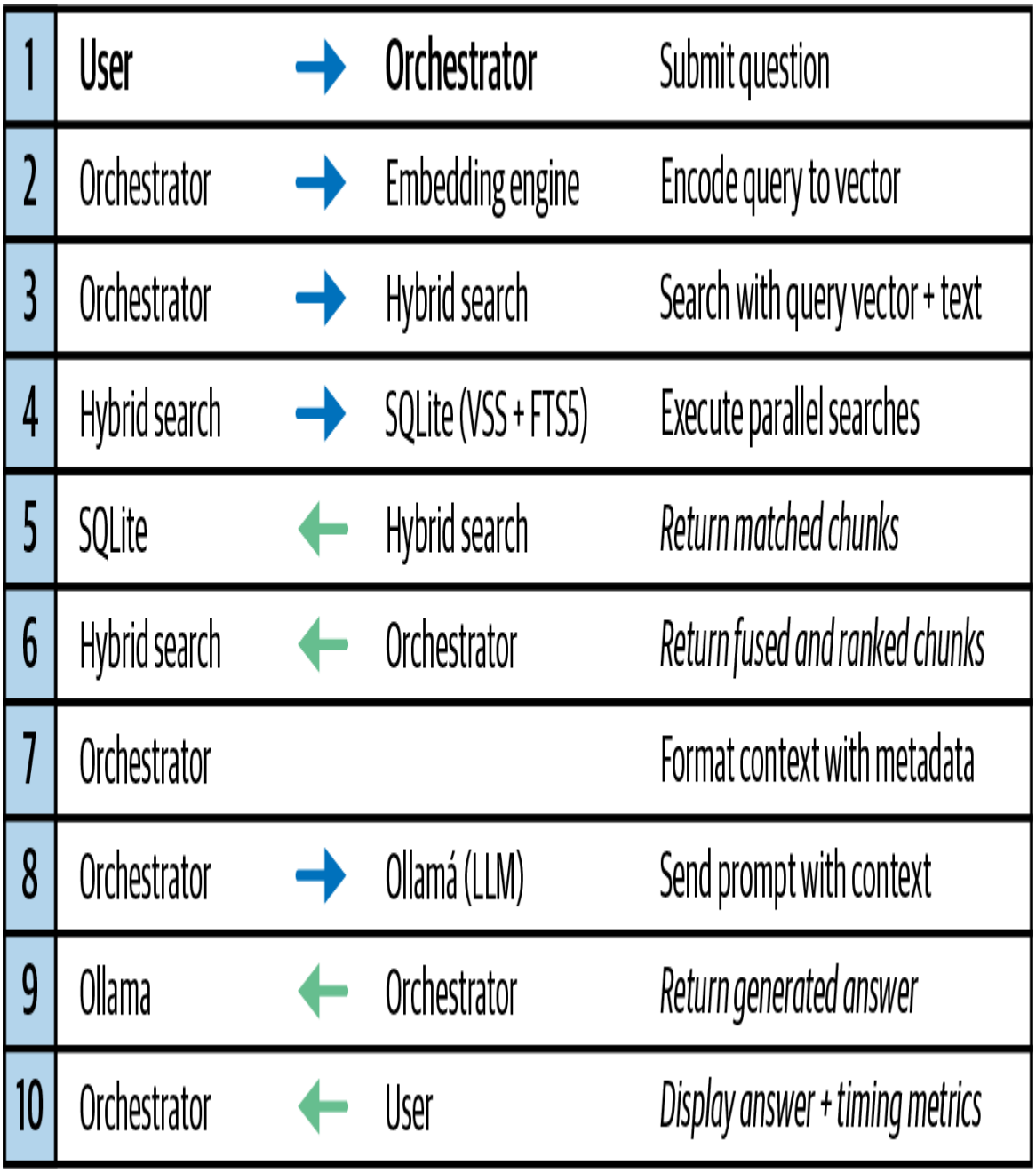


Figure 6-7. RAG pipeline sequence

Context Formatting

Once we have our top candidates, we must format them into a context window for our local LLM:

```
def format_context(chunks, conn):
```

```
    """Format retrieved chunks for the LLM"""
    cursor = conn.cursor()
    formatted_chunks = []

    for i, chunk in enumerate(chunks, 1):
        # Get post metadata
        cursor.execute("""
            SELECT title, subreddit, author, score
            FROM posts
            WHERE post_id = ?
        """, (chunk['post_id'],))

        post = cursor.fetchone()
        if post:
            formatted_chunks.append(f"""
SOURCE {i}:
From r/{post['subreddit']} by u/{post['author']} (Score:
{post['score']})
Title: {post['title']}

Content:
{chunk['content']}
""")

    return "\n---\n".join(formatted_chunks)
```

Note that we include the author and the original post in our context.

In this experimental app, we include the post's score. While this helps the LLM identify "quality" answers, be aware that high scores reflect popularity, not necessarily factual accuracy. For critical domains, you may wish to omit these to avoid popularity bias.

Question-Answering Pipeline

The pipeline starts with retrieval, measuring performance for optimization insights:

```
def answer_question(conn, question, num_chunks=5):
    """Complete RAG pipeline to answer a question"""
    print(f"\nQuestion: {question}")

    # Step 1: Retrieve relevant chunks
```

```
    start_time = time.time()
    chunks = hybrid_search(conn, question, limit=num_chunks)
    retrieval_time = (time.time() - start_time) * 1000

    if not chunks:
        return "No relevant information found in the database."
        print(f"Retrieved {len(chunks)} chunks in
{retrieval_time:.1f}ms")
```

The prompt is carefully crafted to constrain the LLM to use only the provided context, preventing hallucinations and ensuring factual accuracy by requiring citations:

```
    # Step 2: Format context
    context = format_context(chunks, conn)

    # Step 3: Create prompt
    prompt = f"""
    You are a helpful assistant. Answer the user's question using
ONLY the
    retrieved information provided below.

    Instructions:
    1. If the information is not in the context, state that you
do not know.
    2. Do not use outside knowledge.
    3. For every claim you make, you MUST cite the source number
(e.g.,
    [Source 1]) used to find that information.

    RETRIEVED INFORMATION:
    {context}

    USER QUESTION:
    {question}

    ANSWER:
    """

    response = ollama.generate(model='llama3', prompt=prompt)
    return response['response']
```

Performance metrics help identify bottlenecks—typically retrieval is fast (tens of milliseconds) while generation takes longer (hundreds of milliseconds to seconds):

```
    # Step 4: Generate answer
    start_time = time.time()
    answer = call_ollama(prompt)
    generation_time = (time.time() - start_time) * 1000

    print(f"Generated answer in {generation_time:.1f}ms")
    print(f"Total time: {retrieval_time +
generation_time:.1f}ms")

    return answer
```

See Figure 6-8 for a nested view of the prompt structure.

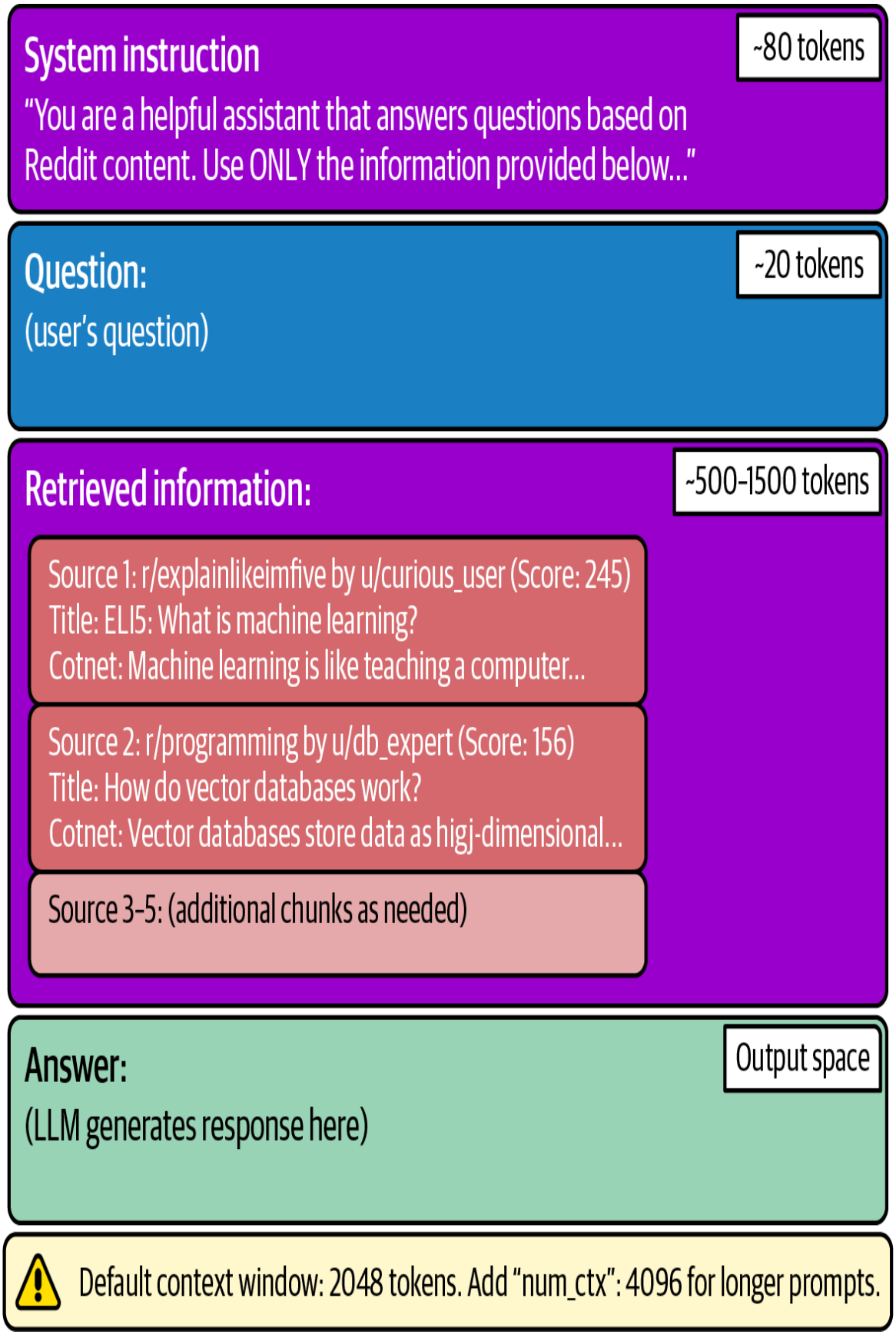

System instruction
~80 tokens
"You are a helpful assistant that answers questions based on Reddit content. Use ONLY the information provided below..."
Question:
~20 tokens
(user's question)
Retrieved information:
~500-1500 tokens
Source 1: r/explainlikeimfive by u/curious_user (Score: 245)
Title: ELI5: What is machine learning?
Cotnet: Machine learning is like teaching a computer...
Source 2: r/programming by u/db_expert (Score: 156)
Title: How do vector databases work?
Cotnet: Vector databases store data as higj-dimensional...
Source 3-5: (additional chunks as needed)
Answer:
Output space
(LLM generates response here)
Default context window: 2048 tokens. Add "num_ctx": 4096 for longer prompts.

Figure 6-8. Prompt assembly structure

Demonstration and Testing

This final section provides sample data and interactive interfaces for testing the system.

Sample Data Loading

Sample data covers diverse topics to demonstrate the system's ability to handle different domains:

```
def load_sample_data(conn):
    """Load some sample Reddit data for testing"""
    sample_posts = [
        {
            'post_id': 'post001',
            'title': 'ELI5: What is machine learning?',
            'content': """Machine learning is like teaching a
computer to
            recognize patterns by showing it many examples.
            Instead of programming exact rules, you feed it data
and it learns
            the patterns itself. For example, to teach it to
recognize cats,
            you show it thousands of cat pictures until it learns
what makes a
            cat a cat.
            It's used in spam filters, recommendation systems,
and voice
            assistants.""",
            'subreddit': 'explainlikeimfive',
            'author': 'curious_user',
            'score': 245
        },
        {
            'post_id': 'post002',
            'title': 'Best Python libraries for beginners?',
            'content': """I recommend starting with: NumPy for
numerical
            computing, Pandas for data manipulation, Matplotlib
for basic
```

```
            plotting, Requests for HTTP requests, and Flask for
simple web
            apps. These libraries cover most beginner needs and
have
            excellent documentation. Don't try to learn them all
at once –
            start with one based on your project needs.""",
            'subreddit': 'learnpython',
            'author': 'python_mentor',
            'score': 189
        },
        {
            'post_id': 'post003',
            'title': 'How do vector databases work?',
            'content': """Vector databases store data as high-
dimensional vectors
            (lists of numbers) that represent the semantic
meaning of the data.
            When you search, your query is converted to a vector
and the database
            finds the most similar vectors using distance metrics
like cosine
            similarity. This enables semantic search where you
find content by
            meaning rather than exact keyword matches. Popular
vector databases
            include Pinecone, Weaviate, and pgvector for
PostgreSQL.""",
            'subreddit': 'programming',
            'author': 'db_expert',
            'score': 156
        }
    ]

    print("Loading sample data...")
    for post in sample_posts:
        store_post_with_chunks(conn, post)
    print(f"Loaded {len(sample_posts)} sample posts")
```

Main Demonstration Function

The main function provides a guided walk-through of the system setup and capabilities:

```
def main():
```

```
    """Main demonstration of the RAG system"""
    print("=== Minimal RAG System with SQLite VSS and Ollama
===\n")

    # Step 1: Setup database
    print("1. Setting up database...")
    conn = setup_database()

    # Step 2: Check Ollama
    print("\n2. Checking Ollama...")
    if not test_ollama():
        print("Please start Ollama first: ollama serve")
        print("Then pull a model: ollama pull llama3.1:8b")
        return

    # Step 3: Load sample data
    print("\n3. Loading sample data...")
    cursor = conn.cursor()
    cursor.execute("SELECT COUNT(*) FROM posts")
    if cursor.fetchone()[0] == 0:
        load_sample_data(conn)
    else:
        print("Data already loaded")
```

Interactive Q&A Interface

The interactive mode allows users to experiment with different questions and see how the system responds:

```
    # Step 4: Demo questions
    print("\n4. Demonstrating RAG system...")

    demo_questions = [
        "What is machine learning and how does it work?",
        "What Python libraries should a beginner learn?",
        "How do vector databases enable semantic search?"
    ]

    for question in demo_questions[:1]:  # Just show one example
        answer = answer_question(conn, question)
        print(f"\nAnswer: {answer}\n")
        print("=" * 50)

    # Step 5: Interactive mode
    print("\n5. Interactive Q&A (type 'quit' to exit)")
```

```
    print("-" * 50)

    while True:
        question = input("\nYour question: ").strip()

        if question.lower() in ['quit', 'exit', 'q']:
            break

        if not question:
            continue

        answer = answer_question(conn, question)
        print(f"\nAnswer: {answer}")

    conn.close()
    print("\nGoodbye!")
```

Quick Testing Utility

The `quick_start` function enables testing individual components without running the full pipeline, useful for debugging and development:

```
def quick_start():
    """Quick start for testing individual components"""
    conn = setup_database()

    # Quick test of search
    print("Testing hybrid search for 'machine learning'...")
    results = hybrid_search(conn, "machine learning", limit=3)

    for i, result in enumerate(results, 1):
        print(f"\n{i}. Score: {result['score']:.3f}")
        print(f"   Content: {result['content'][:150]}...")

    conn.close()

if __name__ == "__main__":
    # Run main demo
    main()

    # Or run quick test
    # quick_start()
```

Next Steps: Extending the System

This implementation demonstrates the core concepts of a RAG system but intentionally keeps functionality limited for clarity. The system currently only indexes Reddit post titles and content, not the rich discussions in comments. Here are the key areas where you could extend this foundation.

Missing Reddit Data Features

Reddit has much more content than just posts. A more featured data store would take these into account:

Comment threads

> The most valuable insights often appear in comments, not the main posts. Adding comment support would require fetching comment trees via PRAW, storing them with proper parent–child relationships, and deciding how to chunk threaded conversations while preserving context.

Post metadata

> Currently ignored metadata like timestamps, awards, cross-post information, and edit history could improve retrieval relevance and answer quality.

Media content

> Reddit posts often include images, videos, and external links that our text-only system cannot process.

Live data ingestion

> The system uses static sample data rather than fetching real Reddit content, which would require proper API authentication and rate limiting.

Retrieval enhancements

Often, whether RAG is useful or frustrating depends on the chunking—and a more robust implementation would devote far more attention to chunking, especially semantic chunking. Other factors include reranking retrieved results, which noticeably improves response quality.

Semantic chunking

Our simple word-count chunking splits text arbitrarily. Semantic chunking would keep related ideas together by analyzing sentence embeddings and natural paragraph boundaries.

Query expansion

Using the LLM to generate alternative phrasings of queries before searching could improve recall for complex questions.

Cross-encoder reranking

After initial retrieval, a cross-encoder model could rerank results based on query-document relevance, significantly improving precision.

Metadata filtering

Allow filtering by subreddit, date range, author karma, or post score before semantic search.

Dimensionality mismatch

Embedding dimension can get out of sync with the stored dimension when trying a new embedding model. Check the dimension of the embedding before attempted insertion.

Duplicate insertions

Unique indexes on appropriate columns can catch these.

Deletion orphans

Deleting from posts but not the chunks can create accumulated junk. Using a script that deletes from both tables in a transaction will help.

Aside from these, robust error handling at each stage should be added, where the text might have skimmed over it in the interest of focusing on core content. Logging both at the ingest phase and the retrieval/generation phase will prove invaluable during one-off corner cases, as happens in real-world scenarios.

Performance Optimizations

A number of standard system design considerations apply here as well, and these are some of the more relevant ones:

Embedding cache

: Repeatedly embedding the same text wastes compute. A simple cache mapping text hashes to embeddings would speed up duplicate processing.

Connection pooling

: Our single database connection becomes a bottleneck under load. Connection pooling would enable concurrent operations.

Batch processing

: Processing posts one at a time is inefficient. Batching embeddings and database operations would improve throughput.

Asynchronous pipelines

: The sequential retrieval-then-generation flow could be parallelized for multiple queries.

Production Considerations

While this book is not meant to provide recipes for production applications —only examples—here are some considerations worth mentioning:

Incremental updates

The system lacks the ability to add new content without reprocessing everything. Implementing incremental indexing would enable continuous updates.

Version management

As embedding models improve, you need strategies for migrating existing embeddings or maintaining multiple versions.

Monitoring and analytics

Production systems need query logging, performance metrics, and user feedback loops to identify retrieval failures and improve over time.

Scale considerations

SQLite and local Ollama work for demos but have limits. Consider PostgreSQL with pgvector for larger datasets and API-based LLMs for concurrent users.

Advanced RAG Patterns

It's also worth considering the following RAG patterns:

Multihop reasoning

For complex questions requiring information from multiple sources, implement iterative retrieval where the LLM can request additional searches.

Source diversity

Ensure that retrieved chunks come from different posts/authors to avoid echo chamber effects.

Confidence scoring

Have the LLM indicate confidence in its answers based on the quality and relevance of retrieved content.

Fact verification

> Add a verification step where claims in the generated answer are checked against the source material.

These extensions would transform our educational demonstration into a system capable of handling real-world complexity and scale. The modular architecture makes it straightforward to add these features incrementally based on your specific needs.

Conclusion

This RAG system demonstrates how vector databases enable a new paradigm in information retrieval and question answering. By combining semantic search with LLM generation, we create a system that can:

- Understand the meaning behind queries, not just keywords.
- Retrieve the most relevant information from large document collections.
- Generate accurate, contextual answers grounded in real data.
- Minimize hallucinations by constraining responses to retrieved information.

The hybrid search approach ensures that we get the best of both worlds—semantic understanding and precise keyword matching. The local deployment using SQLite and Ollama provides privacy and cost-effectiveness while maintaining good performance.

This foundation can be extended with more sophisticated chunking strategies, better ranking algorithms, query expansion techniques, and multihop reasoning chains—all building on the core RAG pattern demonstrated here.

OceanofPDF.com

Chapter 7. Building a Scientific RAG System with PostgreSQL and pgvector

The explosion of scientific publications presents researchers with an overwhelming challenge: how to efficiently discover, understand, and synthesize relevant knowledge from millions of papers. ArXiv alone publishes over 15,000 papers monthly across physics, mathematics, computer science, and other fields. Traditional keyword search fails to capture the semantic richness of scientific discourse, where the same concept may be expressed in countless ways across different domains and research communities.

This chapter builds a RAG system specifically designed for scientific literature. Unlike general-purpose RAG systems, scientific RAG must handle unique challenges:

Technical terminology

: Papers use precise, domain-specific language that requires semantic understanding beyond simple keywords.

Structured content

: Scientific papers follow conventions (abstract, methodology, results, conclusions) that can inform retrieval strategies.

Citation networks

: Papers exist in a web of references that provide additional context.

Mathematical notation

Formulas and equations carry meaning that is not captured by this implementation. Handling mathematical notation requires specialized tools like LaTeX OCR.

Evidence quality

Not all sources are equal—peer review, venue, and citation count matter.

System Goals and Capabilities

Our scientific RAG system will enable the following:

Semantic discovery

Find papers based on conceptual similarity, not just keyword matches.

Cross-paper synthesis

Answer questions that require information from multiple papers.

Contextual understanding

Retrieve specific sections (methodology, results) relevant to queries.

Evidence-based responses

Generate answers grounded in actual research, with citations to source papers—formatting requires post-processing.

Technical depth

Handle complex scientific queries requiring domain expertise.

Architecture Overview

The system architecture consists of six interconnected components (see Figure 7-1).

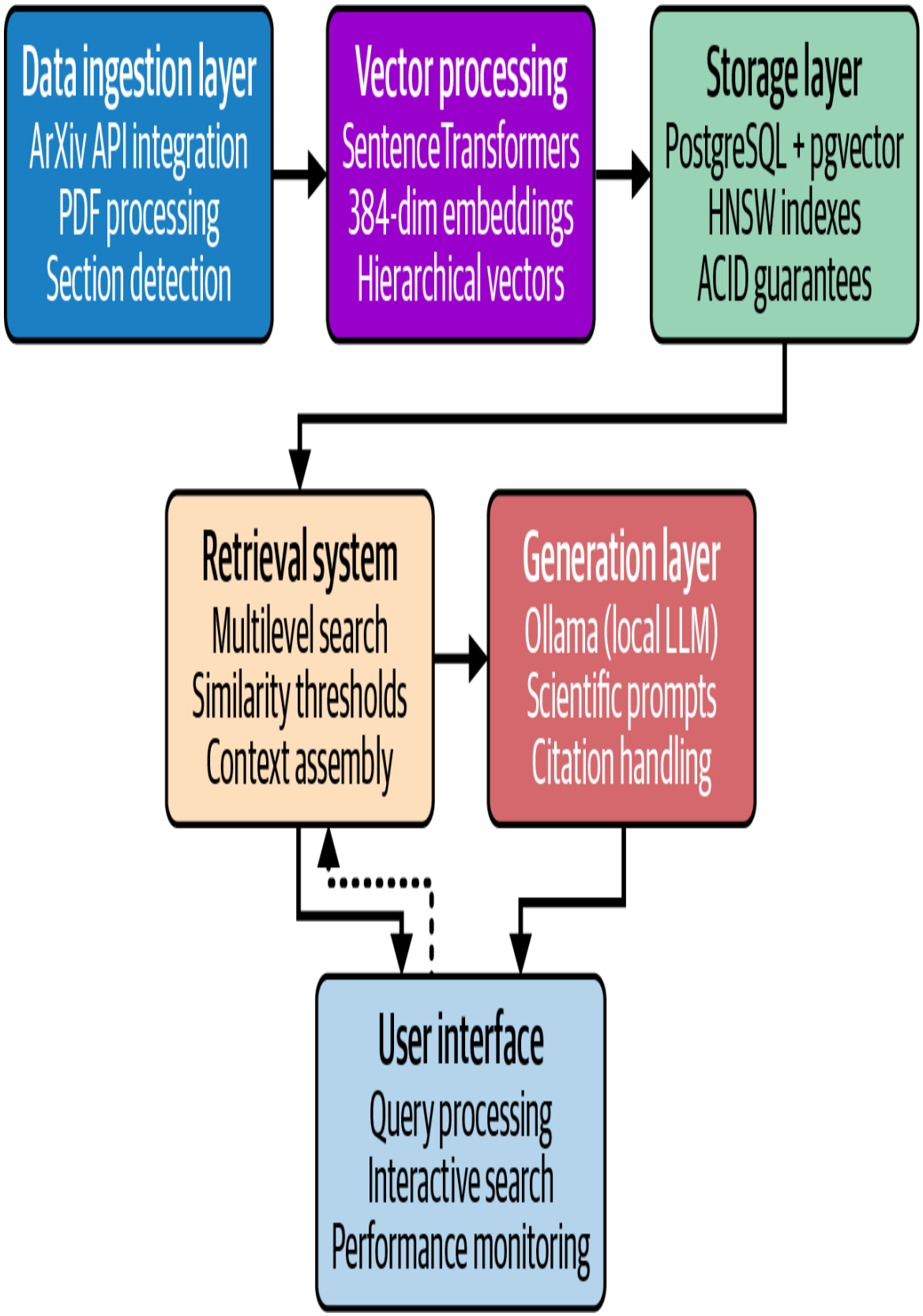


Figure 7-1. System architecture

Let’s look at the components in more detail:

Data ingestion layer

ArXiv API integration

Fetches paper metadata and PDFs

PDF processing

Extracts structured text from scientific PDFs

Section detection

Identifies paper sections (introduction, methods, results)

Vector processing pipeline

Embedding generation

Converts text to dense vectors using `SentenceTransformers`

Hierarchical embeddings

Separate vectors for abstracts and sections

Dimension

Implements 384-dimensional vectors, balancing quality and performance

Storage layer (PostgreSQL + pgvector)

`papers` *table*

Metadata and abstract embeddings

`sections` *table*

Chunked content with section embeddings

HNSW indexes

ANN search at scale

Retrieval system

Multilevel search

Queries both abstracts and sections

Similarity thresholds

Filter results by relevance score

Context assembly

Combines results from multiple sources

Generation layer (Ollama)

Local LLM

Privacy-preserving inference

Scientific prompts

Templates designed for academic responses

Citation handling

Maintains paper references in answers

User interface

Query processing

Natural language scientific questions

Interactive search

Different modes for different needs

Performance monitoring

Tracks retrieval and generation times

The next section discusses how to build each component and includes detailed explanations of the design decisions and implementation.

Database Foundation with pgvector

PostgreSQL with the pgvector extension provides robust vector storage with ACID guarantees. Unlike pure vector databases, this approach gives us the best of both worlds—relational data integrity and vector similarity search:

```
# Chapter 7: Minimal Scientific RAG with PostgreSQL and pgvector
# Simplified version demonstrating scientific paper search and
RAG
#
# Dependencies: pip install psycopg2-binary arxiv sentence-
transformers requests
PyMuPDF

import psycopg2
import arxiv
import requests
import time
from sentence_transformers import SentenceTransformer
from typing import List, Dict, Optional
from dataclasses import dataclass
import numpy as np
```

Database Configuration and Setup

The database setup creates a schema optimized for scientific paper storage and retrieval:

```
def setup_database():
    """Setup PostgreSQL with pgvector for scientific papers"""
```

```
    # Hardcoded database connection - update as needed.
    # The config values are typically imported from environment
variables

    DB_CONFIG = {
        'host': HOST,
        'port': PORT,
        'database': 'scientific_rag',
        'user': USER,
        'password': PASSWORD
    }

    conn = psycopg2.connect(**DB_CONFIG)
    cursor = conn.cursor()

    # Enable pgvector extension
    cursor.execute("CREATE EXTENSION IF NOT EXISTS vector;")
```

The pgvector extension adds the vector data type to PostgreSQL, enabling native vector operations. This is crucial for performance—vector operations happen in the database rather than requiring data transfer to application code.

Schema Design for Scientific Papers

The schema reflects the hierarchical nature of scientific documents:

```
    # Create papers table
    cursor.execute("""
        CREATE TABLE IF NOT EXISTS papers (
            paper_id TEXT PRIMARY KEY,
            title TEXT NOT NULL,
            abstract TEXT,
            authors TEXT[],
            published_date DATE,
            pdf_url TEXT,
            abstract_embedding vector(384)
        );
    """)
```

The `papers` table stores high-level metadata. Note the use of PostgreSQL arrays for authors—this maintains data structure without requiring a

separate table. The abstract embedding enables paper-level similarity search:

```
    # Create paper sections table for chunked content
    cursor.execute("""
        CREATE TABLE IF NOT EXISTS paper_sections (
section_id SERIAL PRIMARY KEY,
paper_id INTEGER REFERENCES papers(paper_id) ON DELETE CASCADE,
section_title TEXT,
section_number INTEGER, -- Scheme: (section_i * 100 + chunk_j)
content TEXT,
embedding vector(384)
);
    """)
```

We use the `ON DELETE CASCADE` constraint to maintain referential integrity; if a paper is removed from the `papers` table, its associated sections are automatically deleted. Additionally, note the use of the `TEXT[]` array in the `papers` table. This allows us to store multiple authors in a single field without needing a complex join table. You can query these using the contains operator: `authors @> ARRAY['Einstein']`.

The `paper_sections` table enables fine-grained retrieval. Scientific papers often contain diverse content—the methodology might be irrelevant to someone interested in results. Section-level indexing allows precise retrieval. See the entity-relationship diagram in Figure 7-2.

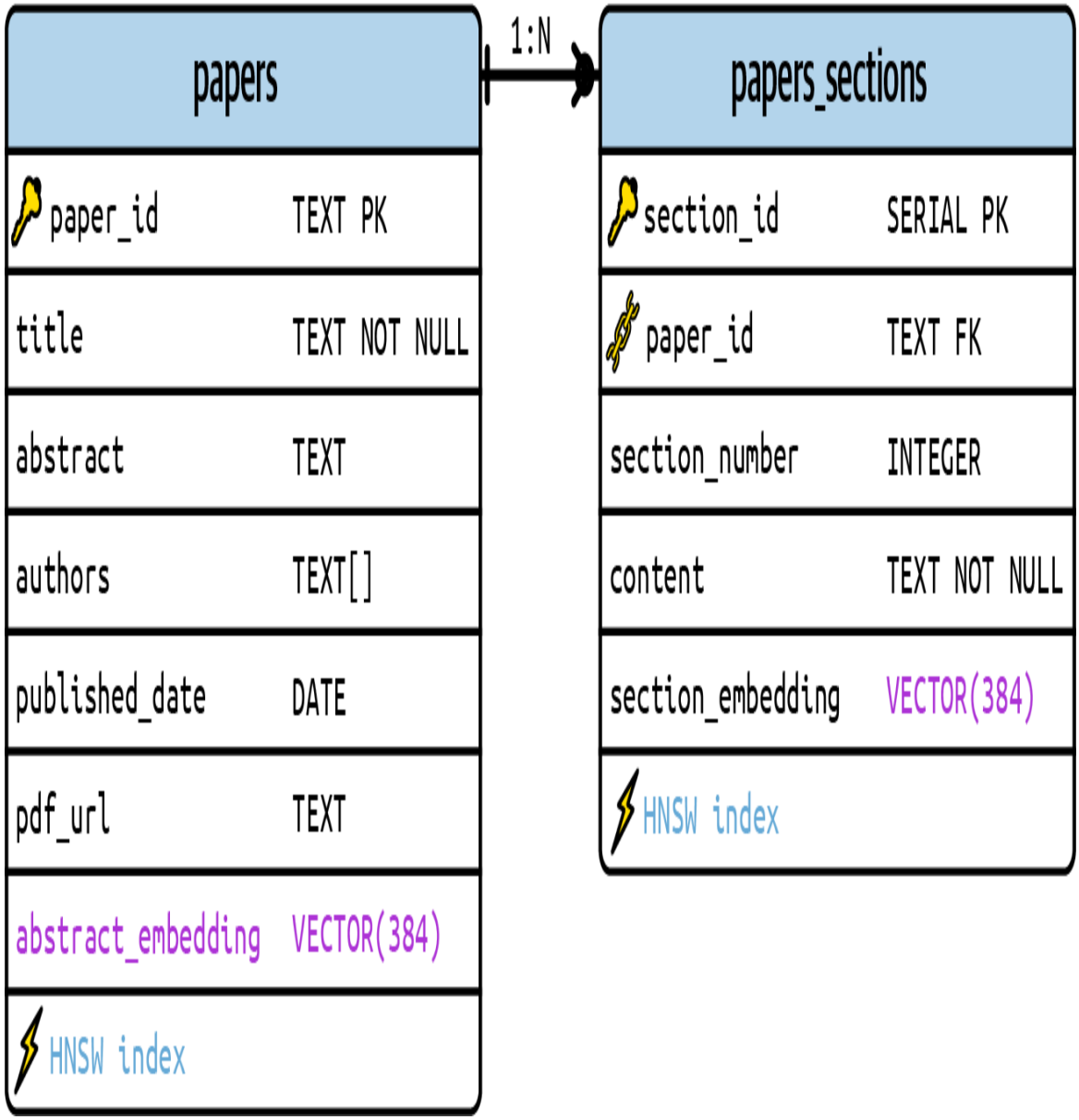


Figure 7-2. Entity-relationship diagram

With our database schema ready, it's important to understand why we are splitting the data into two tables. In a scientific context, a paper's abstract acts as a high-level semantic summary, while the sections contain the granular evidence. By indexing both, we allow our RAG system to find papers either by their overall theme (abstract) or by a specific technical detail (section). This multilevel indexing is a key differentiator of scientific RAG systems compared to simple document search.

High-Performance Vector Indexes

HNSW indexes provide ANN search with excellent recall and query speed:

```
    # Create indexes for fast similarity search
    cursor.execute("""
        CREATE INDEX IF NOT EXISTS idx_abstract_embeddings
        ON papers USING hnsw (abstract_embedding
vector_cosine_ops);
    """)

    cursor.execute("""
        CREATE INDEX IF NOT EXISTS idx_section_embeddings
        ON paper_sections USING hnsw (section_embedding
vector_cosine_ops);
    """)

    conn.commit()
    cursor.close()
    conn.close()

    print("Database setup completed")
    return DB_CONFIG
```

The `vector_cosine_ops` parameter specifies cosine similarity, ideal for normalized embeddings.

HNSW builds a multilayer graph structure that enables ANN search with sublinear complexity. While faster than a brute-force $O(n)$ scan, building HNSW indexes on large datasets is resource-intensive. If you are importing thousands of papers at once, it is 10x to 100x faster to `DROP` the index, perform the bulk insert, and then `CREATE` the index afterward.

For a detailed discussion of HNSW index performance, see “Technical Note on HNSW”.

Embedding Generation Strategy

Embeddings are the bridge between human language and mathematical similarity. The choice of embedding model and strategy significantly impacts retrieval quality:

```
# Global embedding model
embedding_model = None

def get_embedding_model():
    """Get or initialize the embedding model"""
    global embedding_model
    if embedding_model is None:
        embedding_model = SentenceTransformer('all-MiniLM-L6-v2')
        print(f"Loaded embedding model (dimension: \

{embedding_model.get_sentence_embedding_dimension()})")
    return embedding_model

def generate_embedding(text):
    """Generate embedding for text"""
    model = get_embedding_model()
    return model.encode(text, convert_to_numpy=True)
```

The singleton pattern prevents model reloading, crucial for performance. We use the all-MiniLM-L6-v2 model because it provides an excellent balance of speed and performance for general-purpose text on consumer hardware. While it is trained on general datasets (like Quora and Reddit), it serves as a strong baseline. For specialized scientific tasks requiring deeper domain knowledge, you might consider models like allenai/specter2, though they require more VRAM and compute.

ArXiv Integration and PDF Processing

Scientific RAG requires access to full paper content, not just abstracts. This section handles paper discovery and content extraction (see Figure 7-3).

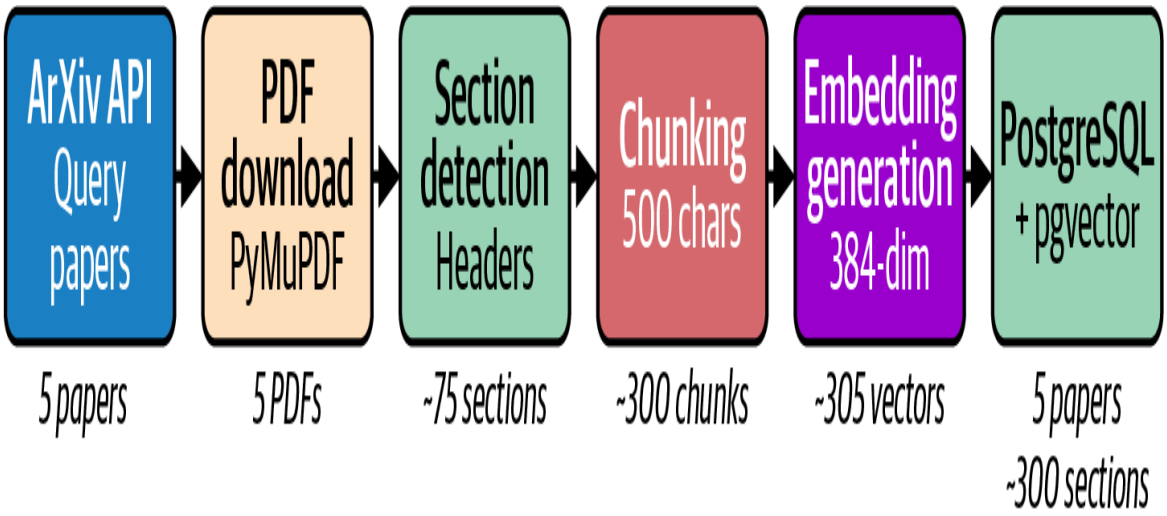


Figure 7-3. Data ingestion pipeline

Paper Discovery with ArXiv API

The dataclass provides type safety and clean data structure. ArXiv queries support complex Boolean logic and field-specific searches (e.g., `cat:cs.LG` for machine learning papers). Rate limiting respects ArXiv's servers and prevents blocking:

```
@dataclass
class Paper:
    paper_id: str
    title: str
    abstract: str
    authors: List[str]
    published_date: str
    pdf_url: str

def fetch_arxiv_papers(query, max_results=5):
    """Fetch papers from ArXiv"""
    print(f"Fetching papers from ArXiv: {query}")

    client = arxiv.Client()
    search = arxiv.Search(
        query=query,
        max_results=max_results,
        sort_by=arxiv.SortCriterion.SubmittedDate
    )

    papers = []
```

```
    for result in client.results(search):
        paper = Paper(
            paper_id=result.entry_id.split('/')[-1],
            title=result.title,
            abstract=result.summary,
            authors=[author.name for author in result.authors],
            published_date=result.published.date(),
            pdf_url=result.pdf_url
        )
        papers.append(paper)

        # Rate limiting
        time.sleep(0.5)

    print(f"Retrieved {len(papers)} papers")
    return papers
```

Intelligent PDF Text Extraction

PDF processing is notoriously difficult, especially for scientific papers with complex layouts, formulas, and figures. Our approach focuses on robust text extraction with section detection:

> **TECHNICAL NOTE: THE LIMITS OF PDF PARSING**
>
> This implementation uses PyMuPDF for simplicity. However, scientific PDFs are notoriously difficult to parse due to multicolumn layouts, complex equations, and nested tables. In a production environment, consider tools like Grobid or Facebook's Nougat, which use machine learning to reconstruct the logical structure of a paper more reliably than standard text extraction.

The `User-Agent` header prevents blocking by CDNs that filter automated requests. The timeout prevents hanging on slow connections:

```
def download_pdf_text(pdf_url):
    """Download and extract text from PDF using PyMuPDF"""
    try:
        import fitz  # PyMuPDF

        print(f"Downloading PDF from {pdf_url[:50]}...")
```

```
        # Download PDF
        # Identify yourself to ArXiv as per their API guidelines
headers = {'User-Agent': 'ScientificRAGDemo/1.0 (contact: \
    your-email@example.com)'}
response = requests.get(pdf_url, headers=headers, timeout=30)
if response.status_code != 200:
            print(f"Failed to download PDF: HTTP
{response.status_code}")
            return []
```

Section detection uses common academic paper structure. This heuristic approach works well for most papers following standard formats. The headers list covers variations across different fields:

```
        # Open PDF from bytes
        doc = fitz.open(stream=response.content, filetype="pdf")

        # Extract text from each page
        sections = []
        current_section = ""

        for page_num in range(len(doc)):
            page = doc[page_num]
            text = page.get_text()

            # Simple section detection: look for common headers
            lines = text.split('\n')
            section_headers = ['introduction', 'abstract',
'methodology',
                               'method', 'results', 'discussion',
'conclusion',
                               'references', 'related work',
'experiments',
                               'background', 'approach',
'evaluation',
                               'analysis']
```

The header detection logic handles various formats: standalone headers, numbered sections (“1. Introduction”), and short headers with the keyword. This captures most academic paper structures:

```
            for line in lines:
                line_lower = line.lower().strip()
```

```
                # Check if this line is a section header
                is_header = False
                for header in section_headers:
                    if (line_lower == header or
                        (len(line.split()) <= 3 and header in
line_lower) or
                        line_lower.startswith(f"{header}.")):
                        is_header = True
                        break

                if is_header and current_section.strip():
                    # Save current section and start new one
                    sections.append(current_section.strip())
                    current_section = line + "\n"
                else:
                    current_section += line + "\n"
```

The fallback to page-based splitting ensures that we extract something, even from poorly formatted PDFs. The 100-character minimum filters out headers, footers, and page numbers. Limiting to 20 sections prevents memory issues with very long papers:

```
        # Add final section
        if current_section.strip():
            sections.append(current_section.strip())

        # If no sections detected, split by pages
        if len(sections) <= 1:
            sections = []
            for page_num in range(min(10, len(doc))):  # Limit to
first 10 pages
                page = doc[page_num]
                text = page.get_text()
                if text.strip():
                    sections.append(text.strip())

        doc.close()
        print(f"Extracted {len(sections)} sections from PDF")

        # Filter out very short sections (likely headers/footers)
        filtered_sections = [s for s in sections if len(s) > 100]

        return filtered_sections[:20]  # Limit to first 20
sections
```

```
    except ImportError:
        print("PyMuPDF not installed. Install with: pip install
PyMuPDF")
        return []
    except Exception as e:
        print(f"Error extracting PDF text: {e}")
        return []
```

This simple PDF extraction works for basic papers but fails on multicolumn layouts, complex equations/figures, and papers with nonstandard structures. Production systems should use specialized tools like Grobid or Nougat for robust scientific PDF parsing.

Advanced Text Chunking

Chunking strategy significantly impacts retrieval quality. Too small and we lose context; too large and we dilute relevance. Our approach balances these concerns:

```
# Fixed character-based overlap
def chunk_text(text, chunk_size=500, overlap=50):
    chunks = []
    start = 0
    while start < len(text):
        end = start + chunk_size
        chunk = text[start:end]
        chunks.append(chunk)
        # Move the start pointer forward by (size - overlap)
        start += (chunk_size - overlap)
    return chunks
```

The 500-character chunks roughly correspond to a paragraph, maintaining semantic coherence. The overlap ensures that information at chunk boundaries isn’t lost. This simple approach works well for scientific text where paragraph boundaries often align with conceptual boundaries.

Storage Pipeline with Embeddings

The storage pipeline coordinates PDF processing, chunking, embedding generation, and database insertion.

The `ON CONFLICT` clause enables idempotent operations—reprocessing a paper updates rather than duplicates. This is essential for maintaining data consistency when refreshing papers or recovering from failures:

```
def store_paper_with_embeddings(db_config, paper, sections):
    """Store paper and sections with embeddings"""
    conn = psycopg2.connect(**db_config)
    cursor = conn.cursor()

    try:
        # Generate abstract embedding
        abstract_embedding = generate_embedding(paper.abstract)

        # Store paper
        cursor.execute("""
            INSERT INTO papers (paper_id, title, abstract,
authors, \
                published_date, pdf_url, abstract_embedding)
            VALUES (%s, %s, %s, %s, %s, %s, %s)
            ON CONFLICT (paper_id) DO UPDATE SET
                title = EXCLUDED.title,
                abstract = EXCLUDED.abstract,
                abstract_embedding = EXCLUDED.abstract_embedding
        """, (
            paper.paper_id,
            paper.title,
            paper.abstract,
            paper.authors,
            paper.published_date,
            paper.pdf_url,
            abstract_embedding.tolist()
        ))
```

The section numbering scheme (`i * 100 + j`) preserves both section order and chunk order within sections. This enables reconstruction of the original document structure if needed. Transaction handling ensures atomicity—either the entire paper is stored or nothing is:

```
        # Store sections with embeddings
        for i, section_text in enumerate(sections):
            if section_text.strip():  # Skip empty sections
                # Chunk the section
                chunks = simple_chunk_text(section_text)

                for j, chunk in enumerate(chunks):
                    section_embedding = generate_embedding(chunk)

                    cursor.execute("""
                        INSERT INTO paper_sections (paper_id,
section_number, \
                            content, section_embedding)
                        VALUES (%s, %s, %s, %s)
                    """, (
                        paper.paper_id,
                        i * 100 + j,  # Simple section numbering
                        chunk,
                        section_embedding.tolist()
                    ))

        conn.commit()
        print(f"Stored paper {paper.paper_id} with
{len(sections)} sections")

    except Exception as e:
        conn.rollback()
        print(f"Error storing paper: {e}")
        raise
    finally:
        cursor.close()
        conn.close()
```

For processing multiple papers at a time for personal use, we can do this sequentially. Processing papers sequentially prevents memory issues with large PDFs and respects rate limits. In production, this could be parallelized with proper resource management:

```
def process_and_store_papers(db_config, papers):
    """Process and store multiple papers"""
    for paper in papers:
        print(f"\nProcessing: {paper.title[:60]}...")

        # Download and extract sections
        sections = download_pdf_text(paper.pdf_url)
```

```
            if sections:
                store_paper_with_embeddings(db_config, paper,
sections)
            else:
                print(f"  Skipping - no content extracted")

            # Rate limiting
            time.sleep(1)
```

Multilevel Semantic Search

Scientific queries require different granularities of search. Sometimes we want paper-level summaries (abstracts), other times specific technical details (sections). See Figure 7-4, which schematically shows the following sequence:

1. User query → Generate Embedding (384-dim)
2. Branch to parallel searches (abstract and section)
3. Result counts (3 and 5 results)
4. Merge point (8 total)
5. Sort by similarity DESC
6. Top 5 contexts output

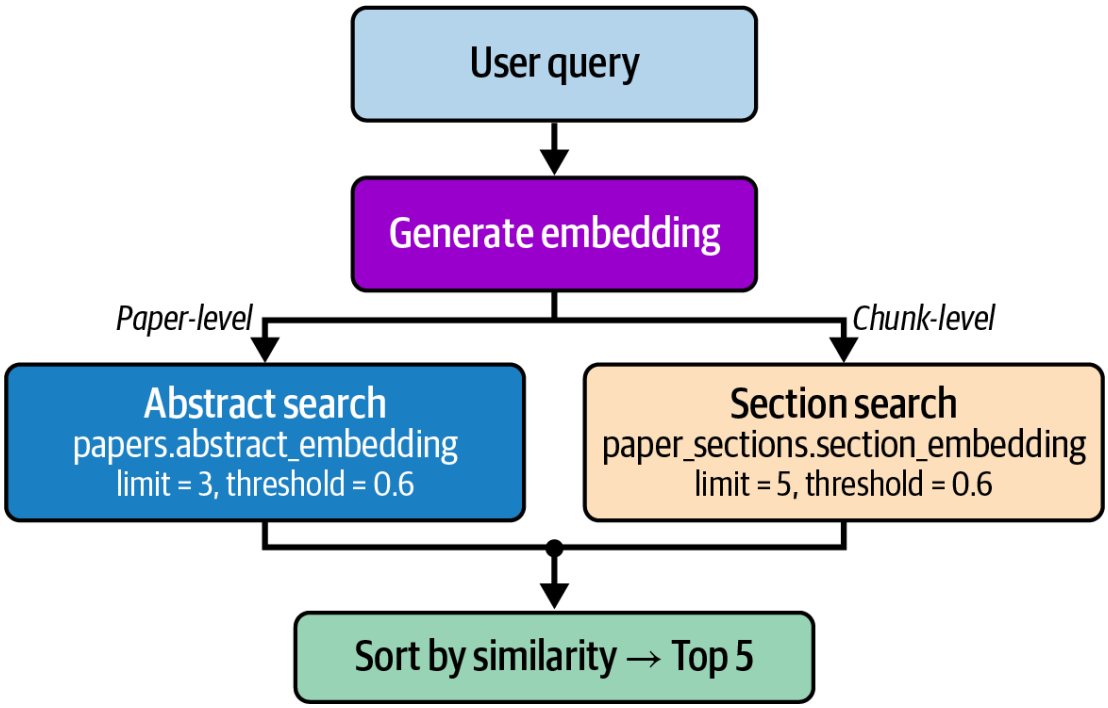


Figure 7-4. Multilevel search strategy

Abstract-Level Search

Abstract search is ideal for discovering relevant papers and understanding the research landscape:

```
def search_papers_by_abstract(db_config, query, limit=3,
threshold=0.7):
    """Search papers by abstract similarity"""
    conn = psycopg2.connect(**db_config)
    cursor = conn.cursor()

    # Generate query embedding
    query_embedding = generate_embedding(query)

    # Search abstracts
    cursor.execute("""
        SELECT
            paper_id,
            title,
            abstract,
            authors,
            1 - (abstract_embedding <=> %s::vector) as similarity
        FROM papers
        WHERE abstract_embedding IS NOT NULL
```

```
          AND 1 - (abstract_embedding <=> %s::vector) > %s
        ORDER BY similarity DESC
        LIMIT %s
    """, (query_embedding.tolist(), query_embedding.tolist(),
threshold, limit))

    results = cursor.fetchall()
    cursor.close()
    conn.close()

    return [{
        'paper_id': row[0],
        'title': row[1],
        'abstract': row[2],
        'authors': row[3],
        'similarity': row[4]
    } for row in results]
```

The `<=>` operator computes cosine distance in pgvector. Converting to similarity (`1 - distance`) provides intuitive scores where `1.0` means identical and `0.0` means orthogonal. The `threshold` parameter filters out marginally relevant results, improving precision.

Section-Level Search

Section search enables finding specific technical details, methodologies, or results:

```
def search_paper_sections(db_config, query, limit=5,
threshold=0.7):
    """Search within paper sections"""
    conn = psycopg2.connect(**db_config)
    cursor = conn.cursor()

    # Generate query embedding
    query_embedding = generate_embedding(query)

    # Search sections
    cursor.execute("""
        SELECT
            ps.paper_id,
            p.title,
            ps.section_number,
            ps.content,
```

```
            1 - (ps.section_embedding <=> %s::vector) as
similarity
        FROM paper_sections ps
        JOIN papers p ON ps.paper_id = p.paper_id
        WHERE ps.section_embedding IS NOT NULL
          AND 1 - (ps.section_embedding <=> %s::vector) > %s
        ORDER BY similarity DESC
        LIMIT %s
    """, (query_embedding.tolist(), query_embedding.tolist(),
threshold, limit))

    results = cursor.fetchall()
    cursor.close()
    conn.close()

    return [{
        'paper_id': row[0],
        'title': row[1],
        'section_number': row[2],
        'content': row[3],
        'similarity': row[4]
    } for row in results]
```

The join with the `papers` table provides context (paper title) for each section result. This helps users understand where the information comes from. Section search typically returns more results than abstract search since there are many more sections than papers.

You will notice we use a similarity threshold (e.g., `1 - (embedding <=> query) > 0.7`). In pgvector, cosine distance ranges from 0 to 2, where 0 is identical. By subtracting from 1, we get a similarity score:

- 0.8+: Very high similarity; likely the same specific method or paper.
- 0.7: A “sweet spot” for finding relevant but diverse scientific papers.
- 0.6 or lower: Higher risk of “hallucinated” relevance. The system may pull in papers that share vocabulary but not concepts.

The RAG Pipeline: Deep Dive

The RAG pipeline is where retrieval meets generation. This is the most critical component for answer quality, requiring careful orchestration of search, context assembly, and prompt engineering (see Figure 7-5).

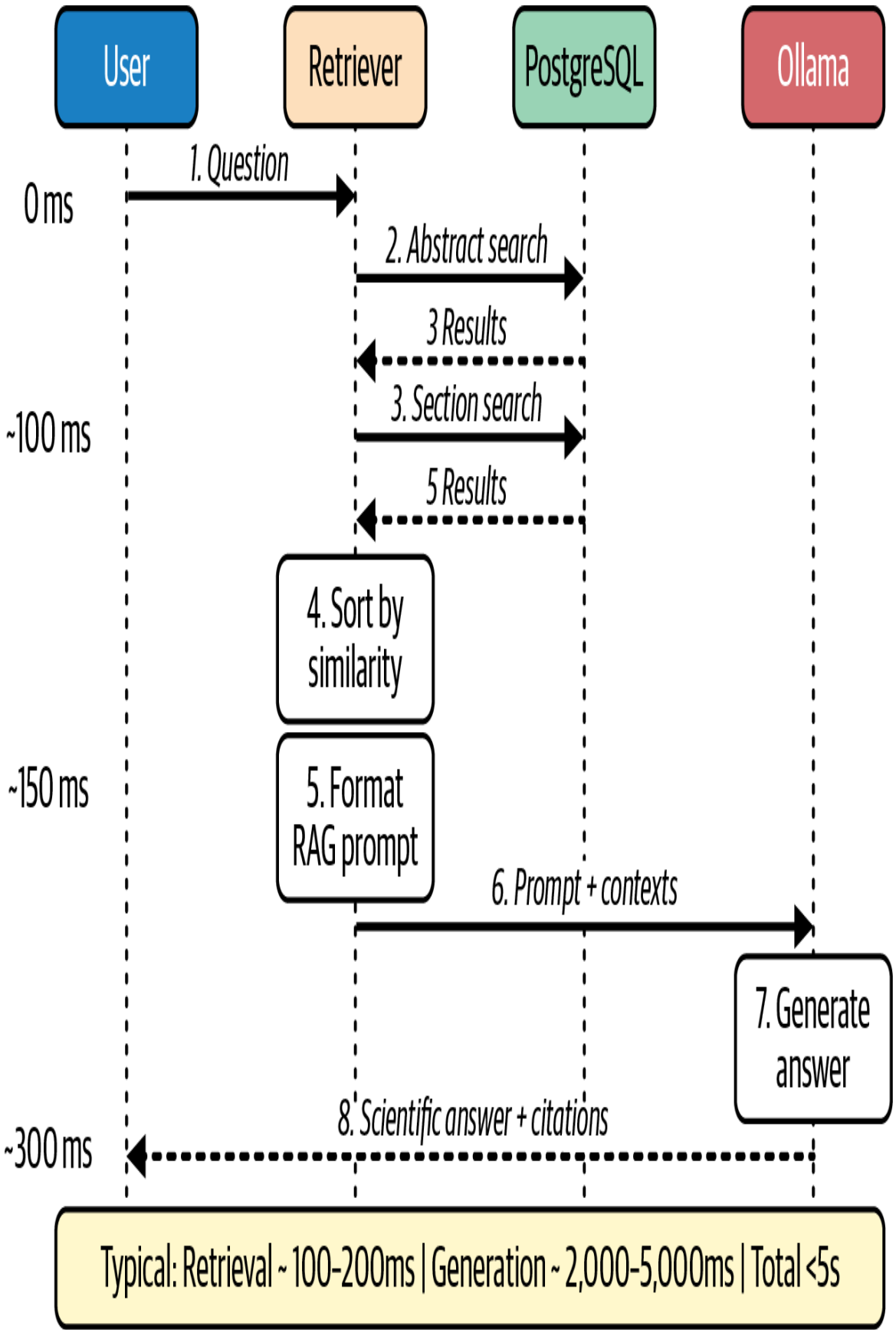

User
Retriever
PostgreSQL
Ollama
1. Question
0 ms
2. Abstract search
3 Results
3. Section search
~100 ms
5 Results
4. Sort by similarity
5. Format RAG prompt
~150 ms
6. Prompt + contexts
7. Generate answer
8. Scientific answer + citations
~300 ms
Typical: Retrieval ~ 100-200ms | Generation ~ 2,000-5,000ms | Total <5s

Figure 7-5. RAG pipeline sequence

Local LLM Integration with Ollama

Ollama provides privacy-preserving local inference, crucial for sensitive research:

```
def call_ollama(prompt, model="llama3.1:8b"):
    """Call Ollama for text generation"""
    url = http://localhost:11434/api/generate

    payload = {
        "model": model,
        "prompt": prompt,
        "stream": False,
        "options": {
            "temperature": 0.1,
            "top_p": 0.9
        }
    }

    try:
        response = requests.post(url, json=payload, timeout=60)
        response.raise_for_status()
        return response.json()['response']
    except Exception as e:
        return f"Error calling Ollama: {str(e)}"
```

Low temperature (0.1) ensures consistent, factual responses based on retrieved context. The 60-second timeout accommodates longer generation for complex scientific explanations. Streaming is disabled for simplicity but could be enabled for better user experience.

Health Check and Model Discovery

Ollama is often used on a personal machine for multiple purposes and is single-tasking, handling one request at a time, so we check availability first:

```
def test_ollama():
    """Test if Ollama is available"""
    try:
        response =
```

```
requests.get("http://localhost:11434/api/tags", timeout=5)
        models = response.json()
        print("Available Ollama models:", [m['name'] for m in \
            models.get('models', [])])
        return True
    except:
        print("Ollama not running. Start with: ollama serve")
        return False
```

This utility function helps with deployment and debugging. It shows available models, helping users choose appropriate models for their domain (e.g., specialized science models).

Intelligent Context Retrieval

The retrieval strategy combines multiple search types for comprehensive context:

```
def get_balanced_context(abstract_results, section_results,
max_total=5):
    # Ensure we get at least 2 abstracts and fill the rest with
sections
    # while preventing duplicate papers from appearing twice
    seen_paper_ids = set()
    combined_context = []

    # 1. Add top abstracts first
    for res in abstract_results:
        if res['paper_id'] not in seen_paper_ids:
            combined_context.append(f"Abstract of {res['title']}:
\
            seen_paper_ids.add(res['paper_id'])
        if len(combined_context) >= 2: break

    # 2. Add sections until max_total is reached
    for res in section_results:
        if len(combined_context) >= max_total: break
        # Even if we saw the paper's abstract, the section is
unique info
        combined_context.append(f"Excerpt from {res['title']}:
{res['content']}")

    return combined_context
```

The dual-search strategy ensures that both broad context (abstracts) and specific details (sections) are available. Lower thresholds (0.6) for retrieval than display cast a wider net, letting the LLM determine relevance. Sorting by similarity ensures that the most relevant contexts appear first in the prompt, improving answer quality since LLMs often give more weight to earlier content. This phenomenon is called primacy bias or lost in the middle (Liu et al., 2023).

Retrieving the data is only half the battle. The final step is “context assembly.” Unlike a web search where you click one link, our LLM needs to “read” all the snippets we found simultaneously. However, scientific text is token-heavy. If we simply dump five 10-page papers into a prompt, we will exceed the context window of most local LLMs (like Llama 3). This is why we carefully limit our chunks to 500 characters and prioritize the most relevant snippets.

Scientific Prompt Engineering

Prompt design significantly impacts answer quality. Scientific prompts need special consideration:

```
def format_scientific_rag_prompt(question, contexts):
    """Format scientific contexts for RAG"""
    if not contexts:
        return f"Question: {question}\n\nI don't have relevant
scientific
                                          literature to answer
this question."

    context_parts = []
    for i, ctx in enumerate(contexts, 1):
        source_type = "Abstract" if ctx['type'] == 'abstract'
else "Section"
        context_parts.append(
            f"Source {i} ({source_type} from '{ctx['title']}',
similarity: \
                {ctx['similarity']:.3f}):\n"
            f"{ctx['content'][:500]}..."
        )

    context_section = "\n\n".join(context_parts)
```

```
    prompt = f"""You are a scientific assistant helping
researchers understand
                academic literature.
Answer the question based ONLY on the provided research paper
excerpts.
Cite the specific papers when referencing information.

Question: {question}

Relevant research findings:
{context_section}

Answer based on the scientific literature above:"""

    return prompt
```

The prompt explicitly instructs the model to:

1. Act as a scientific assistant (sets appropriate tone and expertise level).
2. Use only provided contexts (prevents hallucination).
3. Cite sources (maintains academic rigor).
4. Include similarity scores (helps users assess relevance).

Context truncation (500 characters) balances information density with token limits. Including paper titles and types helps the LLM understand source credibility and relevance.

Complete RAG Execution Pipeline

The main RAG function orchestrates the entire process with performance monitoring:

```
def scientific_rag_answer(db_config, question, max_contexts=5):
    """Generate scientific answer using RAG"""
    print(f"\nScientific Question: {question}")

    # Step 1: Retrieve relevant contexts
    start_time = time.time()
```

```
    #contexts = retrieve_relevant_contexts(db_config, question,
max_contexts)
    get_balanced_context(abstract_results, section_results,
max_total=5)

    retrieval_time = (time.time() - start_time) * 1000

    print(f"Retrieved {len(contexts)} contexts in
{retrieval_time:.1f}ms")

    if not contexts:
        return "I couldn't find relevant scientific literature to
answer this
               question."

    # Show retrieved contexts
    for i, ctx in enumerate(contexts, 1):
        print(f"  {i}. {ctx['title'][:50]}... (similarity: \
            {ctx['similarity']:.3f})")

    # Step 2: Format prompt
    prompt = format_scientific_rag_prompt(question, contexts)

    # Step 3: Generate answer
    start_time = time.time()
    answer = call_ollama(prompt)
    generation_time = (time.time() - start_time) * 1000

    print(f"Generated answer in {generation_time:.1f}ms")
    print(f"Total time: {retrieval_time +
generation_time:.1f}ms")

    return {
        'answer': answer,
        'contexts': contexts,
        'stats': {
            'retrieval_time_ms': retrieval_time,
            'generation_time_ms': generation_time,
            'num_contexts': len(contexts)
        }
    }
```

Note that no token counting was implemented. Long scientific contexts can exceed model limits (e.g., 4,096 tokens for many Ollama models). Monitor token usage and implement truncation strategies when approaching limits.

Performance monitoring helps identify bottlenecks. Retrieval typically takes 50–200 ms with indexes. Generation takes 500–5,000 ms depending on answer length. Total response time under 5 seconds is good for scientific queries.

Returning contexts and statistics enables answer verification by users as well as debugging retrieval issues and providing input for system performance improvement.

While we estimate 50–200 ms for retrieval, remember that this is a local desktop application. Performance will vary based on your hardware. If you are running PostgreSQL on a standard HDD rather than an SSD, or if your CPU is busy generating embeddings, latency will increase. In this "experimental" phase, we prioritize correctness over millisecond latency; once your logic is sound, you can move toward optimizations like async drivers or GPU-accelerated PostgreSQL.

Demonstration and Interactive Interface

The demonstration showcases the system's capabilities and provides interactive tools for exploration.

> **NOTE**
>
> The demo fetches only three papers for quick setup. For meaningful evaluation, index at least 50 to 100 papers covering your domain of interest.

Main Demonstration Flow

This is a rudimentary scaffold for running the RAG system which will fetch papers from ArXiv if we don't have any:

```
def main():
    """Main demonstration of scientific RAG system"""
    print("=== Scientific RAG with PostgreSQL and pgvector
===\n")
```

```
    # Step 1: Setup database
    print("1. Setting up database...")
    db_config = setup_database()

    # Step 2: Check if we have papers
    conn = psycopg2.connect(**db_config)
    cursor = conn.cursor()
    cursor.execute("SELECT COUNT(*) FROM papers")
    paper_count = cursor.fetchone()[0]
    cursor.close()
    conn.close()

    if paper_count == 0:
        print("\n2. Fetching papers from ArXiv...")
        # Fetch papers on machine learning and AI see
        #https://arxiv.org/help/api/user-manual for full query
syntax
        papers = fetch_arxiv_papers('cat:cs.LG AND (abs:"machine
learning" OR \
            abs:"neural network")', max_results=3)

        if papers:
            print("\n3. Processing and storing papers...")
            process_and_store_papers(db_config, papers)
        else:
            print("No papers retrieved from ArXiv")
            return
    else:
        print(f"Found {paper_count} existing papers in database")
```

The demonstration intelligently checks for existing data, avoiding unnecessary reprocessing. The ArXiv query uses category filters (`cat:cs.LG`) and abstract searches (`abs:`) for precise paper selection.

ArXiv queries support Boolean operators (AND, OR, NOT) and field-specific searches—`cat:` specifies category, `abs:` searches abstracts. See the ArXiv API User's Manual for full query syntax.

Search Demonstrations

Here we demo the semantic search capability by giving phrases that we may be interested in and finding papers with similar concepts in the abstract

and/or sections:

```
    # Step 4: Demo semantic search
    print("\n4. Demonstrating semantic search...")

    search_queries = [
        "neural network architecture",
        "machine learning optimization",
        "deep learning applications"
    ]

    for query in search_queries[:1]:  # Show one example
        print(f"\nSearching abstracts for: '{query}'")
        results = search_papers_by_abstract(db_config, query,
limit=3)

        for i, result in enumerate(results, 1):
            print(f"\n{i}. {result['title']}")
            print(f"   Similarity: {result['similarity']:.3f}")
            print(f"   Authors: {', '.join(result['authors']
[:2])}")
            print(f"   Abstract: {result['abstract'][:150]}...")

        print(f"\nSearching sections for: '{query}'")
        section_results = search_paper_sections(db_config, query,
limit=3)

        for i, result in enumerate(section_results, 1):
            print(f"\n{i}. From: {result['title']}")
            print(f"   Similarity: {result['similarity']:.3f}")
            print(f"   Content: {result['content'][:200]}...")
```

Demonstrating both abstract and section search shows the different granularities of retrieval. Limiting output prevents information overload while showing system capabilities.

RAG Demonstration

In addition to search, if we want to synthesize answers to natural language questions, we use the RAG interface, assuming Ollama or some other LLM runner is available:

```
    # Step 5: Demo RAG if Ollama is available
    print("\n5. Checking Ollama...")
    if test_ollama():
        print("\n6. Demonstrating Scientific RAG...")

        questions = [
            "What are the main challenges in machine learning?",
            "How do neural networks learn from data?",
            "What are current trends in AI research?"
        ]

        for question in questions[:1]:  # Show one example
            result = scientific_rag_answer(db_config, question)

            if isinstance(result, dict):
                print(f"\nAnswer: {result['answer'][:300]}...")
                print(f"Based on {result['stats']
['num_contexts']} scientific \
                    sources")
            else:
                print(f"Answer: {result}")
    else:
        print("\n6. Skipping RAG demo (Ollama not available)")

    print("\n=== Demo completed! ===")
```

The questions are chosen to demonstrate different types of scientific queries:

Challenges question

Requires synthesis across papers

How question

Needs technical explanation

Trends question

Requires recent paper analysis

Interactive Search Interface

While a one-time search or a batch search allows regular extraction of interesting content, an interactive interface enables:

- Exploratory searches to understand the corpus
- Testing different query formulations
- Comparing abstract versus section search results
- Real-time Q&A for research

```
def interactive_search():
    """Interactive scientific search interface"""
    db_config = setup_database()

    print("=== Interactive Scientific Search ===")
    print("Commands: 'abstract [query]', 'section [query]', 'ask
[question]', \
        'quit'\n")

    while True:
        command = input("\n> ").strip()

        if command.lower() == 'quit':
            break

        parts = command.split(' ', 1)
        if len(parts) < 2:
            print("Please specify a command and query")
            continue

        cmd, query = parts[0].lower(), parts[1]

        if cmd == 'abstract':
            results = search_papers_by_abstract(db_config, query,
limit=3)
            for i, result in enumerate(results, 1):
                print(f"\n{i}. {result['title']}")
                print(f"   Similarity:
{result['similarity']:.3f}")
                print(f"   {result['abstract'][:200]}...")

        elif cmd == 'section':
            results = search_paper_sections(db_config, query,
```

```
limit=5)
            for i, result in enumerate(results, 1):
                print(f"\n{i}. From: {result['title']}")
                print(f"   Similarity:
{result['similarity']:.3f}")
                print(f"   {result['content'][:150]}...")

        elif cmd == 'ask':
            if test_ollama():
                result = scientific_rag_answer(db_config, query)
                if isinstance(result, dict):
                    print(f"\nAnswer: {result['answer']}")
                else:
                    print(f"Answer: {result}")
            else:
                print("Ollama not available for RAG")

        else:
            print("Unknown command. Use: abstract, section, ask,
or quit")

    print("\nGoodbye!")
```

Entry Point with Mode Selection

As mentioned earlier, supporting both demo and interactive modes accommodates different use cases—having a choice between the two is desirable.

We enable this in the following code:

```
if __name__ == "__main__":
    import sys

    if len(sys.argv) > 1 and sys.argv[1] == "interactive":
        interactive_search()
    else:
        main()
```

Technical Note on HNSW

As I mentioned in Chapter 3, HNSW stands for *hierarchical navigable small worlds*. It is a characteristic of many real-world networks, especially

those relating to language and conceptual hierarchies, that they have the HNSW property.

In practice, for approximate nearest-neighbor search over text corpora (and corpora derived from documents), the HNSW index is widely used. There are some questions about its performance and there are often differing statements made. So here I clarify them with some heuristic advice.

See Figure 7-6 for a conceptual picture of the HNSW index structure.

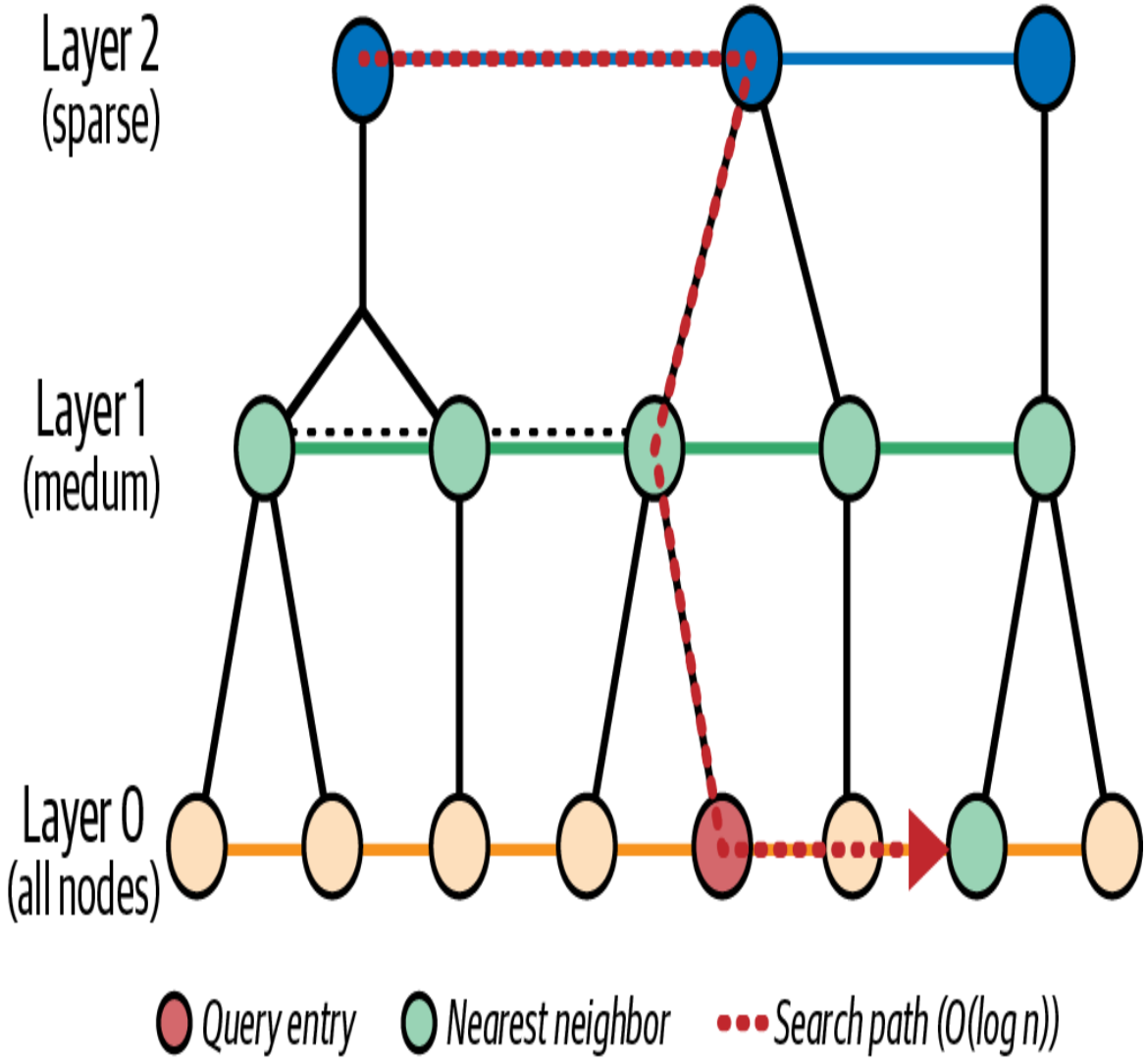


Figure 7-6. HNSW index structure (conceptual)

HNSW search is a best-first/greedy graph traversal with a bounded candidate set:

- You start at an entry point in the top layer and greedily walk to nearer nodes, descending layers.
- In the bottom layer, you run a best-first search maintaining:
    - A candidate queue (nodes to explore)
    - A "top results" set of size controlled by `efSearch`

Two knobs dominate runtime:

- `efSearch` (query-time exploration budget)
- Graph degree parameters (`M`, construction `efConstruction`), which influence navigability

If you hold `efSearch` constant while growing n, runtime can appear close to $O(\log n)$ on many datasets—until recall drops. If instead you scale `efSearch` up to maintain a fixed recall as n increases or as intrinsic dimension gets harder, query work can grow more quickly—sometimes looking closer to $O(n^{\alpha})$ for some α in (0,1), and in pathological cases approaching $O(n)$.

Here's a pragmatic way to think about it:

For a fixed dataset and fixed quality target (recall), HNSW query time is often modeled as:

$T(n) \approx O(L * \text{work per layer}) \approx O(\log(n) * f(\text{efSearch}, M, d, \text{data}))$

But that "$f(\ldots)$" is the real story: it can stay roughly constant (giving near-log scaling) or increase substantially with n, dimension, or distribution.

It trends more "log-like" in the following situations:

- Lower intrinsic dimension/strong cluster structure
- Good construction quality (adequate `efConstruction`, reasonable `M`)
- Moderate recall targets (not pushing to ~0.99+)

- Data not adversarial; distances have good contrast

It trends toward “linear-ish” in the following situations:

- Very high intrinsic dimension (distance concentration)
- Heavy overlap/near duplicates/“hubness” effects
- Poorly tuned construction (too small `M` or `efConstruction`)
- Very high recall requirement forcing large `efSearch`

The bottom line:

- The claim “HNSW is ($O(\log n)$)” is a practical/average-case heuristic, not a theorem.
- Between ($O(\log n)$) and ($O(n)$) is closer to the truth, with the caveat that where you land depends heavily on recall requirements and parameter scaling.

How to Evaluate Your Results

How do you know if your scientific RAG system is actually working? Since we are building this for experimental use, you should perform a spot-check evaluation:

Retrieval depth

Does the system find the exact paper you know exists for a specific query (i.e., recall)?

Context relevance

Does the LLM mention the paper’s title in its answer, or does it give a generic response?

Faithfulness

Does the LLM claim a paper said something that is actually in the retrieved snippet?

For a more rigorous approach, researchers use the RAGAS framework or LLM-as-a-Judge, where a larger model (like GPT-4o) grades the answers provided by your local Llama model. If you plan to rely on this for actual day-to-day use, please investigate more formal and automated ways of evaluation. It is a whole research area in itself.

Next Steps: Extending the Scientific RAG System

This implementation provides a foundation for scientific literature analysis but intentionally limits scope for clarity. Here are key areas for extension to build a more featured and useful research assistant:

Enhanced PDF processing

Formula extraction

Scientific papers contain mathematical equations that our text-only extraction misses. Integrating tools like LaTeX OCR or MathML extraction would capture this crucial information.

Figure and table analysis

Research often communicates key findings through visualizations. Multimodal models could describe figures and extract structured data from tables.

Citation parsing

Papers exist in citation networks. Extracting and following references would enable multihop reasoning across connected research.

Structured extraction

Instead of simple section detection, use machine learning models trained on scientific PDFs to identify methodology, results, and

contributions with high accuracy.

Advanced retrieval techniques

Hybrid search

Combine vector similarity with keyword matching, citation networks, and metadata filters for more precise retrieval.

Query decomposition

Complex scientific questions often have multiple parts. Breaking them down and searching for each component separately could improve coverage.

Iterative refinement

Allow the LLM to request additional searches based on initial results, enabling deeper exploration of topics.

Concept linking

Build a knowledge graph of scientific concepts and use it to expand queries with related terms and synonyms.

Improved RAG pipeline

Reranking

After the initial retrieval, use cross-encoder models specifically trained on scientific text pairs to reorder results by relevance.

Answer verification

Add a fact-checking step that verifies claims in generated answers against the source documents.

Multipaper synthesis

Develop strategies for combining information from multiple papers, handling contradictions, and identifying consensus.

Confidence scoring

Have the system indicate confidence levels based on the number and quality of supporting sources.

Domain specialization

Field-specific models

Use embedding models fine-tuned on specific scientific domains (physics, biology, computer science) for better semantic understanding.

Terminology handling

Build domain-specific dictionaries and acronym expansions to handle technical jargon.

Equation understanding

Integrate symbolic math processing to understand and manipulate mathematical expressions.

Experimental data

Extract and analyze numerical results, statistical tests, and experimental parameters.

Scale and performance

Distributed processing

Parallelize PDF processing and embedding generation across multiple workers for faster ingestion.

Incremental updates

Implement streaming ingestion that can add new papers without reprocessing the entire corpus.

Caching strategies

Cache embeddings, search results, and generated answers to reduce latency for common queries.

Index optimization

Tune HNSW parameters (`M`, `ef_construction`) based on corpus size and query patterns for optimal speed/recall trade-off.

User experience enhancements

Source highlighting

Show exactly which parts of retrieved documents support specific claims in answers.

Interactive refinement

Allow users to provide feedback on retrieval results to improve subsequent searches.

Export capabilities

Generate formatted citations, bibliography entries, and literature review drafts.

Collaborative features

Enable sharing of searches, annotations, and curated paper collections among research teams.

Quality assurance

Evaluation metrics

Implement standard IR metrics (precision, recall, F1) and RAG-specific metrics (answer accuracy, citation correctness).

Test datasets

Create domain-specific question-answer pairs with ground truth for systematic evaluation.

Human-in-the-loop

Add interfaces for expert feedback to continuously improve retrieval and generation quality.

Bias detection

Monitor for biases in paper selection and answer generation, ensuring that diverse perspectives are represented.

Integration capabilities

API endpoints

Expose REST/GraphQL APIs for integration with research tools and workflows.

Reference managers

Connect with Zotero, Mendeley, and EndNote for seamless bibliography management.

Writing assistants

Integrate with LaTeX editors and word processors for in-context literature search.

Continuous monitoring

Set up alerts for new papers matching saved queries, keeping researchers updated on their field.

These extensions would transform our educational demonstration into a comprehensive research platform, capable of accelerating scientific discovery by making vast literature accessible and synthesizable.

Conclusion

This scientific RAG system demonstrates how vector databases and language models can transform academic literature search. By combining semantic understanding with structured retrieval, we've created a system that can:

- Understand scientific concepts through semantic embeddings trained on academic text.
- Navigate document structure with hierarchical indexing of abstracts and sections.
- Synthesize across sources by retrieving from multiple papers and generating coherent answers.
- Maintain academic rigor through source attribution and confidence scoring.
- Scale efficiently using PostgreSQL with pgvector for usable performance up to ~1 million documents.

Future advances in embedding models, retrieval algorithms, and language models will only enhance these capabilities. The modular design ensures that the system can adopt these improvements without architectural changes.

By making scientific literature programmatically accessible and synthesizable, systems like this accelerate research by helping scientists discover connections, identify gaps, and build on existing knowledge more effectively than ever before.

Having looked at familiar content types such as PDF papers, we now move to parsing, storing, and searching data from a topical and popular consumer

application—the LLM chatbot—in the next chapter.

OceanofPDF.com

Chapter 8. Building a Complete Conversation Search and RAG System

In our digital age, we generate vast amounts of conversational data through AI assistants, chat applications, and collaborative platforms. These conversations often contain valuable insights, solutions to problems, creative ideas, and learned knowledge that become increasingly difficult to retrieve as the volume grows. How do you efficiently find that specific discussion about Python decorators from three months ago, or recall the architectural advice given for your web application?

In previous chapters, we explored how vector databases handle static research papers. However, our personal digital lives are messy and conversational. If you use LLMs like Claude or ChatGPT for work, you likely have thousands of messages containing “lost” insights. This chapter moves from searching public data to building a “second brain” for your own chat history.

This system handles private, personal knowledge with the nuanced understanding that conversations have unique characteristics:

Contextual dependencies

> Messages often reference previous exchanges, use pronouns, or build upon earlier concepts. A response like “That approach works well but consider the performance implications” is meaningless without the surrounding context.

Conversational flow

> Ideas develop across multiple message exchanges. A single search result might not capture the complete reasoning or solution that emerged from

a back-and-forth discussion.

Personal language patterns

Conversations with AI assistants develop their own vocabulary, shorthand, and reference patterns unique to the individual user's communication style and domain interests.

Privacy and control

Personal conversations require local storage and processing, not cloud-based solutions that compromise privacy or require ongoing API costs.

Temporal relevance

Recent conversations might be more relevant than older ones, but breakthrough insights from months ago shouldn't be lost in the noise.

System Goals and Capabilities

Our conversation search and RAG system will provide the following, but beyond the focus on LLM conversations with Claude, the system can be modified to extract and manage all kinds of personal online conversation threads:

Core search capabilities

Semantic discovery

Find relevant conversations based on meaning, not just keyword matches.

Contextual retrieval

Return conversation segments with surrounding context to maintain dialogue flow.

Cross-conversation synthesis

Answer questions by combining insights from multiple conversation threads.

Temporal navigation

Efficiently browse conversation history with both semantic and chronological organization.

RAG-powered insights

Knowledge synthesis

Generate comprehensive answers by combining relevant exchanges from across your conversation history.

Pattern recognition

Identify recurring themes, solutions, and evolving thoughts across time.

Decision support

Surface past decisions, reasoning, and outcomes to inform current choices.

Learning reinforcement

Rediscover explanations and concepts previously discussed but potentially forgotten.

Scalable system architecture

Web API interface

Utilize RESTful endpoints for integration with other applications and with interfaces.

Scalable storage

Implement PostgreSQL with pgvector for handling large conversation archives.

Batch processing

Enable efficient embedding generation and indexing for thousands of messages.

Real-time search

Achieve subsecond response times for interactive exploration.

System Architecture Overview

Our architecture consists of seven integrated components that work together to transform raw conversation data into an intelligent, searchable knowledge system (see Figure 8-1):

Data ingestion pipeline

JSON import system

Processes Claude conversation exports and other chat formats

Schema validation

Ensures data consistency and handles various export formats

Incremental updates

Supports adding new conversations without reprocessing existing data

Conversation database design

Hierarchical storage

Separate tables for conversations, messages, and embeddings

Relationship preservation

Maintains conversation threading and message ordering

Metadata management

Stores timestamps, sender information, and conversation context

Vector processing engine

Embedding generation

Converts messages to semantic vectors using `SentenceTransformers`

Batch optimization

Processes large message volumes efficiently

Vector indexing

HNSW indexes for fast similarity search across thousands of conversations

Contextual search system

Semantic similarity

Finds conceptually related messages across conversation boundaries

Context window retrieval

Returns surrounding messages to preserve conversational flow

Relevance scoring

Balances semantic similarity with conversational coherence

RAG integration layer

Context assembly

Combines relevant conversation segments into coherent prompts

Prompt engineering

Templates optimized for conversational knowledge synthesis

Local LLM integration

Ollama-powered generation for privacy and cost control

Web Service API

RESTful endpoints

Search, RAG, and analytics APIs for external integration

Real-time processing

Live search and answer generation

Statistics dashboard

System health and usage metrics

Demonstration interface

Interactive CLI

Command-line exploration of conversation data

Sample data generation

Testing with realistic conversation examples

Deployment tools

Complete system setup and configuration

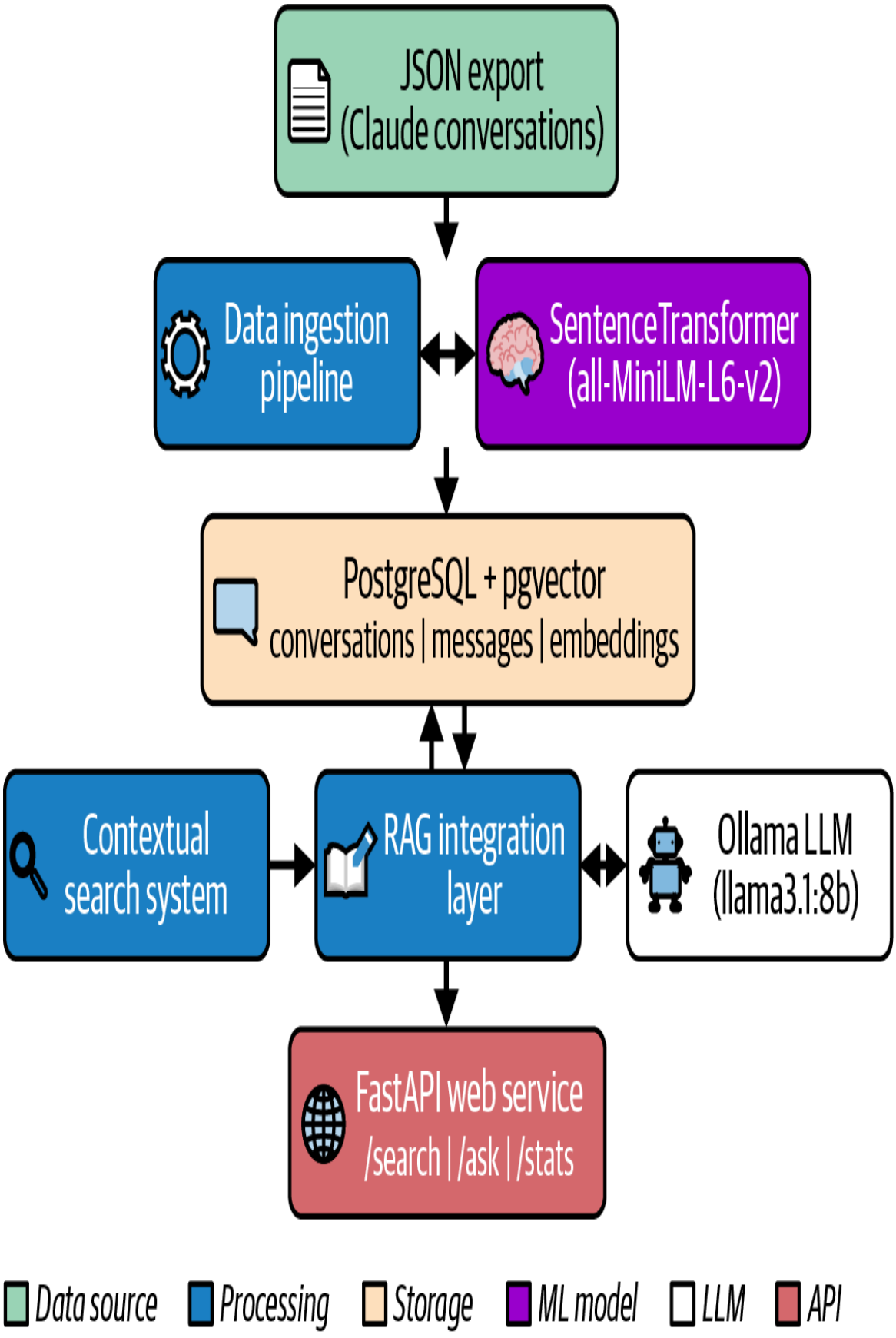

JSON export
(Claude conversations)
Data ingestion
pipeline
SentenceTransformer
(all-MiniLM-L6-v2)
PostgreSQL + pgvector
conversations | messages | embeddings
Contextual
search system
RAG integration
layer
Ollama LLM
(llama3.1:8b)
FastAPI web service
/search | /ask | /stats
Data source
Processing
Storage
ML model
LLM
API

Figure 8-1. High-level system architecture

What We'll Build Together

This chapter takes a complete systems approach, building not just the core RAG functionality but a fully featured application. We'll start with database design optimized for conversation storage, implement efficient batch processing for large-scale embedding generation, and culminate with a web API that can serve as the foundation for chat applications, personal knowledge management tools, or research platforms.

Unlike previous chapters that focused on specific techniques, this chapter demonstrates how all the pieces fit together in a real application. You'll see how to handle the messy realities of data import, optimize for both speed and accuracy, and build systems that can grow with your conversation history over months and years.

The FastAPI web service we'll build provides robust endpoints that can be integrated into other applications, making your conversation history a living, queryable knowledge base rather than static archived data.

By the end of this chapter, you'll have a complete system that not only demonstrates advanced RAG techniques but also provides immediate practical value for managing and extracting insights from your own AI conversation history. Let's begin building this personal knowledge assistant.

Database Foundation for Conversation Storage

Conversation data has unique characteristics that require specialized database design. Unlike documents or posts, conversations have inherent structure (threads, message ordering), metadata (timestamps, participants), and relationships (responses, context dependencies) that must be preserved for effective search and retrieval:

```
# Chapter 8: Minimal LLM Conversation Search & RAG System
# Simplified version demonstrating conversation storage, search,
and RAG with
pgvector

import psycopg2
import json
import requests
import time
from pathlib import Path
from sentence_transformers import SentenceTransformer
from datetime import datetime
from typing import List, Dict, Optional
from dataclasses import dataclass
```

The imports reveal our technology stack: PostgreSQL with pgvector for vector storage, sentence-transformers for embeddings, FastAPI for the web service, and standard libraries for data processing.

Designing the Conversation Schema

The database design reflects the hierarchical nature of conversations while optimizing for both relational queries and vector search:

```
def setup_database():
    """Setup PostgreSQL with pgvector extension and create
tables"""
    # Hardcoded database connection - update as needed
    DB_CONFIG = {
        'host': HOST,
        'port': PORT,
        'database': 'claude_conversations',
        'user': USER,
        'password': PASSWORD
    }

    conn = psycopg2.connect(**DB_CONFIG)
    cursor = conn.cursor()

    # Enable pgvector extension
    cursor.execute("CREATE EXTENSION IF NOT EXISTS vector;")
```

The dedicated database name `claude_conversations` reflects this system's focus on AI conversation history, though the schema could handle any conversational data.

Three-Table Architecture for Optimal Performance

Our schema uses three tables to balance normalized data integrity with query performance:

```
    # Create simple schema
    cursor.execute("""
        CREATE TABLE IF NOT EXISTS conversations (
            uuid TEXT PRIMARY KEY,
            name TEXT NOT NULL,
            created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
        );
    """)
```

The `conversations` table stores high-level metadata. Using UUIDs as primary keys matches Claude's export format and ensures global uniqueness. The conversation name provides human-readable context for search results:

```
    cursor.execute("""
        CREATE TABLE IF NOT EXISTS messages (
            uuid TEXT PRIMARY KEY,
            conversation_uuid TEXT REFERENCES
conversations(uuid),
            sender TEXT NOT NULL,
            text TEXT NOT NULL,
            created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
            message_index INTEGER
        );
    """)
```

The `messages` table stores individual exchanges with foreign key relationships to maintain conversation context. The `message_index` preserves message ordering within conversations—crucial for retrieving contextual windows around matching results:

```
    cursor.execute("""
        CREATE TABLE IF NOT EXISTS message_embeddings (
            message_uuid TEXT PRIMARY KEY REFERENCES
messages(uuid),
            embedding vector(384) NOT NULL
        );
    """)
```

You might wonder why we use two tables (`messages` and `message_embeddings`) instead of one. This is a classic normalization strategy. Text data is bulky and frequently accessed for display, while vectors are fixed-width numerical data used only for math. By separating them, we keep our primary message table lean and allow for easier re-indexing if we decide to change embedding models later (see Figure 8-2).

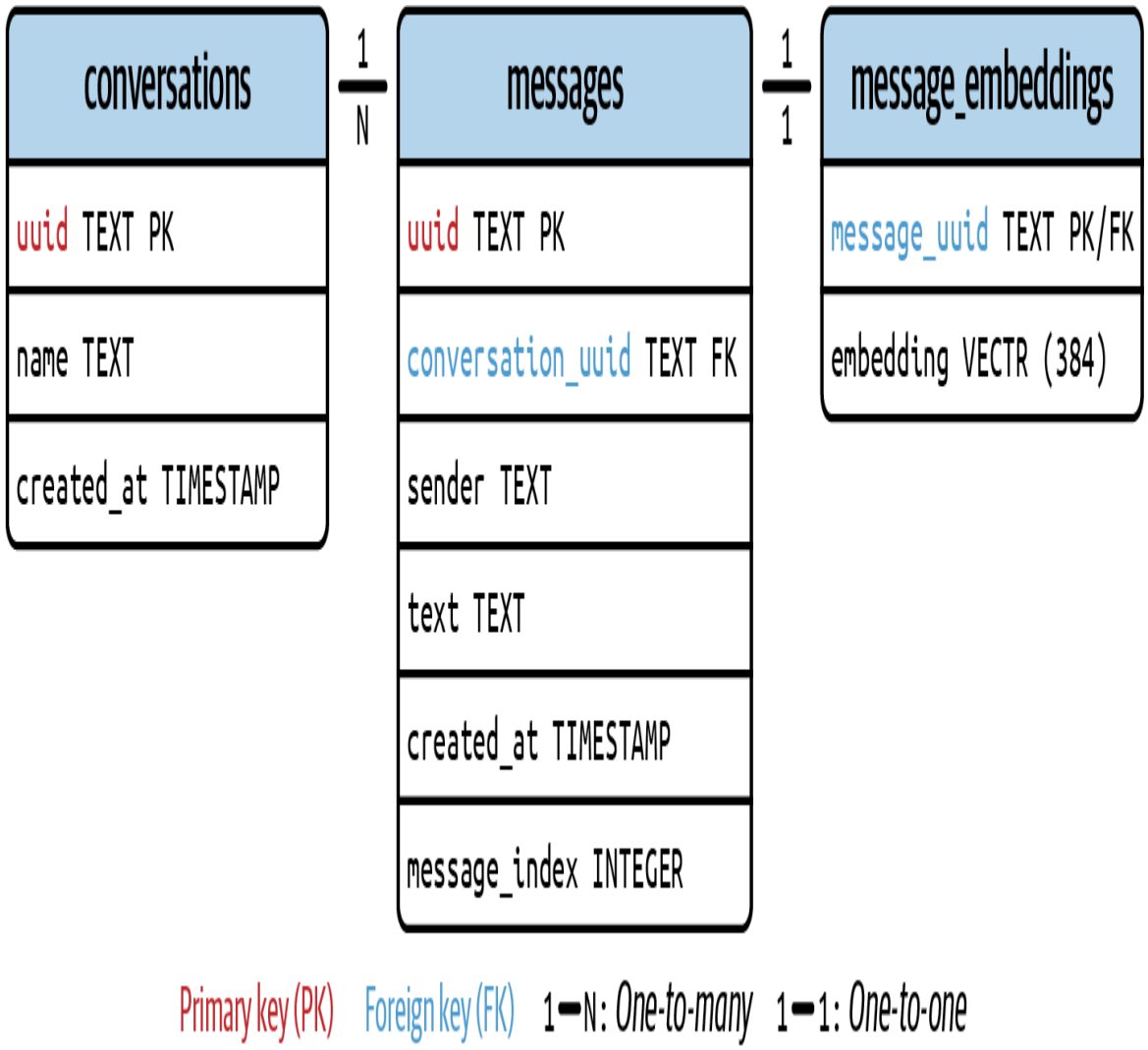


Figure 8-2. Entity-relationship diagram

High-Performance Vector Indexing

The HNSW index is crucial for subsecond search across large conversation archives:

```
# Create index for fast similarity search
cursor.execute("""
    CREATE INDEX IF NOT EXISTS idx_message_embeddings_vector
    ON message_embeddings
    USING hnsw (embedding vector_cosine_ops);
""")

conn.commit()
cursor.close()
```

```
    conn.close()

    print("Database setup completed")
    return DB_CONFIG
```

The `vector_cosine_ops` operator class optimizes for cosine similarity, ideal for normalized text embeddings.

The HNSW index provides sublinear search complexity, allowing it to scale efficiently as your conversation history grows. While default parameters work well for small datasets, performance can be tuned using `m` (maximum connections per node) and `ef_construction` (search buffer during index creation). For a desktop application under 100,000 messages, the defaults are sufficient, but larger archives may require increasing these values to maintain high recall.

Conversation Import and Data Processing Pipeline

Real-world conversation data comes in various formats and quality levels. Our import system handles Claude's JSON export format while providing extensibility for other conversation sources. Note that this is often a moving target as vendors change such formats with little to no warning—so be aware that this part of the system is the most brittle and may need occasional revisiting when it starts failing. At that point there will be a need for ingester redesign. Hopefully the change will be small enough that we can still maintain the database schema as is. See Figure 8-3 for current conversation data structure as of early 2026.

```
{ "conversations": [ // Array of conversation objects { "uuid": "550e8400-e29b...", // Unique conversation ID "name": "Python Learning", // User-visible title "created_at": "2024-01-15T10:00:00Z", // ISO 8601 timestamp "messages": [ // Ordered message array { "uuid": "msg-001-abc...", // Unique message ID "sender": "Human", // "Human" or "Assistant" "text": "Message content...", // Full message text "created_at": "2024-01-15T10:00:00z" }, { "uuid": "msg-002-def...", "sender": "Assistant", "text": "Response content...", "created_at": "2024-01-15T10:01: 002" } ] } ] }
```



Figure 8-3. Claude conversation export JSON structure

Robust JSON Import with Error Handling

The import function processes Claude's conversation export format, handling the nested structure of conversations containing message arrays:

```
def import_conversations(db_config, conversations_file):
    """Import conversations from Claude export JSON"""
    if not Path(conversations_file).exists():
        print(f"File {conversations_file} not found")
        return 0, 0

    conn = psycopg2.connect(**db_config)
    cursor = conn.cursor()

    with open(conversations_file, 'r') as f:
        data = json.load(f)

    conversations = data.get('conversations', [])
    total_conversations = 0
    total_messages = 0

    print(f"Processing {len(conversations)} conversations...")
```

The function returns counts of processed items, enabling the caller to track import progress and detect issues. Using `data.get('conversations', [])` provides graceful degradation for malformed files.

Atomic Transaction Processing

Each conversation is processed in its own transaction to ensure data consistency while allowing partial recovery from errors:

```
      for conv in conversations:
          try:
              # Insert conversation
              cursor.execute("""
                  INSERT INTO conversations (uuid, name,
  created_at)
                  VALUES (%s, %s, %s)
                  ON CONFLICT (uuid) DO NOTHING
              """, (
                  conv['uuid'],
                  conv['name'],

  datetime.fromisoformat(conv['created_at'].replace('Z', '+00:00'))
              ))

              if cursor.rowcount > 0:
                  total_conversations += 1
```

The `ON CONFLICT DO NOTHING` clause enables idempotent imports—reprocessing the same file won't create duplicates. The `rowcount` check tracks actually inserted records versus skipped duplicates.

Timestamp Handling and Data Validation

ISO timestamp processing requires careful handling of time zone information:

```
              # Insert messages
              messages = conv.get('messages', [])
              for idx, msg in
```

```
enumerate(conversation_data['messages']):
                cursor.execute("""
                    INSERT INTO messages (uuid,
conversation_uuid, sender, \
                        text, created_at, message_index)
                    VALUES (%s, %s, %s, %s, %s, %s)
                    ON CONFLICT (uuid) DO NOTHING
                """, (
                    msg['uuid'],
                    conv['uuid'],
                    msg['sender'],
                    msg['text'],

datetime.datetime.fromisoformat(msg['created_at'].replace('Z',
'+00:00')),
  idx
))

                if cursor.rowcount > 0:
                    total_messages += 1

            conn.commit()
```

> **NOTE**
>
> Python's `isoformat` handles the common Z UTC time zone starting with Python 3.11. For older Python versions, import `parser` from `dateutil` and use `parser.parse(msg['created_at'] )` instead of `datetime.datetime`.

Using `enumerate()` generates the message index automatically, preserving conversation order.

Error Recovery and Logging

Robust error handling ensures partial import success even when individual conversations have issues:

```
        except Exception as e:
            print(f"Error processing conversation
{conv.get('uuid', \
                'unknown')}: {e}")
```

```
                conn.rollback()
                continue

        cursor.close()
        conn.close()

        print(f"Imported {total_conversations} conversations with
    {total_messages} \
            messages")
        return total_conversations, total_messages
```

Per-conversation error logging with UUID identification helps diagnose import issues. Rolling back failed transactions prevents partial data corruption while allowing subsequent conversations to process successfully.

See Figure 8-4 for conversation import data flow.

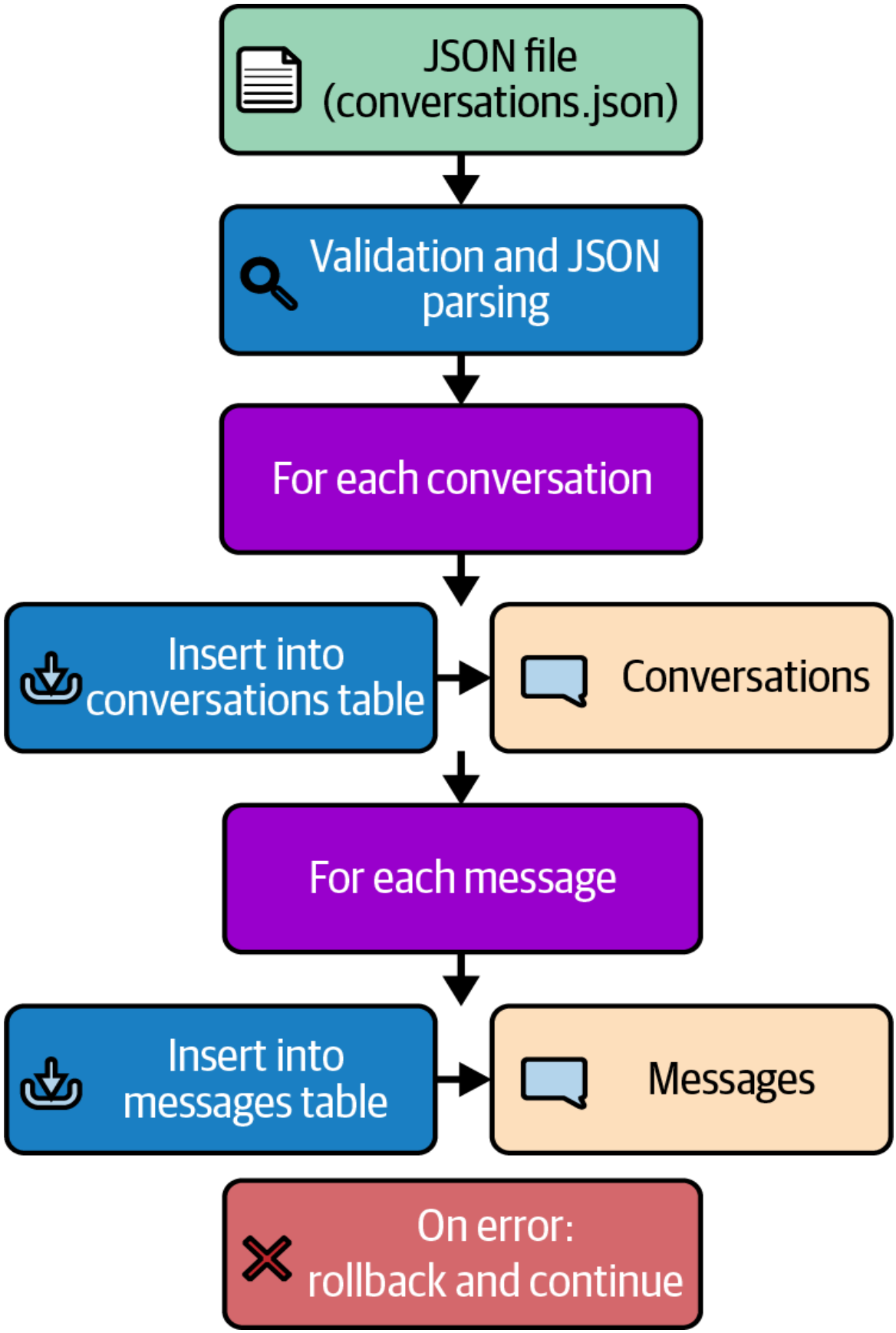

JSON file (conversations.json)
Validation and JSON parsing
For each conversation
Insert into conversations table
Conversations
For each message
Insert into messages table
Messages
On error: rollback and continue

Figure 8-4. Conversation import data flow

Efficient Embedding Generation and Batch Processing

Generating embeddings for large conversation archives requires careful attention to memory usage, processing speed, and progress tracking. Our approach balances throughput with system stability.

Singleton Pattern for Model Management

The embedding model is expensive to load and should be reused across all operations:

```
# Global embedding model
embedding_model = None

def get_embedding_model():
    """Get or initialize the embedding model"""
    global embedding_model
    if embedding_model is None:
        embedding_model = SentenceTransformer('all-MiniLM-L6-v2')
        print(f"Loaded embedding model (dimension: \

{embedding_model.get_sentence_embedding_dimension()})")
    return embedding_model
```

The all-MiniLM-L6-v2 model provides an excellent balance of quality and speed for conversational text. Its 384-dimensional output is compact enough for large-scale storage while rich enough to capture semantic nuances.

Incremental Processing Strategy

The embedding generation function only processes messages that don’t already have embeddings, enabling incremental updates as new conversations are added:

```
def generate_message_embeddings(db_config, batch_size=100):
    """Generate embeddings for messages that don't have them"""
    conn = psycopg2.connect(**db_config)
    cursor = conn.cursor()

    # Find messages without embeddings
    cursor.execute("""
        SELECT m.uuid, m.text
        FROM messages m
        LEFT JOIN message_embeddings me ON m.uuid =
me.message_uuid
        WHERE me.message_uuid IS NULL
        ORDER BY m.created_at
    """)

    messages = cursor.fetchall()

    if not messages:
        print("All messages already have embeddings")
        cursor.close()
        conn.close()
        return 0
```

The `LEFT JOIN` identifies messages without corresponding embeddings. Ordering by `created_at` processes conversations chronologically, which can be helpful for debugging and provides predictable progress tracking.

Batch Processing for Optimal Performance

Processing embeddings in batches dramatically improves performance by amortizing model overhead across multiple texts:

```
    print(f"Generating embeddings for {len(messages)}
messages...")

    model = get_embedding_model()
    total_generated = 0

    # Process in batches
    for i in range(0, len(messages), batch_size):
        batch = messages[i:i + batch_size]
        uuids = [msg[0] for msg in batch]
        texts = [msg[1] for msg in batch]
```

```
        # Generate embeddings
        embeddings = model.encode(texts, convert_to_numpy=True)
```

We chose the batch size of 100 to balance memory usage with processing efficiency. `SentenceTransformers` can process multiple texts simultaneously, significantly faster than individual encoding calls.

Database Insertion with Conflict Handling

The embedding insertion handles potential conflicts while tracking actual progress:

```
        # Insert embeddings
        data = [(uuid, emb.tolist()) for uuid, emb in zip(uuids,
embeddings)]
  cursor.executemany("""
                  INSERT INTO message_embeddings (message_uuid,
embedding)
                  VALUES (%s, %s)
                  ON CONFLICT (message_uuid) DO NOTHING
""", data)
  conn.commit()
  total_generated += len(batch)

  if i % (batch_size * 5) == 0:  # Progress update every 5
batches
            print(f"Generated {total_generated} embeddings...")

  cursor.close()
  conn.close()

    print(f"Generated {total_generated} message embeddings")
    return total_generated
```

See Figure 8-5 for the batch embedding pipeline.

Using `executemany()` reduces database round-trips by sending the entire batch in a single network call. This can improve ingestion speed by 10x to 50x compared to individual inserts.

Converting embeddings to lists with `.tolist()` is necessary for PostgreSQL's `vector` type. The progress updates every 500 messages (5

batches × 100) to provide feedback during long processing runs without overwhelming the console.

With our messages vectorized and indexed, we have effectively built a specialized library. But a library is only useful if the librarian understands the question. In the next section, we will implement the “retrieval” logic that translates a human question into a mathematical coordinate for our HNSW index.

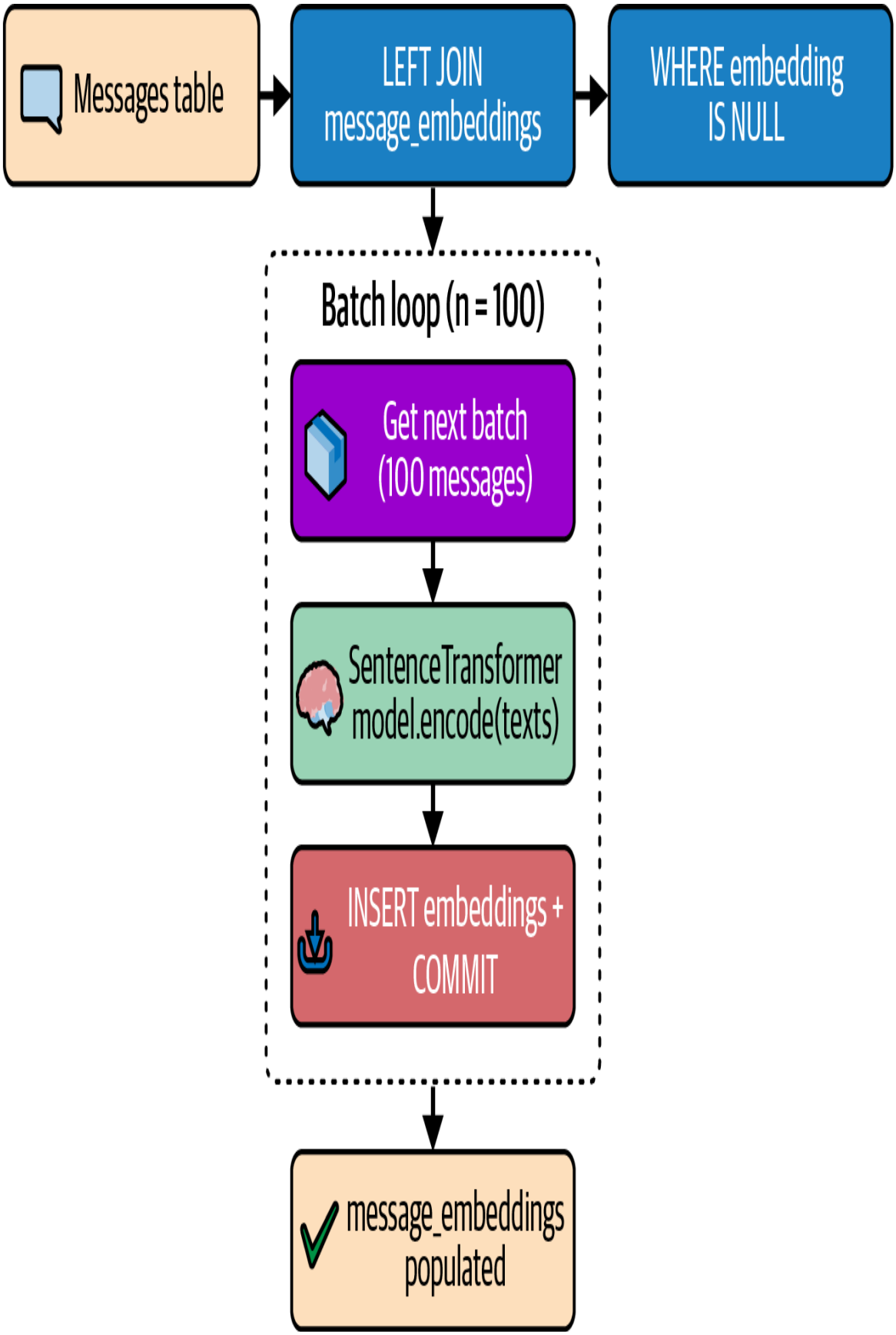

Messages table
LEFT JOIN message_embeddings
WHERE embedding IS NULL
Batch loop (n = 100)
Get next batch (100 messages)
SentenceTransformer model.encode(texts)
INSERT embeddings + COMMIT
message_embeddings populated

Figure 8-5. Batch embedding generation pipeline

Contextual Search with Conversational Understanding

Searching conversations requires more than finding semantically similar messages—it needs to understand and preserve the conversational context that makes individual messages meaningful.

Semantic Similarity Search

The core search function finds messages based on semantic similarity while retrieving essential context information:

```
def search_messages(db_config, query, limit=10, threshold=0.7):
    """Search for messages using semantic similarity"""
    conn = psycopg2.connect(**db_config)
    cursor = conn.cursor()

    # Generate query embedding
    model = get_embedding_model()
    query_embedding = model.encode([query])[0]
```

Encoding the query as a single-item list and accessing `[0]` ensures consistent array dimensionality with the stored embeddings.

Multitable Joins for Rich Context

The search query demonstrates the power of our normalized schema by joining across all three tables to provide comprehensive result context:

```
    # Search using pgvector cosine similarity
    cursor.execute("""
        WITH similarities AS (
            SELECT me.message_uuid,
1 - (me.embedding <=> %s::vector) as score
            FROM message_embeddings me
  )
  SELECT m.uuid, m.text, s.score
```

```
FROM similarities s
JOIN messages m
ON
s.message_uuid = m.uuid
  WHERE s.score > %s
  ORDER BY s.score DESC
  LIMIT %s;""" (query_embedding.tolist(), threshold, limit))
```

The `join` sequence starts with embeddings (for efficient vector filtering), then joins messages (for content and metadata) and conversations (for context). The similarity threshold filters out weakly related results, improving precision.

Result Formatting and Structure

The search results are formatted into a consistent structure that downstream components can rely on:

```
    results = cursor.fetchall()

    cursor.close()
    conn.close()

    # Format results
    formatted_results = []
    for row in results:
        formatted_results.append({
            'message_uuid': row[0],
            'text': row[1],
            'sender': row[2],
            'created_at': row[3],
            'conversation_name': row[4],
            'similarity': row[5]
        })

    return formatted_results
```

Returning dictionaries instead of raw tuples makes the code more maintainable and provides clear field names for consuming code.

Conversation Context Retrieval

Finding similar messages is only half the challenge—we need surrounding context to understand what each message means in its conversational flow:

```
def get_message_context(db_config, message_uuid, context_size=3):
    """Get surrounding context for a message"""
    conn = psycopg2.connect(**db_config)
    cursor = conn.cursor()

    # Get the target message details
    cursor.execute("""
        SELECT conversation_uuid, message_index, text, sender
        FROM messages
        WHERE uuid = %s
    """, (message_uuid,))

    target = cursor.fetchone()
    if not target:
        cursor.close()
        conn.close()
        return None

    conv_uuid, target_index, target_text, target_sender = target
```

The function first retrieves the target message's position information, which is essential for calculating the context window bounds.

> **NOTE**
>
> Context windows may be asymmetric at conversation boundaries (e.g., only one message before a match instead of the requested three). Applications should handle variable-sized context appropriately.

Context Window Calculation

The context retrieval uses the message index to extract a symmetric window around the target message:

```
    # Get surrounding messages
    cursor.execute("""
```

```
            SELECT
                uuid,
                text,
                sender,
                message_index,
                created_at
            FROM messages
            WHERE conversation_uuid = %s
              AND message_index >= %s
              AND message_index <= %s
            ORDER BY message_index
        """, (conv_uuid, target_index - context_size, target_index +
    context_size))

        context_messages = cursor.fetchall()

        cursor.close()
        conn.close()

        return {
            'target_message_uuid': message_uuid,
            'conversation_uuid': conv_uuid,
            'context_messages': [{
                'uuid': msg[0],
                'text': msg[1],
                'sender': msg[2],
                'message_index': msg[3],
                'created_at': msg[4],
                'is_target': msg[0] == message_uuid
            } for msg in context_messages]
        }
```

The `is_target` flag helps consuming code identify which message in the context window was the actual search match. A context size of 3 provides 7 total messages (3 before, the target, 3 after), usually sufficient to understand the conversational flow.

RAG Integration for Conversation History

The RAG pipeline transforms retrieved conversation fragments into coherent, contextual answers that synthesize knowledge from across your conversation history.

Structured Context Management

We use dataclasses to provide clean interfaces between the search and generation components:

```
@dataclass
class Context:
    text: str
    conversation_name: str
    sender: str
    similarity: float
```

This structure encapsulates all the information needed for both prompt construction and result attribution in this minimal implementation. Production systems might include additional fields such as `message_uuid` (for deduplication), `created_at` (for temporal awareness), and `message_index` (for conversation reconstruction).

Local LLM Integration with Ollama

Our LLM integration prioritizes privacy and control by using locally hosted models:

```
def call_ollama(prompt, model="llama3.1:8b"):
    """Simple Ollama API call"""
    url = http://localhost:11434/api/generate

    payload = {
        "model": model,
        "prompt": prompt,
        "stream": False,
        "options": {
            "temperature": 0.1,
            "top_p": 0.9
        }
    }

    try:
        response = requests.post(url, json=payload, timeout=30)
        response.raise_for_status()
        return response.json()['response']
```

```
    except Exception as e:
        return f"Error calling Ollama: {str(e)}"
```

The low temperature (0.1) encourages factual responses based on the provided context rather than creative generation. The 30-second timeout balances response time with the ability to process complex conversational contexts.

However, larger query payloads could cause longer generation times and 30 seconds may turn out to be too little. If you see repeated error messages at this point in the message flow, increase timeout to 60 or 90 seconds or even more.

Health Monitoring and Model Discovery

The health check function provides visibility into available models and helps with deployment troubleshooting:

```
def test_ollama():
    """Test if Ollama is running"""
    try:
        response =
requests.get("http://localhost:11434/api/tags", timeout=5)
        models = response.json()
        print("Available Ollama models:", [m['name'] for m in \
            models.get('models', [])])
        return True
    except:
        print("Ollama not running. Start with: ollama serve")
        return False
```

Listing available models helps users verify their preferred model is loaded and provides guidance for model management.

Context Retrieval and Assembly

The retrieval function bridges the search system and the generation pipeline:

```
def retrieve_contexts(db_config, question, max_contexts=5,
threshold=0.7):
    """Retrieve relevant contexts for RAG"""
    results = search_messages(db_config, question,
limit=max_contexts, \
        threshold=threshold)

    contexts = []
    for result in results:
        contexts.append(Context(
            text=result['text'],
            conversation_name=result['conversation_name'],
            sender=result['sender'],
            similarity=result['similarity']
        ))

    return contexts
```

This function transforms search results into the structured `Context` objects that the prompt formatting function expects, providing a clean abstraction layer.

Conversational Prompt Engineering

Prompts for conversation-based RAG require special consideration for the personal nature of the data and the conversational context:

```
def format_rag_prompt(question, contexts):
    """Format contexts into a prompt for the LLM"""
    if not contexts:
        return f"Question: {question}\n\nI don't have any
relevant information to
                                        answer this question."

    context_parts = []
    for i, ctx in enumerate(contexts, 1):
        context_parts.append(
            f"Context {i} (from '{ctx.conversation_name}' -
{ctx.sender}, \
                similarity: {ctx.similarity:.3f}):\n"
            f"{ctx.text}"
        )

    context_section = "\n\n".join(context_parts)
```

```
    prompt = f"""You are a helpful assistant answering questions
based on the
                user's conversation history with Claude.
Use ONLY the information provided in the contexts below to answer
the question.
If the answer cannot be found in the contexts, say so clearly.

Question: {question}

Relevant contexts from your conversations:
{context_section}

Answer based on the above contexts:"""

    return prompt
```

The prompt design includes several important elements:

Source attribution

Shows which conversation and sender each context comes from

Similarity scores

Helps the LLM weight information appropriately

Constraint enforcement

Explicitly instructs the model to use only provided context

Conversation framing

Acknowledges this is the user’s personal conversation history

Complete RAG Pipeline with Performance Monitoring

The main RAG function orchestrates the entire retrieval-to-generation pipeline:

```
def answer_question(db_config, question, max_contexts=5,
threshold=0.7):
    """Complete RAG pipeline to answer a question"""
    print(f"\nQuestion: {question}")
```

```
    # Step 1: Retrieve relevant contexts
    start_time = time.time()
    contexts = retrieve_contexts(db_config, question,
max_contexts, threshold)
    retrieval_time = (time.time() - start_time) * 1000

    print(f"Retrieved {len(contexts)} contexts in
{retrieval_time:.1f}ms")

    if not contexts:
        return "I couldn't find any relevant information in your
conversation
               history to answer this question."
```

Performance monitoring helps identify bottlenecks and provides user feedback during processing. Retrieval should typically complete in under 100 ms with proper indexing.

```
    # Step 2: Format prompt
    prompt = format_rag_prompt(question, contexts)

    # Step 3: Generate answer
    start_time = time.time()
    answer = call_ollama(prompt)
    generation_time = (time.time() - start_time) * 1000

    print(f"Generated answer in {generation_time:.1f}ms")
    print(f"Total time: {retrieval_time +
generation_time:.1f}ms")

    return {
        'answer': answer,
        'contexts': contexts,
        'stats': {
            'retrieval_time_ms': retrieval_time,
            'generation_time_ms': generation_time,
            'num_contexts': len(contexts)
        }
    }
```

Returning detailed statistics and the source contexts enables downstream applications to provide transparency about how answers were generated and from which conversations. See Figure 8-6 for the full RAG pipeline.

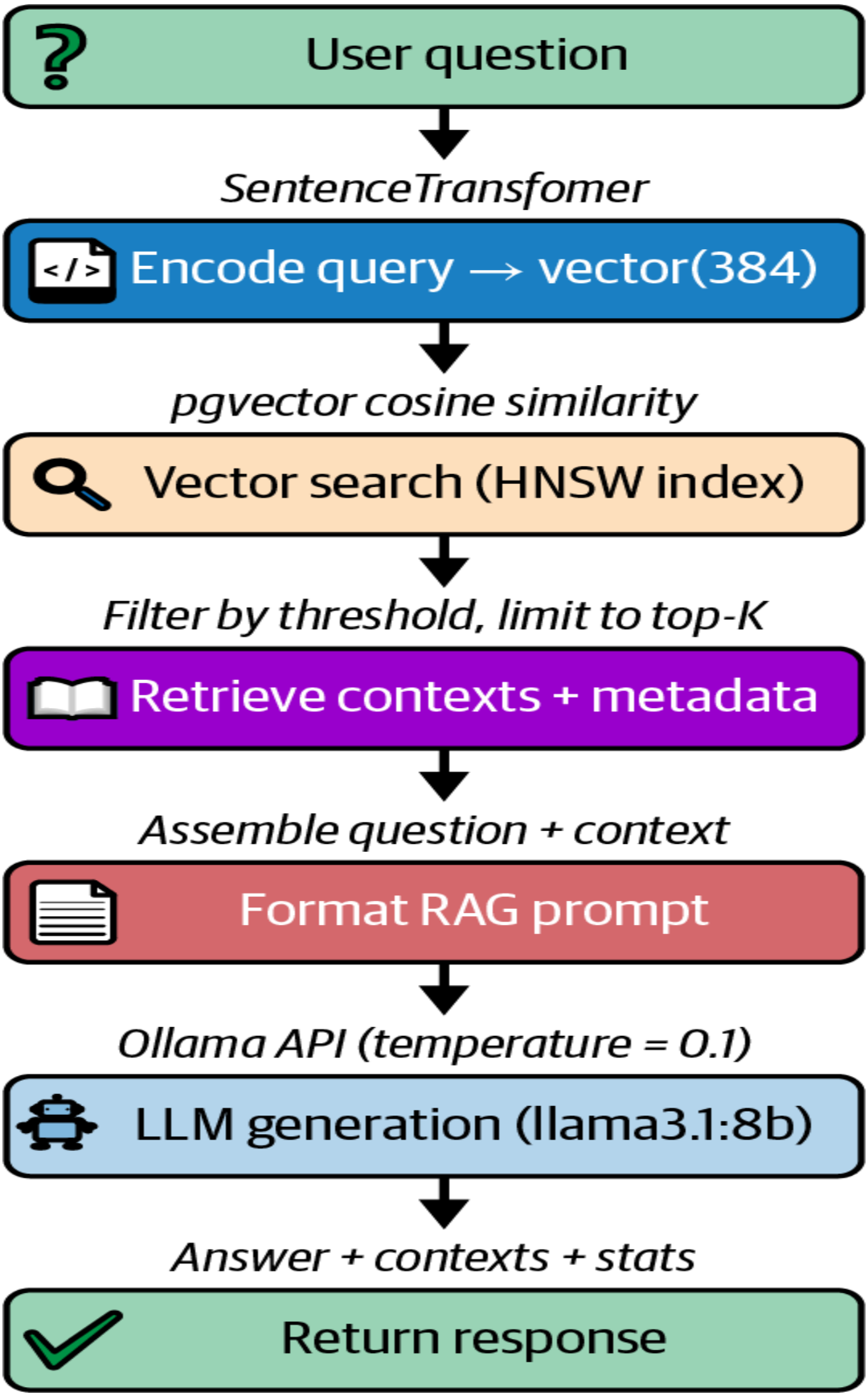

User question
SentenceTransfomer
Encode query → vector(384)
pgvector cosine similarity
Vector search (HNSW index)
Filter by threshold, limit to top-K
Retrieve contexts + metadata
Assemble question + context
Format RAG prompt
Ollama API (temperature = 0.1)
LLM generation (llama3.1:8b)
Answer + contexts + stats
Return response

Figure 8-6. RAG pipeline sequence diagram

Complete Web API with FastAPI

Building a web API transforms our conversation search system from a command-line tool into a reusable service that can power web applications and mobile apps or integrate with other systems.

FastAPI Application Structure

FastAPI provides automatic API documentation, request validation, and high-performance async capabilities:

```
from fastapi import FastAPI, HTTPException
from fastapi.middleware.cors import CORSMiddleware
from pydantic import BaseModel
from typing import List

app = FastAPI(title="Claude Conversation Search",
version="1.0.0")

# Add CORS for web access
app.add_middleware(
    CORSMiddleware,
    allow_origins=["*"],
    allow_methods=["*"],
    allow_headers=["*"],
)

# Global database config
DB_CONFIG = None
```

The CORS middleware enables web browser access from any origin, essential for development and client-side applications. In production, the `allow_origins` should be restricted to known domains.

Request Models with Validation

Pydantic models provide automatic request validation and API documentation:

```
class SearchRequest(BaseModel):
    query: str
    limit: int = 10
    threshold: float = 0.7

class RAGRequest(BaseModel):
    question: str
    max_contexts: int = 5
    threshold: float = 0.7
```

These models define default values for optional parameters and ensure type safety. FastAPI automatically generates OpenAPI documentation from these model definitions.

> **NOTE**
>
> Since we are using psycopg2 (a synchronous database driver), defining our endpoints without async allows FastAPI to run each request in a separate thread, preventing a single search from freezing the entire API.

Search Endpoint Implementation

The search endpoint provides direct access to semantic search capabilities:

```
@app.post("/search")
def search_endpoint(request: SearchRequest):
    """Search messages by semantic similarity"""
    try:
        results = search_messages(DB_CONFIG, request.query,
request.limit, \
            request.threshold)
        return {"results": results}
    except Exception as e:
        raise HTTPException(status_code=500, detail=str(e))
```

Using POST instead of GET allows for complex query parameters and avoids URL length limitations. The try-catch block ensures proper HTTP error responses for debugging.

RAG Question-Answering Endpoint

The RAG endpoint provides the complete question-answering pipeline:

```
@app.post("/ask")
def ask_endpoint(request: RAGRequest):
    """Ask a question using RAG"""
    try:
        result = answer_question(DB_CONFIG, request.question, \
            request.max_contexts, request.threshold)
        return result
    except Exception as e:
        raise HTTPException(status_code=500, detail=str(e))
```

This endpoint returns the full result structure including answer, contexts, and performance statistics, giving clients complete visibility into the response generation process.

System Statistics and Monitoring

The stats endpoint provides operational visibility into the system’s data and health:

```
@app.get("/stats")
def stats_endpoint():
    """Get database statistics"""
    try:
        conn = psycopg2.connect(**DB_CONFIG)
        cursor = conn.cursor()

        cursor.execute("SELECT COUNT(*) FROM conversations")
        total_conversations = cursor.fetchone()[0]

        cursor.execute("SELECT COUNT(*) FROM messages")
        total_messages = cursor.fetchone()[0]

        cursor.execute("SELECT COUNT(*) FROM message_embeddings")
```

```
            total_embeddings = cursor.fetchone()[0]

            cursor.close()
            conn.close()

            return {
                "total_conversations": total_conversations,
                "total_messages": total_messages,
                "total_embeddings": total_embeddings,
                "embedding_coverage": f"
    {(total_embeddings/total_messages) \
                    *100:.1f}%" if total_messages > 0 else "0%"
            }
        except Exception as e:
            raise HTTPException(status_code=500, detail=str(e))
```

The embedding coverage percentage helps operators understand if embedding generation is complete and identify when new messages need processing.

Server Startup and Configuration

The server startup function configures the global database connection and starts the ASGI server:

```
    def start_api(db_config):
        """Start the FastAPI server"""
        global DB_CONFIG
        DB_CONFIG = db_config

        import uvicorn
        print("\nStarting API server at http://localhost:8000")
        print("- Search endpoint: POST /search")
        print("- RAG endpoint: POST /ask")
        print("- Stats endpoint: GET /stats")
        print("- API docs: http://localhost:8000/docs")

        uvicorn.run(app, host="0.0.0.0", port=8000, log_level="info")
```

Binding `to 0.0.0.0` makes the service accessible from other machines on the network, essential for container deployment or remote access. The

automatic documentation at `/docs` provides an interactive interface for testing endpoints. See Figure 8-7 for FastAPI endpoint information.

Method	Endpoint	Request body	Response
POST	/search	{query, limit?, threshold?}	{results: [{message_uuid, text, sender, similarity, ...}]}
POST	/ask	{question, max_contexts?, threshold?}	{answer, contexts: [...], stats: {retrieval_time_ms, ...}}
GET	/stats	*none*	[total_conversations, total_messages, total_embeddings, embedding_coverage}
GET	/docs	*none*	OpenAPI interactive documentation (Swagger UI)

Base URL: http://localhost:8000 • CORS enabled for all origins

Figure 8-7. FastAPI endpoint structure

Demonstration and Sample Data

A complete application needs robust demonstration capabilities and sample data for testing and development.

Realistic Sample Data Generation

The sample data mimics real conversation patterns while covering different topic areas:

```
  def create_sample_data(db_config):
      """Create sample conversation data for testing"""
      sample_conversations = [
          {
              "uuid": "conv_001",
              "name": "Python Learning Discussion",
              "created_at": "2024-01-15T10:00:00Z",
              "messages": [
                  {
                      "uuid": "msg_001",
                      "sender": "Human",
                      "text": "I'm learning Python and struggling
  with list
                              comprehensions. Can you help?",
                      "created_at": "2024-01-15T10:00:00Z"
                  },
                  {
                      "uuid": "msg_002",
                      "sender": "Assistant",
                      "text": "List comprehensions are a concise
  way to create
                              lists. The basic syntax is
  [expression for item in
                              iterable]. For example: squares =
  [x**2 for x in
                              range(10)] creates a list of squares
  from 0 to 81.",
                      "created_at": "2024-01-15T10:01:00Z"
                  },
                  {
                      "uuid": "msg_003",
                      "sender": "Human",
                      "text": "That's helpful! What about filtering
  with
                              conditions?",
                      "created_at": "2024-01-15T10:02:00Z"
                  },
                  {
                      "uuid": "msg_004",
                      "sender": "Assistant",
                      "text": "You can add conditions with 'if'.
```

```
For example:
                            even_squares = [x**2 for x in
range(10) if x % 2
                            == 0] creates squares only for even
numbers.",
                    "created_at": "2024-01-15T10:03:00Z"
                }
            ]
        },
```

This conversation demonstrates a typical learning interaction with follow-up questions and detailed explanations. The realistic timestamps and UUIDs match Claude's export format.

Multitopic Sample Coverage

The second sample conversation covers a different domain to demonstrate cross-topic search capabilities:

```
        {
            "uuid": "conv_002",
            "name": "Machine Learning Concepts",
            "created_at": "2024-01-16T14:00:00Z",
            "messages": [
                {
                    "uuid": "msg_005",
                    "sender": "Human",
                    "text": "Explain the difference between
supervised and
                            unsupervised learning",
                    "created_at": "2024-01-16T14:00:00Z"
                },
                {
                    "uuid": "msg_006",
                    "sender": "Assistant",
                    "text": "Supervised learning uses labeled
training data to
                            learn patterns, like predicting house
prices from
                            features. Unsupervised learning finds
patterns in
                            data without labels, like clustering
customers into
                            groups.",
```

```
                    "created_at": "2024-01-16T14:01:00Z"
                },
                {
                    "uuid": "msg_007",
                    "sender": "Human",
                    "text": "What are some common algorithms for
each type?",
                    "created_at": "2024-01-16T14:02:00Z"
                },
                {
                    "uuid": "msg_008",
                    "sender": "Assistant",
                    "text": "Supervised: Linear regression,
decision trees,
                             random forest, SVM, neural networks.
                             Unsupervised: K-means clustering,
hierarchical
                             clustering, PCA, DBSCAN.",
                    "created_at": "2024-01-16T14:03:00Z"
                }
            ]
        }
    ]
```

Having conversations on different topics (Python programming and machine learning) allows testing cross-domain search and synthesis capabilities.

Sample Data Processing Pipeline

The sample data is converted to the expected JSON format and processed through the normal import pipeline:

```
    # Convert to the format expected by import_conversations
    import tempfile
    import json

    sample_data = {"conversations": sample_conversations}

    with tempfile.NamedTemporaryFile(mode='w', suffix='.json',
delete=False) \
        as f:
        json.dump(sample_data, f)
        temp_file = f.name
```

```
    import_conversations(db_config, temp_file)

    # Clean up
    Path(temp_file).unlink()
    print("Sample data created")
```

Using the standard import pipeline ensures that sample data processing exercises the same code paths as real data, improving test coverage and reliability.

Comprehensive System Demonstration

The main function orchestrates a complete system demonstration:

```
def main():
    """Main demonstration of the system"""
    print("=== Minimal Claude Conversation Search & RAG System
===\n")

    # Step 1: Set up database
    print("1. Setting up database...")
    db_config = setup_database()

    # Step 2: Check if we have data, if not create sample data
    print("\n2. Checking for existing data...")
    conn = psycopg2.connect(**db_config)
    cursor = conn.cursor()
    cursor.execute("SELECT COUNT(*) FROM messages")
    message_count = cursor.fetchone()[0]
    cursor.close()
    conn.close()

    if message_count == 0:
        print("No existing data found. Creating sample data...")
        create_sample_data(db_config)
    else:
        print(f"Found {message_count} existing messages")
```

The demonstration intelligently handles both fresh installations and systems with existing data, making it suitable for both development and production environments.

Progressive Feature Demonstration

Each step of the demonstration builds on the previous ones, showing the full capability of the system:

```
    # Step 3: Generate embeddings
    print("\n3. Generating embeddings...")
    generate_message_embeddings(db_config)

    # Step 4: Check Ollama
    print("\n4. Checking Ollama...")
    ollama_available = test_ollama()

    # Step 5: Demo search
    print("\n5. Demonstrating semantic search...")

    demo_queries = [
        "Python programming help",
        "machine learning algorithms",
        "how to learn data science"
    ]

    for query in demo_queries[:1]:  # Just show one example
        print(f"\nSearching for: '{query}'")
        results = search_messages(db_config, query, limit=3)

        for i, result in enumerate(results, 1):
            print(f"\n{i}. Similarity:
{result['similarity']:.3f}")
            print(f"   From: {result['conversation_name']}")
            print(f"   {result['sender']}: {result['text']
[:100]}...")
```

The search demonstration shows similarity scores and conversation context, helping users understand how the semantic search performs on different query types.

RAG Demonstration with Conditional Execution

The RAG demonstration only runs if Ollama is available, gracefully degrading the demonstration experience:

```
    # Step 6: Demo RAG if Ollama is available
    if ollama_available:
        print("\n6. Demonstrating RAG...")
        question = "What did I learn about Python list
comprehensions?"
        result = answer_question(db_config, question)

        if isinstance(result, dict):
            print(f"\nAnswer: {result['answer'][:200]}...")
            print(f"Used {result['stats']['num_contexts']}
contexts")
        else:
            print(f"Answer: {result}")
    else:
        print("\n6. Skipping RAG demo (Ollama not available)")

    # Step 7: Offer to start API
    print("\n7. Would you like to start the API server? (y/n)")
    if input().lower().startswith('y'):
        start_api(db_config)
    else:
        print("Demo completed!")
```

The interactive API server option allows users to immediately explore the web interface, transitioning seamlessly from command-line demonstration to web-based interaction.

Production Import Functionality

For real-world usage, the system provides a separate import mode for processing actual Claude conversation exports:

```
def import_from_file():
    """Import conversations from a Claude export file"""
    print("=== Import Claude Conversations ===\n")

    file_path = input("Enter path to conversations.json file:
").strip()
    if not file_path:
        print("No file specified")
        return

    # Set up database
    db_config = setup_database()
```

```
    # Import conversations
    total_conversations, total_messages =
import_conversations(db_config, \
        file_path)

    if total_messages > 0:
        print(f"\n3. Generating embeddings...")
        generate_message_embeddings(db_config)

    print("\nImport completed!")
```

This function provides a streamlined workflow for users with real conversation data, handling the complete pipeline from file import to embedding generation.

Application Entry Points

The main function supports multiple execution modes based on command-line arguments:

```
if __name__ == "__main__":
    import sys

    if len(sys.argv) > 1 and sys.argv[1] == "import":
        import_from_file()
    else:
        main()
```

This design pattern allows the same script to serve multiple purposes: demonstration with sample data, or production import of real conversation files.

Conclusion: A Complete Personal Knowledge System

This conversation search and RAG system demonstrates how vector databases can transform personal AI interactions from ephemeral exchanges into a persistent, queryable knowledge base. By combining PostgreSQL’s

relational capabilities with pgvector’s semantic search, we’ve built a system that understands both the structure and meaning of conversational data.

As AI assistants become more capable and conversations more valuable, systems like this bridge the gap between transient interactions and permanent knowledge. Rather than starting each conversation from scratch, we can build on the rich history of previous exchanges, creating a true conversational partnership that grows smarter over time.

The complete system we’ve built here—from database design through web API—provides the foundation for this future, where our conversations with AI become a living, searchable extension of our own memory and reasoning capabilities.

OceanofPDF.com

Chapter 9. Vector Query Language

This chapter is an experiment. It is not a typical technology book chapter. The goal here is to present a prototype of an idea for discussion and consensus building in the community for a longer-term collaborative development project, ideally an official standards body effort.

Vector Query Language (VQL) is a SQL-inspired query language designed for vector databases. It combines the familiar syntax of SQL with powerful vector operations, making it intuitive for developers while providing advanced capabilities for machine learning and AI applications.

This language doesn't currently exist as an implementation. This chapter therefore describes a hypothetical version, including its syntax and supported operations.

Being a first attempt at writing down such a language, there will be areas that may have been overlooked; certainly the scope is larger than what this chapter expresses. For instance, the equivalent of indexing and data definition language (DDL) are missing. The intent is to focus first on those aspects of VQL that are unique to vector spaces, leaving the routine aspects —such as creating collections and indexes, which follow naturally—for later discussion.

The larger intent is to inspire the community to come together and create a consensus implementation and then, in the future, have a standards body that works similarly to the SQL body, with representatives from vector database companies, tool builders, and middleware vendors. While these are still early days in the vector database industry, this is meant to start the initial conversation and let interested parties self-select to build an experimental implementation.

The GitHub site connected with this book has an area dedicated to VQL and serves as the seed for this effort.

VQL is motivated by the needs of three communities:

Application developers—who are used to and expect abstractions for databases and stored collections

> The proliferation of vector databases with different implementation paradigms ties an application tightly to the vector database implementation, and the elements of the database's metaphors now leak into the application and application architecture. In contrast, relational databases have mature abstractions, and application developers are insulated from underlying data structures and can program to the domain objects and relationships. This abstraction and a common access layer across vendors are both missing from the vector database landscape circa 2025. VQL provides these abstractions and a common access layer in one.

Database creators and maintainers—DevOps, SREs, and AI/machine learning data engineers who need to load, maintain, and optimize vector databases, often from multiple vendors

> Scripts to run these activities need to be customized to each vendor's offering, and different DDL/data management language (DML) syntax needs to be incorporated for each vendor. Even with AI, manual inspection of these scripts is necessary, as they have a high, potentially destructive, impact.

AI and machine learning researchers—in GenAI, signals and systems, topological data analysis, and vector embeddings, among others

> These practitioners need to write programs just to manipulate data. VQL serves as a macro language for composing transforms easily and running experimental algorithms on the data that runs in the database. This short-circuits the programming and program maintenance phase and changes it to writing VQL scripts that are easier to read, debug, and

maintain. This automation shortens the cycle time of experiments and increases velocity.

Core Concepts

There are two core concepts in VQL: the data model and the syntax that defines queries (i.e., operations that operate on the data in the model).

Data Model

The data model is the conceptual foundation of VQL. As in SQL, we have nested containers of data. But unlike SQL, the data comes from a homogeneous set—a vector space—and with it comes a vast array of mathematical heavy artillery.

However, in the initial cut of VQL, we will be concerned with only the elementary mathematical operations—and more so with the relationships between the elements of the data model:

Collection

: This is the top-level container for vector data (analogous to a database).

Table

: This is a structured set of vectors with associated metadata (analogous to a database table).

Vector

: This is a numerical array with fixed dimensions.

Embedding

: This is a kind of vector representing an encoded entity.

Field

In the language of SQL, we will consider an element of the vector to be a field, like a column is a field in a row. In the language of vector space, a field is the value of one of the dimensions in the *n*-dimensional space.

Metadata

This is data associated with a vector that is not part of the *n*-dimensional space that is the domain of the vector. Metadata consists of elements such as `create_date`, `author`, and `last_modified_date`. Note that the collection of metadata elements is not homogeneous.

Distance metric

VQL requires a distance metric to be defined on the vector space.

Basic Syntax Structure

While SQL has different categories of syntax—especially DDL and DML—in this initial definition of the language, we will be more concerned with the equivalent of the DML, that is, the operations we can carry out on the core elements of the data model—namely, vectors and their various aggregations (tables, collections, etc.).

We start off with a putative standard template for a VQL query in the following form:

```
SELECT subspace
FROM collection_name.table_name
VECTOR_OPERATION operation_parameters
WHERE metadata_filters
USING METRIC distance_metric
LIMIT top_k
```

This is the general structure for VQL queries, combining SQL's `SELECT`, `FROM`, `WHERE`, `ORDER BY`, and `LIMIT` with a new `VECTOR_OPERATION` clause to specify vector-space-specific operations. However, we will soon find that there are variations of this syntax that we need to accommodate.

For now, we take `VECTOR_OPERATION` to be one of many possible operations on a vector space without requiring us to be more specific about it—but we will define it further soon.

Vector Operations

The following examples of vector operations give us a hook to hang a hat on and convince ourselves that the term *VECTOR_OPERATION* is not vacuous. A fully fleshed-out version of this spec would have many more operations—these are here to give a flavor of what is possible.

Similarity Search

Let's take a look at similarity search queries.

Basic similarity search

This query performs a basic similarity search on the `product_vectors` table within the `ecommerce` collection. It finds the top 10 most similar vectors to the provided query vector `[1.2, 0.8, -0.2, 0.5]` using the cosine similarity metric:

```
-- Basic similarity search
SELECT *
FROM ecommerce.product_vectors
SIMILARITY SEARCH [1.2, 0.8, -0.2, 0.5]
USING METRIC cosine
TOP K 10;
```

Similarity search with metadata filtering

This query refines the similarity search by adding metadata filters. It retrieves the top five products that are similar to the query vector (using Euclidean distance) *and* belong to the `electronics` category and have a `price` less than `1000`. Note that `category` and `price` are metadata.

Additionally, it applies a `THRESHOLD` of 0.8, meaning only results with a similarity score (or distance, depending on metric interpretation) beyond

this threshold will be returned:

```
-- With metadata filtering
SELECT *
FROM ecommerce.product_vectors
SIMILARITY SEARCH [1.2, 0.8, -0.2, 0.5]
THRESHOLD 0.8
USING METRIC euclidean
WHERE category = 'electronics' AND price < 1000
TOP K 5;
```

Hybrid Search

This query performs a hybrid search, combining vector similarity with traditional text search. It searches the `paper_vectors` table in the `research` collection. The search is weighted, giving 70% importance to vector similarity with `[1.2, 0.8, -0.2]` and 30% importance to text matching “machine learning”:

```
-- Combining vector similarity with text search
HYBRID SEARCH (
    VECTOR [1.2, 0.8, -0.2] WEIGHT 0.7,
    TEXT "machine learning" WEIGHT 0.3
)
FROM research.paper_vectors
WHERE publication_year > 2020;
```

> **NOTE**
>
> The weights need to add up to 1.0 and represent the fraction of importance we give to each in the result.

A word about score fusion: Hybrid search combines normalized scores using weighted linear combinations: `final_score = w1 * norm(vector_score) + w2 * norm(text_score)`. Weights must be nonnegative. Text search uses BM25 by default (configurable via `SET TEXT_SEARCH_ALGORITHM`). Results are further filtered to include only papers published after 2020.

Range Search

This query finds all vectors within the `behavior_vectors` table of the `user_data` collection that are within a specified distance (`THRESHOLD` of 0.5) from a given `user_vector`, using the cosine distance metric. This is useful for finding all relevant items, not just the top *K*:

```
-- Find all vectors within a distance threshold
SELECT *
FROM user_data.behavior_vectors
RANGE SEARCH [user_vector]
THRESHOLD 0.5
USING METRIC cosine;
```

Batch Operations

This query performs a batch similarity search. Instead of a single query vector, it takes a set of query vectors retrieved from `user_queries.batch_requests` and finds the top five similar products for each query vector in the `ecommerce.product_vectors` table. This is efficient for processing multiple queries simultaneously:

```
-- Batch similarity search
BATCH SIMILARITY SEARCH (
    SELECT query_vectors FROM user_queries.batch_requests
)
FROM ecommerce.product_vectors
TOP K 5;
```

Batch operations return results grouped by input query vector. The output schema is `(query_id, result_rank, matched_vector, similarity_score)`. Execution may be parallelized (implementation-defined) but result ordering within each query is preserved.

Vector Functions and Aggregations

Here are some vector functions and aggregations.

Vector Functions

The vector functions we're going to look at are basic vector operations and vector arithmetic.

Basic vector operations

This query returns the dimensionality (number of components) of the `embedding` vector as `vector_dim`:

```
-- Get vector dimension
SELECT DIMENSION(embedding) AS vector_dim;
```

This query calculates the `cosine` distance between `vector1` and `vector2`, returning the result as `similarity`:

```
-- Calculate distance between vectors
SELECT DISTANCE(vector1, vector2, 'cosine') AS similarity;
```

This query concatenates `vector1` and `vector2` into a single, longer vector, returned as `combined_vector`:

```
-- Concatenate vectors
SELECT CONCAT(vector1, vector2) AS combined_vector;
```

This query calculates the dot product of `vector1` and `vector2`, returning the scalar result as `dot_product`:

```
-- Dot product
SELECT DOT(vector1, vector2) AS dot_product;
```

Vector arithmetic

This query performs vector addition, adding `influence_vector` to `vector_data` element-wise, returning the result as `adjusted_vector`:

```
-- Vector addition
SELECT vector_data + influence_vector AS adjusted_vector;
```

This query performs scalar multiplication, multiplying each element of `vector_data` by `scalar_value`, returning the result as `scaled_vector`:

```
-- Scalar multiplication
SELECT vector_data * scalar_value AS scaled_vector;
```

Vector Aggregations

The vector aggregations we'll look at are compute centroids and a query to find representative vectors.

Compute centroids

This query computes the average vector (`centroid`) for `user_vector` within each `user_category` in the `user_profiles` table:

```
-- Compute centroids
SELECT AVG(user_vector) AS centroid
FROM analytics.user_profiles
GROUP BY user_category;
```

Find representative vectors

This query computes the median vector (`geometric or L1 median`) for `feature_vector` within each `product_category` in the `product_features` table:

```
-- Find representative vectors
SELECT GEOMETRIC_MEDIAN(feature_vector) AS representative
FROM analytics.product_features
GROUP BY product_category;
```

The geometric median vector is typically more robust to outliers than the mean:

- The geometric median is the point that minimizes the sum of Euclidean distances to all vectors in the collection, whereas the centroid minimizes the sum of squared distances.
- The geometric median is more robust to outliers than the centroid, just as the scalar median is more robust than the scalar mean.
- Unlike the centroid, the geometric median has no closed-form solution and must be computed iteratively.
- A related concept is the *medoid*, which is the actual data point from your collection that minimizes the sum of distances to all other points.

Geometric median versus centroid

The centroid has a clean, closed-form solution (just average the coordinates), but the geometric median requires iterative algorithms:

The centroid solves: $\operatorname{argmin}_x \sum_i ||x - v_i||^2$

The geometric median solves: $\operatorname{argmin}_x \sum_i ||x - v_i||$

That removal of the square fundamentally changes the optimization landscape.

Robustness

If you have 10 vectors clustered together and one extreme outlier, the centroid gets dragged substantially toward the outlier. The geometric median barely moves. This mirrors exactly how the scalar median behaves. The centroid is easier to compute but less stable.

OceanofPDF.com

Index

A

abstract-level search, Abstract-Level Search-Abstract-Level Search

AdditiveQuantizer, Key Components

all-MiniLM-L6-v2, Most popular models and their characteristics

all-mpnet-base-v2, Most popular models and their characteristics

ANN (Approximate Nearest Neighbor) search

- – basics, Nearest-neighbor search
- – in FAISS
  - – avoiding computational cost, Avoid Computational Cost
  - – choosing an index, Choosing an Index in FAISS
  - – code example, Code Example
  - – key techniques, Key ANN Techniques in FAISS-Key ANN Techniques in FAISS

ArXiv, for paper search system with PostgreSQL pgvector, Building an ArXiv Paper Search System with PostgreSQL pgvector-Conclusion

- – advantages of ArXiv as a data source, Why ArXiv Makes an Ideal Data Source
- – architecture overview, Architecture Overview-Design Philosophy
  - – data flow, Data Flow
  - – design philosophy, Design Philosophy

    – system components, System Components

– challenges of searching scientific literature, The Challenge of Searching Scientific Literature-Technology Stack Rationale

– current limitations, Current Limitations

– database design for scientific papers, Database Design for Scientific Papers-Indexing Strategy

    – core tables structure, Core Tables Structure

    – indexing strategy, Indexing Strategy

    – schema design principles, Schema Design Principles

    – vector storage strategy, Vector Storage Strategy

– Docker packaging for local deployment, Docker Packaging for Local Deployment-Cloud-Ready Design

    – cloud-ready design, Cloud-Ready Design

    – container architecture, Container Architecture

    – database initialization scripts, Database Initialization Scripts

    – development workflow, Development Workflow

    – Docker compose configuration, Docker Compose Configuration

– embedding generation/storage, Embedding Generation and Storage-Batch Processing Pipeline

    – batch processing pipeline, Batch Processing Pipeline

    – embedding model strategy, Embedding Model Strategy

– enhancement ideas, Enhancement Ideas

– environment setup and dependencies, Environment Setup and Dependencies-Verification and Testing

    - directory structure and configuration, Directory Structure and Configuration
    - PostgreSQL and pgvector installation, PostgreSQL and pgvector Installation
    - Python environment configuration, Python Environment Configuration
    - verification and testing, Verification and Testing
- integration and PDF management, ArXiv Integration and PDF Management-Batch Processing System
    - ArXiv API client implementation, ArXiv API Client Implementation
    - batch processing system, Batch Processing System
    - PDF download pipeline, PDF Download Pipeline
- interactive application and UI, Interactive Application and UI
- PDF text extraction/processing, PDF Text Extraction and Processing-Intelligent Text Chunking
    - challenges, PDF Extraction Challenges
    - intelligent text chunking, Intelligent Text Chunking
- performance tuning, Basic Performance Tuning
    - index configuration, Index Configuration
    - query performance, Query Performance
    - resource management, Resource Management
- real-world use cases, Real-World Use Cases
- similarity search implementation, Similarity Search Implementation

– technology stack rationale, Technology Stack Rationale

B

bag of words, Word2Vec: The Breakthrough That Changed Everything

BART (Bidirectional and Auto-Regressive Transformers), Encoder-Decoder Transformers (T5, BART)

BERT (Bidirectional Encoder Representations from Transformers), Encoder-Only Transformers (BERT and Its Variants)

BLOBs (binary large objects), Why Do You Need Vector Databases?

C

chunking (text chunking), Intelligent Text Chunking, Intelligent Text Chunking, Advanced Text Chunking

composable components, Foundation

construction factor (efConstruction), efConstruction (construction factor)

continuous bag-of-words model, Word2Vec: The Breakthrough That Changed Everything

conversation search and RAG system, Building a Complete Conversation Search and RAG System-Conclusion: A Complete Personal Knowledge System

– complete web API with FastAPI, Complete Web API with FastAPI-Server Startup and Configuration

– FastAPI application structure, FastAPI Application Structure

– RAG question-answering endpoint, RAG Question-Answering Endpoint

– request models with validation, Request Models with Validation

  - search endpoint implementation, Search Endpoint Implementation
  - server startup and configuration, Server Startup and Configuration
  - system statistics and monitoring, System Statistics and Monitoring
- contextual search with conversational understanding, Contextual Search with Conversational Understanding-Context Window Calculation
  - context window calculation, Context Window Calculation
  - conversation context retrieval, Conversation Context Retrieval
  - multitable joins for rich context, Multitable Joins for Rich Context
  - result formatting and structure, Result Formatting and Structure
  - semantic similarity search, Semantic Similarity Search
- conversation import/data processing pipeline, Conversation Import and Data Processing Pipeline-Error Recovery and Logging
  - atomic transaction processing, Atomic Transaction Processing
  - error recovery and logging, Error Recovery and Logging
  - robust JSON import with error handling, Robust JSON Import with Error Handling
  - timestamp handling/data validation, Timestamp Handling and Data Validation
- database foundation for conversation storage, Database Foundation for Conversation Storage-High-Performance Vector Indexing

    - designing the conversation schema, Designing the Conversation Schema
    - high-performance vector indexing, High-Performance Vector Indexing
    - three-table architecture for optimal performance, Three-Table Architecture for Optimal Performance
- demonstration and sample data, Demonstration and Sample Data-Application Entry Points
- efficient embedding generation/batch processing, Efficient Embedding Generation and Batch Processing-Database Insertion with Conflict Handling
    - batch processing for optimal performance, Batch Processing for Optimal Performance
    - database insertion with conflict handling, Database Insertion with Conflict Handling
    - incremental processing strategy, Incremental Processing Strategy
    - singleton pattern for model management, Singleton Pattern for Model Management
- RAG integration for conversation history, RAG Integration for Conversation History-Complete RAG Pipeline with Performance Monitoring
    - complete RAG pipeline with performance monitoring, Complete RAG Pipeline with Performance Monitoring
    - context retrieval and assembly, Context Retrieval and Assembly
    - conversational prompt engineering, Conversational Prompt Engineering

– health monitoring and model discovery, Health Monitoring and Model Discovery

– local LLM integration with Ollama, Local LLM Integration with Ollama

– structured context management, Structured Context Management

– system architecture overview, System Architecture Overview

– system goals and capabilities, System Goals and Capabilities

cosine similarity, Similarity comparison

curse of dimensionality, Avoid Computational Cost

D

data definition language (DDL), Vector Query Language

data ingestion pipeline, for RAG, Text Processing and Embedding Generation

data management language (DML), Vector Query Language

data validation, Timestamp Handling and Data Validation

DDL (data definition language), Vector Query Language

decoder-only transformers, Decoder-Only Transformers (GPT Family)

dense embeddings, Doc2Vec: From Words to Documents

distance metrics, in vector similarity search, Distance Metrics-Cosine similarity

– cosine similarity, Cosine similarity

– inner-product distance, Inner-product distance

– L2 (Euclidean) distance, L2 (Euclidean) distance

DML (data management language), Vector Query Language

Doc2Vec, Doc2Vec: From Words to Documents-Doc2Vec: From Words to Documents

– Word2Vec versus, Doc2Vec: From Words to Documents

Docker, packaging for local deployment of ArXiv paper search system, Docker Packaging for Local Deployment-Cloud-Ready Design

– cloud-ready design, Cloud-Ready Design

– container architecture, Container Architecture

– database initialization scripts, Database Initialization Scripts

– development workflow, Development Workflow

– Docker compose configuration, Docker Compose Configuration

dual-search strategy, Intelligent Context Retrieval

E

efConstruction (construction factor), efConstruction (construction factor)

efSearch (search factor), efSearch (search factor)

embedding

– as term, Embedding

– varying definitions of, Embedding

embedding layers, in transformer models, The Embedding Layer: The Gateway to Zero-Shot Learning-Latest Developments and Trends

– anatomy of transformer embeddings, Anatomy of Transformer Embeddings

– connection to zero-shot learning, Connection to Zero-Shot Learning

– key characteristics that enable zero-shot learning, Key Characteristics That Enable Zero-Shot Learning

– latest developments and trends, Latest Developments and Trends

– limitations and considerations, Limitations and Considerations

embeddings, Embeddings-Conclusion

– dedicated embedding models, Embedding Models: The Specialized Vector Generators-Error handling

– embedding models versus embeddings as layers of LLMs, Distinction from Traditional Models

– practical applications and use cases, Practical Applications and Use Cases

– role in modern LLM applications, Role in Modern LLM Applications

– sentence-transformers library, The sentence-transformers Library: The Swiss Army Knife of Text Embeddings-Error handling

– simple RAG pipeline, Simple RAG Pipeline-Simple RAG Pipeline

– Doc2Vec, Doc2Vec: From Words to Documents-Doc2Vec: From Words to Documents

– embedding layers as gateway to zero-shot learning, The Embedding Layer: The Gateway to Zero-Shot Learning-Latest Developments and Trends

– anatomy of transformer embeddings, Anatomy of Transformer Embeddings

– bias and fairness, Bias and fairness

– computational costs, Computational costs

– key characteristics that enable zero-shot learning, Key Characteristics That Enable Zero-Shot Learning

– latest developments and trends, Latest Developments and Trends

– limitations and considerations, Limitations and Considerations

– quality of embeddings, Quality of embeddings

– role of embeddings in enabling zero-shot capabilities, Connection to Zero-Shot Learning

– sparse versus dense, Doc2Vec: From Words to Documents

– transformer architecture, From Embeddings to Modern Language Models: The Transformer Connection-Encoder-Decoder Transformers (T5, BART)

– varying definitions of, Embedding

– vector arithmetic with Word2Vec, Vector Arithmetic with Word2Vec: A Hands-On Guide-Final Words on Vector Arithmetic

– why we need them, Understanding Vector Embeddings: Why We Need Them

– Word2Vec, Word2Vec: The Breakthrough That Changed Everything

encoder-decoder transformers (T5, BART), Encoder-Decoder Transformers (T5, BART)

encoder-only transformers (BERT), Encoder-Only Transformers (BERT and Its Variants)

Euclidean (L2) distance, L2 (Euclidean) distance

F

FAISS (Facebook AI Similarity Search), Similarity Search with FAISS-Conclusion

– ANN search, The ANN Problem-Code Example

– avoiding computational cost, Avoid Computational Cost

– choosing an index in FAISS, Choosing an Index in FAISS

– code example, Code Example

– key ANN techniques in FAISS, Key ANN Techniques in FAISS-Key ANN Techniques in FAISS

– architecture and components, FAISS Architecture and Components-Further Exploration

– common workflow, Common Workflow

– core concepts, Core Concepts

– distance metrics, Distance Metrics-Cosine similarity

– example, Illustrative Example

– foundation, Foundation-Foundation

– further exploration, Further Exploration

– key components, Key Components-Key Components

– selection heuristics, Selection Heuristics

– vector representations, Vector Representations

– foundations, Foundations-Selection Heuristics

– indexes, FAISS Indexes-Choosing the Right Index

– choosing the right index, Choosing the Right Index-Choosing the Right Index

– composite and transformative indexes, Composite and Transformative Indexes

– flat indexes (brute force), Flat Indexes (Brute Force)

– HNSW-based indexes, HNSW-Based Indexes

– IVF-based indexes, IVF-Based Indexes

– LSH-based indexes, LSH-Based Indexes

– other specialized indexes, Other Specialized Indexes

– quantization, Quantization-When is PQ a good choice?

FastAPI, Complete Web API with FastAPI-Server Startup and Configuration

– application structure, FastAPI Application Structure

– RAG question-answering endpoint, RAG Question-Answering Endpoint

– request models with validation, Request Models with Validation

– search endpoint implementation, Search Endpoint Implementation

– server startup and configuration, Server Startup and Configuration

– system statistics and monitoring, System Statistics and Monitoring

FTS5 (SQLite Full-text-Search), Keyword Search Component

G

GPT (Generative Pretrained Transformer), Decoder-Only Transformers (GPT Family)

graph degree parameter (M), M (graph degree parameter)

H

hallucinations, by LLMs, Question-Answering Pipeline

health check function

– LLM integration with Ollama, Health Check Function

– RAG pipeline, Health Check and Model Discovery

health monitoring, Health Monitoring and Model Discovery

HNSW (hierarchical navigable small world) indexes, Understanding HNSW Indexes-Best Practices

– best practices, Best Practices

– efConstruction (construction factor), efConstruction (construction factor)

– efSearch (search factor), efSearch (search factor)

– how it works, How HNSW Works

– key parameters, Key Parameters Explained

– M (graph degree parameter), M (graph degree parameter)

– performance characteristics, Performance Characteristics

– practical example, Practical Example: Building a Similarity Search System

– what is HNSW?, What Is HNSW?

hybrid vector database systems (vector databases with traditional SQL databases), Hybrid Approaches: Combining Structured and Vector Data-Benefits of the Hybrid Approach

– benefits of hybrid approach, Benefits of the Hybrid Approach
– database architecture, Hybrid Database Architecture
– hybrid query example, Example of a Hybrid Query
– limitations of pure vector storage, Limitations of Pure Vector Storage
– need for both vector data and metadata, The Need for Both Vector Data and Metadata

I

incremental indexing, Extension: Incremental Indexing

Index (abstract base class), Key Components

IndexBinary, Key Components

IndexFlat, Key Components

IndexHNSW, Key Components

IndexIVF, Key Components

IndexIVFPQ, Key Components

IndexPQ, Key Components

ISO timestamp processing, Timestamp Handling and Data Validation

K

keyword searches, Keyword search (literal word matching)

king–queen analogy, Step 4: Classic King–Queen Analogy

L

L2 (Euclidean) distance, L2 (Euclidean) distance

large language models (LLMs)

– dedicated embedding models and, Embedding Models: The Specialized Vector Generators-Error handling

– hallucinations, Question-Answering Pipeline

– integration with Ollama, LLM Integration with Ollama, Local LLM Integration with Ollama, Local LLM Integration with Ollama

– health check function, Health Check Function

– Ollama API client, Ollama API Client

– intelligent context retrieval and, Intelligent Context Retrieval

– RAG systems and, Building a Retrieval-Augmented Generation System with SQLite VSS and Ollama

– transformer architecture, From Embeddings to Modern Language Models: The Transformer Connection

– vector similarity search and, Similarity Search

M

M (graph degree parameter), M (graph degree parameter)

metadata

– in hybrid vector database systems, The Need for Both Vector Data and Metadata-Benefits of the Hybrid Approach

– in VQL, Data Model

model discovery, Health Monitoring and Model Discovery

multilevel semantic search, Multilevel Semantic Search-Section-Level Search

– abstract-level search, Abstract-Level Search-Abstract-Level Search

– section-level search, Section-Level Search

N

Natural language processing (NLP), Where Do You Use Vector Databases?

nearest-neighbor search, Nearest-neighbor search

– (see also ANN (Approximate Nearest Neighbor) search)

NoSQL databases, NoSQL Versus Vector Databases

– limitations of vector extensions in, Limitations of Vector Extensions in NoSQL Databases

– vector storage in, NoSQL Databases and Vector Storage

– when to choose NoSQL databases with vector extensions, When to Choose NoSQL with Vector Extensions

O

OAI-PMH protocol, ArXiv API Client Implementation

Ollama, Building a Retrieval-Augmented Generation System with SQLite VSS and Ollama-Conclusion

P

paraphrase-multilingual-mpnet-base-v2, Most popular models and their characteristics

PDF parsing, limits of, Intelligent PDF Text Extraction

pgvector extension, Building a Scientific RAG System with PostgreSQL and pgvector-Conclusion

– (see also ArXiv, for paper search system with PostgreSQL pgvector)

– database foundation with, Database Foundation with pgvector-High-Performance Vector Indexes

– setup/installation, PostgreSQL and pgvector Installation

PostgreSQL, Building a Scientific RAG System with PostgreSQL and pgvector-Conclusion

– (see also ArXiv, for paper search system with PostgreSQL pgvector)

– setup and installation, PostgreSQL and pgvector Installation

PQ (product quantization), PQ-When is PQ a good choice?

– details of PQ in FAISS, Details of PQ in FAISS

– key advantages of PQ in FAISS, Key advantages of PQ in FAISS

– problems, The problem

– walk-through of PQ, Walk-through of PQ

– when to use, When is PQ a good choice?

product quantization (see PQ)

ProductQuantizer, Key Components

PyMuPDF, PDF Extraction Challenges

Python Reddit API Wrapper (PRAW)

– connecting to Reddit with, Connecting to Reddit with the Python Reddit API Wrapper-Usage Example

– client implementation, PRAW Client Implementation

– creating Reddit API credentials, Creating Reddit API Credentials

– example, Usage Example

Python, environment configuration for ArXiv paper search system, Python Environment Configuration

Q

quantization, Quantization-When is PQ a good choice?

– about, Foundations

– PQ, PQ-When is PQ a good choice?

– SQ, SQ-Trade-offs of SQ in FAISS

R

RAG pattern, embedding models and, Role in Modern LLM Applications

RAG system, building with SQLite VSS and Ollama, Building a Retrieval-Augmented Generation System with SQLite VSS and Ollama-Conclusion

– architecture overview, System Architecture Overview

– database foundation with vector support, Database Foundation with Vector Support-Creating Search Indexes

– schema design for RAG, Schema Design for RAG

– search index creation, Creating Search Indexes

– vector-enabled database setup, Setting Up the Vector-Enabled Database

– demonstration and testing, Demonstration and Testing-Quick Testing Utility

– interactive Q&A interface, Interactive Q&A Interface

– main demonstration function, Main Demonstration Function

– quick testing utility, Quick Testing Utility

    – sample data loading, Sample Data Loading

– extending the system, Next Steps: Extending the System-Advanced RAG Patterns

    – advanced RAG patterns, Advanced RAG Patterns

    – missing Reddit data features, Missing Reddit Data Features

    – performance optimizations, Performance Optimizations

    – product considerations, Production Considerations

– hybrid search implementation, Hybrid Search Implementation-Score Fusion and Ranking

    – hybrid search algorithm, Hybrid Search Algorithm

    – keyword search component, Keyword Search Component

    – score fusion and ranking, Score Fusion and Ranking

    – semantic search component, Semantic Search Component

– LLM integration with Ollama, LLM Integration with Ollama

– RAG pipeline, The RAG Pipeline-Question-Answering Pipeline

    – context formatting, Context Formatting

    – question-answering pipeline, Question-Answering Pipeline

– system architecture overview, System Architecture Overview

– text processing/embedding generation, Text Processing and Embedding Generation-Storing Content with Embeddings

    – embedding model management, Embedding Model Management

    – intelligent text chunking, Intelligent Text Chunking

    – storing content with embeddings, Storing Content with Embeddings

RAG system, with PostgreSQL and pgvector for scientific literature, Building a Scientific RAG System with PostgreSQL and pgvector-Conclusion

– advanced text chunking, Advanced Text Chunking

– architecture overview, Architecture Overview

– ArXiv integration and PDF processing, ArXiv Integration and PDF Processing-Intelligent PDF Text Extraction

    – intelligent PDF text extraction, Intelligent PDF Text Extraction-Intelligent PDF Text Extraction

    – paper discovery with ArXiv API, Paper Discovery with ArXiv API

– challenges of, Building a Scientific RAG System with PostgreSQL and pgvector

– database foundation with pgvector, Database Foundation with pgvector-High-Performance Vector Indexes

    – database configuration and setup, Database Configuration and Setup

    – high-performance vector indexes, High-Performance Vector Indexes

    – schema design for scientific purposes, Schema Design for Scientific Papers

– demonstration/interactive interface, Demonstration and Interactive Interface-How to Evaluate Your Results

    – entry point with mode selection, Entry Point with Mode Selection

    – evaluating results, How to Evaluate Your Results
    – interactive search interface, Interactive Search Interface
    – main demonstration flow, Main Demonstration Flow
    – RAG demonstration, RAG Demonstration
    – search demonstrations, Search Demonstrations
    – technical note on HSNW, Technical Note on HNSW
– embedding generation strategy, Embedding Generation Strategy
– extending the scientific RAG system, Next Steps: Extending the Scientific RAG System-Next Steps: Extending the Scientific RAG System
– multilevel semantic search, Multilevel Semantic Search-Section-Level Search
– RAG pipeline, The RAG Pipeline: Deep Dive-Complete RAG Execution Pipeline
    – complete RAG execution pipeline, Complete RAG Execution Pipeline
    – health check and model discovery, Health Check and Model Discovery
    – intelligent context retrieval, Intelligent Context Retrieval
    – local LLM integration with Ollama, Local LLM Integration with Ollama
    – scientific prompt engineering, Scientific Prompt Engineering
– storage pipeline with embeddings, Storage Pipeline with Embeddings-Storage Pipeline with Embeddings
– system goals and capabilities, System Goals and Capabilities

Reddit, semantic search over content (see semantic search, with SQLite3)

relational database management system (RDBMS)

– vector representation in, Vector Representation in a Relational Database Management System

relational databases, The Foundation of Business Math: Accounting Arithmetic

– (see also SQL (Structured Query Language) databases)

retrieval-augmented generation (RAG) (see RAG entries)

S

scalar quantization (SQ), SQ-Trade-offs of SQ in FAISS

ScalarQuantizer, Key Components

scientific literature (see ArXiv, for paper search system with PostgreSQL pgvector; RAG system, with PostgreSQL and pgvector for scientific literature)

score dominance, in hybrid search, Score Fusion and Ranking

search factor (efSearch), efSearch (search factor)

section-level search, Section-Level Search

semantic search, defined, Semantic search (meaning matching)

semantic search, multilevel, Multilevel Semantic Search-Section-Level Search

– abstract-level search, Abstract-Level Search-Abstract-Level Search

– section-level search, Section-Level Search

semantic search, with SQLite3, Semantic Search with SQLite3-Conclusion

– building the vector index, Building the Vector Index
  – index management, Index Management
  – understanding VSS index, Understanding VSS Indexing
– complete system architecture, Putting It All Together-Example Output
– connecting to Reddit with Python Reddit API Wrapper, Connecting to Reddit with the Python Reddit API Wrapper-Usage Example
  – client implementation, PRAW Client Implementation
  – creating Reddit API credentials, Creating Reddit API Credentials
  – example, Usage Example
– content extraction and preprocessing, Content Extraction and Preprocessing-Quality Filtering
  – quality filtering, Quality Filtering
  – text cleaning pipeline, Text Cleaning Pipeline
– database schema design, Designing the Database Schema-Schema Design Decisions
  – schema design decisions, Schema Design Decisions
  – schema requirements, Schema Requirements
  – table definitions, Table Definitions
– demonstration and sample data, Demonstration and Sample Data-Application Entry Points
  – application entry points, Application Entry Points
  – comprehensive system demonstration, Comprehensive System Demonstration
  – multitopic sample coverage, Multitopic Sample Coverage

  - production import functionality, Production Import Functionality
  - progressive feature demonstration, Progressive Feature Demonstration
  - RAG demonstration with conditional execution, RAG Demonstration with Conditional Execution
  - realistic sample data generation, Realistic Sample Data Generation
  - sample data processing pipeline, Sample Data Processing Pipeline
- generating and storing embeddings, Generating and Storing Embeddings-Batch Processing Pipeline
  - batch processing pipeline, Batch Processing Pipeline
  - database storage, Database Storage
  - embedding generator, Embedding Generator
- implementing semantic search, Implementing Semantic Search-Search Engine
  - search engine, Search Engine
  - search result container, Search Result Container
- incremental indexing, Extension: Incremental Indexing
- setting up the development environment, Setting Up the Development Environment-Operational Pragmas
  - installing dependencies, Installing Dependencies
  - operational pragmas, Operational Pragmas
  - verifying installation, Verifying the Installation

– sqlite-vss, Understanding the SQLite Vector Similarity Search Extension-Limitations
– architecture overview, Architecture Overview
– core capabilities, Core Capabilities
– limitations, Limitations

semantic similarity
– defined, Understanding Vector Embeddings: Why We Need Them

semantics, defined, Why Do You Need Vector Databases?

sentence-transformers library, The sentence-transformers Library: The Swiss Army Knife of Text Embeddings-Best practices when using sentence-transformers
– all-MiniLM-L6-v2 for RAG applications, all-MiniLM-L6-v2 for RAG applications
– best practices, Best practices when using sentence-transformers-Error handling
– batch processing, Batch processing
– error handling, Error handling
– GPU utilization, GPU utilization
– length handling, Length handling
– memory management, Memory management
– normalization, Normalization
– popular models and their characteristics, Most popular models and their characteristics
– training approach, Training approach

sequence-to-sequence models, Encoder-Decoder Transformers (T5, BART)

similarity comparison, Similarity comparison

similarity search, Similarity Search-Embedding, Similarity Search

– ArXiv paper search system and, Similarity Search Implementation

– basic search, Basic similarity search

– embedding, Embedding

– example, Similarity search example

– FAISS (see FAISS (Facebook AI Similarity Search))

– keyword search, Keyword search (literal word matching)

– metadata filtering, Similarity search with metadata filtering

singleton pattern, for model management, Singleton Pattern for Model Management

skip-gram model, Word2Vec: The Breakthrough That Changed Everything

sparse embeddings, Doc2Vec: From Words to Documents

SQ (scalar quantization), SQ-Trade-offs of SQ in FAISS

SQL (Structured Query Language) databases, SQL Versus Vector Databases

– accounting arithmetic, The Foundation of Business Math: Accounting Arithmetic

– combining with vector databases (see hybrid vector database systems)

– need for vector-specific capabilities as limitation of, The Need for Vector-Specific Capabilities

– vector representation in relational database management system, Vector Representation in a Relational Database Management System

sqlite-vss (SQLite Vector Similarity Search extension), Understanding the SQLite Vector Similarity Search Extension-Limitations

– architecture overview, Architecture Overview

– building a RAG system with (see RAG system, building with SQLite VSS and Ollama)

– core capabilities, Core Capabilities

– limitations, Limitations

structured data, combining with vector data (see hybrid vector database systems)

T

T5 (Text-to-Text Transfer Transformer), Encoder-Decoder Transformers (T5, BART)

term frequency–inverse document frequency (TF-IDF), Word2Vec: The Breakthrough That Changed Everything

timestamps, Timestamp Handling and Data Validation

transformers, From Embeddings to Modern Language Models: The Transformer Connection-Encoder-Decoder Transformers (T5, BART)

– decoder-only transformers (GPT Family), Decoder-Only Transformers (GPT Family)

– encoder-decoder transformers (T5, BART), Encoder-Decoder Transformers (T5, BART)

– encoder-only transformers (BERT), Encoder-Only Transformers (BERT and Its Variants)

U

UPSERT, Database Storage

V

vector addition and subtraction, Vector addition and subtraction

vector aggregations, Vector Aggregations-Robustness

– compute centroids, Compute centroids

– find representative vectors, Find representative vectors

– geometric median versus centroid, Geometric median versus centroid

– robustness, Robustness

vector arithmetic with Word2Vec, Vector Arithmetic with Word2Vec: A Hands-On Guide-Final Words on Vector Arithmetic

– classic king–queen analogy, Step 4: Classic King–Queen Analogy

– implementing vector arithmetic functions, Step 3: Implement Vector Arithmetic Functions

– interactive exploration tool, Step 6: Interactive Exploration Tool

– loading pretrained Word2Vec model, Step 2: Load Pretrained Word2Vec Model

– setup and installation, Step 1: Setup and Installation

– various analogies, Step 5: More Interesting Analogies

vector data type, A New Data Type: Vector

– nearest-neighbor search, Nearest-neighbor search

– operations with, What’s Different About the Vector Type?

– similarity comparison, Similarity comparison

– vector addition and subtraction, Vector addition and subtraction

vector databases (generally), Introduction to Vector Databases-Conclusion

– combining structured and vector data (see hybrid vector database systems)

– importance of, Why Do You Need Vector Databases?

– NoSQL versus, NoSQL Versus Vector Databases

– operations with, What's Different About the Vector Type?

– similarity search, Similarity Search-Embedding

– SQL versus, SQL Versus Vector Databases

– vector data type, A New Data Type: Vector

– where to use, Where Do You Use Vector Databases?

vector embeddings (see embeddings)

vector functions, Vector Functions

– basic vector operations, Basic vector operations

– vector arithmetic, Vector arithmetic

vector operations, Vector Operations-Batch Operations

– batch operations, Batch Operations

– hybrid search, Hybrid Search

– range search, Range Search

– similarity search, Similarity Search

Vector Query Language (VQL), Vector Query Language-Robustness

– core concepts, Core Concepts-Basic Syntax Structure
  – basic syntax structure, Basic Syntax Structure
  – data model, Data Model
– vector functions and aggregations, Vector Functions and Aggregations-Robustness
– vector operations, Vector Operations-Batch Operations

vector0 extension, Core Capabilities

vss0 extension, Core Capabilities

W

web API (see FastAPI)

Word2Vec, Word2Vec: The Breakthrough That Changed Everything, Doc2Vec: From Words to Documents

Z

zero-shot learning, embedding layers as gateway to, The Embedding Layer: The Gateway to Zero-Shot Learning-Latest Developments and Trends
– anatomy of transformer embeddings, Anatomy of Transformer Embeddings
– key characteristics that enable zero-shot learning, Key Characteristics That Enable Zero-Shot Learning
– latest developments and trends, Latest Developments and Trends
– limitations and considerations, Limitations and Considerations
– role of embeddings in enabling zero-shot capabilities, Connection to Zero-Shot Learning

OceanofPDF.com

About the Author

Nitin Borwankar is a seasoned data scientist and database professional with a background in the development and implementation of enterprise data solutions. With a career spanning over three decades, Nitin is known for his work on data science education, advocacy for the use of open source tools for data science, and contributions to open source machine learning curricula. He is a frequent speaker at conferences and approaches AI and LLMs from a pragmatic data application perspective.

OceanofPDF.com

Colophon

The animal on the cover of *Vector Databases* is a Telomian, a dog native to the Malay Peninsula and named after the Telom River. The Telomian is considered an Indian Pariah dog, also known as a Pye dog or village dog. These are free-roaming dogs with wild habits that scavenge in human settlements.

Telomians have a terrier-like temperament. They are affectionate and protective of people they know, but wary of strangers. They kill pests such as rats and snakes, for which villagers reward them with food. They can also hunt their own prey for sustenance. Since villagers along the Telom River build their huts on stilts above the ground, Telomians are known for their ability to climb ladders. They can also hold objects and open doors with their paws.

These small to midsized dogs grow to 15–19 inches tall and weigh 18–28 pounds. They have erect ears, curly tails, a blue-black tongue, and short red or yellow fur. They are muscular and athletic. Telomians howl rather than bark. These hardy dogs have a life expectancy of up to 15 years. They are usually healthy and unaffected by genetic diseases that are associated with breeding.

The Telomian dog is not a specific breed, but refers to Pye dogs in a specific geographic range. It is estimated that there may be around 800 million Pye dogs globally. Many of the animals on O'Reilly covers are endangered; all of them are important to the world.

The cover illustration is by José Marzan Jr., based on an antique line engraving from Lydekker's *Royal Natural History*. The series design is by Edie Freedman, Ellie Volckhausen, and Karen Montgomery. The cover fonts are Gilroy Semibold and Guardian Sans. The text font is Adobe Minion Pro; the heading font is Adobe Myriad Condensed; and the code font is Dalton Maag's Ubuntu Mono.

OceanofPDF.com